Java Core

Course notes

Nicola Bicocchi

Contents

A0 — Computing in the Age of AI 1

E1.1 — From Functions to Classes 6

E1.2 — Environment and Build Tools 19

E2.1 — Types and Expressions 27

E2.2 — Control Flow 36

E2.3 — Methods and References 40

E2.4 — Arrays 50

E2.5 — Strings 60

E2.6 — Wrapper Classes 66

E2.7 — Text Input and Output 72

E2.8 — Unit Testing with JUnit 75

E3.1 — Classes and Constructors 82

E3.2 — Encapsulation and Invariants 92

E3.3 — Inheritance and Polymorphism 98

E3.4 — Interfaces 114

E3.5 — Abstract Classes and Composition 125

E3.6 — Object Equality and Hashing 134

E3.7 — Immutability and Records 142

E3.8 — Enums and State Modelling 146

E3.9 — Nested Classes 148

E3.10 — SOLID Principles 152

E4.1 — Collections Lists and Iterators 158

E4.2 — Sets 174

E4.3 — Queues and Deques 179

E4.4 — Maps 183

E4.5 — Collection Ordering and Search 188
E5.1 — Exceptions and Propagation 193
E5.2 — Error Handling and Contracts 202
E5.3 — Resource Ownership 207
E6.1 — Generic Classes and Methods 209
E6.2 — Bounds and Wildcards 219
E6.3 — Type Erasure 225
E7.1 — Lambdas and Functional Interfaces 228
E7.2 — Function Composition and Optional 242
E7.3 — Stream Pipelines 247
E7.4 — Stream Collection and Reduction 255
E8.1 — Threads and Execution 262
E8.2 — Shared State and Synchronization 272
E8.3 — Thread Coordination 281
E8.4 — Executors Tasks and Futures 289

A0 — Computing in the Age of AI

Ten years from now: an open question

AI tools can generate code, explain programs, propose tests, and automate parts of software development.

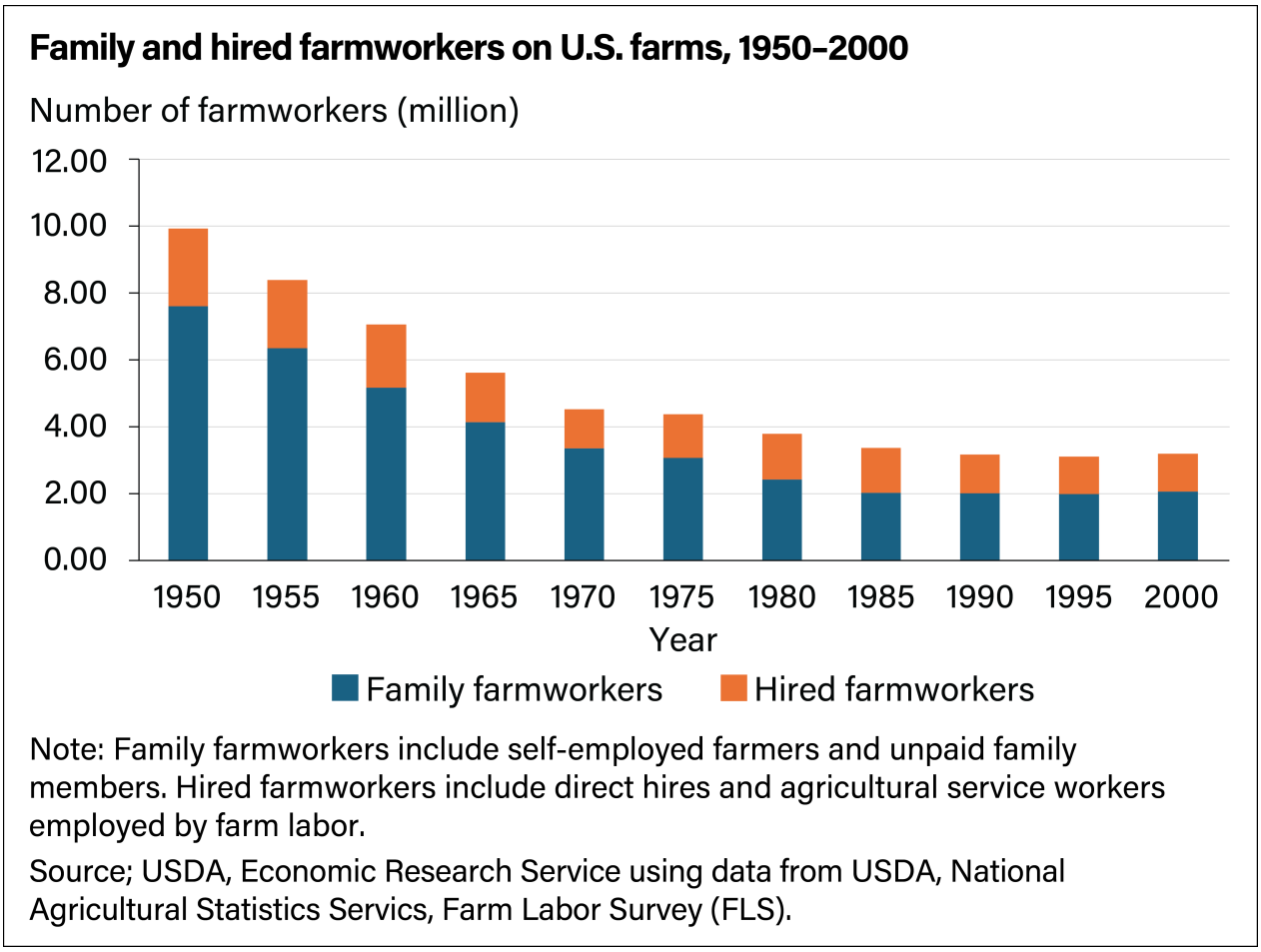
But **we do not know what computing jobs will look like in ten years**. Technical progress, reliability, adoption costs, regulation, and demand all affect the outcome.

We can examine how earlier technologies changed work and use those mechanisms to reason about possible futures.

Historical analogies are tools for thinking, not predictions.

Bear case: the tractor

Agricultural mechanization made it possible to produce much more with far less human labor. Over time, agricultural employment fell sharply in many industrialized economies.



Source: USDA Economic Research Service.

The tractor illustrates a possible mechanism:

> **A large increase in productivity can reduce the number of people needed, even when the product remains essential.**

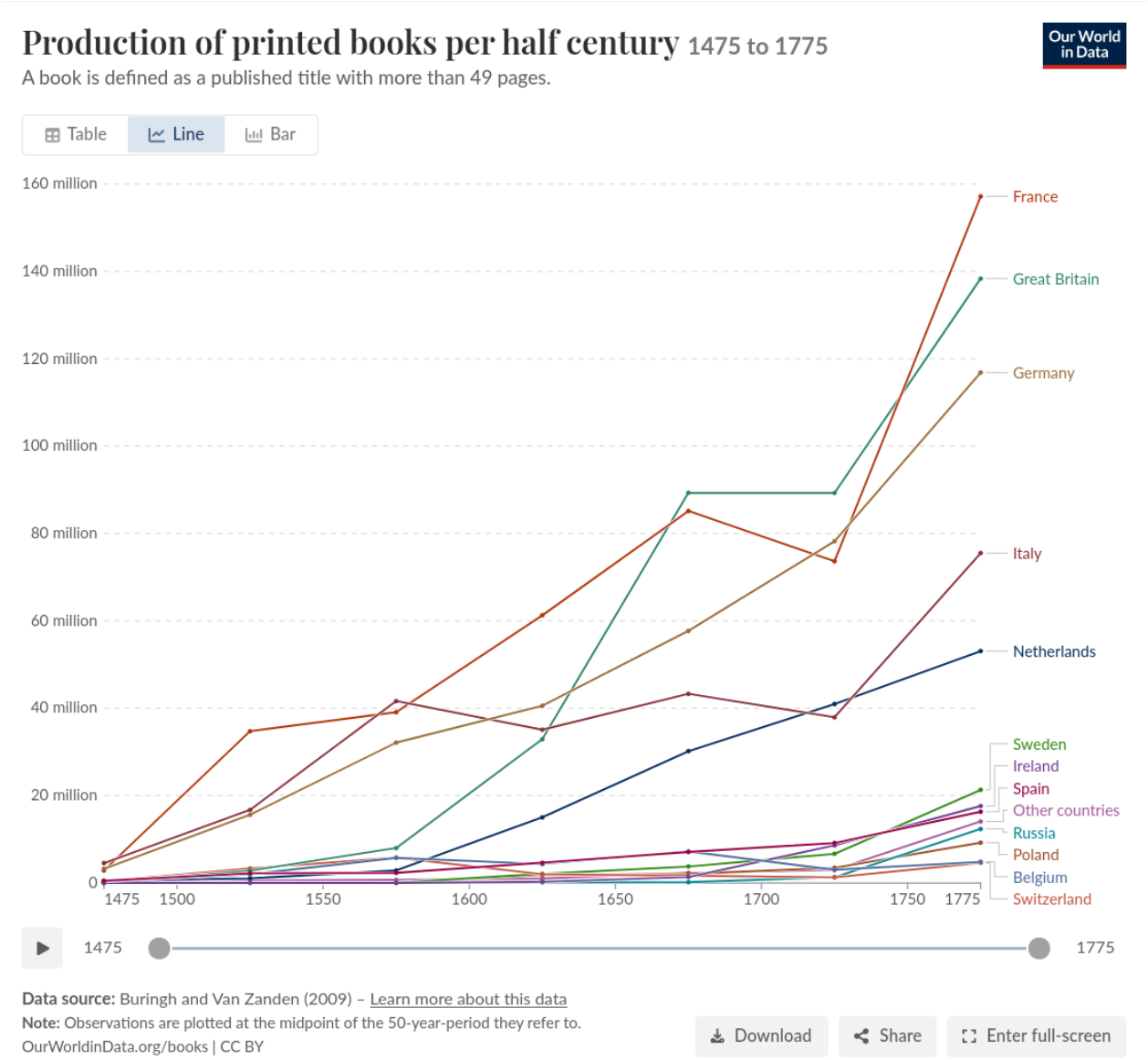
Applied to software: if AI reduces the labor needed per project and demand does not expand enough to compensate, fewer developers may be needed overall.

The analogy has limits. Agricultural change took decades and involved much more than tractors, including fertilizers, changes in farm organization, and wider economic transformation.

Bull case: the printing press

Printing with movable type greatly reduced the labor needed to reproduce texts. Its long-term significance was not just producing the same manuscripts with fewer workers.

Cheaper reproduction helped support a much larger market for books and, over time, newspapers and other publications, alongside expanding literacy and new institutions.



Source: Our World in Data, based on Buringh and Van Zanden (2009). CC BY.

The printing press illustrates another possible mechanism:

> **Lower production costs can make previously uneconomic uses viable and help create a much larger market.**

Applied to software: small organizations, specialized professions, and individual users may obtain tools that are currently too expensive to build and maintain.

What previously uneconomic software might look like

Consider:

- A small association that needs a tool for managing volunteers and equipment.
- A workshop that needs to connect an old machine to its planning system.
- A research group that needs a specialized data-processing application.
- A person who needs an accessibility tool adapted to a particular workflow.

The benefit may be real but too small to justify a conventional custom-software project. Lower development costs could change that calculation.

Generating the first version is not the whole cost: integration, security, hosting, support, and maintenance still matter. The optimistic scenario requires useful systems to become cheaper over their lifetime, not just cheaper code generation.

More software does not automatically mean more jobs

The printing press did not preserve the occupation of manuscript copying simply because publishing expanded.

Likewise, a growing software market would not guarantee that every existing computing role survives—or that the number of workers grows as quickly as output.

Both scenarios can occur at once:

- Much more software is produced.
- Each project needs fewer people for some tasks.
- Some roles shrink, while other activities and roles expand.
- Outcomes differ across sectors, organizations, and career stages.

Sector growth, employment, wages, and the security of a particular job are different outcomes.

Tasks change before job titles tell the whole story

Software work includes more than producing source code:

- Understand a problem and decide whether software is an appropriate solution.
- Turn incomplete requests into explicit requirements.
- Choose designs and integrate systems under real constraints.
- Check correctness, security, performance, and failure behavior.
- Operate systems and take responsibility for their consequences.

AI may assist or automate parts of all these activities. This is not a list of permanently protected tasks.

The useful distinction is **between activities and responsibilities**, not simply between “programmers” and “everyone else”.

Is computer engineering still a sensible degree choice?

Yes: it remains a well-motivated choice, including for employment. That is not a promise of an unchanged career or a guaranteed job.

There are concrete reasons to choose it:

- **Breadth:** computing skills apply across manufacturing, healthcare, energy, finance, public services, and many other sectors.
- **Participation in automation:** understanding computing helps us build, integrate, and evaluate the tools changing those sectors, rather than only use them.
- **Different career directions:** software, infrastructure, security, data, and embedded systems offer different paths, each requiring further learning and specialization.

These are reasons for adaptability, not immunity to disruption. Entry-level opportunities may also become harder to access if organizations automate tasks through which beginners traditionally learned.

> **The payoff of studying computer engineering is being able to take on harder, better-paid problems—and build products of your own. Use the classroom to develop skills that are difficult to acquire by prompting alone: understanding systems, diagnosing failures, and making things work under real constraints. AI can help you build a demo; those skills help you turn it into something people pay for.**

Resources

E1.1 — From Functions to Classes

Java in the real world

Java is used in several application areas:

- **Enterprise systems:** Java, often with Spring Boot, is widely used for APIs and services. It powers major backend systems at **Netflix, LinkedIn, Amazon, and Uber**, alongside other languages.
- **Event streaming: Apache Kafka**, originally developed at LinkedIn, is a widely used platform for event streams, built primarily with Java and Scala.
- **Applications and developer tools: JetBrains software**, and **Jenkins** are familiar examples of software built with Java.
- **Android and the JVM ecosystem:** Java remains relevant through existing Android code and interoperability with Kotlin. Android has its own runtime; on the JVM, Java also shares an ecosystem with Kotlin, Scala, and other languages.

Beyond these applications, **Java is one of the most established expressions of class-based object-oriented programming**. Its language features, standard libraries, and ecosystem provide a rich foundation for studying how to organize software around objects, responsibilities, and clear boundaries.

That is why we use Java in this course: not just to learn a widely used language, but to practice structured object-oriented design. Using Java does not automatically produce good design; learning to make those design choices is our goal.

Why this course?

The program works. Then something changes.

Imagine a program that stores a student's marks and calculates their average. It works.

Then the requirements evolve:

1. **Validate marks:** only values from 0 to 30 are accepted. A value such as 35 must be rejected, not included in the average.
2. **Calculate a weighted average:** an exam counts twice as much as a short quiz. Adding all marks and dividing by their number is no longer the correct calculation.
3. **Import marks from a file:** another developer adds an alternative to manual entry. Imported marks must follow the same validation rules as marks entered manually.

- Where should the validation rules live?

- Does every way of entering a mark apply the same rules?
- How many parts must change when the calculation changes?
- How do we know that existing behavior still works?

Each calculation may be simple. **Keeping the whole program consistent as it changes is the harder problem.**

From solving a problem to maintaining a program

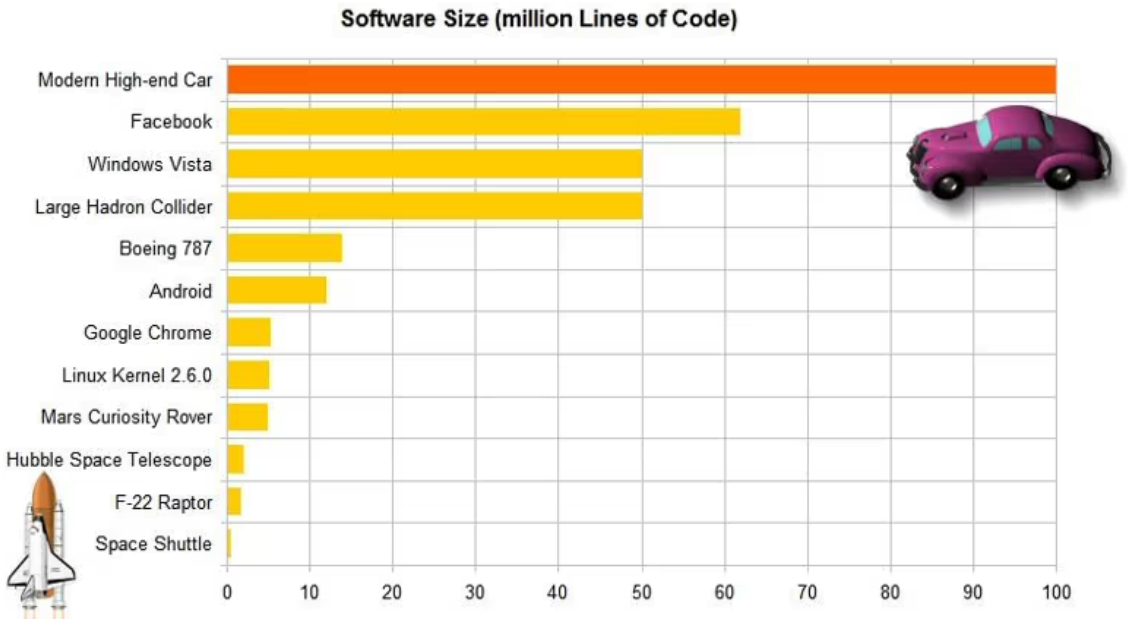
Knowing variables, loops, and functions lets us express algorithms. As programs grow, we also need to decide:

- Which part is responsible for each rule?
- What does one part need to know about another?
- What can change independently?
- How can we check each part without understanding everything at once?

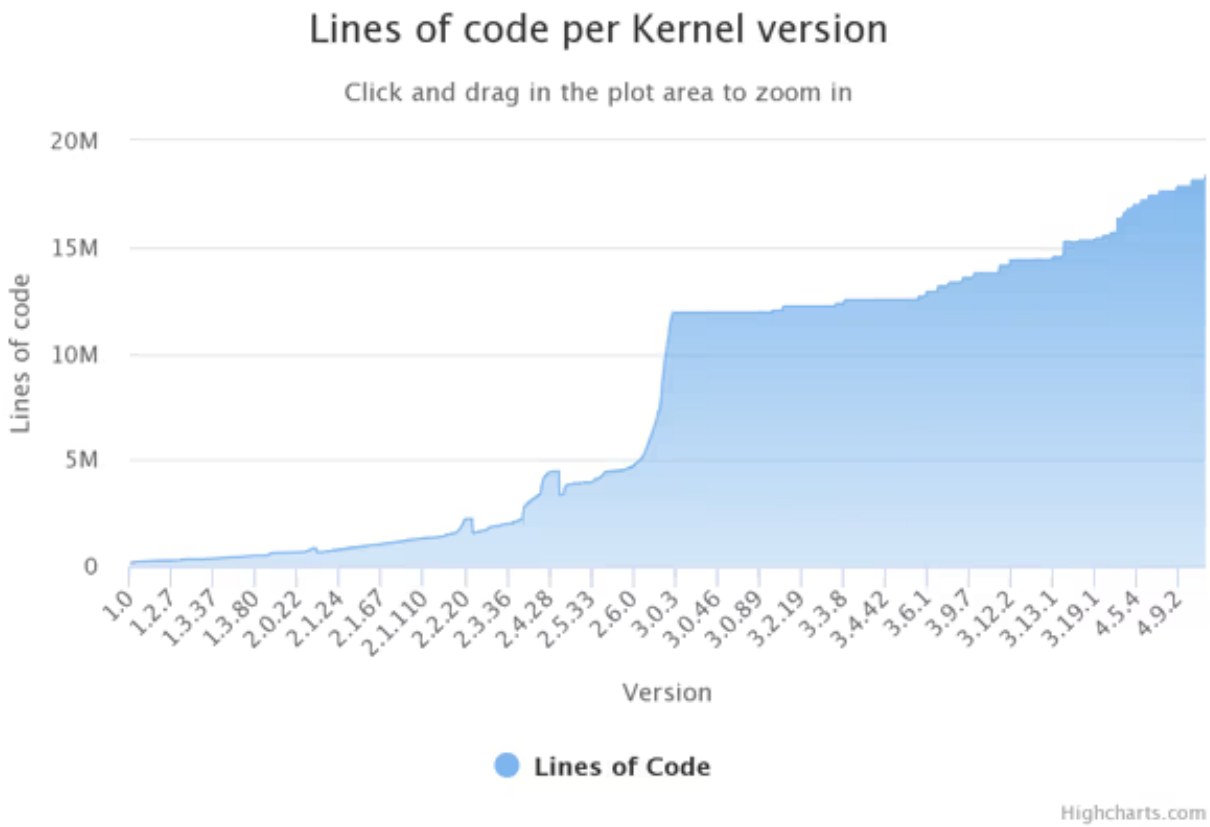
This course is about making that transition: from writing a working computation to organizing software that we and others can understand, verify, and modify.

Objects and functional techniques offer different tools for doing this. First, let us look at why the problem matters.

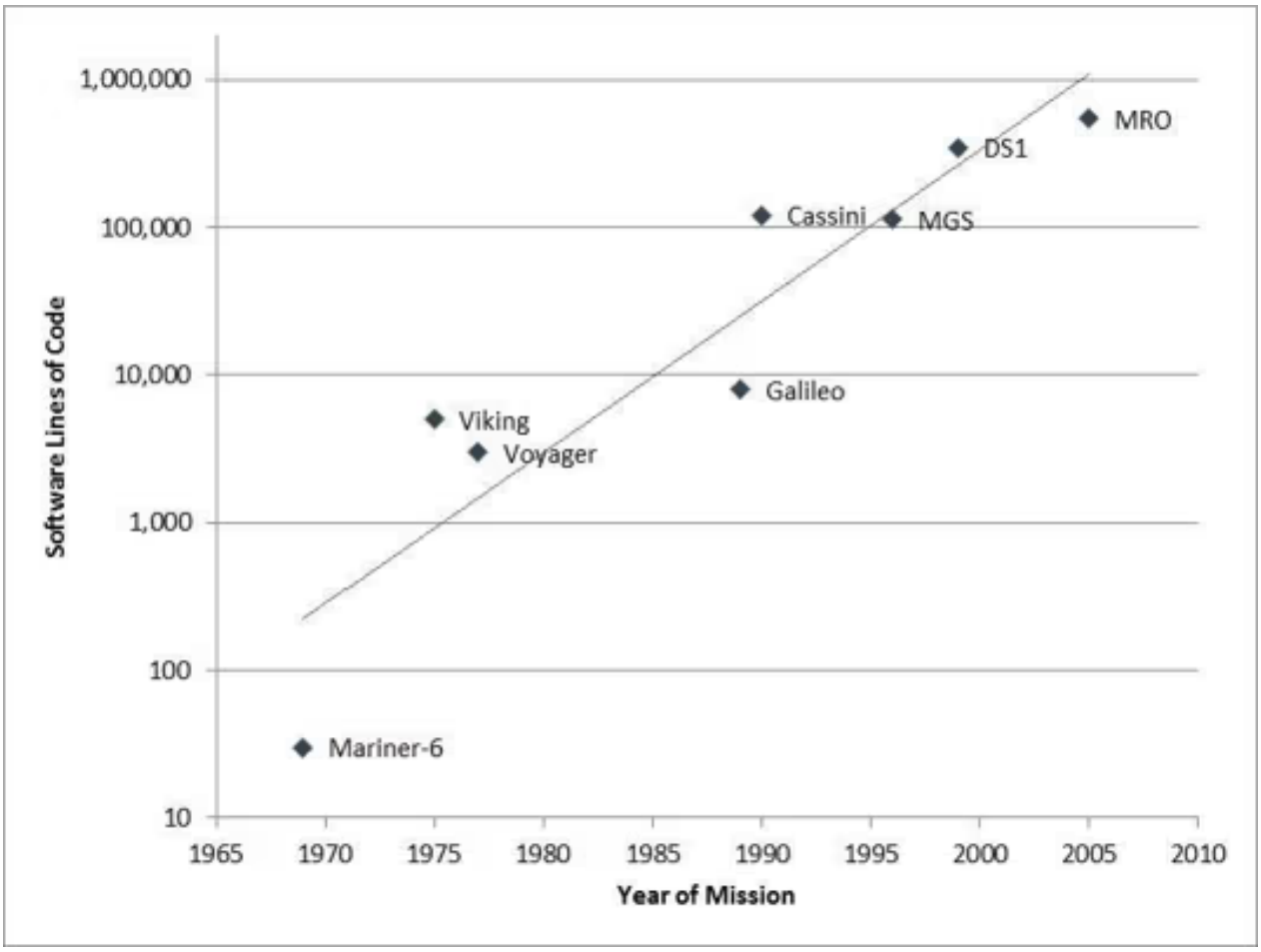
Lines of code



Lines of code per product



Lines of code per kernel



Lines of code per space mission

(see more: Million Lines of Code — Information is Beautiful)

More lines are not necessarily better. Dependencies, interactions, and the difficulty of changing behavior also matter.

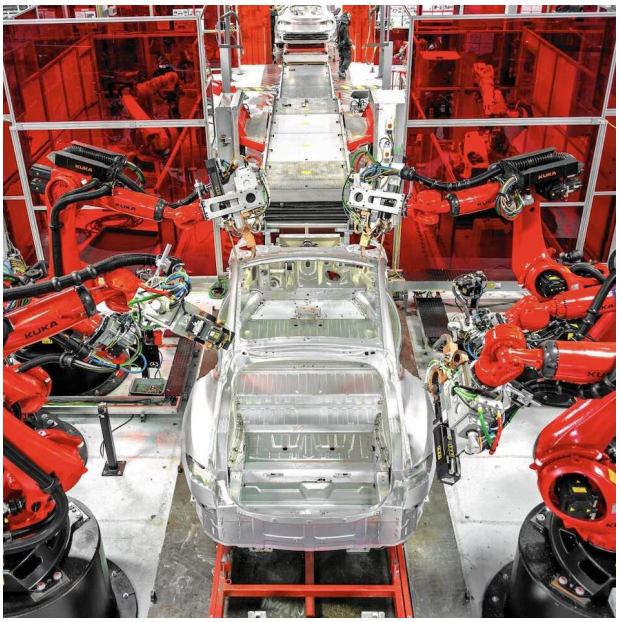
> If a program no longer fits in one person's head, how can we still understand and change it safely?

Tools and artifacts

As software grows, keeping it understandable becomes increasingly difficult.

The software crisis highlighted recurring problems:

- Projects running over budget and over time
- Software failing to meet requirements
- Systems that were difficult to test and maintain

Tools	Artifacts
	
	

Today we have IDEs, build tools, automated tests, version control, and AI assistants.

Better tools help manage complexity; they do not remove the need for software design. Generating more code without reviewing its structure can increase the maintenance burden.

Errors and software quality

It is not enough to ask how much code was produced, by a developer or an AI assistant.

More useful questions include:

- Does the program satisfy its requirements?
- What happens at boundaries and with invalid inputs?
- Do tests cover important failure cases?
- Can another developer understand and change it?

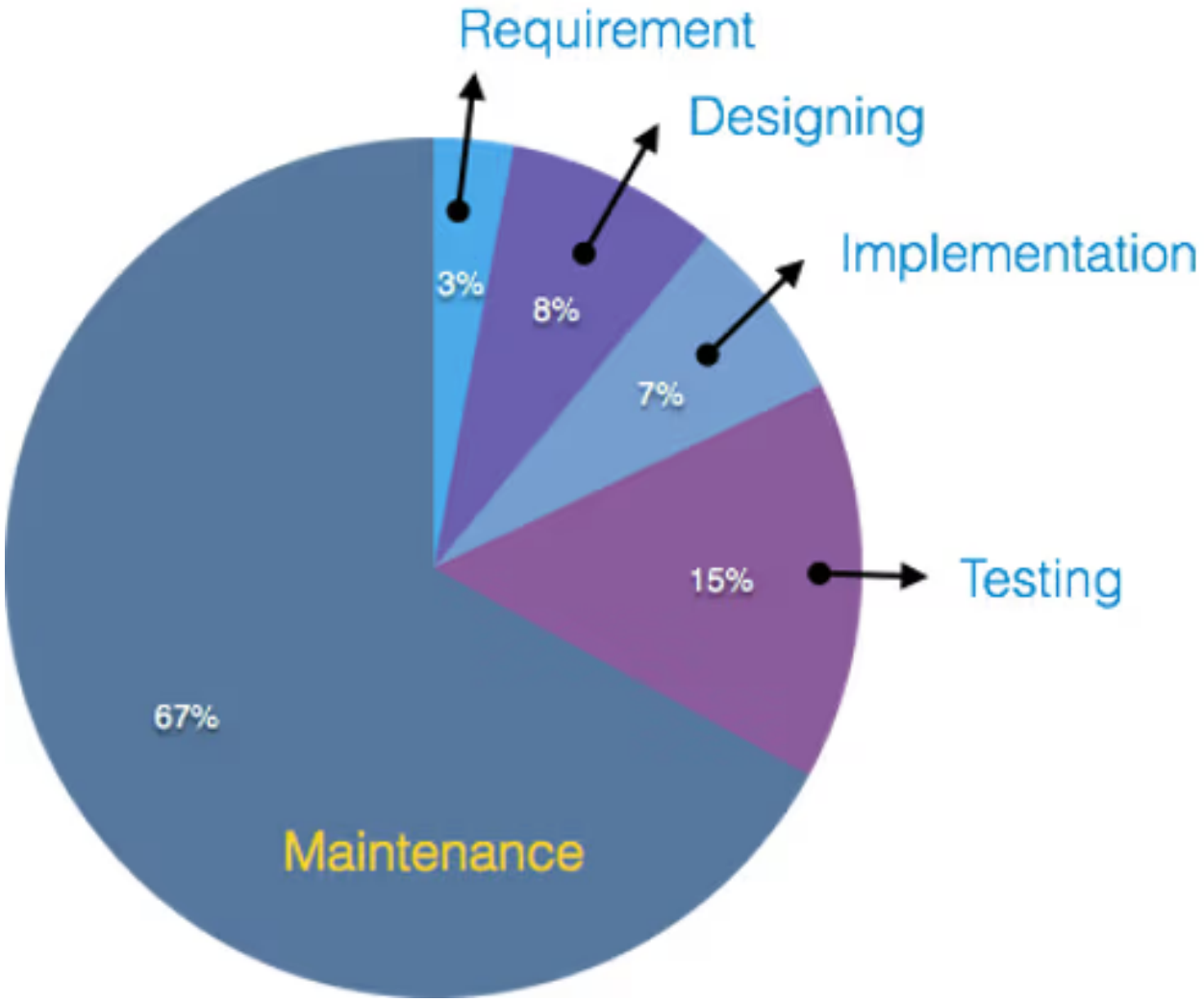
> Compilation checks language rules. Tests check selected behaviors. Neither automatically establishes that the requirements are correct or complete.

Software Maintenance Costs

Software is rarely written once and forgotten.

Over its lifetime, significant effort goes into:

- Fixing bugs
- Adapting to new requirements
- Extending existing functionality
- Understanding and modifying code written by others



Historical illustration: the balance of costs varies across projects.

> The larger the system, the more important clear responsibilities, stable interfaces, and verification become.

From algorithms to responsibilities

A small program is easy to hold in your head

Suppose a program stores numbers, sorts them, and searches for a value.

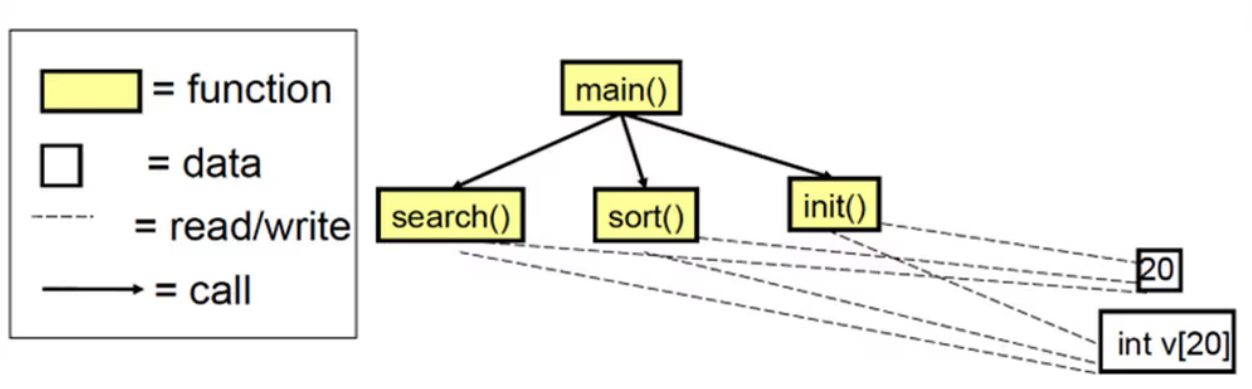
In a procedural organization, functions operate on data supplied by the caller:

```
init(values, size)
sort(values, size)
index = search(values, size, 10)
```

These are schematic calls, not a complete program. The important point is the organization: **data and the functions that operate on it are represented separately**.

For a small program, this can be clear and effective.

Where complexity appears



In this design:

- The array and its size are passed separately: callers must keep them consistent.
- Any code with access to the array can change its contents.
- Callers must know which operations are needed and in which order.

These are manageable responsibilities in a small program. They become harder to coordinate as more components use the same data.

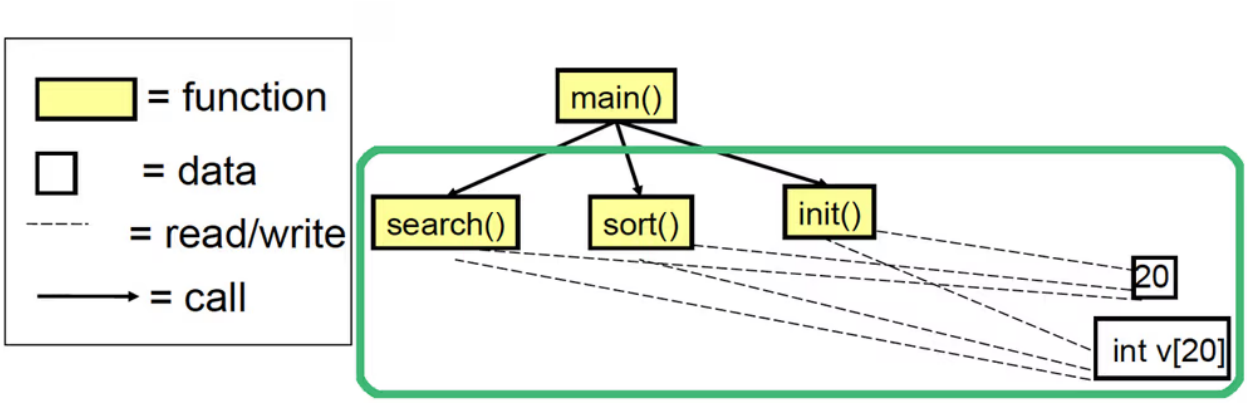
Procedural programming also supports modularity, data hiding, and reuse. These limitations concern this design, not every procedural program.

The question is not “which paradigm always wins?” but **“where should responsibility for this data live?”**

The object-oriented idea

Instead of making every caller coordinate the data, give that responsibility to an object:

```
vector = a new Vector containing 100 numbers
vector.sort()
index = vector.search(10)
```



The object keeps track of its own data and offers operations through which other parts of the program can use it.

- **State:** the numbers it currently contains.
- **Operations:** sorting and searching those numbers.
- **Responsibility:** keeping the representation consistent and controlling how it is used.

Callers ask the object to do something, rather than manage all its internal details themselves.

Class and object: just enough vocabulary

A **class** describes a kind of object: the state it holds and the operations it offers.

An **object** is a particular instance, with its own state.

Two vectors can offer the same operations while containing different numbers. Sorting one should not change the other when they own independent data.

For this conceptual example, focus on **who is responsible for what**, rather than the syntax for declaring a class.

What changed—and what did not?

- The object owns the numbers and keeps track of their size.
- Callers use a small set of operations instead of manipulating the internal representation directly.
- Responsibility for managing that representation is concentrated in one place.
- Sorting and searching still need algorithms: putting them in a class does not automatically make them faster.

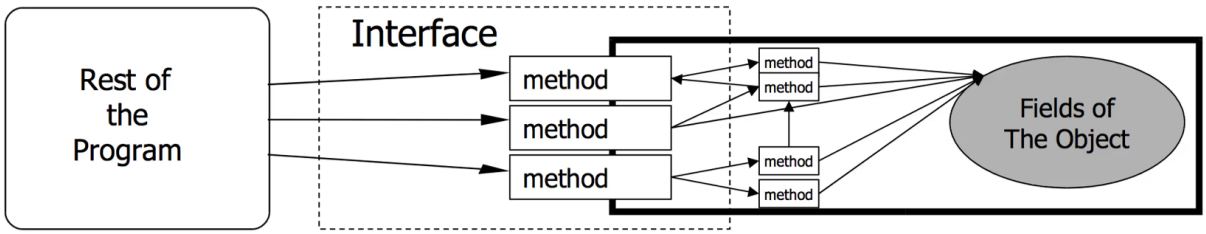
We changed ownership and responsibilities, not algorithmic complexity. Searching does not inherently require sorting first; that depends on the chosen algorithm.

The difference is not simply C versus Java: procedural organization is possible in Java too.

Why this helps with change

Encapsulation: a boundary around a responsibility

Encapsulation groups state and operations within a boundary and controls how that state can be accessed.



For example, callers of our vector should not need to update a separate size whenever its representation changes.

The object handles those details. Its public operations form an **API**: the ways other code is allowed to interact with it.

The benefit is not simply “hiding variables”. It is **reducing the number of places that must know and preserve the same rules**.

Abstraction: use an operation without knowing its implementation

A point can offer an operation for calculating the distance to another point:

```
Point a = new Point(0, 0);
Point b = new Point(3, 4);
IO.println(a.distance(b)); // 5.0
```

This is a preview of Java syntax, not an exercise in defining a class. Assume a `Point` class that keeps its coordinates private and provides `distance`.

The caller knows what a point represents and what distance means. It does not need direct access to the coordinates or the calculation formula.

Abstraction exposes a useful model without requiring callers to know every implementation detail. Encapsulation helps maintain that separation.

Change the implementation, preserve the promise

Suppose we replace our vector's handwritten sorting algorithm with a standard library implementation.

Callers still ask:

```
vector.sort()
```

If the operation preserves its promised behavior, callers do not need to change.

That promise is a **contract**: what an operation requires and what it guarantees. For example, `sort` leaves the same numbers in ascending order.

A boundary is useful when it lets us change one part without making unrelated parts change too.

A faster implementation can still be the wrong change

Suppose `search` promises to work even before sorting and to return the first matching index, or `-1` when the value is absent.

An AI assistant proposes replacing a linear search with binary search.

Before accepting, ask:

- Does the new algorithm require sorted input?
- Does it preserve the promised results for duplicates and missing values?
- Would sorting first change the data or the meaning of its indices?

Hiding an implementation does not make every replacement correct.

Objects are not a guarantee of good design

Adding a class around existing code is not enough.

- If every caller can freely modify the internal data, the same coordination problems remain.
- If one object does everything, it can still be difficult to understand and change.
- If a change requires inspecting the whole program, the boundaries may not be helping.

> We introduce objects to limit how much we must know at once and how many parts must change together—not to write more code.

Another approach: functional programming

Focus on transformations rather than changing shared state

The functional approach emphasizes **composing functions that transform values**, favoring immutable data and limiting or making effects explicit.

- **Immutable data** is not modified after it is created.
- An **effect** is an interaction beyond returning a result, such as changing shared data or printing output.
- A **pure function** depends only on its inputs and produces a result without such effects.

For a qualitative contrast:

```
Change an existing vector:
    vector.sort()

Produce a sorted result while keeping the original:
    sortedValues = sortedCopy(values)
```

The second operation illustrates a functional idea: reason about an input and a result without having to track a modification to the original data. One function alone does not make an entire program functional.

Different ways to manage complexity

Approach	Main emphasis
Procedural	Organize procedures that operate on data.
Object-oriented	Assign responsibilities to objects that encapsulate state and operations.
Functional	Compose transformations of values, favoring immutability and controlled effects.

Objects help us ask: **“Who is responsible for this state and its rules?”**

Functional programming helps us ask: **“Can I express this computation as transformations without changing shared state?”**

These are not successive stages where one makes the others obsolete. They offer different tools and trade-offs for the same broad challenge: making software easier to reason about and change.

Complementary, not incompatible

An object can be immutable. A method can compute a result without changing anything. An object-oriented application can use functional transformations internally.

Real applications also need effects: reading files, displaying results, or storing data. The question is how to organize and control them, not pretend they do not exist.

Java allows us to combine these approaches: objects can define responsibilities and boundaries while functions express transformations within those boundaries.

Resources

- https://dev.java/learn/
- https://dev.java/learn/classes-objects/
- https://docs.oracle.com/en/java/javase/25/docs/api/index.html

E1.2 — Environment and Build Tools

Start with a Java 25 JDK and an editor or IDE. Verify `java -version` and `javac -version`; a build can still target an older release even when the installed runtime is newer.

Compact source files

Java 25 supports **compact source files**: small programs can declare methods and fields without an explicit enclosing class. This reduces the syntax needed to get started; the compiler still creates a class implicitly.

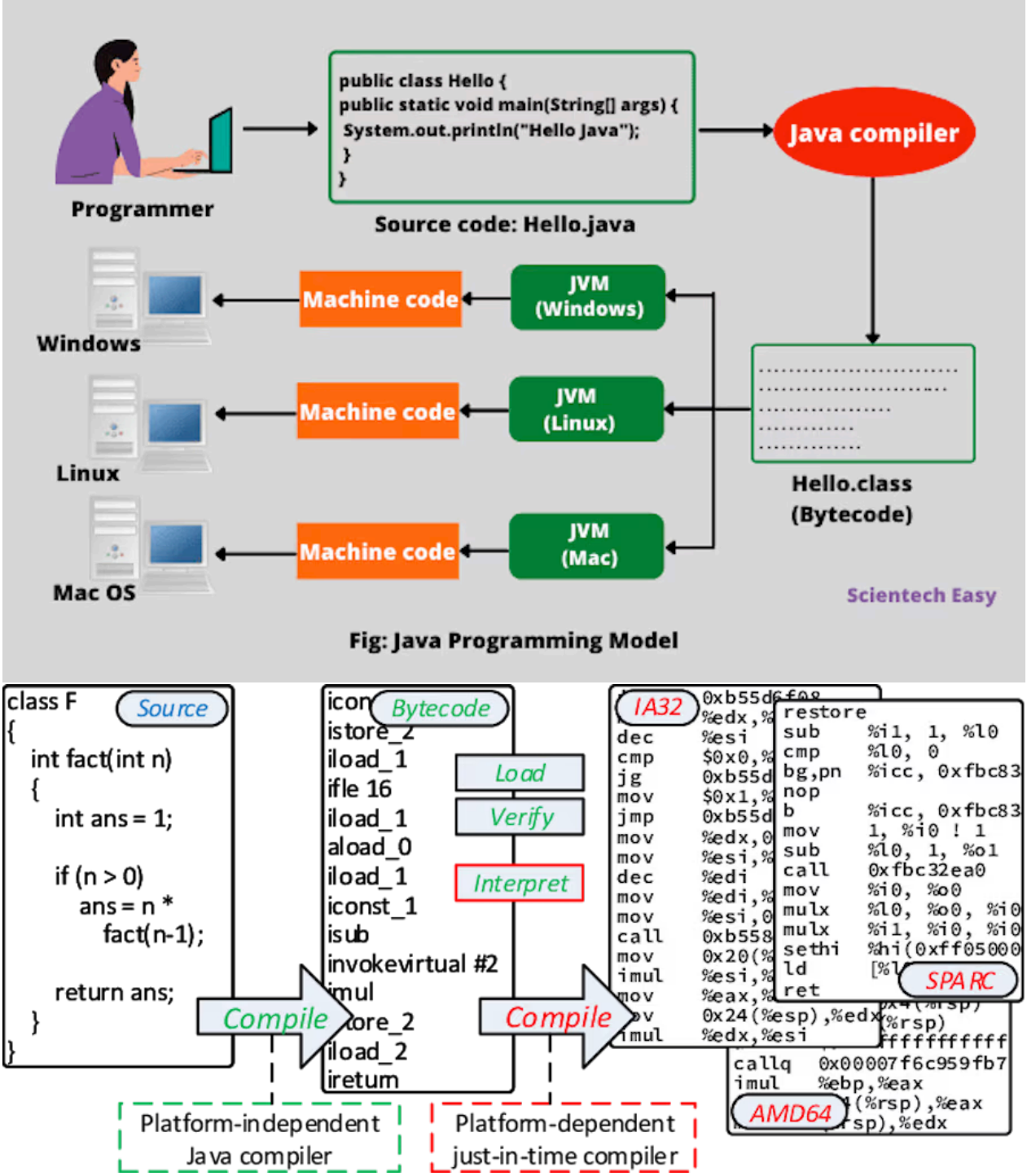
```
void main() {
    IO.println("Hello world!");
}
```

- `void main()` is the program's entry point. It can be an **instance method**, without `public`, `static`, or a `String[] args` parameter; the launcher creates an instance to invoke it.
- The compiler implicitly places the method in a class named `Hello`, based on the filename. There is no need to write `class Hello { ... }` around it.
- `IO.println` prints a line. `IO` belongs to `java.lang`, so it needs no explicit import in either compact source files or explicit classes.

For comparison, the same program in the traditional form, without a compact source file, is:

```
public class Hello {
    public static void main(String[] args) {
        IO.println("Hello world!");
    }
}
```

Building and running



The compiler produces **bytecode** stored in `.class` files. The JVM loads and executes it; modern JVMs use **Just-In-Time (JIT) compilation** to translate frequently executed code into native machine code.

Using the same `Hello.java` shown above, run these commands from the directory containing

the file (with JDK 25):

```
javac Hello.java
java Hello
```

- `javac Hello.java` compiles the source file and creates `Hello.class` containing bytecode.
- `java Hello` starts the JVM and runs the compiled class. Use the class name, **without `.java` or `.class`**.

The output is:

```
Hello world!
```

By contrast, `java Hello.java` uses **source-file mode**: it compiles the source in memory and runs it without writing a `Hello.class` file to disk. After changing the source, run `javac Hello.java` again before using `java Hello`, otherwise you will run the previously compiled version.

Distinguish three kinds of problem:

Problem	Example	How to investigate
Compilation error	Undefined variable	Read the compiler diagnostic
Runtime failure	Integer division by zero	Read the exception and stack trace
Incorrect behavior	Wrong average	Compare actual results with requirements

A successful compilation does not establish that a program is correct.

History

Reference — background, not a list of dates to memorize.

- **1991–1996:** the Green project at Sun Microsystems led to Java's introduction and JDK 1.0.
- **2004 — Java 5:** generics, annotations, enums, and enhanced `for`.
- **2014 — Java 8:** lambdas and streams.
- **2018 / 2021 / 2023 / 2025:** Java 11, 17, 21, and 25 LTS releases.

Java has a six-month feature-release cycle. Recent LTS releases follow a two-year cadence; long-term support is provided according to the JDK vendor's policy.

These examples use Java 25 as their baseline. Compact source files, instance main methods, flexible constructor bodies, and scoped values are stable in this release. Not every feature discussed online is final: structured concurrency and primitive patterns are still preview in Java 25 and are not required by these examples.

See: Java version history, The Java Story.

Build tools

Compiling one file manually is simple. A project with dependencies, resources, and tests needs a repeatable build process.

- **Maven:** declarative configuration in `pom.xml`, standard directory layout, dependency management.
- **Gradle:** build configuration in Kotlin or Groovy, dependency management, incremental builds and caching.

Both can automate compilation, testing, and packaging. This repository uses **Maven**.

From a module directory containing `pom.xml`, with Maven installed:

```
mvn compile
mvn test
mvn package
```

Example — build, test and package `deploy-cli`

The deploy-cli module converts temperatures between Celsius, Fahrenheit and Kelvin. `App` parses arguments and selects a conversion; `TemperatureConverter` contains numeric conversion methods and checks on temperature values.

Run these commands from the **repository root**, with JDK 25 and Maven installed:

```
mvn -f projects/deploy-cli/pom.xml compile
mvn -f projects/deploy-cli/pom.xml test
mvn -f projects/deploy-cli/pom.xml clean package
java -jar projects/deploy-cli/target/deploy-cli-1.0-SNAPSHOT.jar 25 C F
```

- `-f` selects the module's `pom.xml`, so there is no need to change directory.
- `compile` builds the application classes in `target/classes/`.
- `test` compiles and runs the JUnit tests. Inspect the test count, failures and errors; reports are in `target/surefire-reports/`.
- `clean` removes previous build output; `package` compiles the code, runs the configured test phase and creates the JAR in `target/`. Check the build summary to confirm which tests actually ran.
- The Maven Shade Plugin runs during `package` and writes `com.nbicocchi.app.App` as the JAR's entry point in its manifest. This lets `java -jar` find the application's `main` method.

The application prints:

```
25.00 C = 77.00 F
```

The filename follows the POM's artifact ID and version: `deploy-cli` and `1.0-SNAPSHOT`. This module targets Java 25 through the POM's `maven.compiler.release` property; building it requires JDK 25 or newer, and running it requires a Java 25-compatible runtime.

Scales are case-insensitive: `C`, `F`, or `K`. Use a dot for decimal values. Invalid arguments, non-finite numbers, and temperatures below absolute zero produce an error and a nonzero exit status. Pass `--help` to display usage instructions.

The distributable artifact is **`deploy-cli-1.0-SNAPSHOT.jar`**. Copy it to another machine with a compatible Java runtime and run it with `java -jar`. The JAR does not include the runtime itself.

Example — build, test and package `deploy-gui`

The deploy-gui module provides a JavaFX Celsius/Fahrenheit converter. `App` loads the FXML view, `ConverterController` handles its events, and `TemperatureConverter` contains numeric conversions. This project requires **JDK 25**; running the interface also requires a graphical desktop.

From the **repository root**, compile, test and launch it during development:

```
mvn -f projects/deploy-gui/pom.xml compile
mvn -f projects/deploy-gui/pom.xml test
mvn -f projects/deploy-gui/pom.xml javafx:run
```

- Maven downloads the JavaFX dependencies declared in `pom.xml`.
- Resources such as `temperature-view.fxml` are copied to `target/classes/` alongside the compiled classes.
- The JUnit tests check numeric conversion and validation without opening a window. They do not replace manual checks of the interface.
- `javafx:run` compiles as needed and starts the application with its JavaFX modules and configured JVM options; it does not create a distribution.

To create a distributable artifact with its **own Java runtime**:

```
mvn -f projects/deploy-gui/pom.xml clean verify javafx:jlink
```

`verify` includes compilation, tests and JAR packaging. The JavaFX plugin then invokes `jlink` to assemble the application and required runtime modules:

- **`projects/deploy-gui/target/app/`** — runnable application directory, including the runtime.
- **`projects/deploy-gui/target/app.zip`** — archive to distribute; extract it before launching.

On Linux or macOS, run:

```
projects/deploy-gui/target/app/bin/app
```

On Windows, use `projects\deploy-gui\target\app\bin\app.bat`.

Distribute the **whole runtime image**, not just its launcher or the plain JAR produced by `package`. Unlike the CLI JAR, this image needs no separately installed Java runtime, but it is **OS- and architecture-specific**: build it on each target platform. JavaFX still requires the platform's graphical system libraries.

Maven project structure

```
project-root/
 ├─ pom.xml
 └─ src/
    ├─ main/
    │  ├─ java/        (application source code)
    │  └─ resources/   (non-code application resources)
    └─ test/
       ├─ java/        (test source code)
       └─ resources/   (resources used by tests)
```

- `pom.xml` defines dependencies, plugins, and build configuration.
- Resources can include configuration files or test data.

> **Note:** unit and integration tests may require different execution configuration; a directory name alone does not make a test run.

Standard conventions help IDEs, build servers, developers, and assistants work with the same project structure.

Conventions

Coding conventions

- **Packages:** lowercase, e.g. `com.nbicocchi.myapp`.
- **Classes and interfaces:** `UpperCamelCase`, e.g. `Point`, `List`, `Comparable`.
- **Fields, parameters, and local variables:** `lowerCamelCase`, e.g. `eventCount`.
- **Methods:** descriptive `lowerCamelCase`, e.g. `findMedian`, `distance`, `isEmpty`.
- **Constants:** typically `UPPER_SNAKE_CASE`, e.g. `MAX_SIZE`.
- **Braces:** opening brace on the declaration or statement line; closing brace on its own line.
- **Indentation:** use four spaces per level and keep formatting consistent throughout the project.
- **Spacing:** put spaces around binary operators and after commas; separate methods with a blank line.
- **Control flow:** use braces even for a single-statement `if` or loop, so later edits do not accidentally change its scope.
- **Source files:** in an ordinary source file, a public top-level class such as `ItemCounter` belongs in `ItemCounter.java`. In Maven's standard layout, its package corresponds to the directory under `src/main/java`.

Interfaces may have noun or adjective names. Getter/setter naming is common, but not every API needs getters and setters; records, for example, use component names for accessors.

Prefer names that describe meaning: `remainingCapacity` rather than `rc`, and `itemCount` rather than `n`. Boolean methods often read as questions, such as `isEmpty`, `hasNext`, or `canAccept`. Include a unit when it prevents ambiguity, for example `timeoutSeconds` instead of `timeout`.

Not every `final` field is a named constant: an instance field such as `private final String customerName` still uses `lowerCamelCase`. Uppercase names are normally reserved for fixed values such as a `static final int MAX_ITEMS`.

These are readability conventions, not all compiler requirements. For example, `ItemCounter` is a naming convention, while matching a public top-level class to its ordinary source filename is a language requirement. Follow the project's existing style and use IntelliJ **Reformat Code** (`Ctrl+Alt+L`) to apply its formatting settings.

Example — consistent names and formatting

Save this class as `src/main/java/com/nbicocchi/myapp/ItemCounter.java` in a Maven project. It illustrates package, class, constant, field, parameter and local-variable names, together with indentation and braces. This example focuses on naming and layout, not a complete validation policy.

```java
package com.nbicocchi.myapp;

public final class ItemCounter {
    private static final int MAX_ITEMS = 100;
    private int itemCount;

    public void addItems(int quantity) {
        int remainingCapacity = MAX_ITEMS - itemCount;
        if (quantity < 0 || quantity > remainingCapacity) {
            throw new IllegalArgumentException("Quantity outside allowed range");
        }
        itemCount += quantity;
    }

    public boolean isEmpty() {
        return itemCount == 0;
    }
}
```

`MAX_ITEMS` is a shared fixed limit; `itemCount` is each object's state. `quantity` is a method parameter and `remainingCapacity` is a local variable. The names reveal their roles without abbreviations or comments that merely repeat the instructions. There is no setter for `itemCount`: callers use the operation `addItems` instead.

Comments

```java
// A single-line comment

/*
 * A block comment
 */
```

```java
/**
 * Doubles a small integer without overflowing.
 * @param number a value between -100 and 100 (caller precondition)
 * @return twice number
 */
public static int multiplyByTwo(int number) {
    return 2 * number;
}
```

Comments are useful for **contracts, reasons, and constraints**. Do not merely translate every statement into English. A generated comment can be wrong or become outdated just like any other comment.

Here, the documented input range is a responsibility of the caller; the method does not enforce it. Documentation and runtime validation are different things.



"Don't comment bad code—rewrite it."

—Brian W. Kernighan and P. J. Plaugher[1]

Resources

E2.1 — Types and Expressions

A first program

Compact source file

Save this complete Java 25 program as `HelloCompact.java`:

```
void main() {
    IO.println("Hello world!");
}
```

Run it directly with the JDK source launcher:

```
java HelloCompact.java
```

No class declaration, static modifier, command-line parameter, or preview flag is required. The compiler supplies an implicit class; the program still consists of a method belonging to a class. `IO` is the new `java.lang.IO` helper for simple line-oriented input/output.

Explicit class and static main

The conventional form remains valid and useful for larger examples:

Save the following as `HelloWorld.java`:

```
public class HelloWorld {
    public static void main(String[] args) {
        IO.println("Hello world!");
    }
}
```

From the directory containing the file:

```
javac HelloWorld.java
java HelloWorld
```

- `javac` compiles the source into `HelloWorld.class`.
- `java` launches the program using the class name, without the `.class` suffix.

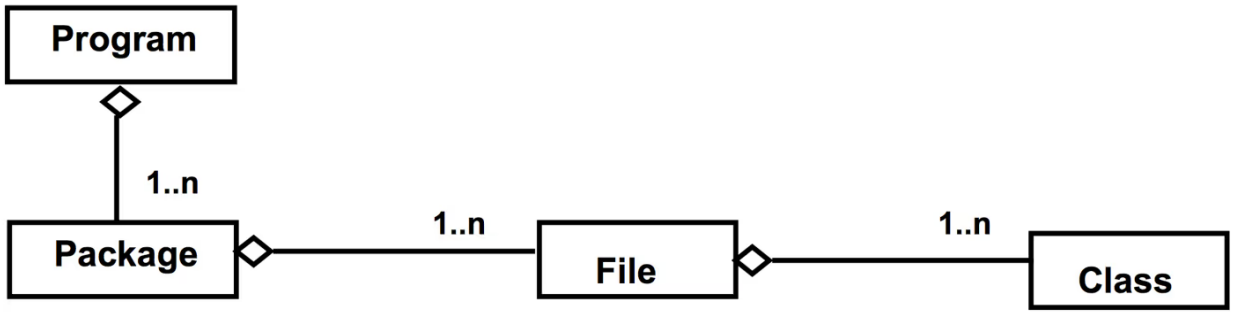
- `IO.println` prints a line to standard output. Java 25 provides `java.lang.IO`, automatically imported in both compact source files and explicit classes; no import or preview flag is needed. `System.out.println` remains valid and is not deprecated.

Try: change the message, save, compile, and run again. What happens if you save but do not recompile before running `java HelloWorld`?

Programs, files and classes

A Java program can contain multiple packages and source files. An ordinary source file can declare multiple top-level types, such as classes, interfaces, enums, and records. A compact source file instead declares its class implicitly and cannot declare a package.

With the conventional file-based compilation used here, at most one top-level type in a file is `public`, and **the file name must match that public type's name**.



The explicit `HelloWorld` example has one named class and no package declaration. Larger examples use named packages to organize related types.

Imports and the Java 25 source forms

Ordinary source files automatically import `java.lang` types, including `String`, `Math`, and `IO`. Compact source files additionally import the public top-level types of packages exported by `java.base`, making types such as `List` available without explicit imports.

Java 25 also supports **module import declarations** in ordinary files:

```
import module java.base;

class NameLengths {
    void main() {
        List<String> names = List.of("Ada", "Grace");
        names.forEach(name -> IO.println(name.length()));
    }
}
```

`void main()` is also valid inside an explicit class. For an instance main, the launcher needs a non-private no-argument constructor; this class receives one automatically. A module import imports types, not static methods, and does not add a dependency or replace a module descriptor. Explicit type imports remain useful for clarity and resolving name conflicts.

Primitive types

Core: distinguish integer, floating-point, character, and boolean values; use `int`, `long`, `double`, and `boolean` appropriately.

Reference — sizes, ranges, and less frequently used types:

Type	Size (bits)	Range or meaning	Example
`byte`	8	−2^7 to 2^7 − 1	`byte b = 100;`
`short`	16	−2^15 to 2^15 − 1	`short s = 30_000;`
`int`	32	−2^31 to 2^31 − 1	`int i = 100_000_000;`
`long`	64	−2^63 to 2^63 − 1	`long l = 100_000_000_000_000L;`
`float`	32	IEEE 754 binary32	`float f = 1.456f;`
`double`	64	IEEE 754 binary64	`double d = 1.456789012345678;`
`char`	16	UTF-16 code unit, 0 to 65,535	`char c = 'c';`
`boolean`	Not specified as a storage size here	`true` or `false`	`boolean b = true;`

For floating-point types, distinguish:

Type	Largest positive finite value	Smallest positive nonzero value
`float`	approximately 3.4 × 10^38	approximately 1.4 × 10^−45
`double`	approximately 1.8 × 10^308	approximately 4.9 × 10^−324

> **Note:** the most negative finite value is `-Float.MAX_VALUE` or `-Double.MAX_VALUE`. `MIN_VALUE` means the smallest positive nonzero value for these two types, not the most negative value.

> **Note:** floating-point values also include signed zero, infinities, and `NaN`. A `char` is a UTF-16 code unit and does not necessarily represent an entire Unicode character.

Literals

Reference: focus first on decimal numbers, strings, booleans, and `null`. Other notations can be consulted when encountered.

Kind	Examples	Notes
Decimal integer	`42`, `30_000`, `42L`	`L` denotes `long`
Binary integer	`0b11011`	Prefix `0b`
Hexadecimal integer	`0x9A`	Prefix `0x`
Octal integer	`0275`	Leading `0`; avoid accidental octal notation
Floating-point	`3.14`, `1.456f`, `3.14e2`	Default is `double`; `f` denotes `float`
Character	`'A'`, `'\n'`, `'\u0041'`	Single quotes
String	`"John Doe"`	Double quotes; `String` is not primitive
Boolean	`true`, `false`	Not interchangeable with `0` and `1`
Null	`null`	No object is referenced

Constants

A `final` variable can be assigned only once. For a reference variable, the object itself may still be mutable.

```java
public class ConstantsDemo {
    private static final int MAX_ATTEMPTS = 3;

    public static void main(String[] args) {
        final int attempts = MAX_ATTEMPTS;
        // attempts++; // Does not compile
        IO.println(attempts);
    }
}
```

Use named constants to explain meaningful values, not to give a name to every literal.

> **Note:** not every `final` variable is a compile-time constant; that distinction is not needed for the examples here.

Working with numbers

The Math library

`java.lang.Math` provides numeric operations and is available without an explicit import.

```
public class MathDemo {
    public static void main(String[] args) {
        IO.println(Math.sqrt(25)); // 5.0
        IO.println(Math.max(10, 20)); // 20
        IO.println(Math.log(-1)); // NaN
    }
}
```

Not every invalid numeric operation throws an exception. Check the API's documented behavior.

Integer division

Predict: all four variables below are `double`. Do they all contain the same value?

```
double average = 5 / 2;
double correct = 5 / 2.0;
double alsoCorrect = (double) 5 / 2;
double tooLate = (double) (5 / 2);
```

Print each value to test your prediction.

Check your prediction

The values are `2.0`, `2.5`, `2.5`, and `2.0`, respectively.

The operand types determine the division. Assigning the result to `double` does not restore a discarded fractional part.

Integer division truncates toward zero: `-5 / 2` is `-2`. Integer division by zero throws `ArithmeticException`; floating-point division by zero can produce infinity or `NaN`.

Overflow and numeric promotion

```
long wrong = 100_000 * 100_000;   // 1410065408: int overflow first
long correct = 100_000L * 100_000; // 10000000000
long tooLate = (long) (100_000 * 100_000); // Still wrong

byte a = 10;
byte b = 20;
int sum = a + b; // byte operands are promoted to int
```

Ordinary integer arithmetic does not automatically report overflow. When overflow must be detected, APIs such as `Math.multiplyExact` can help.

Predict: what does `Integer.MAX_VALUE + 1` produce? Does assigning it to `long` prevent overflow?

Floating-point precision

```
IO.println(0.1 + 0.2);        // 0.30000000000000004
IO.println(0.1 + 0.2 == 0.3); // false
```

Many decimal fractions cannot be represented exactly in binary floating-point.

For approximate computations, choose a comparison tolerance appropriate to the problem and scale.

> **Note:** exact decimal requirements may call for `BigDecimal` and an explicit rounding policy. We do not develop that API in this lesson.

Random numbers

```
import java.util.random.RandomGenerator;

public class RandomDemo {
    public static void main(String[] args) {
        RandomGenerator random = RandomGenerator.getDefault();
        int anyInt = random.nextInt();       // Any int value
        int below100 = random.nextInt(100);  // [0, 100)
        int between = random.nextInt(10, 20); // [10, 20)
        long below100Long = random.nextLong(100); // [0, 100)
        double fraction = random.nextDouble();   // [0.0, 1.0)
        double scaled = random.nextDouble(10.0, 20.0); // [10.0, 20.0)
        boolean flag = random.nextBoolean();
        IO.println(between);
    }
}
```

The upper bound is exclusive. Do not assume an exact output for this example. Reproducible tests should control their inputs or generator.

> **Note:** security-sensitive randomness requires an appropriate API such as `SecureRandom`; the generator above is not a security recommendation.

Operators

Operators combine operands to produce values. The table below is a **reference**, not a memorization exercise. Higher rows bind more tightly.

Category	Operators
Postfix	`expr++ expr--`
Unary	`++expr --expr +expr -expr ~ !`
Multiplicative	`* / %`
Additive	`+ -`
Shift	`<< >> >>>`
Relational	`< > <= >= instanceof`
Equality	`== !=`
Bitwise AND	`&`
Bitwise XOR	`^`
Bitwise OR	`|`
Logical AND	`&&`
Logical OR	`||`
Conditional	`? :`
Assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`

Grouping versus evaluation order

- **Precedence** and **associativity** determine expression grouping.
- Java evaluates operands from left to right, subject to conditional evaluation rules.
- Parentheses make intended grouping explicit; avoid expressions with several side effects.

```
int result = 2 + 3 * 4;       // 14
int grouped = (2 + 3) * 4;    // 20
int leftGrouped = 10 - 3 - 2; // (10 - 3) - 2 = 5
```

Short-circuit evaluation

`&&` evaluates its right operand only if the left operand is `true`. `||` evaluates its right operand only if the left operand is `false`.

```
int count = 0;
int total = 10;
boolean acceptable = count != 0 && total / count > 2; // false; no division
```

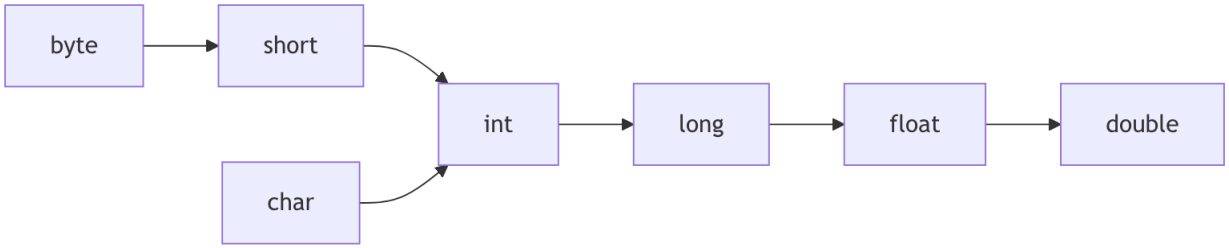
Review: replacing && with & here still compiles, but both operands are evaluated and integer division by zero fails. These operators are not interchangeable optimizations.

Casting

Implicit casting

The compiler permits **widening primitive conversions** without an explicit cast.

Reference — main conversion paths:




```
int number = 10;
long larger = number;
double decimal = larger;
```

Widening is not synonymous with exactness. `int` to `float`, `long` to `float`, and `long` to `double` can lose precision.

```
int original = 16_777_217;
float rounded = original;
IO.println((int) rounded); // 16777216
```

Explicit casting

A narrowing primitive conversion generally requires a cast and may lose information.

```
double decimal = 2.9;
int truncated = (int) decimal; // 2, not rounded to nearest
int negative = (int) -2.9;    // -2
byte wrapped = (byte) 130;    // -126
char letter = (char) 65;      // 'A'
```

> **Note:** some representable integer constant expressions can be assigned to smaller types without a cast, such as `byte value = 100;`.

A cast is not input validation. It requests a conversion; it does not guarantee that the value fits the application's requirements.

Resources

- https://dev.java/learn/language-basics/
- https://docs.oracle.com/en/java/javase/25/docs/api/index.html
- https://openjdk.org/jeps/512
- https://openjdk.org/jeps/511

E2.2 — Control Flow

Code blocks and scope

Braces { } delimit blocks. A local variable is available only within its scope and must be definitely assigned before use.

```
public class ScopeDemo {
    public static void main(String[] args) {
        int number = 3;
        {
            int doubled = 2 * number;
            IO.println(doubled); // 6
        }
        IO.println(number); // 3
        // IO.println(doubled); // Does not compile: out of scope
    }
}
```

```
int count;
// IO.println(count); // Does not compile: not initialized
count = 3;
IO.println(count);
```

Fields receive default values; local variables do not receive the same automatic initialization. Keep variables close to where they are needed.

Flow-control statements

Decision statements

if statement

Only the first matching branch is executed:

```
int time = 22;
if (time < 10) {
    IO.println("Good morning.");
} else if (time < 20) {
    IO.println("Good day.");
```

```
} else {
    IO.println("Good evening.");
}
```

Check boundaries: which branch runs for `9`, `10`, `19`, and `20`? What policy should apply to an invalid hour such as `-1`?

Modern switch statement

Use arrow labels when selecting an action without fall-through:

```
char grade = 'B';
switch (grade) {
    case 'A' -> IO.println("Excellent!");
    case 'B', 'C' -> IO.println("Well done");
    case 'D' -> IO.println("Danger zone");
    default -> IO.println("Invalid grade");
}
```

An arrow branch does not fall through into the next branch, so no `break` is required.

Switch expression

When selecting a value, use a switch expression:

```
char grade = 'B';
String message = switch (grade) {
    case 'A' -> "Excellent!";
    case 'B', 'C' -> "Well done";
    case 'D' -> "Danger zone";
    default -> "Invalid grade";
};
IO.println(message);
```

A switch expression must be exhaustive. A block branch uses `yield` to supply its value.

Classic switch statement

Reference — reading existing code. You will also encounter the older colon form:

```
switch (grade) {
    case 'A':
        IO.println("Excellent!");
        break;
    case 'B':
    case 'C':
```

```
        IO.println("Well done");
        break;
    default:
        IO.println("Check your grade");
}
```

Execution can fall through unless interrupted by `break` or another transfer of control. Be able to read this form; prefer the arrow form when fall-through is not intended.

Iterative statements

while statement

The condition is checked **before** the body, which may run zero times:

```
int i = 0;
while (i < 5) {
    IO.println(i);
    i++;
}
```

do-while statement

The condition is checked **after** the body, which runs at least once:

```
int i = 0;
do {
    IO.println(i);
    i++;
} while (i < 5);
```

Predict: how do these two examples differ if `i` starts at 5?

for statement

A `for` loop groups initialization, condition, and update:

```
for (int i = 0; i < 5; i++) {
    IO.println(i); // 0, 1, 2, 3, 4
}
```

The update runs after the body, before the next condition check. Ask how the loop makes progress and why it eventually terminates.

Review: an assistant changes `< 5` to `<= 5`. How many iterations run now? Is the extra iteration intended?

Enhanced for statement

Use enhanced `for` to visit elements of an array or an `Iterable`:

```
String[] cars = {"Supra", "Lancer Evo", "Impreza"};
for (String car : cars) {
    IO.println(car);
}
```

The enhanced `for` visits each element of an array or iterable value. It does not expose an index; choose a conventional `for` when the position is needed.

break and continue

- `break` exits the nearest enclosing loop or switch statement.
- `continue` starts the next iteration of the nearest enclosing loop. In a conventional `for`, its update expression still executes.

```
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        break;
    }
    IO.println(i);
} // Prints 0, 1, 2, 3
```

```
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        continue;
    }
    IO.println(i);
} // Prints 0, 1, 2, 3, 5, 6, 7, 8, 9
```

Be careful with `continue` in a `while`: skipping the statement that updates the loop variable can prevent termination.

Resources

E2.3 — Methods and References

Methods

Behavior is organized into **methods** belonging to classes or interfaces, rather than standalone functions. In a compact source file, methods outside an explicit class body are members of the implicitly declared class, not free functions or top-level executable statements.

A method can receive parameters and return a value. `void` means that it does not return a value.

Replace the first `HelloWorld` example with:

```
public class HelloWorld {
    public static void main(String[] args) {
        int result = multiplyByTwo(20);
        IO.println(result); // 40
    }

    public static int multiplyByTwo(int number) {
        return 2 * number;
    }
}
```

- `int number` declares a parameter.
- `20` is the argument supplied by the caller.
- `return` supplies the result and ends that invocation.
- `static` allows this method to be called without creating a `HelloWorld` object.

The method name describes an operation, but not its complete contract: for large `int` inputs, multiplication can overflow. Integer arithmetic wraps around when the result exceeds the type's range; it does not automatically report an error.

Reference variables

A primitive variable holds a primitive value. A reference variable holds a reference value identifying an object, or `null`.

For the following examples, save this teaching type as `MutablePoint.java`. Public mutable fields deliberately make state changes visible here; this is an execution-model example, not a recommendation for API design.

```java
public class MutablePoint {
    public int x;
    public int y;

    public MutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

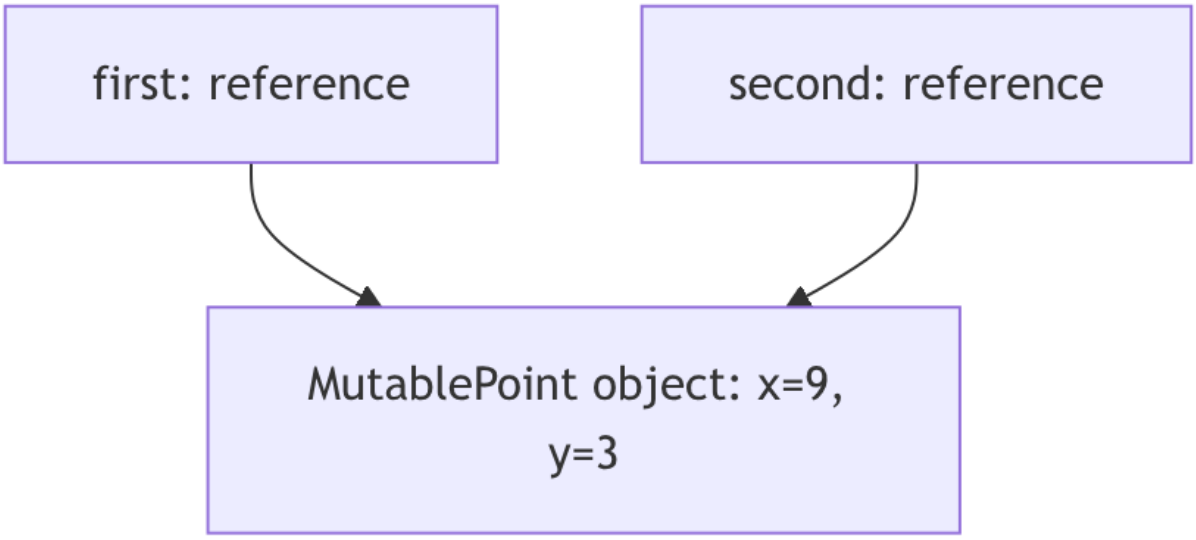
Predict before running: inside a `main` method, what does the first `println` print? Draw how many objects are created.

```java
String message = "Hello world!";
MutablePoint first = new MutablePoint(2, 3);
MutablePoint second = first;

second.x = 9;
IO.println(first.x);
IO.println(message.length());
```

Check your prediction

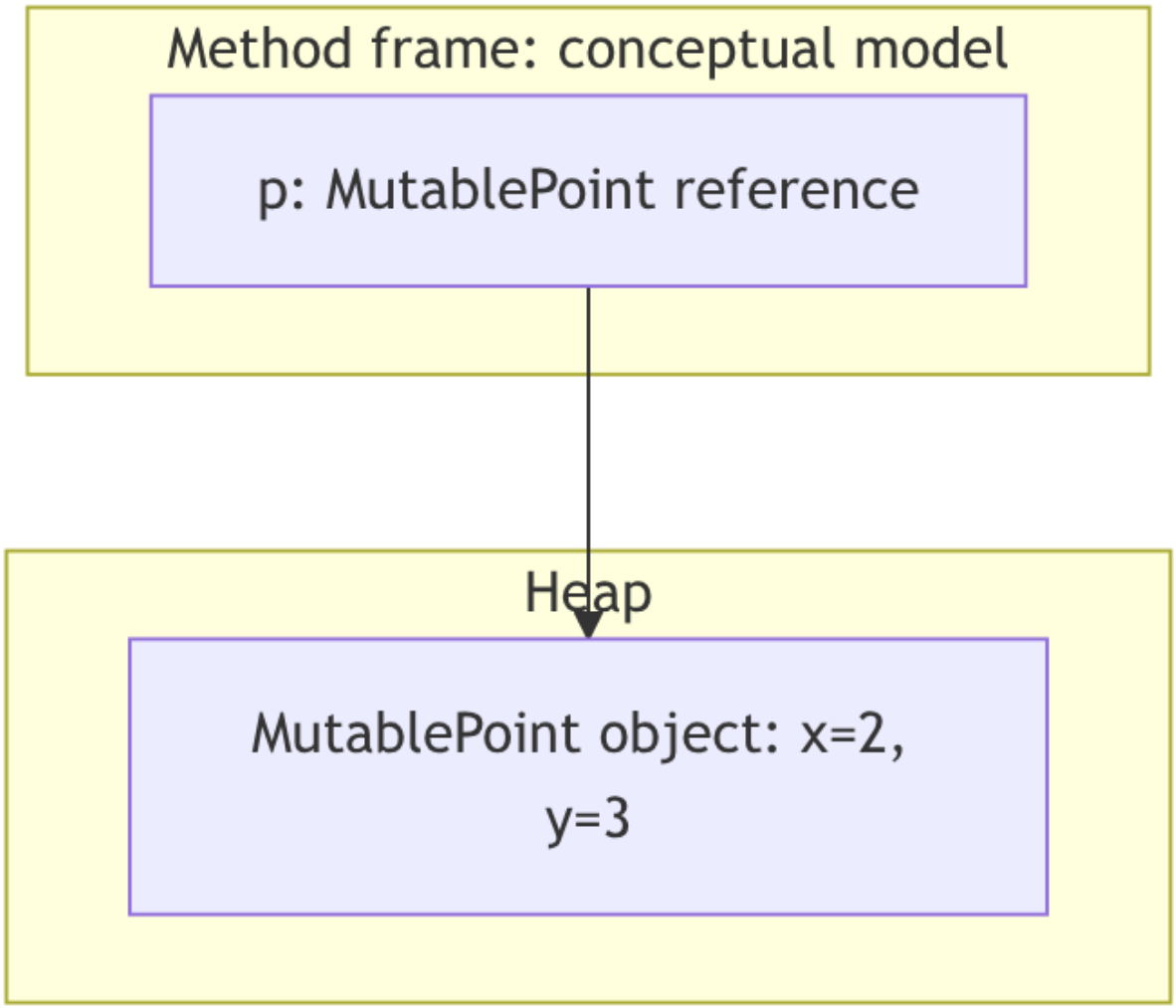
The output is 9, then 12. `first` and `second` refer to one shared `MutablePoint` object; assigning the reference did not copy that object.



This is **aliasing**: two references identify the same object. Assigning a reference does not copy the object.

References and memory

The JVM manages object allocation and reclamation. A useful introductory model places objects in the heap and local variables in method stack frames.



> **Note:** this is a conceptual model, not a promise about optimized physical memory layout. Reference fields can also be stored inside other objects.

Compared with C pointers, Java references do not support pointer arithmetic or explicit `free`. Garbage collection does not replace closing files or other external resources.

Null and final references

```
MutablePoint missing = null;
// IO.println(missing.x); // Throws NullPointerException
```

```
final MutablePoint point = new MutablePoint(2, 3);
point.x = 10;                 // Allowed: mutate the object
// point = new MutablePoint(5, 7); // Does not compile: reassign final variable
```

Predict: after `MutablePoint a = new MutablePoint(1, 2); MutablePoint b = a; b = new MutablePoint(8, 9);`, what are `a.x` and `b.x`? Draw the references before running it.

Passing parameters to methods

Java is **always pass-by-value**. Each argument value is copied into a separate parameter variable.

- For primitives, the primitive value is copied.
- For references, the reference value is copied—not the object.

Changing primitive parameters

```
public class PrimitiveParameters {
    public static void swap(int a, int b) {
        int temporary = a;
        a = b;
        b = temporary;
    }

    public static void main(String[] args) {
        int a = 2;
        int b = 3;
        swap(a, b);
        IO.println(a + ", " + b); // 2, 3
    }
}
```

Only local copies are swapped. The caller's variables are unchanged.

Reassigning a reference parameter

Compile this class alongside `MutablePoint.java`. **Before running it, predict the two printed values.**

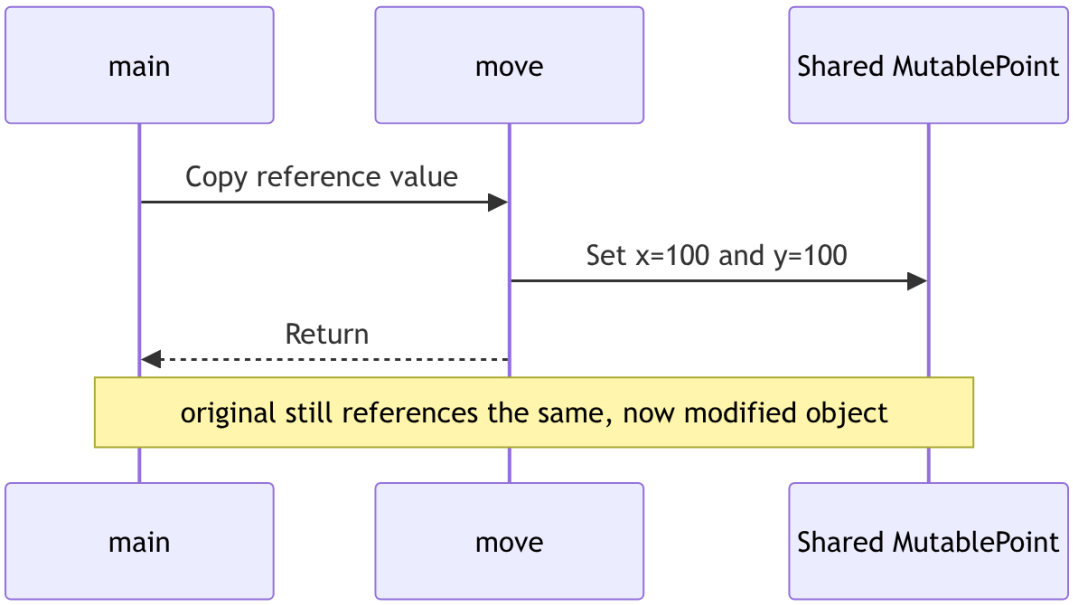
```
public class ReferenceParameters {
    public static void replace(MutablePoint point) {
        point = new MutablePoint(100, 100);
    }
```

```
    public static void move(MutablePoint point) {
        point.x = 100;
        point.y = 100;
    }

    public static void main(String[] args) {
        MutablePoint original = new MutablePoint(2, 3);
        replace(original);
        IO.println(original.x);
        move(original);
        IO.println(original.x);
    }
}
```

Check your prediction and explanation

The output is 2, then 100. replace changes only its parameter. move uses its copy of the reference to mutate the shared object.



Swapping object state is not swapping caller references

This method swaps coordinates, not the reference values in the caller:

```
public static void swapCoordinates(MutablePoint a, MutablePoint b) {
    int temporaryX = a.x;
    int temporaryY = a.y;
    a.x = b.x;
    a.y = b.y;
    b.x = temporaryX;
    b.y = temporaryY;
```

```
}
```

Operation inside a method	Changes the caller's variable?	Can change a shared object?
`number = 100`	No	Not applicable
`point = new MutablePoint(100, 100)`	No	No mutation of the original object
`point.x = 100`	No	Yes

Review: an assistant claims that `move` works because objects are passed by reference. Explain why the observed behavior is right but the explanation is wrong.

Reference identity and reachability

For the next independent identity example, use this minimal teaching type:

```
class Patient { }
```

Operations on references

The equality operators == and != compare reference identity, not object content. Java also provides operations such as `instanceof`; do not confuse identity comparison with all possible reference operations.

- More than one variable can refer to the same object.
- A reference value can be **null**, meaning it identifies no object. This is a value, not the same as an uninitialized local variable.

```
public static void main(String[] args) {
    Patient p1 = new Patient();
    Patient p2 = p1;
    Patient p3 = null;

    if (p1 == p2) {
        // true
    }

    if (p1 == p3) {
        // false
    }
}
```

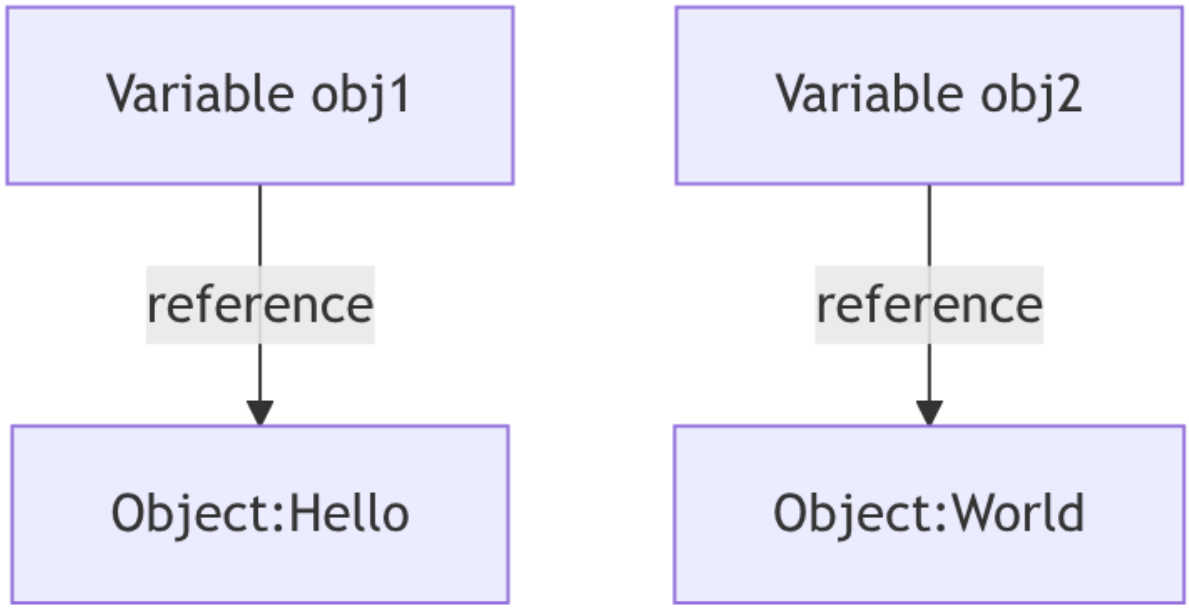
Garbage collector

Garbage collectors reclaim memory from objects that are no longer reachable in the relevant reachability model. An object may be eligible for collection without being reclaimed immediately. Cycles alone do not keep unreachable objects alive.

```java
class Example {
    String name;

    Example(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj1 = new Example("Hello");
        Example obj2 = new Example("World"); // different object
    }
}
```



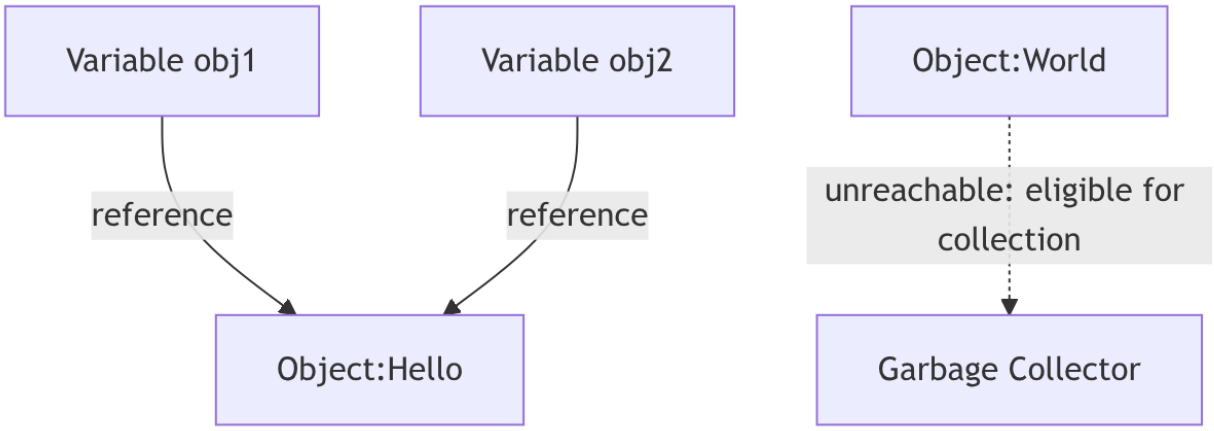

```java
class Example {
    String name;

    Example(String name) {
        this.name = name;
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Example obj1 = new Example("Hello");
        Example obj2 = new Example("World"); // different object

        obj2 = obj1;
    }
}
```



Garbage collection — further details

Reference — optional further reading. The following material describes memory-management costs and implementation choices, not additional guarantees about collection timing. The core concept remains reachability: an object may become eligible for collection without being reclaimed immediately. Collection is not a substitute for explicitly closing resources: code that owns a file or connection must arrange its closure, typically with try-with-resources.

Memory reclamation trade-offs

Advantages:

Memory reclamation is automatic: ordinary Java code does not manually free objects. Allocation still occurs through constructs such as new. Java avoids ordinary use-after-free errors, but **retaining unwanted reachable objects can still cause memory leaks**.

In the example below, removing the last strong reference can make the collection eligible for reclamation; it does not synchronously free memory.

```
import java.util.ArrayList;
import java.util.List;

public class LargeCollectionExample {
    public static void main(String[] args) {
```

```
        // Create a large collection
        List<String> bigList = new ArrayList<>();
        for (int i = 0; i < 1_000_000; i++) {
            bigList.add("Item " + i);
        }

        // If no other references retain it, the list is eligible for GC.
        bigList = null;
    }
}
```

Disadvantages:

Collection has CPU, memory, and pause-time costs that depend on the workload, collector, and configuration. Neither manual allocation nor garbage collection is universally faster.

Developers have no direct control over when the garbage collector runs. As a result, CPU time may be spent on freeing unused objects at unpredictable moments, potentially impacting performance.

Some examples in which the issue is discussed:

- 9 Tips to Reduce Long Garbage Collection Pauses – Practical strategies to minimize GC pause times by tuning JVM settings and selecting appropriate GC algorithms.
- How I Reduced GC Pause from 1s to 50ms in a High-Load Java Application – A case study detailing the reduction of GC pauses in a high-throughput Java service.
- Reducing Garbage Collection Pause Time – Methods to reduce GC pause times, including parallel and concurrent garbage collection strategies.

Garbage collectors in Java 25

Reference — implementation and deployment choices. G1 is the usual default on server-class HotSpot Java 25 configurations; defaults and collector availability depend on the JVM distribution and environment.

GC	Key characteristics
G1 (default)	Parallel, low-latency, suitable for large heaps
ZGC	Generational, concurrent, designed for low pause times
Shenandoah GC	Low pause times, concurrent collection
Parallel GC	High throughput, longer pauses
Serial GC	Single-threaded, for small or embedded apps

- **G1 GC** is designed for **server-side applications** with large heaps.
- It is **incremental and parallel**, aiming to reduce **stop-the-world (STW) pauses** compared to the older **Parallel GC**.

- Although G1 is the default, you can change it using JVM options:

```
java -XX:+UseZGC -jar MyApp.jar
java -XX:+UseShenandoahGC -jar MyApp.jar
```

These commands require an executable JAR and a JDK supporting the chosen collector. Tune using measured evidence, not a generated recommendation detached from the workload.

> **Note:** ZGC's non-generational mode was removed in JDK 24. For Java 25 use `-XX:+UseZGC`; do not copy older `ZGenerational` flags into a new configuration. Generational Shenandoah is a JDK 25 feature where the collector is available; its deployment modes remain an advanced, distribution-dependent choice. None of these collectors provides deterministic reclamation or resource closure.

Resources

- https://www.baeldung.com/jvm-garbage-collectors
- https://openjdk.org/jeps/490
- https://openjdk.org/jeps/521

E2.4 — Arrays

An array is an object containing a **fixed-size sequence of elements of one component type**.

- Indices start at `0`.
- Length is fixed at creation; elements can change.
- Components can be primitive values or references.
- A multidimensional array is an array of arrays.

Unless a complete class is shown, put statements inside `main` and method declarations inside a class. Examples using `Arrays`, `List`, or `Scanner` require the corresponding `java.util` imports.

Creating an array

Declaration

```
int[] numbers;
String[] names;
```

Declaring a local reference variable does not create an array or initialize that variable. Assign it before use.

Declaration and initialization with default values

```
int[] numbers = new int[5];
String[] names = new String[4];
```

Array elements receive default values:

Component type	Default
Numeric primitive	`0` or `0.0`
`boolean`	`false`
`char`	`'\u0000'`

Component type	Default
Reference	`null`

`new String[4]` creates one array with four null references, not four `String` objects.

Declaration and initialization with custom values

```
int[] numbers = {1, 2, 3, 4, 5};
String[] names = {"John", "Mary", "David", "Sarah"};
```

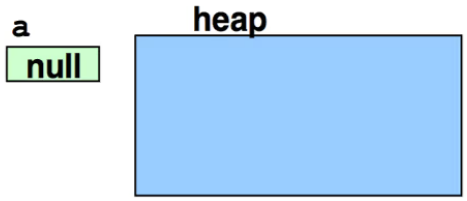
Declaration and initialization in two statements

```
int[] numbers;
numbers = new int[]{1, 2, 3};
// numbers = {1, 2, 3}; // Does not compile outside a declaration
```

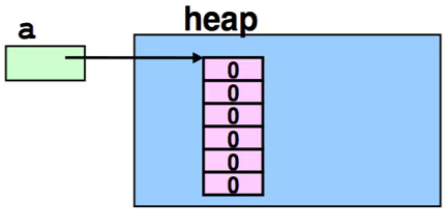
> **Note:** length zero is valid. A negative length throws `NegativeArraySizeException`. A very large allocation may fail even when its length is non-negative.

Arrays in memory

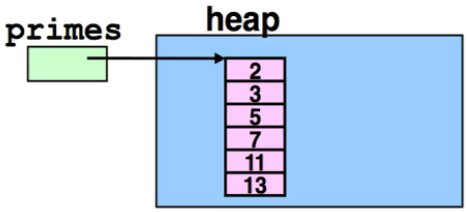
```
int[] a;
```



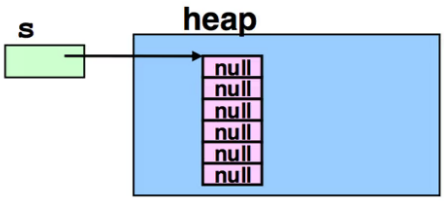

```
a = new int[6];
```



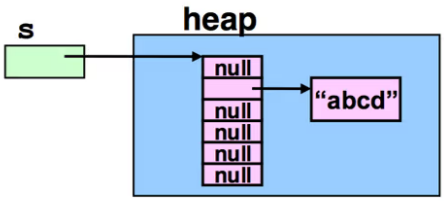

```
int[] primes =
  {2,3,5,7,11,13};
```



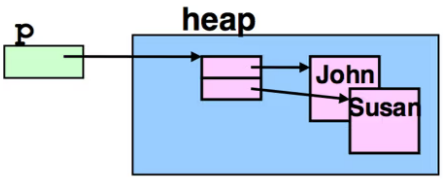

```
String[] s = new
  String[6];
```




```
s[1] = new
  String("abcd");
```




```
Person[] p =
{new Person("John") ,
 new Person("Susan")};
```



Primitive arrays contain primitive values. Reference arrays contain references, not the referenced objects themselves.

Assigning an array reference does **not** copy its elements.

Predict: what are the contents of `first` after this code?

```java
int[] first = {1, 2, 3};
int[] second = first;
second[0] = 99;
IO.println(Arrays.toString(first));
```

Check your prediction

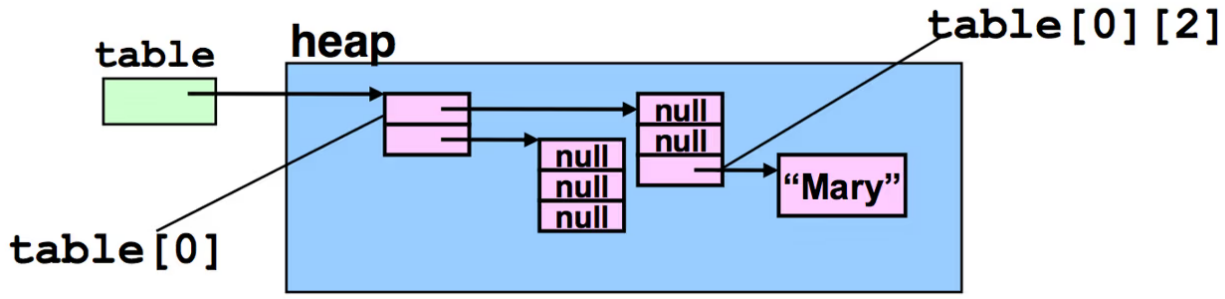
`[99, 2, 3]`. There is one array and two references to it: this is **aliasing**.

`final int[]` prevents reassigning the reference, not modifying the array's elements.

Multi-Dimensional arrays

A two-dimensional array is an **array whose components are row-array references**.

```java
String[][] table = new String[2][3];
table[0][2] = "Mary";
```



Rows are separate arrays and need not have the same length:

```java
int[][] matrix = {{1, 2}, {3}, {4, 5, 6}};
for (int row = 0; row < matrix.length; row++) {
    for (int column = 0; column < matrix[row].length; column++) {
        IO.println(matrix[row][column]);
    }
}
```

Use each row's own length, not an assumed common width.

```java
int[][] rows = new int[3][];
rows[0] = new int[]{1, 2};
// rows[1] and rows[2] are still null
```

> **Note:** `new int[3][4]` creates an outer array and three row arrays initialized with zeros. It is not an uninitialized block or a guarantee of contiguous matrix storage.

Switching rows

```
String[][] table = {{"Alice", "Bob"}, {"Dave", "Eve"}};
String[] temporary = table[0];
table[0] = table[1];
table[1] = temporary;
```

No row contents or strings are copied: swapping row references is **O(1)**.

Assigning and accessing elements

Valid indices satisfy:

```
0 <= index < array.length
```

```
import java.util.Arrays;

public class ArrayAccess {
    public static void main(String[] args) {
        int[] numbers = new int[5];
        numbers[0] = 42;
        numbers[4] = 1;
        IO.println(Arrays.toString(numbers)); // [42, 0, 0, 0, 1]
    }
}
```

`length` is a field: use `numbers.length`, not `numbers.length()`.

An invalid index throws `ArrayIndexOutOfBoundsException`. An empty array has no valid index. Access through a null array reference throws `NullPointerException` instead.

Review: an assistant writes `i <= numbers.length` as a traversal condition. Which access fails for a nonempty array? What happens for an empty array?

Iterating over an array

Using a while loop

```
int[] numbers = {10, 20, 30};
int index = 0;
while (index < numbers.length) {
    IO.println(numbers[index]);
    index++;
}
```

Using a for loop

Use indices when you need positions or want to replace elements:

```
int[] numbers = {10, 20, 30};
for (int index = 0; index < numbers.length; index++) {
    numbers[index]++;
}
```

Using an enhanced for loop

```
int[] numbers = {10, 20, 30};
for (int number : numbers) {
    IO.println(number);
}
```

Predict: does this alternative increment the array elements?

```
int[] numbers = {10, 20, 30};
for (int number : numbers) {
    number++;
}
IO.println(java.util.Arrays.toString(numbers));
```

Check your prediction

No: the output is `[10, 20, 30]`. Each iteration assigns a component value to a local variable; changing that variable does not update the component.

For reference components, the local variable contains a copied reference: mutating the referenced object can still be visible through the array.

Filling an array from input

This complete example assumes that input tokens are valid integers and that enough values are supplied:

```
import java.util.Arrays;
import java.util.Scanner;

public class FillArray {
    public static void main(String[] args) {
        Scanner reader = new Scanner(System.in);
        IO.print("How many numbers (0 to 1000)? ");
        int count = reader.nextInt();
        if (count < 0 || count > 1000) {
            System.err.println("Unsupported size");
```

```
            return;
        }
        int[] numbers = new int[count];
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = reader.nextInt();
        }
        IO.println(Arrays.toString(numbers));
    }
}
```

The application limit of 1000 is an explicit policy, not a Java limit. This demo assumes valid integer input; a robust input loop must also check for malformed input and end-of-input. It does not close the scanner wrapping `System.in`, because closing that scanner would also close the process-wide input stream.

Passing arrays to methods

An array parameter receives **a copy of the array reference**. Java is still pass-by-value.

```
import java.util.Arrays;

public class ArrayParameters {
    public static void addOne(int[] values) {
        for (int i = 0; i < values.length; i++) {
            values[i]++;
        }
    }

    public static void replaceArray(int[] values) {
        values = new int[]{10, 20, 30};
    }

    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};
        replaceArray(numbers);
        IO.println(Arrays.toString(numbers));
        addOne(numbers);
        IO.println(Arrays.toString(numbers));
    }
}
```

Predict both outputs, then use **Step Into** to inspect the parameter and caller references.

Check your prediction

The outputs are `[1, 2, 3]` and `[2, 3, 4]`. Reassignment affects only the parameter; element mutation affects the shared array.

If a method produces a different array, return it and let the caller explicitly assign the result.

Array Utilities

java.util.Arrays

Reference — choose operations by their contract:

Task	API	Important detail
Print contents	`toString`, `deepToString`	Nested arrays need the deep form
Compare contents	`equals`, `deepEquals`	== compares array identity
Copy	`copyOf`, `copyOfRange`	Reference components remain references to the same objects
Fill	`fill`	Modifies the target array
Sort	`sort`	Modifies the array; ascending order for `int[]`
Search	`binarySearch`	Requires sorted input
List view	`asList`	Fixed-size view; not an independent copy

Copying is not assignment

```java
import java.util.Arrays;

public class CopyArray {
    public static void main(String[] args) {
        int[] source = {1, 2, 3};
        int[] copy = Arrays.copyOf(source, source.length);
        copy[0] = 99;
        IO.println(Arrays.toString(source)); // [1, 2, 3]
        IO.println(source == copy); // false
        IO.println(Arrays.equals(source, copy)); // false
    }
}
```

Review: is `matrix.clone()` an independent copy of all rows?

```java
int[][] original = {{1, 2}, {3, 4}};
int[][] copy = original.clone();
copy[0][0] = 99;
IO.println(original[0][0]);
```

Check your prediction

It prints 99. Only the outer array was copied; the row arrays are shared. This is a **shallow copy**.

For an `int[][]` with non-null rows, clone each row as well when independent row contents are required. A general deep-copy policy depends on the component types and on whether shared references should be preserved.

Sorting, searching, and printing

```
int[] numbers = {5, 2, 8, 1, 3};
java.util.Arrays.sort(numbers);
IO.println(java.util.Arrays.toString(numbers)); // [1, 2, 3, 5, 8]
IO.println(java.util.Arrays.binarySearch(numbers, 5)); // 3
IO.println(java.util.Arrays.binarySearch(numbers, 4)); // -4
```

A missing value is encoded as `-(insertionPoint) - 1`, not always `-1`. With duplicate values, the first matching index is not guaranteed. Replacing a linear search with this method is safe only if sorted input and these result conventions satisfy the required contract.

For nested arrays:

```
int[][] matrix = {{1, 2}, {3, 4}};
IO.println(java.util.Arrays.deepToString(matrix)); // [[1, 2], [3, 4]]
```

Arrays.asList

Note — preview of collections:

```
String[] names = {"Alice", "Bob"};
java.util.List<String> list = java.util.Arrays.asList(names);
list.set(0, "Carol");
IO.println(names[0]); // Carol: the view is backed by the array
// list.add("Dave"); // Throws UnsupportedOperationException
```

Fixed-size does not mean immutable. `Arrays.asList(new int[]{1, 2})` produces a list containing one `int[]`, not two boxed integers.

System.arraycopy

Copies a range into an **existing** destination array:

```
int[] source = {10, 20, 30, 40};
int[] destination = {0, 0, 0, 0, 0};
System.arraycopy(source, 1, destination, 2, 3);
IO.println(java.util.Arrays.toString(destination)); // [0, 0, 20, 30, 40]
```

Reference — argument order: source, source position, destination, destination position, element count. Ranges must fit both arrays, and component types must be compatible. Reference elements are not cloned.

> **Note:** overlapping ranges within the same array are supported. A naive forward-copy loop is not always an equivalent replacement.

Resources

- https://dev.java/learn/language-basics/arrays/
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/System.html#arraycopy(java.lang.Object,int,java.lang.Object,int,int)

E2.5 — Strings

`String` represents immutable text. It provides operations for inspecting, comparing, searching, and transforming content.

Unless a complete class is shown, place statements inside `main` and methods inside a class. Imports are shown where needed or fully qualified names are used.

Creating and using strings

```
public class StringBasics {
    public static void main(String[] args) {
        String text = "abc";
        IO.println(text.length());
        IO.println(text.toUpperCase(java.util.Locale.ROOT));
        IO.println(text.startsWith("d"));
        IO.println(text.indexOf('c'));
        IO.println(text.substring(1, 3));
        IO.println(text.equals("ABC"));
    }
}
```

Predict the six results, then run the program.

Check your prediction

The output is 3, ABC, `false`, 2, bc, and `false`, each on its own line.

Prefer literals for ordinary fixed text:

```
String greeting = "hello";
String description = """
        Java strings are immutable.
        Text blocks make multiline literals easier to read.
        """;
```

A text block is still a `String`, not a separate type. It supports escapes and removes incidental indentation; the example ends with a newline.

> **Note:** `new String("hello")` creates a distinct object, but is normally unnecessary. We use it later only to make reference identity explicit.

String main methods

Reference — method names are less important than their contracts:

Method	Behavior or detail
`length()`	Number of UTF-16 code units
`charAt(index)`	One UTF-16 code unit at a valid index
`isEmpty()`	Length is zero
`isBlank()`	Empty or composed of whitespace code points as defined by the API
`equals(other)`	Same string content
`compareTo(other)`	Lexicographic comparison; negative, zero, or positive
`contains`, `startsWith`, `endsWith`	Test for literal sequences
`indexOf`	First matching position, or `-1`
`substring(begin, end)`	Begin inclusive, end exclusive
`replace`	Literal character or sequence replacement
`split`	Splitting using a regular expression
`strip()`	Remove leading and trailing whitespace as defined by the API

These operations do not change the original `String`. Methods returning strings may return an existing object when appropriate; do not assume every call allocates a new one.

Indices and boundaries

```
String text = "Java";
IO.println(text.substring(0, 2)); // Ja
IO.println(text.substring(4, 4)); // Empty string
// text.charAt(4); // Throws IndexOutOfBoundsException
```

For `charAt`, `0 <= index < length()`. For `substring(begin, end)`, `0 <= begin <= end <= length()`.

Review: a method takes the first three units using `substring(0, 3)`. What contract should it have for shorter input? Rejecting it and accepting the whole short string are different policies; do not silently choose one without a requirement.

Literal text versus regular expressions

```
String filename = "report.csv";
String[] parts = filename.split("\\.");
IO.println(java.util.Arrays.toString(parts)); // [report, csv]
IO.println("a,b,".split(",").length);     // 2
IO.println("a,b,".split(",", -1).length); // 3
```

`split(".")` does not mean “split on a literal dot”: `.` has a regex meaning. Default splitting discards trailing empty fields; a negative limit preserves them.

> **Note:** `replace` uses literal text, whereas `replaceAll` uses a regular expression and special replacement syntax. Regular expressions are an API topic to learn deliberately, not infer from a method name.

Text length and Unicode

Note — representation versus visible characters:

```
String symbol = "\uD83D\uDE00"; // One supplementary Unicode code point
IO.println(symbol.length()); // 2
IO.println(symbol.codePointCount(0, symbol.length())); // 1
```

An index addresses a UTF-16 code unit. A Unicode code point can require two units, and a user-perceived character can contain multiple code points. Neither `length()` nor code-point count universally measures visible characters.

For introductory indexing exercises, explicitly assume simple text when that is the intended scope.

Strings in memory

Strings are immutable: their content cannot be modified after construction.

Predict: does this print HELLO?

```
String original = "hello";
String alias = original;
original.toUpperCase(java.util.Locale.ROOT);
IO.println(original);
original = original.toUpperCase(java.util.Locale.ROOT);
IO.println(original);
IO.println(alias);
```

Check your prediction

The outputs are `hello`, `HELLO`, and `hello`. Discarding the returned string has no effect on the original. Reassignment changes a variable, not the content of the object shared with `alias`.

String pool

Note: identical string literals are interned and refer to the same `String` object. This explains identity results, but is not a rule for comparing text.

```
String first = "Java";
String second = "Java";
String separate = new String("Java");
IO.println(first == second);   // true: identical interned literals
IO.println(first == separate); // false: distinct objects
```

Do not depend on how runtime text happens to be allocated or reused. Use `equals` for content comparison regardless of object reuse.

Comparing strings

- == compares reference identity.
- `equals` compares string content.
- `compareTo` supplies a lexicographic order, not a language-sensitive dictionary order.

```
String expected = "Java";
String supplied = new String("Java");
IO.println(expected == supplied);      // false
IO.println(expected.equals(supplied)); // true
```

Null and comparison policy

```
String input = null;
IO.println("yes".equals(input)); // false
IO.println(java.util.Objects.equals(input, null)); // true
// input.equals("yes"); // Throws NullPointerException
```

`Objects.equals` compares possibly null values. It does not decide whether null is valid input for your application.

Review: an assistant replaces `"yes".equals(input)` with `input == "yes"` because a test using a literal passes. Add a test with `new String("yes")` and explain why the replacement is incorrect.

> **Note:** `equalsIgnoreCase` does not implement every language's text-comparison rules. Case conversion without an explicit locale uses the default locale. `Locale.ROOT` is useful for locale-independent operations; user-facing text may need an appropriate user locale and collation policy.

Concatenating strings

The + operator

Use + for a small number of values in one expression:

```
int students = 3;
String message = "Students: " + students;
IO.println(message); // Students: 3
```

Predict: why do these expressions produce different output?

```
IO.println("Total: " + 2 + 3);
IO.println("Total: " + (2 + 3));
IO.println(2 + 3 + " total");
```

Check your prediction

The outputs are `Total: 23`, `Total: 5`, and `5 total`. Grouping and operand types determine whether each + performs addition or concatenation.

StringBuilder

For incremental construction in a loop, `StringBuilder` provides a mutable buffer:

```
public class BuildText {
    public static String repeatWithConcatenation(int count) {
        String result = "";
        for (int i = 0; i < count; i++) {
            result += 'a';
        }
        return result;
    }

    public static String repeatWithBuilder(int count) {
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < count; i++) {
            builder.append('a');
        }
        return builder.toString();
    }

    public static void main(String[] args) {
        String first = repeatWithConcatenation(5);
        String second = repeatWithBuilder(5);
        IO.println(first); // aaaaa
        IO.println(first.equals(second)); // true
    }
}
```

These teaching methods assume a non-negative count. The builder changes during construction; the returned `String` is immutable.

Repeatedly extending an immutable accumulated result can require copying a growing amount of text. A builder avoids that pattern. This does not mean every + expression should be rewritten.

> **Note:** a single `nanoTime` measurement is not a reliable Java microbenchmark. JIT warm-up, allocation, garbage collection, and unused results can affect measurements. Use a suitable harness such as JMH when performance needs evidence; do not turn this introductory example into a universal timing claim.

Resources

- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Scanner.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/IO.html

E2.6 — Wrapper Classes

Each primitive type has a corresponding **wrapper class**. Wrappers are immutable reference types, not alternative names for primitive types.

Unless a complete class is shown, place statements inside `main`.

Wrappers are useful when:

- An API requires an object, such as a generic collection.
- Absence is represented by `null`, with an explicit policy for handling it.
- You need conversion methods or constants associated with a primitive type.

Prefer primitives for ordinary numeric computation when absence and an object-based API are not required.

Reference — type mapping:

Primitive	Wrapper
`boolean`	`Boolean`
`byte`	`Byte`
`short`	`Short`
`char`	`Character`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`

Boxing and unboxing

Boxing converts a primitive value to a corresponding wrapper reference. It does not necessarily allocate a fresh object.

Unboxing extracts the primitive value from a wrapper object.

```
int primitive = 100;
Integer wrapped = Integer.valueOf(primitive);
int extracted = wrapped.intValue();
```

The compiler can insert these conversions automatically:

```java
Integer wrapped = 100; // Autoboxing
int extracted = wrapped; // Auto-unboxing
```

Use factories or autoboxing, not deprecated wrapper constructors such as `new Integer(...)`.

Immutable object, reassignable variable

Predict: do both variables become 11?

```java
Integer first = 10;
Integer second = first;
first++;
IO.println(first);
IO.println(second);
```

Check your prediction

The outputs are `11` and `10`. Incrementing unboxes, adds one, and boxes the result back into `first`. It does not mutate the old `Integer` object shared with `second`.

Wrappers in collections

Note — wrappers in collections:

```java
java.util.List<Integer> scores = new java.util.ArrayList<>();
scores.add(10); // Boxing
int score = scores.get(0); // Unboxing
```

Generic type arguments must be reference types: `List<Integer>`, not `List<int>`. `int[]` is nevertheless a reference type: an array object whose components are primitive integers.

Converting Strings to Primitive Types

```java
int count = Integer.parseInt("42");
double value = Double.parseDouble("3.14");
boolean enabled = Boolean.parseBoolean("true");
IO.println(count);
IO.println(value);
IO.println(enabled);
```

`parseInt` returns an `int`; `Integer.valueOf("42")` returns an `Integer`. Choose the return type your API needs.

Parsing is not validation of the whole requirement

```
String supplied = " 42 ";
int count = Integer.parseInt(supplied.strip());
if (count < 0 || count > 100) {
    throw new IllegalArgumentException("Count must be between 0 and 100");
}
```

Here we explicitly accept surrounding whitespace and restrict the application range. `parseInt` alone would not establish that range.

Reference — important differences:

Input or operation	Outcome
`Integer.parseInt("hello")`	`NumberFormatException`
`Integer.parseInt("2147483648")`	`NumberFormatException`: outside int range
`Integer.parseInt(" 42 ")`	`NumberFormatException`: no automatic trimming
`Double.parseDouble("NaN")`	`NaN`, not a parsing failure
`Boolean.parseBoolean("TRUE")`	`true`
`Boolean.parseBoolean("yes")`	`false`, not a parsing failure
`Boolean.parseBoolean(null)`	`false`

Review: an assistant claims `Boolean.parseBoolean` validates that input is either `"true"` or `"false"`. Which input disproves that? If other input must be rejected, validate it explicitly before converting.

> **Note:** `Double.parseDouble` uses Java's numeric syntax, not locale-specific decimal separators. Successful parsing can produce non-finite values; use `Double.isFinite` if the requirement excludes them. Decimal overflow can produce infinity rather than `NumberFormatException`.

Converting Primitive Types to Strings

```
String count = Integer.toString(42);
String value = Double.toString(3.14);
String enabled = Boolean.toString(true);
String alternative = String.valueOf(42);
```

Conversion is not display formatting. For a fixed number of displayed fractional digits, use a format string, for example `String.format(java.util.Locale.ROOT, "%.2f", 3.5)` produces `"3.50"`.

> **Note:** converting a nullable reference through `String.valueOf(Object)` produces the text `"null"` when the reference is null. Decide whether that text is suitable rather than allowing it to leak into a user-facing message accidentally.

Comparing wrappers

For two wrapper references, == checks identity, not numeric value. `equals` compares values **according to the wrapper's type-specific contract**.

```
Integer first = Integer.valueOf("2000");
Integer second = Integer.valueOf("2000");
IO.println(first == second); // Identity: do not rely on the result
IO.println(first.equals(second)); // true
```

Do not infer value-comparison behavior from a particular JVM's allocation or caching choices.

Caching

Note — explaining identity, not choosing a comparison strategy:

```
Integer first = Integer.valueOf(100);
Integer second = Integer.valueOf(100);
IO.println(first == second); // true: guaranteed valueOf cache range
```

`Integer.valueOf(int)` guarantees caching from **−128 to 127**, and may cache additional values. The language also guarantees shared identity for boxing certain small constant values. These guarantees should not be generalized into an identity rule for every wrapper expression.

Core rule: compare values using the intended value operation, not reference identity. The string-pool and wrapper-cache mechanisms are different; neither makes == a general content comparison.

Same number, different wrapper type

```
Integer integer = 42;
Long longValue = 42L;
IO.println(integer.equals(longValue)); // false
```

`Integer.equals` requires another `Integer`, not merely a numerically equivalent `Number`. Normalize to an agreed numeric type when cross-type comparison is part of the requirement.

Mixed primitive and wrapper comparison

```
Integer wrapped = 2000;
int primitive = 2000;
IO.println(wrapped == primitive); // true: numeric comparison after unboxing
```

The operand types matter: this is not a comparison between two wrapper references. If wrapped is null, the same expression fails during unboxing.

> **Note:** floating-point wrappers have additional equality rules: `Double.equals` treats NaN values as equal and distinguishes positive and negative zero. Do not assume its behavior is identical to primitive floating-point ==.

NPE when unboxing

Unboxing a null reference throws `NullPointerException`, even when no explicit unboxing method appears.

```
Integer count = null;
// int value = count;       // NPE: assignment requires unboxing
// boolean empty = count == 0; // NPE: comparison requires unboxing
// count++;                 // NPE: increment requires unboxing
```

Predict: does the following comparison merely return `false`?

```
Boolean enabled = null;
if (enabled) {
    IO.println("Enabled");
}
```

Check your prediction

No: evaluating the condition unboxes null and throws `NullPointerException` before either branch can run.

Choose a policy for absence

```
Boolean enabled = null;
boolean active = Boolean.TRUE.equals(enabled); // Explicit policy: null is not true

Integer count = null;
if (count != null && count > 0) {
    IO.println("Positive count");
}
```

These examples treat absence as “not active” or skip processing. That is not always the right policy: a required missing value may need to be rejected instead. Do not silently replace every null with zero or false.

For equality of two possibly null references, `java.util.Objects.equals` avoids dereferencing null, but retains each wrapper's type-specific equality rules.

Resources

- https://dev.java/learn/numbers-strings/autoboxing/
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Integer.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Boolean.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Double.html

E2.7 — Text Input and Output

Standard streams

Stream	Usual console role	Type
`System.in`	Input	`InputStream`
`System.out`	Normal output	`PrintStream`
`System.err`	Diagnostics	`PrintStream`

The runtime initializes these streams. They can be redirected, so console or keyboard connections are not guaranteed.

Printing and formatting

For simple output in Java 25, prefer `IO.print` and `IO.println`; `java.lang.IO` needs no explicit import, even in an explicit class. The following example uses `System.out` to illustrate its `PrintStream` methods. Keep `System.out.printf` for formatted output: `IO` has no `printf` method.

```
System.out.print("Hello");
System.out.println(" world!");
System.out.printf(java.util.Locale.ROOT, "Price: %.2f%n", 12.5);
String label = String.format(java.util.Locale.ROOT, "Count: %d", 3);
System.out.println(label);
```

Reference: `print` adds no newline, `println` adds one, and `printf` formats output. `String.format` returns a string rather than printing it. `%n` denotes a platform line separator; `%d`, `%s`, and `%.2f` format integers, text, and two fractional digits respectively.

Formatting a rounded display does not change the stored numeric value.

Reading from the keyboard

Simple line input with java.lang.IO

For simple interactive programs, Java 25 provides `IO.readln` without an explicit import. Save the following as `ReadGreeting.java` and run it with `java ReadGreeting.java`:

```
void main() {
    String name = IO.readln("Full name: ");
    if (name != null) {
        IO.println("Hello, " + name);
    }
}
```

`readln` reads a whole line without its separator and returns null at end-of-input. It returns text; numeric conversion still requires parsing. The API can throw `java.io.IOError` on an input failure.

Core: choose one owner and reading strategy for standard input. After using `IO.readln`, do not also read `System.in` through a scanner or another reader in the same program: buffering makes such mixing unspecified. The scanner examples below are independent programs, not additions to this one.

Token and line input with Scanner

`Scanner` reads text and parses tokens. By default, whitespace separates tokens.

```
import java.util.Scanner;

public class ReadName {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        IO.print("Full name: ");
        if (scanner.hasNextLine()) {
            String name = scanner.nextLine();
            IO.println("Hello, " + name);
        }
    }
}
```

`next()` reads one token; `nextLine()` reads the rest of the current line, excluding its line separator.

Mixing token and line input

Use fixed input first so the behavior is reproducible:

```
try (java.util.Scanner scanner = new java.util.Scanner("42\nAda Lovelace\n")) {
    int age = scanner.nextInt();
    String name = scanner.nextLine();
    IO.println("[" + name + "]");
}
```

Predict: does `name` contain `Ada Lovelace`?

Check your prediction

No: the output is `[]`. `nextInt()` consumes the integer token but leaves the line separator. `nextLine()` consumes the empty remainder of that line.

If the input protocol puts the name on the next line, consume the remainder first, then read that next line. Alternatively, consistently read whole lines and parse the numeric lines. Do not add extra `nextLine()` calls blindly: they can discard legitimate input.

Invalid input and ownership

- Invalid numeric tokens can cause `InputMismatchException`; missing input can also fail.
- `hasNextInt()` lets you check the next token, but an invalid token must be consumed or otherwise handled to make progress.
- Numeric parsing is locale-sensitive; set the scanner's locale when the input format requires a particular convention.

> **Note:** closing a scanner also closes its underlying source when applicable. The console demo deliberately does not close `System.in`; the application should decide its ownership. In the fixed-string example, try-with-resources automatically closes the scanner when the `try` block exits, including on failure.

Reference — reading a file:

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.util.Scanner;

public class ReadFile {
    public static void main(String[] args) throws IOException {
        try (Scanner scanner = new Scanner(Path.of("names.txt"),
        ↪  StandardCharsets.UTF_8)) {
            while (scanner.hasNextLine()) {
                IO.println(scanner.nextLine());
            }
        }
    }
}
```

The example requires a UTF-8 `names.txt` in the working directory. The `throws IOException` declaration lets file-opening failures propagate to the caller; try-with-resources closes the scanner on exit. A scanner can read sources other than the keyboard.

Resources

E2.8 — Unit Testing with JUnit

From output to automated checks

Compilation checks whether a program follows the language rules, not whether it meets its requirements. Printing a result and inspecting it manually is useful while exploring, but it is easy to overlook a wrong value or forget to repeat a check.

A **unit test** automatically checks a small unit of behavior, such as one method, against an expected result. A **test suite** is a set of tests that can be rerun after a change. A test that protects against the return of a previously fixed bug is a **regression test**.

Tests provide evidence for the cases they exercise, not proof that a program is correct for every input. Unit tests also do not replace integration tests between components or manual checks of a user interface.

A small Maven project

The examples use **JUnit Jupiter**, the programming model used by the repository's JUnit tests. Create this independent practice project:

```
testing-basics/
 ├─ pom.xml
 └─ src/
    ├─ main/java/com/example/Basics.java
    └─ test/java/com/example/BasicsTest.java
```

Use this complete `pom.xml`, with JDK 25 and Maven installed. The versions match the course's `deploy-cli` example; they are not a claim about the latest release.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
          ↪  https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>testing-basics</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.12.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.2</version>
            </plugin>
        </plugins>
    </build>
</project>
```

- `test` scope makes JUnit available to test code, not to application code.
- The Jupiter dependency includes support for the parameterized tests used below.
- Maven Surefire discovers and runs the tests. Keep the conventional `BasicsTest` class name.

Start with a contract

Before writing assertions, decide what the method promises:

Method	Contract for this example
`clamp(value, min, max)`	Return `min` below the interval, `max` above it, otherwise `value`. Requires `min <= max`.
`isPalindrome(text)`	Compare characters exactly, including case and spaces. Empty and single-character strings are palindromes. Requires non-null text.
`reversedCopy(values)`	Return a new array in reverse order without changing the input. Requires a non-null array.

Inputs outside these preconditions have no specified behavior here. Defining and testing explicit rejection with exceptions comes in E5.

Save this as `src/main/java/com/example/Basics.java`:

```
package com.example;

public class Basics {
    public static int clamp(int value, int min, int max) {
        if (value < min) {
            return min;
        }
        if (value > max) {
            return max;
        }
        return value;
    }

    public static boolean isPalindrome(String text) {
        for (int left = 0, right = text.length() - 1; left < right; left++, right--) {
            if (text.charAt(left) != text.charAt(right)) {
                return false;
            }
        }
        return true;
    }

    public static int[] reversedCopy(int[] values) {
        int[] result = new int[values.length];
        for (int i = 0; i < values.length; i++) {
            result[i] = values[values.length - 1 - i];
        }
        return result;
    }
}
```

These methods return values rather than reading the keyboard or printing. Keeping computation separate from console interaction makes it easy to supply fixed inputs and check results.

The first test: Arrange–Act–Assert

Save this as `src/test/java/com/example/BasicsTest.java`:

```
package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class BasicsTest {
```

```
    @Test
    void clampReturnsUpperBoundWhenValueIsTooLarge() {
        // Arrange: choose input and expected result
        int value = 12;
        int expected = 10;

        // Act: invoke the behavior under test
        int actual = Basics.clamp(value, 0, 10);

        // Assert: compare the result with the requirement
        assertEquals(expected, actual);
    }
}
```

- `@Test` marks a method for JUnit to run; it is an **annotation**, not a method call.
- A basic test method returns `void`, takes no arguments and is not `static`. The class and method do not need to be `public`.
- JUnit invokes the test; no `main` method is needed. Treat the test class as a container for now; class design comes in E3.
- `import static` lets us write `assertEquals(...)` instead of the fully qualified assertion name.
- Put the **expected** value first and the **actual** result second. Derive the expected result from the contract, not by copying the implementation.

From the `testing-basics` directory, run:

```
mvn test
```

Check that the summary reports **1 test**, with zero failures and errors. Reports are in `target/surefire-reports/`. A build that runs zero tests has not verified this behavior.

Experiment: change the expected value to 9 and rerun. Read the expected/actual diagnostic and locate the failing assertion. Restore `10` afterward. This checks that the test is really being executed.

Assertions for different results

Add the following methods **inside the existing `BasicsTest` class**:

```
@Test
void palindromeUsesExactCharacters() {
    assertTrue(Basics.isPalindrome("level"));
    assertFalse(Basics.isPalindrome("Level"));
}

@Test
void emptyAndSingleCharacterStringsArePalindromes() {
    assertTrue(Basics.isPalindrome(""));
    assertTrue(Basics.isPalindrome("x"));
```

```
}

@Test
void reversedCopyReversesContentsWithoutChangingInput() {
    int[] input = {2, 5, 8};

    int[] actual = Basics.reversedCopy(input);

    assertArrayEquals(new int[]{8, 5, 2}, actual);
    assertArrayEquals(new int[]{2, 5, 8}, input);
    assertNotSame(input, actual);
}

@Test
void reversedCopyHandlesEmptyAndSingleElementArrays() {
    assertArrayEquals(new int[]{}, Basics.reversedCopy(new int[]{}));
    assertArrayEquals(new int[]{7}, Basics.reversedCopy(new int[]{7}));
}

@Test
void floatingPointComparisonUsesATolerance() {
    double actual = 0.1 + 0.2;

    assertEquals(0.3, actual, 1.0e-12);
}
```

Assertion	Purpose
`assertEquals(expected, actual)`	Compare values; for strings, compare contents rather than reference identity.
`assertTrue(condition)` / `assertFalse(condition)`	Check a boolean result.
`assertArrayEquals(expected, actual)`	Compare array contents, not just their references.
`assertNotSame(first, second)`	Check that two references identify different objects.
`assertEquals(expected, actual, delta)`	Compare floating-point values with an absolute tolerance appropriate to the requirement.

JUnit assertions are not Java's `assert` statement: they do not require the JVM's `-ea` flag. A printed value alone does not make a test fail when the value is wrong.

Several assertions can describe one coherent behavior, such as returning a reversed copy without mutating the input. Avoid combining unrelated behaviors into one large test.

Choosing cases and testing boundaries

For `clamp(value, 0, 10)`, choose inputs from distinct regions:

Region	Input	Expected
Below the interval	`-1`	`0`
Lower boundary	`0`	`0`
Inside the interval	`5`	`5`
Upper boundary	`10`	`10`
Above the interval	`11`	`10`

Also check an interval with equal bounds: `clamp(5, 3, 3)` must return 3. For strings and arrays, consider empty, one-element, and multi-element inputs. Tests should distinguish plausible mistakes, not merely repeat one easy case.

Parameterized tests

When the same behavior needs several input/expected pairs, use a parameterized test. Add these imports to `BasicsTest.java`:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
```

Add this method inside `BasicsTest`:

```
@ParameterizedTest
@CsvSource({
        "-1, 0, 10, 0",
        "0, 0, 10, 0",
        "5, 0, 10, 5",
        "10, 0, 10, 10",
        "11, 0, 10, 10",
        "5, 3, 3, 3"
})
void clampHandlesRegionsAndBoundaries(int value, int min, int max, int expected) {
    assertEquals(expected, Basics.clamp(value, min, max));
}
```

Each CSV row supplies arguments for a separate invocation. Do not also add `@Test` to this method. With all examples included, `mvn test` should report **12 tests**: six ordinary tests and six parameterized invocations.

A useful testing workflow

1. Write down the behavior and select representative inputs, including boundaries.
2. Add a test with a clear name describing the expected behavior.
3. Run it and inspect any failure; either the implementation or the test's expectation may be wrong.
4. Fix the problem, then rerun the whole suite to detect regressions.
5. Refactor while preserving the behavior checked by the tests.

Keep tests independent: do not rely on execution order or shared mutable data. Prefer fixed inputs to keyboard interaction, current time, or network access in these unit tests.

Practice: add tests for an even-length palindrome, a non-palindrome, an array with repeated values, and a negative interval for `clamp`. Then deliberately break the reverse-index calculation and check which tests detect the defect.

Next: in E3, use the same structure to check constructors, object state, and invariants. In E5.1 — Exceptions and Propagation, extend assertions to expected exceptions with `assertThrows`.

Resources

- JUnit 5 user guide
- Maven Surefire Plugin
- Existing temperature converter tests

E3.1 — Classes and Constructors

Examples progressively redefine `Car`: compile each version separately. Put method fragments inside the indicated class and use statements inside `main`. Short examples may deliberately omit validation to focus on initialization; they are not complete domain models.



Declaring new classes

A new class is declared with the `class` keyword followed by a name. A class body can include fields, methods, constructors, and other members.

Fields store the state of an object, **methods** define its behavior, and **constructors** initialize new objects of the class.

```
public class Car {
    private String licensePlate;
    private double price;
    private double hp;

    public Car(String licensePlate, double price, double hp) {
        this.licensePlate = licensePlate;
```

```
        this.price = price;
        this.hp = hp;
    }

    public String show() {
        return "Car{" +
                "licensePlate='" + licensePlate + '\'' +
                ", price=" + price +
                ", hp=" + hp +
                '}';
    }

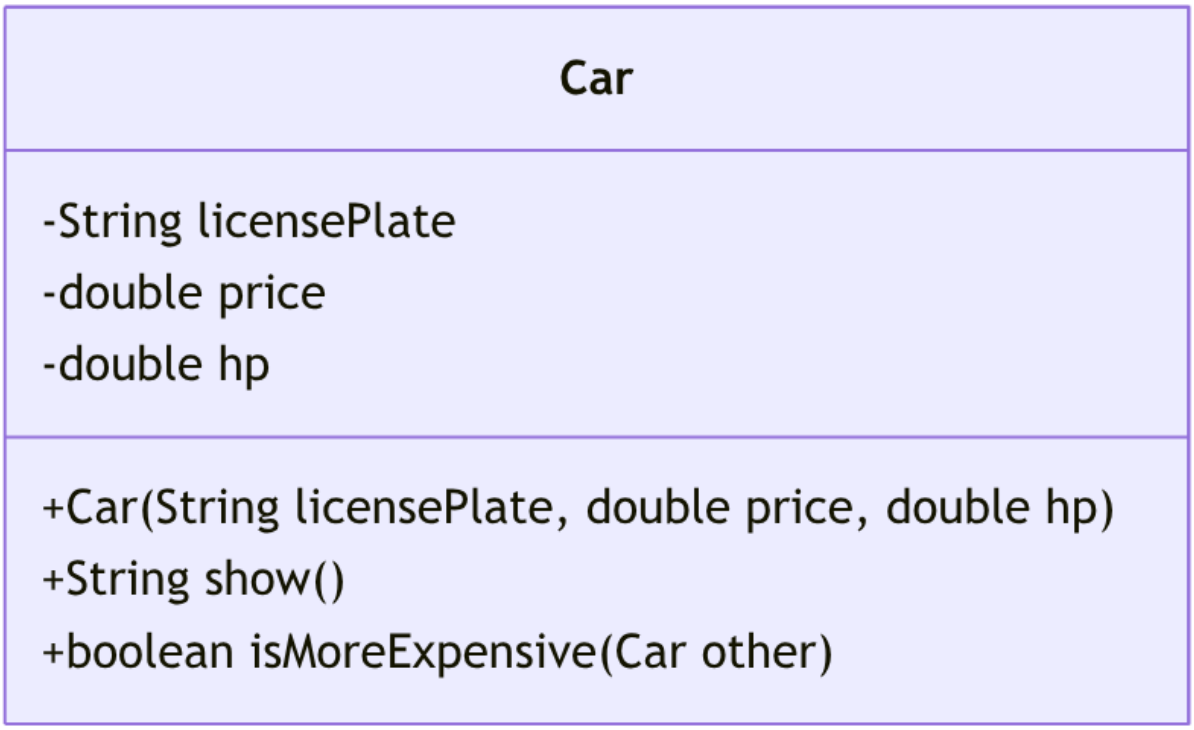
    public boolean isMoreExpensive(Car other) {
        return price > other.price;
    }
}
```

Let's create **instances** of the class `Car`:

```
public static void main(String[] args) {
    Car economy = new Car("DD654ZF", 3000, 60.0);
    Car expensive = new Car("GH345II", 72000, 460.0);

    IO.println(economy.show()); // Car{licensePlate='DD654ZF', price=3000.0, hp=60.0}
    IO.println(expensive.isMoreExpensive(economy)); // true
    IO.println(economy.isMoreExpensive(expensive)); // false
}
```

Reference — UML class diagram of `Car`:



Constructors

Constructors are special members used to initialize a **new object** of a class.

- A constructor is invoked when an instance is created using the keyword `new`.
- Constructors initialize the state of newly created objects.
- Constructors can initialize fields using values provided as parameters.
- A constructor has the same name as the class.
- A constructor has **no return type**, not even `void`.

For example:

```
public Car(String licensePlate, double price, double hp) {
    this.licensePlate = licensePlate;
    this.price = price;
    this.hp = hp;
}
```

To initialize the fields, the keyword `this` refers to the **current object**.

It is especially useful when a constructor parameter has the same name as an instance field:

```java
public Car(String licensePlate, double price, double hp) {
    this.licensePlate = licensePlate;
    this.price = price;
    this.hp = hp;
}
```

Here:

- `this.licensePlate` refers to the instance field;
- `licensePlate` refers to the constructor parameter.

Default and no-argument constructor

If a class declares **no constructor**, the compiler provides a **default constructor** with no parameters.

For example:

```java
public class Car {
    private String licensePlate;
    private double price;
    private double hp;
}
```

We can create an instance of the class using the compiler-provided constructor:

```java
public static void main(String[] args) {
    Car car = new Car();
}
```

The fields are initialized to their default values:

```
licensePlate = null
price        = 0.0
hp           = 0.0
```

A class does **not** receive a compiler-provided default constructor if it declares any constructor itself.

For example, after declaring:

```java
public Car(String licensePlate, double price, double hp) {
    this.licensePlate = licensePlate;
    this.price = price;
    this.hp = hp;
}
```

the expression:

```
new Car();
```

is no longer valid unless we explicitly provide a no-argument constructor.

> **NOTE:** A **default constructor** is specifically the no-argument constructor automatically provided by the compiler when no constructor is declared. A programmer-defined no-argument constructor is simply a **no-argument constructor**, not a default constructor in the strict sense.

Constructor overloading

You can define multiple constructors within a class. Each constructor must have the same name as the class but must differ in its parameter list.

This practice is known as **constructor overloading**.

It can provide different ways to initialize objects, for example by allowing some attributes to be omitted.

```
public class Car {
    String licensePlate;
    int price;
    double hp;

    public Car(String licensePlate, int price, double hp) {
        this.licensePlate = licensePlate;
        this.price = price;
        this.hp = hp;
    }

    // overloaded constructor
    public Car(String licensePlate, int price) {
        this.licensePlate = licensePlate;
        this.price = price;
        this.hp = 0.0;
    }

    // overloaded constructor
    public Car(String licensePlate) {
        this.licensePlate = licensePlate;
        this.price = 0;
        this.hp = 0.0;
    }

    public String show() {
        return "Car{" +
                "licensePlate='" + licensePlate + '\'' +
                ", price=" + price +
                ", hp=" + hp +
                '}';
    }
```

```
    public boolean isMoreExpensive(Car other) {
        return price > other.price;
    }
}
```

We can also invoke one constructor from another constructor using `this(...)`.

This is useful because it avoids duplicating initialization logic.

```
public class Car {
    String licensePlate;
    int price;
    double hp;

    public Car(String licensePlate, int price, double hp) {
        this.licensePlate = licensePlate;
        this.price = price;
        this.hp = hp;
    }

    // overloaded constructor invoking another constructor
    public Car(String licensePlate, int price) {
        this(licensePlate, price, 60.0);
    }

    // overloaded constructor invoking another constructor
    public Car(String licensePlate) {
        this(licensePlate, 10, 60.0);
    }

    public String show() {
        return "Car{" +
                "licensePlate='" + licensePlate + '\'' +
                ", price=" + price +
                ", hp=" + hp +
                '}';
    }

    public boolean isMoreExpensive(Car other) {
        return price > other.price;
    }
}
```

When using `this(...)` to invoke another constructor:

- Java 25 allows a restricted prologue before the constructor invocation;
- it invokes another constructor of the same class;
- it is commonly used to avoid duplicated initialization code.

Default values are application decisions: zero horsepower or an unspecified price is not automatically a sensible domain model.

Flexible constructor bodies in Java 25

Validation and argument preparation can appear before `this(...)` or `super(...)`, without enabling preview features. This independent version illustrates validation before constructor delegation:

```java
public final class Car {
    private final String licensePlate;
    private final int price;

    public Car(String licensePlate, int price) {
        if (price < 0) {
            throw new IllegalArgumentException("Negative price");
        }
        this.licensePlate = licensePlate;
        this.price = price;
    }

    public Car(String licensePlate) {
        String normalized = java.util.Objects.requireNonNull(licensePlate).strip();
        if (normalized.isEmpty()) {
            throw new IllegalArgumentException("Blank plate");
        }
        this(normalized, 0);
    }
}
```

Core: the prologue must not freely use the partially constructed instance: do not invoke its instance methods, read its fields, or publish `this`. Ordinary access follows the constructor invocation. The explicit invocation is still a special constructor operation, not something that can be called repeatedly or conditionally like a normal method.

> **Note:** Java 25 permits certain early assignments to fields declared in the current class without initializers. This is a restricted initialization rule, not permission to use the whole object before superclass initialization. The earlier examples keep constructor delegation first because they do not need a prologue.

Predict: after declaring only `Car(String licensePlate)`, does `new Car()` still compile? Add the expression in your IDE and explain the diagnostic rather than asking a tool to add constructors blindly.

Static methods and fields

Static methods

A **static method** belongs to the class rather than to a particular object. It can be invoked using the class name without creating an instance.

Static methods are often used for operations that do not depend on the state of a particular object, such as utility methods.

The Java library provides many static methods. For example:

- the `Math` class provides methods such as `Math.min(a, b)`, `Math.abs(val)`, and `Math.pow(x, y)`;
- the `Arrays` class provides utility methods for working with arrays;
- the `Collections` class provides utility methods for working with collections;
- the `Objects` class provides utility methods for working with objects.

Static methods have some important characteristics:

- a static method can directly access only static fields;
- a static method can directly invoke another static method;
- a static method cannot directly access instance fields or invoke instance methods without an object reference;
- a static method cannot use `this`, because `this` refers to a specific object and a static context is not associated with an instance.

```
public class SomeClass {

    public SomeClass() {
        invokeAnInstanceMethod(); // possible: this object is being constructed
        invokeAStaticMethod();    // possible
    }

    public static void invokeAStaticMethod() {
        // Cannot directly invoke invokeAnInstanceMethod() here
    }

    public void invokeAnInstanceMethod() {
        invokeAStaticMethod();  // possible
    }
}
```

To invoke the methods:

```
public static void main(String[] args) {
    // static method
    SomeClass.invokeAStaticMethod();

    // instance method
    SomeClass sc = new SomeClass();
    sc.invokeAnInstanceMethod();
}
```

The distinction is:

```
Class.method()       -> static method
object.method()      -> instance method
```

A static method can still work with objects if they are explicitly passed as parameters or accessed through an object reference.

For example:

```
public static double calculatePrice(Car car) {
    return car.getPrice();
}
```

The method is static, but it can access the `Car` object's state through the `car` reference.

Static fields

A **static field** is a field declared with the `static` keyword.

Unlike an instance field, which belongs to each individual object, a static field belongs to the **class**. There is normally one class-level field shared by all instances of that class.

For example:

```
public class Car {
    private static int numberOfCars = 0;

    public Car() {
        numberOfCars++;
    }

    public static int getNumberOfCars() {
        return numberOfCars;
    }
}
```

All `Car` objects share the same `numberOfCars` field.

This counts constructor calls in the example, not currently live objects. It is not a thread-safe counter: `count++` is a read-modify-write sequence, so concurrent updates can be lost without synchronization or an atomic counter.

Static fields can be accessed using the class name:

```
IO.println(Car.getNumberOfCars());
```

When a static field represents a value that should not be reassigned, it can also be declared `final`.

The Java `Math` class, for example, provides mathematical constants:

```java
// Simplified excerpt, not a replacement for java.lang.Math.
public final class Math {
    public static final double E = 2.718281828459045;
    public static final double PI = 3.141592653589793;
    public static final double TAU = 2 * PI;
    // ...
}
```

To use the constants:

```java
public static void main(String[] args) {
    IO.println(Math.E);
    IO.println(Math.PI);
}
```

Static fields declared with `final` are commonly used as **constants**.

By convention, constants are named using uppercase letters with underscores separating words:

```java
public static final int MAX_SPEED = 200;
public static final double TAX_RATE = 0.22;
```

> **NOTE:** `final` prevents the field from being assigned a different value after initialization. For a primitive constant, this means the value cannot change. For a reference type, `final` prevents the reference from being reassigned, but it does **not** make the referenced object immutable.

Resources

E3.2 — Encapsulation and Invariants

Members visibility

An **access modifier** is a keyword that specifies where a class member can be accessed.

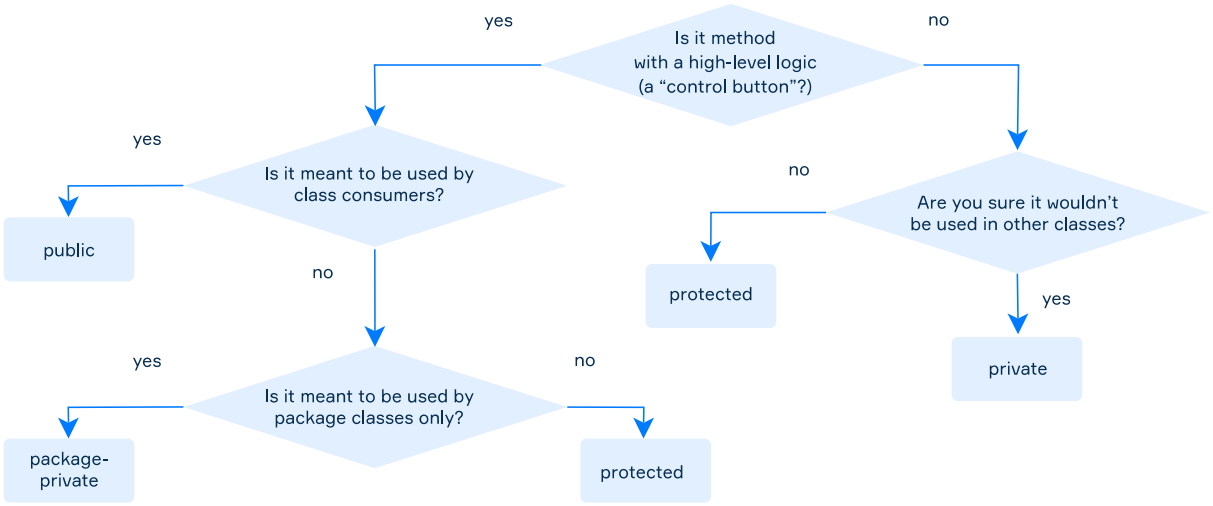
For top-level classes, only `public` and package-private access are allowed. For members such as fields and methods, all four access levels are available.

The four access levels are:

- **private** — accessible only within the declaring class;
- **package-private** — accessible by classes in the same package. It is the implicit access level when no modifier is specified;
- **protected** — accessible by classes in the same package and by subclasses, including subclasses in other packages under specific access rules;
- **public** — accessible wherever the containing type is accessible.

Reference — simplified member-access table. The subclass column means a subclass in another package; the same-package rule still applies to subclasses in the declaring package. Nested classes and cross-package protected access have additional rules.

Modifier	Class	Package	Subclass	All
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes	No
package-private	Yes	Yes	No	No
`private`	Yes	No	No	No



> **NOTE:** The table above describes member access. A top-level class cannot be declared `private` or `protected`; a top-level class can be `public` or package-private.

Access control improves:

- **Code clarity**: implementation details can be hidden while exposing only the operations that clients need.
- **Code safety**: direct access to internal state can be restricted, allowing the class to enforce its own invariants.
- **Maintainability**: the internal implementation can change without requiring users of the class to access its fields directly.

Getters and Setters

A common technique for encapsulating fields is to provide **getters** and **setters**.

- A **getter** reads the value of a field.
- A **setter** changes the value of a field.

Getters and setters are usually `public`, but they do not have to be. Their visibility should depend on how the class is intended to be used.

Encapsulation does **not** mean that every field must have both a getter and a setter. A class can deliberately expose:

- a getter without a setter, making the state externally read-only;
- a setter without a getter, when write-only access is appropriate;
- neither, when the field should remain completely internal;
- both, when both reading and controlled modification are required.

Using methods instead of exposing fields directly gives the class control over how its state is accessed and modified.

For example:

```
public class Car {
    private String licensePlate;
    private int price;
    private double hp;

    public Car(String licensePlate, int price, double hp) {
        this.licensePlate = licensePlate;
        this.price = price;
        this.hp = hp;
    }

    public String getLicensePlate() {
        return licensePlate;
    }

    public void setLicensePlate(String licensePlate) {
        this.licensePlate = licensePlate;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    public double getHp() {
        return hp;
    }

    public void setHp(double hp) {
        this.hp = hp;
    }

    // ...
}
```

See this interesting discussion on the topic, in particular answer #2.

In the following example, we define a setter that implements actual validation.

```
public final class Car {
    private String licensePlate;
    private int price;
    private final double hp;

    public Car(String licensePlate, int price, double hp) {
        this.licensePlate = validatedPlate(licensePlate);
        if (price < 0 || !Double.isFinite(hp) || hp < 0) {
            throw new IllegalArgumentException("Invalid price or horsepower");
        }
        this.price = price;
```

```
        this.hp = hp;
    }

    private static String validatedPlate(String plate) {
        // Teaching format: two ASCII capitals, three digits, two capitals.
        if (plate == null || !plate.matches("[A-Z]{2}[0-9]{3}[A-Z]{2}")) {
            throw new IllegalArgumentException("License plate invalid");
        }
        return plate;
    }

    public String getLicensePlate() {
        return licensePlate;
    }

    public void setLicensePlate(String licensePlate) {
        this.licensePlate = validatedPlate(licensePlate);
    }

    public int getPrice() {
        return price;
    }

    public void changePrice(int price) {
        if (price < 0) {
            throw new IllegalArgumentException("Negative price");
        }
        this.price = price;
    }
}
```

The setter does more than simply assign a value: it **enforces a rule** about which values are valid.

This is one of the main advantages of encapsulation. The class can guarantee that its state satisfies the rules defined by the class itself.

The constructor and later updates enforce the same plate rule through a private helper. Validation happens **before assignment**, so a rejected update leaves the old state intact. The format is a teaching policy, not a complete vehicle-registration validator. The price uses integer units with no currency or fractional-price model.

> **Note:** do not call overridable methods from constructors: subclass code can run before its fields are initialized. Private helpers avoid that dispatch. A real monetary model also needs a currency and a rounding policy.

Review: an assistant generates setters for every field. Which operations do clients actually need? Test valid creation, a negative price, a null plate, and a rejected plate update. Verify that the last operation preserves the previous plate.

Testing object state

Use the JUnit practice project from E2.8. Save the **validated, final version** of `Car` above as `src/main/java/com/example/Car.java`, adding `package com.example;` before the class. The earlier version with unrestricted setters is a separate example, not an additional class to copy.

Save this as `src/test/java/com/example/CarTest.java`:

```java
package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CarTest {
    @Test
    void constructorSetsObservableState() {
        Car car = new Car("AB123CD", 12_000, 90.0);

        assertEquals("AB123CD", car.getLicensePlate());
        assertEquals(12_000, car.getPrice());
    }

    @Test
    void priceCanReachZeroWithoutChangingThePlate() {
        Car car = new Car("AB123CD", 12_000, 90.0);

        car.changePrice(0);

        assertEquals(0, car.getPrice());
        assertEquals("AB123CD", car.getLicensePlate());
    }

    @Test
    void changingOneCarDoesNotChangeAnother() {
        Car first = new Car("AB123CD", 12_000, 90.0);
        Car second = new Car("EF456GH", 8_000, 75.0);

        first.setLicensePlate("XY789ZT");

        assertEquals("XY789ZT", first.getLicensePlate());
        assertEquals("EF456GH", second.getLicensePlate());
    }
}
```

Each test creates its own objects and observes behavior through the public API. Do not expose private fields just to test them. These tests check valid construction and state transitions; E5.2 adds rejection tests and checks that failed updates preserve the previous state.

Resources

- https://dev.java/learn/classes-objects/
- https://openjdk.org/jeps/513
- https://www.baeldung.com/java-information-hiding-vs-encapsulation
- https://www.baeldung.com/java-oop

E3.3 — Inheritance and Polymorphism

Several sections redefine `Person` or `Mammal` to isolate one concept. Compile those versions separately, not together. Add method fragments to their indicated class; imports and prerequisite types must be available.

Inheritance

Inheritance is a mechanism for deriving a new class from another class. It allows developers to build class hierarchies and reuse existing code. Inheritance represents an **IS-A** relationship.

A subclass can reuse accessible members of its superclass and can define additional fields and methods or override inherited methods.

A class **derived** from another class is called:

- **subclass**
- **derived class**
- **child class**

The class from which the subclass is derived is called:

- **superclass**
- **base class**
- **parent class**

The following example represents a simple inheritance hierarchy:

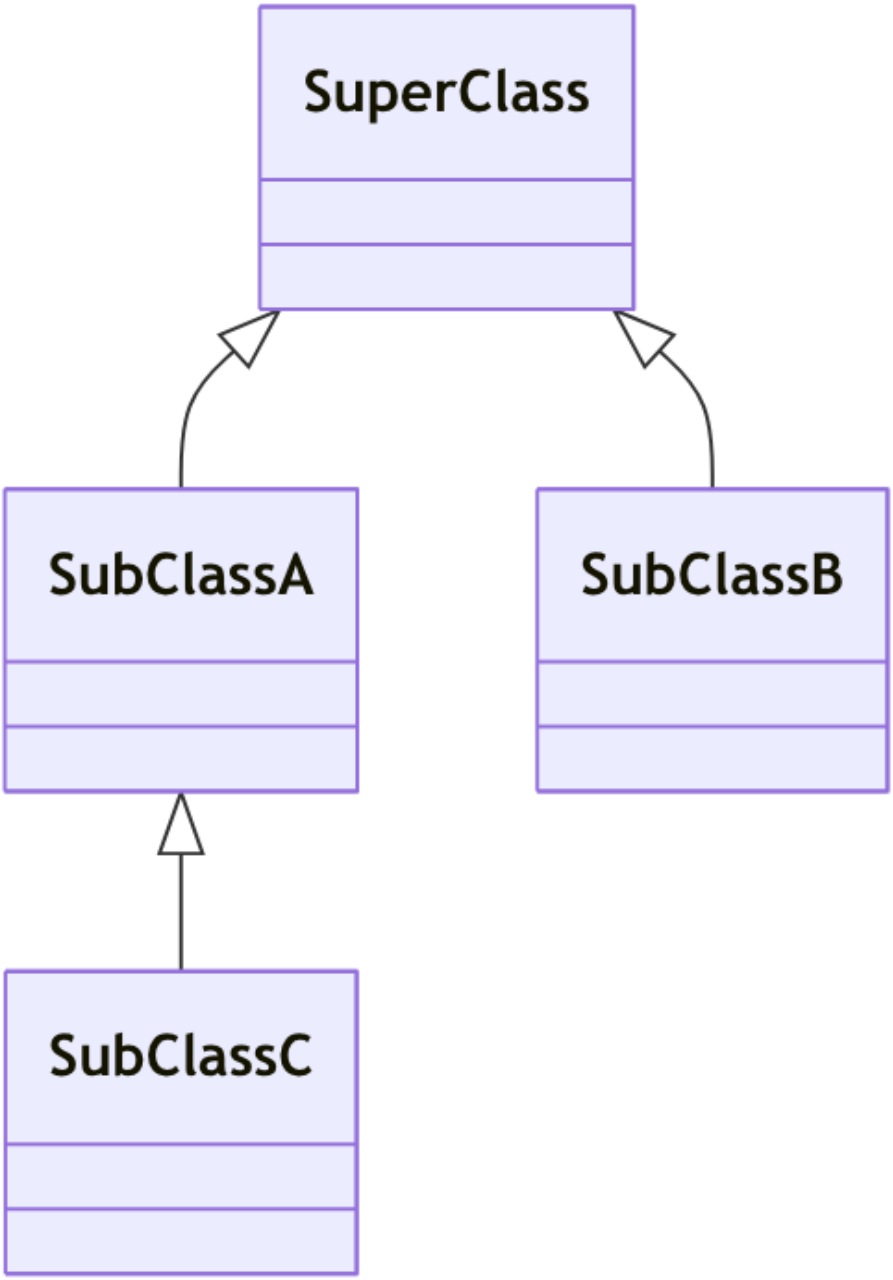
1. **SuperClass** is the base (or parent) class.
2. **SubClassA** and **SubClassB** both inherit from **SuperClass**, meaning they are direct subclasses of it.
3. **SubClassC** inherits from **SubClassA**, making it a subclass of **SubClassA** and an indirect subclass of **SuperClass**.

```
class SuperClass { }

class SubClassA extends SuperClass { }

class SubClassB extends SuperClass { }

class SubClassC extends SubClassA { }
```



There are important points about inheritance in Java:

- **Multi-Level Inheritance**: A class can extend another class that itself extends a third class (e.g., `C extends B`, where `B extends A`).
- **Single Inheritance for Classes**: Java does not support multiple inheritance for classes. A class can directly extend only one superclass.
- **Multiple Children**: A single superclass can have multiple subclasses.
- **Inheritance of Members**:

  - A subclass inherits accessible members of its superclass.
  - `public` and `protected` members are accessible from the subclass according to Java's access rules.
  - Package-private members are accessible only when the subclass is in the same package.
  - `private` members are not directly accessible from the subclass.
  - A subclass can define new fields and methods in addition to the inherited ones.

- **Constructor Inheritance**:

  - Constructors are **not inherited**.
  - A subclass can invoke a superclass constructor using the `super(...)` constructor invocation.
  - If a constructor has no explicit `this(...)` or `super(...)` invocation, an implicit `super()` occurs at the start of its body. Java 25 permits a restricted prologue before an explicit invocation: it can validate arguments and perform permitted early field assignments, but cannot use the object as fully initialized.
  - Therefore, the superclass must have an accessible no-argument constructor unless the subclass explicitly invokes another accessible superclass constructor.

- **Final Classes**: If a class is declared with the `final` keyword, it **cannot be extended**.

An example of a class hierarchy

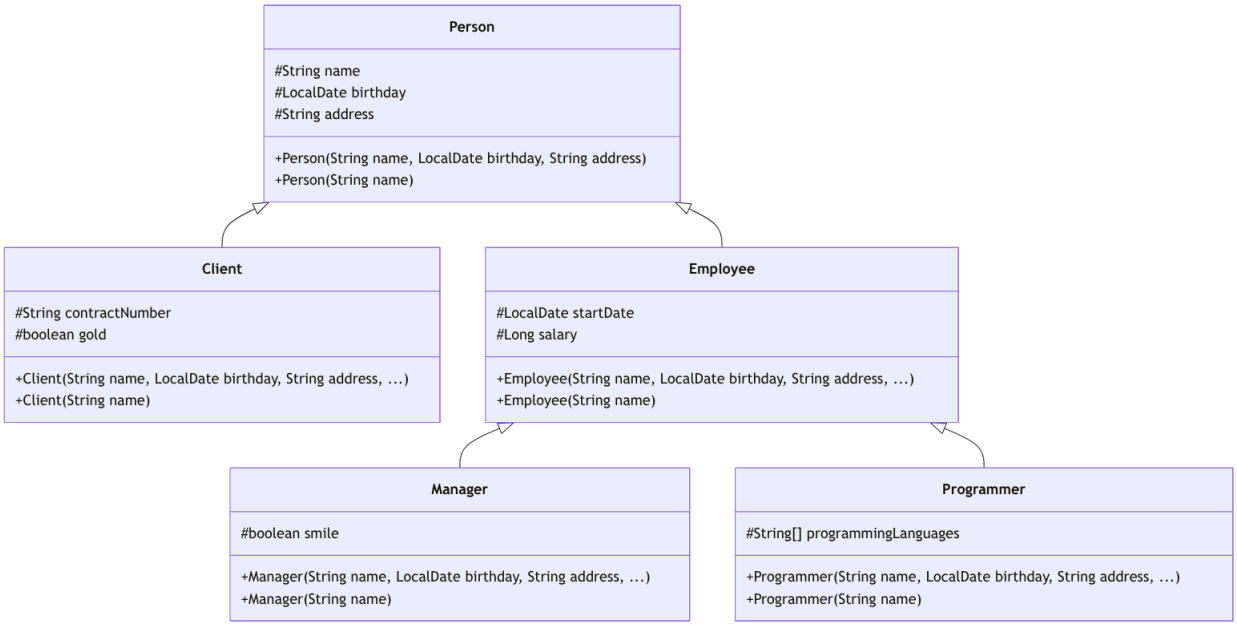
The example shows constructor chaining. Accessors are included where needed; a production API should not automatically expose a setter for every field.

- the base class `Person` has fields for storing common data: name, birthday, and address;
- the class `Client` has additional fields to store the contract number and status (gold or not);
- the class `Employee` stores the start date of work for the company and the salary;
- the class `Programmer` has an array of the programming languages a programmer uses;
- the class `Manager` has a boolean field representing whether the manager has a smile.

The fields are `protected`, which allows direct access from subclasses and from classes in the same package. For a subclass in a different package, `protected` access is more restricted: the inherited member can be accessed through the subclass relationship, not through an arbitrary superclass reference.

In UML (Unified Modeling Language):

- + → **public**
- - → **private**
- # → **protected**
- ~ → **package** or **default**




```java
import java.time.LocalDate;

public class Person {
  protected String name;
  protected LocalDate birthday;
  protected String address;

  public Person(String name, LocalDate birthday, String address) {
    this.name = name;
    this.birthday = birthday;
    this.address = address;
  }

  public Person(String name) {
    this(name, null, null);
  }

  public String getName() {
    return name;
  }
}
```

```java
import java.time.LocalDate;

public class Client extends Person {
  protected String contractNumber;
  protected boolean gold;

  public Client(
          String name,
          LocalDate birthday,
          String address,
```

```
          String contractNumber,
          boolean gold) {

    super(name, birthday, address);
    this.contractNumber = contractNumber;
    this.gold = gold;
  }

  public Client(String name) {
    super(name);
  }

  // getters and setters here
}
```

```
import java.time.LocalDate;

public class Employee extends Person {
  protected LocalDate startDate;
  protected Long salary;

  public Employee(
          String name,
          LocalDate birthday,
          String address,
          LocalDate startDate,
          Long salary) {

    super(name, birthday, address);
    this.startDate = startDate;
    this.salary = salary;
  }

  public Employee(String name) {
    super(name);
  }

  public Long getSalary() {
    return salary;
  }
}
```

```
import java.time.LocalDate;

public class Manager extends Employee {
  protected boolean smile;

  public Manager(
          String name,
          LocalDate birthday,
          String address,
          LocalDate startDate,
```

```
          Long salary,
          boolean smile) {

    super(name, birthday, address, startDate, salary);
    this.smile = smile;
  }

  public Manager(String name) {
    super(name);
  }

  // getters and setters here
}
```

```
import java.time.LocalDate;

public class Programmer extends Employee {
  protected String[] programmingLanguages;

  public Programmer(
          String name,
          LocalDate birthday,
          String address,
          LocalDate startDate,
          Long salary,
          String[] programmingLanguages) {

    super(name, birthday, address, startDate, salary);
    this.programmingLanguages = programmingLanguages;
  }

  public Programmer(String name) {
    super(name);
  }

  // getters and setters here
}
```

Let's create an object of the `Programmer` class and get its name using a getter method inherited from the `Person` class.

> **Note:** the hierarchy deliberately demonstrates `protected`; private state plus controlled methods is usually a stronger boundary. Name-only constructors leave some fields null, including salary. This example neither computes with missing salaries nor models overlapping roles: a real person may be both a client and an employee, which may favor composition over this hierarchy.

```
import java.time.LocalDate;

public class CompanyApp {
  public static void main(String[] args) {
```

```
    Programmer programmer1 = new Programmer(
            "John Elephant",
            LocalDate.of(2000, 6, 15),
            "Some street 15",
            LocalDate.of(2023, 6, 15),
            500_000L,
            new String[] { "Java", "Scala", "Kotlin" });

    // or...
    Programmer programmer2 = new Programmer("Henry McCarty");

    IO.println(programmer1.getName()); // John Elephant
    IO.println(programmer2.getName()); // Henry McCarty
  }
}
```

Choosing references

The basic rule is:

> If class A is a superclass of class B, then a reference of type A can refer to an object whose type is B or any subclass of B.

For example, because `Programmer` extends `Employee` and `Employee` extends `Person`, a `Person` reference can refer to a `Programmer` object.

Same class reference

```
Person person = new Person("John Doe");
// reference type: Person
// object type:    Person

Employee employee = new Employee("John Doe");
// reference type: Employee
// object type:    Employee

Programmer programmer = new Programmer("John Doe");
// reference type: Programmer
// object type:    Programmer
```

Superclass reference

```
Person person = new Person("John Doe");
// reference type: Person
// object type:    Person

Person employee = new Employee("John Doe");
// reference type: Person
// object type:    Employee
```

```
Person programmer = new Programmer("John Doe");
// reference type: Person
// object type:    Programmer
```

Wrong reference

```
Client whoIsIt = new Employee("John Doe"); // compile-time error
Client client = new Person("John Doe");    // compile-time error
```

The reason is that an `Employee` is not necessarily a `Client`, and a `Person` is not necessarily a `Client`.

Upcast

Upcasting means assigning a subclass object to a superclass reference.

It is safe and performed **implicitly** by Java.

Upcasting is particularly useful when working with collections or arrays containing objects of different types belonging to the same hierarchy.

```
public static void showPersons(Person[] persons) {
  for (Person person : persons) {
    IO.println(person.getName());
  }
}

public static void main(String[] args) {
  Person person = new Employee("Ginger R. Lee");
  Client client = new Client("Pauline E. Morgan");
  Employee employee = new Employee("Lawrence V. Jones");

  Person[] persons = { person, client, employee };
  // automatic upcasting of the elements

  showPersons(persons);
}
```

We can use a superclass reference for any subclass object derived from it.

However, **the reference type determines which members can be accessed at compile time**.

```
public static void showPersons(Person[] persons) {
  for (Person person : persons) {
    IO.println(person.getName()); // OK

    // IO.println(person.getSalary());
    // Does not compile: Person does not declare getSalary()
  }
}
```

The object may actually be an `Employee`, but the compiler only knows that `person` has type `Person`.

This distinction between the **reference type** and the **actual object type** is fundamental to understanding polymorphism.

Downcast

Downcasting means converting a superclass reference to a subclass reference.

It requires an explicit cast and can throw a `ClassCastException` at runtime if the object is not actually an instance of the target subclass.

```
public static void showPersons(Person[] persons) {
  for (Person person : persons) {
    IO.println(person.getName());

    Employee employee = (Employee) person;
    IO.println(employee.getSalary());
  }
}
```

The cast is valid only when `person` actually refers to an `Employee` object or to an object of a subclass of `Employee`.

For example, the following array would cause a `ClassCastException` when `client` is processed:

```
Person[] persons = {
        new Employee("Alice"),
        new Client("Bob")
};
```

A safer approach is to use the `instanceof` operator.

```
public static void showPersons(Person[] persons) {
  for (Person person : persons) {
    IO.println(person.getName());

    if (person instanceof Employee) {
      Employee employee = (Employee) person;
      IO.println(employee.getSalary());
    }
  }
}
```

The `instanceof` operator checks at runtime whether an object is compatible with a given type.

Modern Java also provides **pattern matching for `instanceof`**, which can combine the type check and cast:

```
if (person instanceof Employee employee) {
    IO.println(employee.getSalary());
}
```

After successfully casting a superclass reference to a subclass type, we can access subclass-specific members.

Predict: does casting a `Person` reference to `Employee` transform the object? What happens when the object is a `Client`, or when the reference is null?

Check your prediction

A cast changes the usable reference type, not the object. An incompatible non-null object causes `ClassCastException`; casting null yields null, while `null instanceof Employee` is false.

Overriding methods

Java allows a subclass to provide a new implementation of an **instance method** inherited from its superclass. This is known as **method overriding**.

Method overriding allows a subclass to provide behavior that is more specific to that subclass.

For an instance method to override a superclass method, it must have:

- the same method name;
- the same parameter types;
- a compatible return type (the same type or a **covariant return type**);
- an access level that is not more restrictive than the overridden method.

The overriding method must also satisfy Java's other rules for overriding, including restrictions involving `final`, `static`, and private methods.

```
class Mammal {

  public String sayHello() {
    return "random stuff";
  }
}
```

```
class Cat extends Mammal {

  @Override
  public String sayHello() {
    return "meow";
  }
}
```

```
class Human extends Mammal {

  @Override
  public String sayHello() {
    return "hello";
  }
}
```

The hierarchy includes three classes: `Mammal`, `Cat`, and `Human`.

The class `Mammal` declares the method `sayHello()`. Each subclass overrides this method.

The `@Override` annotation indicates that a method is intended to override a superclass method. The annotation is optional, but strongly recommended because the compiler can detect accidental mistakes.

Let's create instances and invoke the method:

```
public static void main(String[] args) {
  Mammal mammal = new Mammal();
  IO.println(mammal.sayHello()); // random stuff

  Cat cat = new Cat();
  IO.println(cat.sayHello()); // meow

  Human human = new Human();
  IO.println(human.sayHello()); // hello
}
```

Overriding static methods?

Static methods cannot be overridden.

If a subclass declares a static method with the same signature as a static method in the superclass, the subclass method **hides** the superclass method.

This is different from method overriding because static method calls are resolved according to the reference or class type, not according to the runtime type of the object.

```
class SuperClass {
  public static void staticMethod() {
    IO.println("super");
  }
}
```

```
class SubClass extends SuperClass {
  public static void staticMethod() {
    IO.println("sub");
  }
}
```

```java
class Main {
  public static void main(String[] args) {
    SuperClass.staticMethod();          // super
    new SuperClass().staticMethod();    // super

    SubClass.staticMethod();            // sub
    new SubClass().staticMethod();      // sub

    SuperClass ss = new SubClass();
    ss.staticMethod();                  // super
  }
}
```

The last call prints `super` because the static method is selected based on the **reference type** (`SuperClass`), not the runtime type (`SubClass`).

> **NOTE:** Although Java allows static methods to be called through an object reference, this is discouraged because it can make the code misleading. Prefer `ClassName.staticMethod()`.

Polymorphism

Polymorphism is the ability to use a common type to work with objects of different concrete types.

In Java, polymorphism is particularly visible when a superclass or interface reference refers to an object of a subclass and an overridden instance method is invoked.

Polymorphism allows the same method call to produce different behavior depending on the object's actual runtime type.

Two commonly discussed forms are:

- **compile-time polymorphism**, usually associated with method overloading;
- **run-time polymorphism**, implemented through method overriding and dynamic method dispatch.

Static polymorphism

Reference — distinguish overloading from overriding. Overload selection happens before runtime dispatch; it is not a substitute for dynamic polymorphism.

Compile-time polymorphism is commonly associated with **method overloading**.

Method overloading occurs when a class declares multiple methods with the same name but different parameter lists.

```java
class Mammal {

  public String sayHello() {
    return "hi, i'm a mammal";
  }

  public String sayHello(int count) {
    return "hi, i'm a mammal".repeat(Math.max(0, count));
  }

  public String sayHello(String name) {
    return "hi " + name + ", i'm a mammal";
  }

  public String sayHello(boolean reverse) {
    return reverse
            ? "lammam a m'i ,ih"
            : "hi, i'm a mammal";
  }
}
```

The compiler determines which overloaded method to call based on the **compile-time types and number of the arguments**.

Dynamic polymorphism

Run-time polymorphism allows an overridden instance method to be selected according to the **runtime type of the object**.

```java
class Mammal {

  public String sayHello() {
    return "hi, i'm a mammal";
  }
}
```

```java
class Cat extends Mammal {

  @Override
  public String sayHello() {
    return "meow";
  }
}
```

```java
class Human extends Mammal {

  @Override
  public String sayHello() {
    return "hello";
```

```
  }
}
```

```
public static void main(String[] args) {
  Mammal[] mammals = {
          new Mammal(),
          new Cat(),
          new Human()
  };

  for (Mammal mammal : mammals) {
    IO.println(mammal.sayHello());
  }
}
```

Will print:

```
hi, i'm a mammal
meow
hello
```

The important point is that all three objects are accessed through a `Mammal` reference, but the overridden `sayHello()` implementation is selected according to the actual object.

Consequences of Dynamic Polymorphism

Flexibility and extensibility: Dynamic polymorphism allows code to depend on a common abstraction instead of explicitly checking every concrete type.

This is closely related to the **Tell, Don't Ask** principle, which suggests that objects should be **told what to do rather than asked for their internal state and then manipulated externally**.

Instead of checking an object's type and deciding externally what action to perform, the program can invoke a common method and let the actual object provide the appropriate behavior.

For example, consider the following design:

```
public class Mammal {
  String type;

  public Mammal(String type) {
    this.type = type;
  }
}
```

The following approach centralizes type-specific behavior in a `switch`. For an open family of types with their own behavior, that can create repeated edits in client code.

Every time a new type is introduced, the `switch` statement must be modified.

> **Note:** a switch is not inherently poor design. Closed domains, enums, and sealed hierarchies with pattern matching can make centralized operations appropriate. Choose based on which kinds of change are expected.

```
public static void main(String[] args) {
  Mammal mammal = new Mammal("mammal");
  Mammal cat = new Mammal("cat");
  Mammal human = new Mammal("human");

  Mammal[] mammals = { mammal, cat, human };

  for (Mammal m : mammals) {
    switch (m.type) {
      case "dog" -> IO.println("woof");
      case "cat" -> IO.println("meow");
      case "human" -> IO.println("hello!");
    }
  }
}
```

With polymorphism, the behavior can instead be associated with the objects themselves:

```
class Mammal {
  public void sayHello() {
    IO.println("hi, i'm a mammal");
  }
}

class Cat extends Mammal {
  @Override
  public void sayHello() {
    IO.println("meow");
  }
}

class Human extends Mammal {
  @Override
  public void sayHello() {
    IO.println("hello!");
  }
}
```

Now the calling code does not need to know the concrete type:

```
public static void makeItSpeak(Mammal mammal) {
  mammal.sayHello();
}
```

Abstraction and decoupling: Polymorphism encourages developers to write code that depends on **abstract types**, such as superclasses or interfaces, rather than concrete implementations.

For instance, a `List<Mammal>` can store objects of different subclasses such as `Cat`, `Human`, or `Dog`, while the program interacts with them through the common `Mammal` type.

This can reduce coupling between components and make systems easier to extend and maintain.

```
/**
 * This method does not depend on any specific mammal.
 *
 * @param mammal the mammal that should speak
 */
public void makeItSpeak(Mammal mammal) {
  mammal.sayHello();
}
```

The method does not need to know whether `mammal` is a `Cat`, `Human`, `Dog`, or another subclass. It simply relies on the behavior guaranteed by the `Mammal` type.

Polymorphism promotes extensibility by allowing new types to provide new behavior without modifying the code that uses the common type.

Resources

- https://dev.java/learn/inheritance/
- https://www.programiz.com/java-programming/inheritance
- https://www.programiz.com/java-programming/method-overriding
- https://www.programiz.com/java-programming/super-keyword

E3.4 — Interfaces

The drawing examples evolve through alternative implementations. Compile each version separately. Use `import java.awt.Color;` for color and this small prerequisite type:

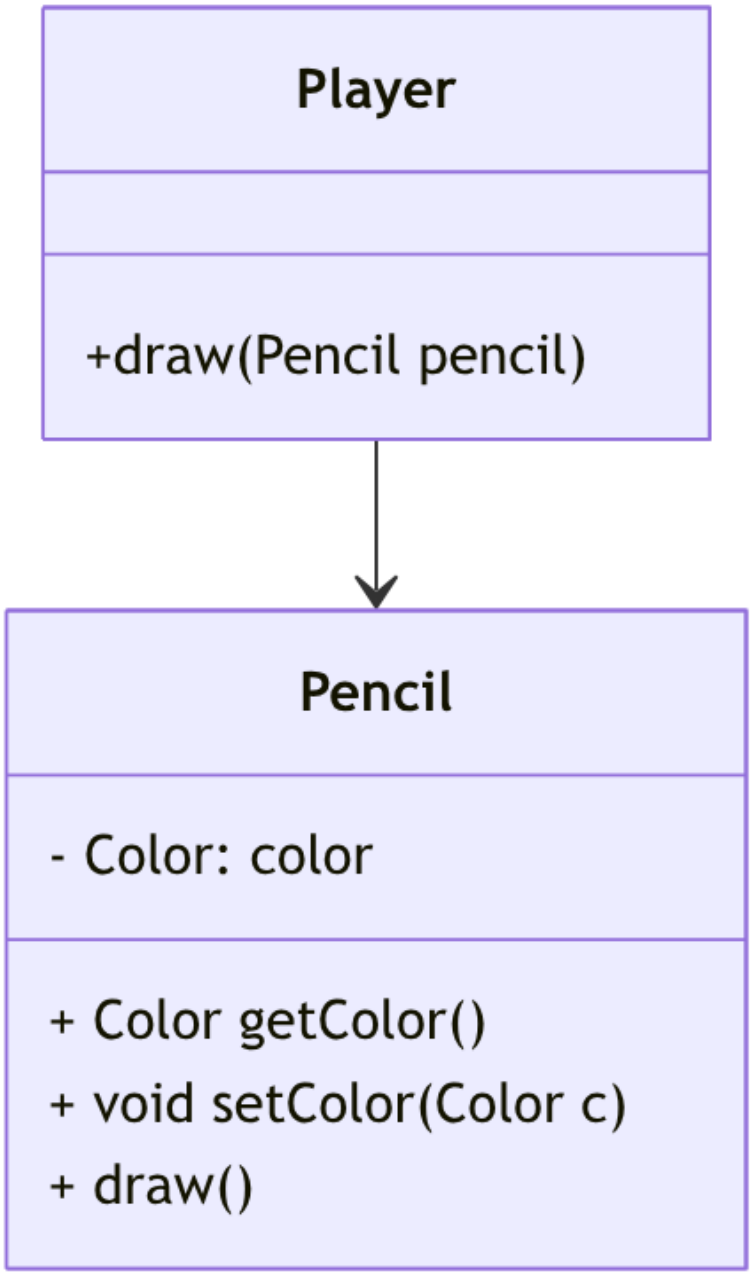
```
record Curve(String description) {
    public String draw() {
        return description;
    }
}
```

The examples describe drawing using strings; they do not implement graphical rendering.

Motivation

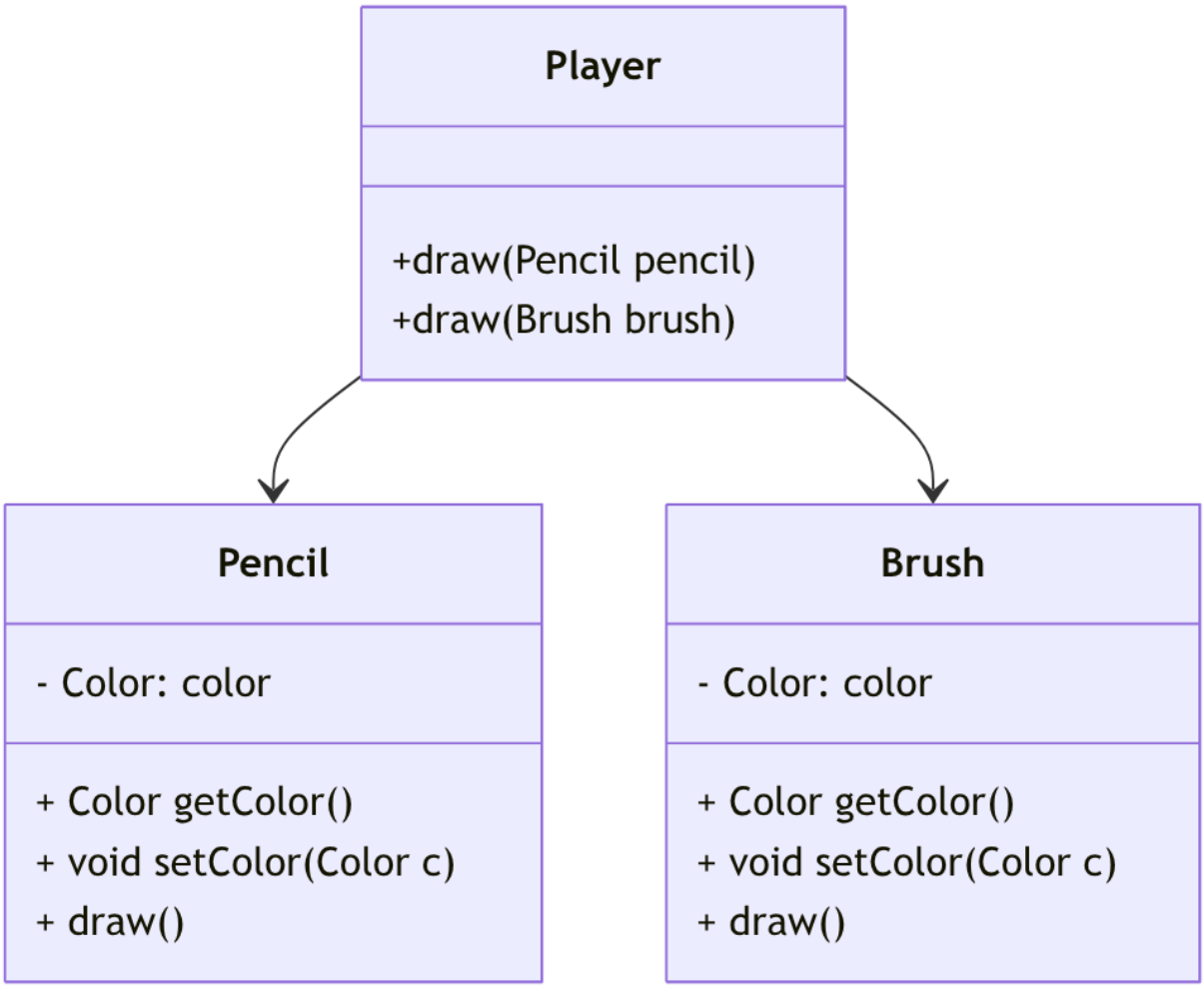
We aim to develop a drawing game. As an initial proof of concept, we will focus exclusively on pencil-based drawing, allowing users to create sketches using a variety of colors.

- The **Pencil** class provide methods to draw and change its color.
- The **Player** class can use a Pencil to perform drawing operations.



We implement everything, and it works like a charm. However, we get our next task. On top of drawing with a Pencil, we must support drawing with a Brush, as well.

No problem, we introduce the Brush class and the Player.draw(Brush) method:

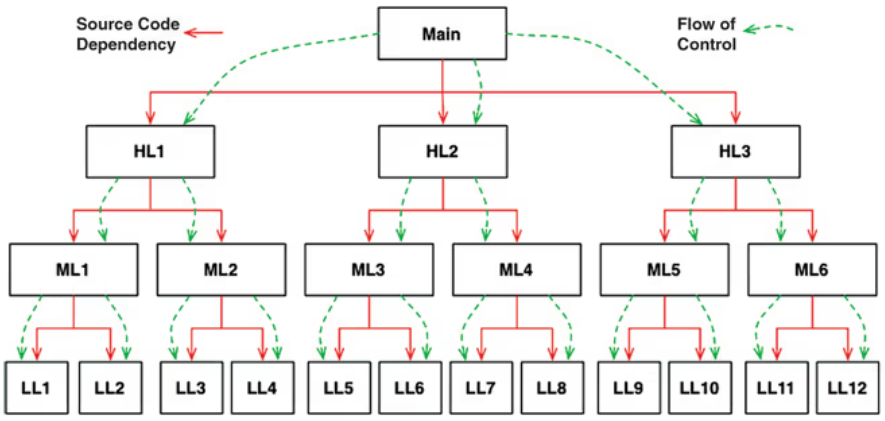


So far, so good. But then we face the next task: adding more drawing tools.

We could repeat the same process we did with the Pencil and Brush. However, we start wondering: how many other tools might we need to support?

Limitations of This Approach:

- **No Common Abstraction**: `Pencil` and `Brush` have no shared abstraction, meaning **we cannot treat all tools generically**.
- **Tight Coupling**: Supporting a new unrelated tool requires another overload or another client edit. A subclass of an already supported tool does not necessarily require a new overload, but the API still names concrete tool families.
- **Concrete dependencies**: the player knows each supported family instead of only the operation it needs. This is about dependency structure, not the order in which instructions execute.

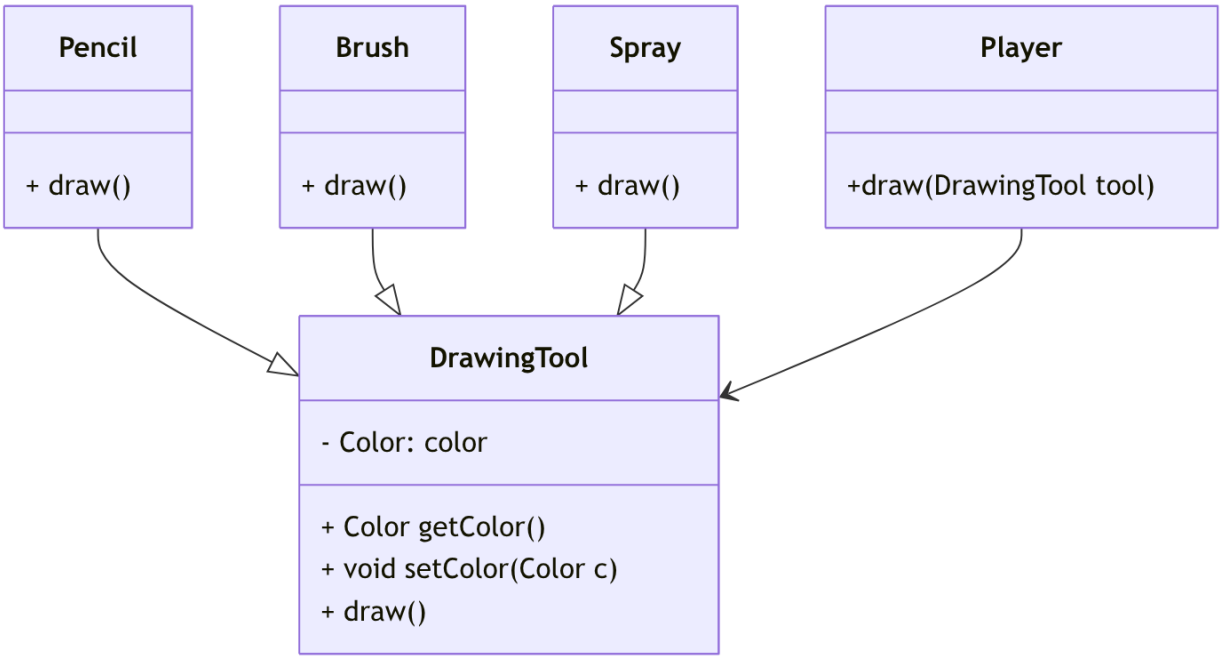


Source code dependencies versus flow of control

Using abstraction

We decide that we'll create an abstract DrawingTool class, which will be the superclass of the Pencil, Brush and all future classes.

Also, this way, we'll need a single draw(DrawingTool) method in the Player class:



By using **abstractions**, OOP separates the *what* (abstract behavior) from the *how* (concrete implementations).

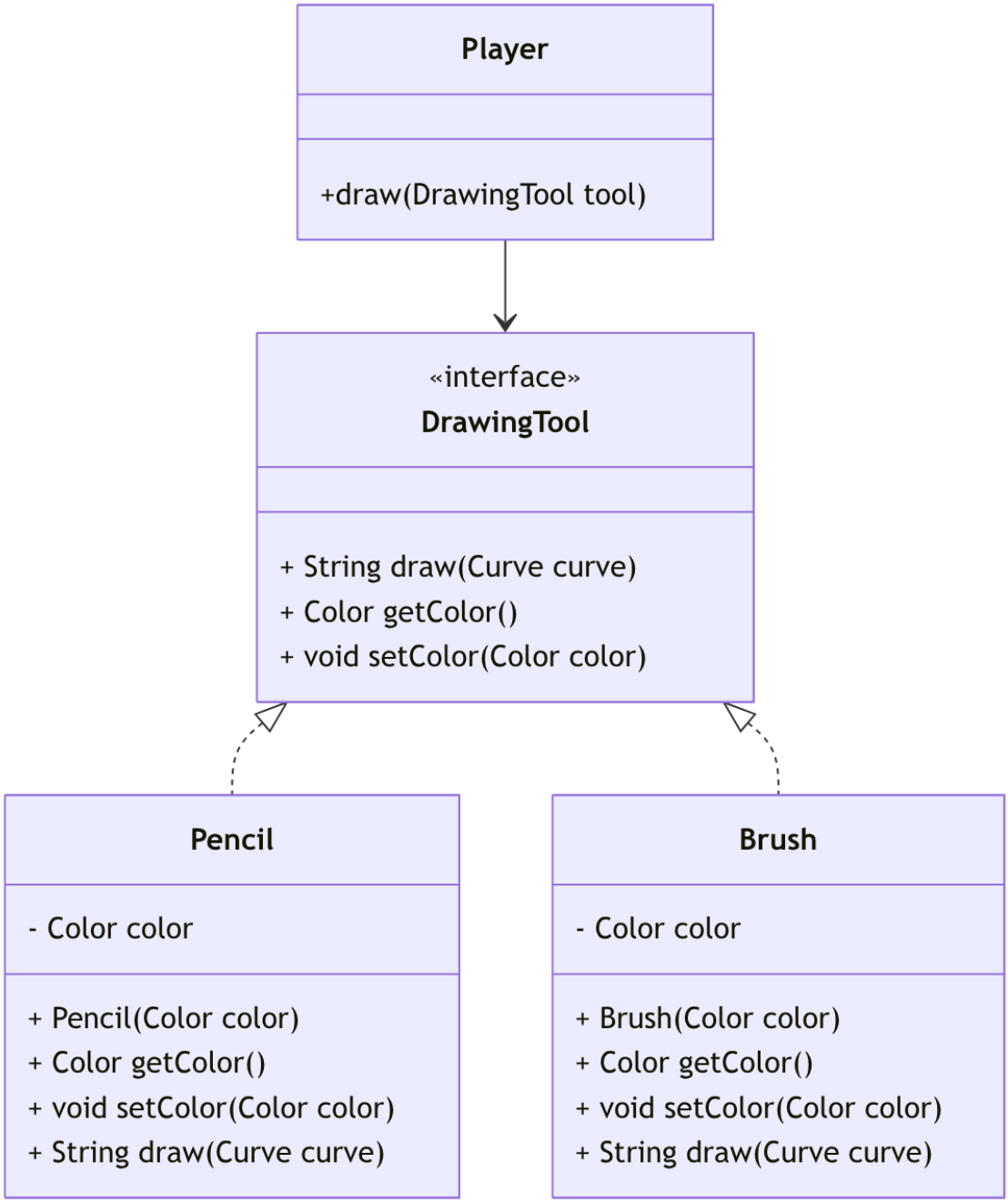
- We can express the **capabilities** of all tools without tying the application to a concrete implementation.

- New tools can be added without modifying existing code, improving **extensibility** and **maintainability**.
- The player depends on a stable contract. Runtime calls still have an order; abstraction changes what the client must know, not the need to execute operations.

Interfaces

An interface is a **reference type distinct from a class**. It defines a contract that multiple classes can implement without sharing an implementation superclass.

- Interfaces cannot have constructors, as they cannot be instantiated.
- A method declared without a body, `private`, `static`, or `default` is implicitly `public abstract`.
- Interfaces can also declare default, static, and private methods with bodies, as shown below. Fields are implicitly `public static final`; there is no per-instance interface state.
- Other classes can **implement** an interface by providing concrete implementations for its abstract methods.




```
interface DrawingTool {
    String draw(Curve curve);
    Color getColor();
    void setColor(Color color);
}
```

```
class Pencil implements DrawingTool {
    Color color;
```

```
    public Pencil(Color color) {
        this.color = color;
    }

    public Color getColor() {
        return color;
    }

    public void setColor(Color color) {
        this.color = color;
    }

    public String draw(Curve curve) {
        return "Pencil drawing a " + curve.draw();
    }
}
```

```
class Brush implements DrawingTool {
   Color color;

   public Brush(Color color) {
      this.color = color;
   }

   public Color getColor() {
      return color;
   }

   public void setColor(Color color) {
      this.color = color;
   }

    public String draw(Curve curve) {
        return "Brush drawing a " + curve.draw();
    }
}
```

Key rules

Reference — type relationships.

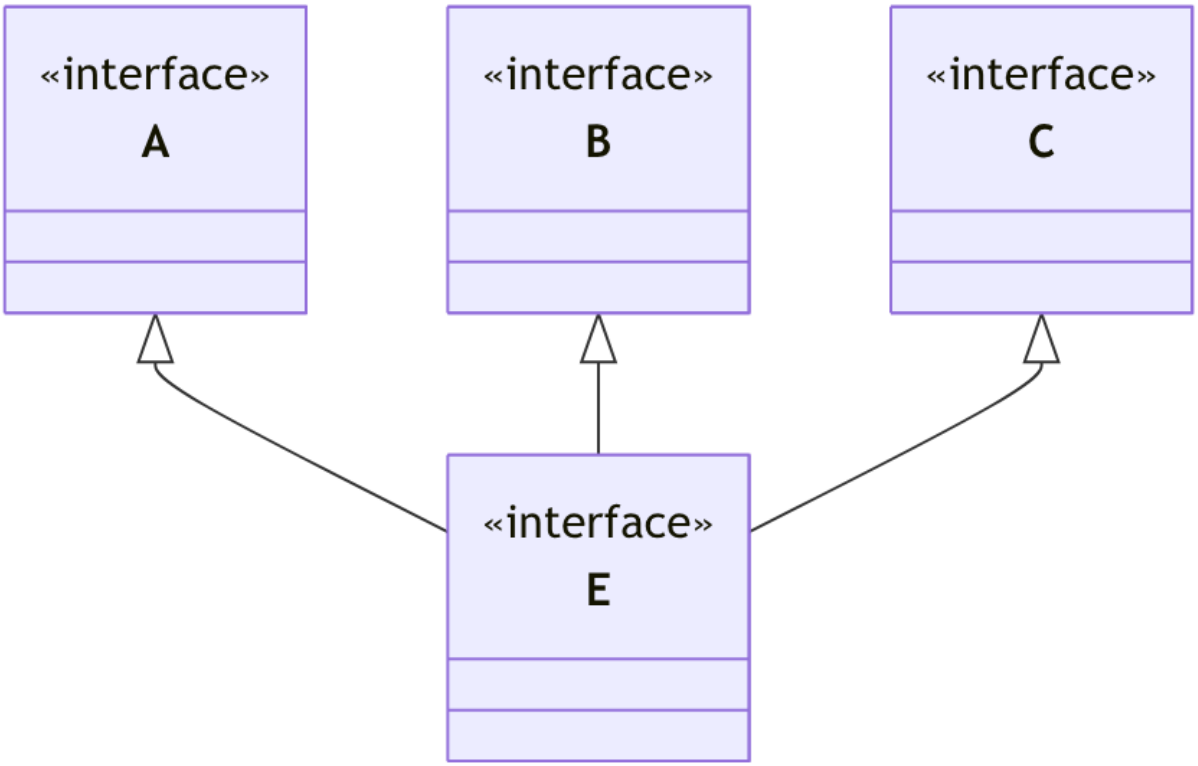
An interface can extend one or more other interfaces using the keyword `extends`:

```
interface A {
    void walk();
}

interface B {
    void run();
}
```

```
interface C {
    void swim();
}

interface E extends A, B, C {
    void talk();
}
```

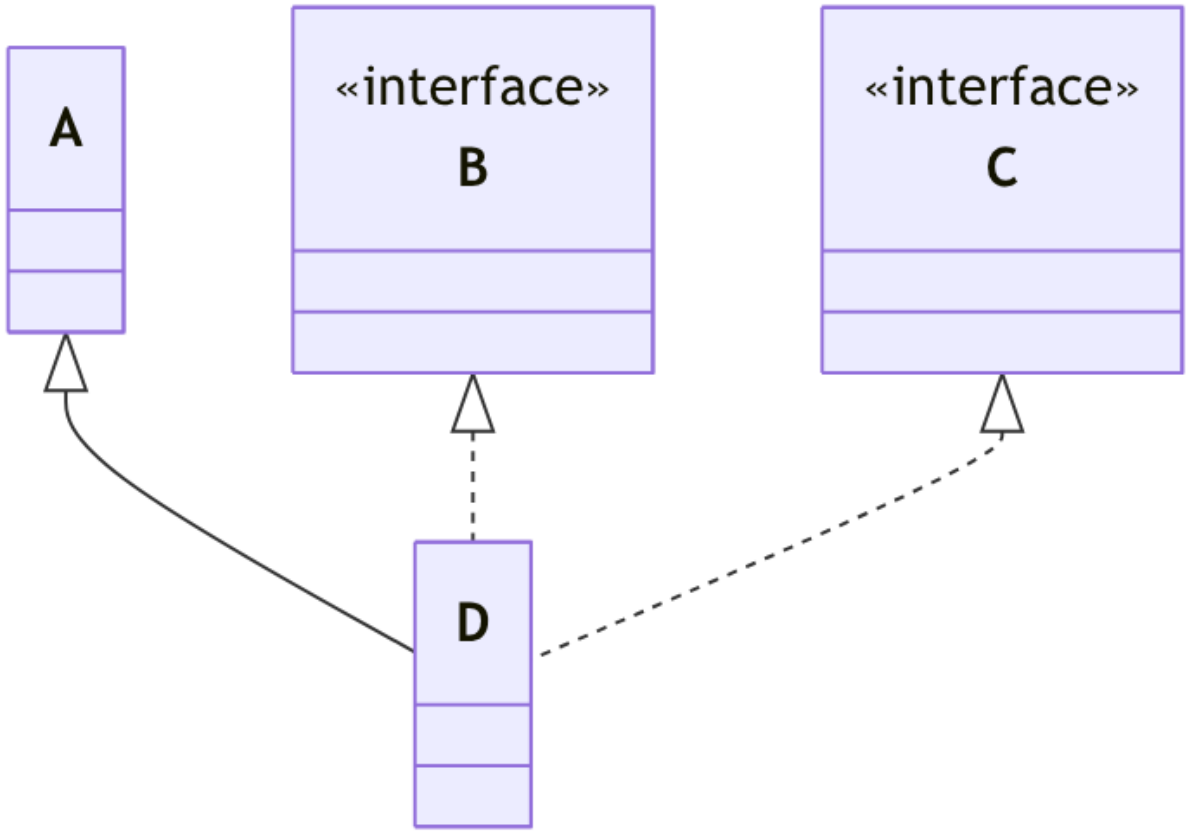


A class can extend only one class and implement multiple interfaces:

```
class A { }

interface B { }
interface C { }

class D extends A implements B, C { }
```



Multiple interfaces are often used in the standard library. A simplified excerpt of `String` shows some of its interfaces (not its complete Java 25 declaration):

```
public final class String implements Serializable, Comparable<String>, CharSequence {
    // ...
}
```

Methods with a body

Static methods

Interface fields are constants. Static methods provide utility behavior and are called on the declaring interface; they are not inherited as instance methods.

```
interface DrawingTool {
    static final Color DEFAULT_COLOR = Color.BLACK;

    static boolean hasSameColor(DrawingTool a, DrawingTool b) {
        return a.getColor().equals(b.getColor());
    }
    // ...
}
```

To use a static method you just need to invoke it directly from an interface

```
DrawingTool.hasSameColor(pencil, brush);
```

The main purpose of interface static methods is to define utility functionality that is common for all classes implementing the interface. They help to avoid code duplication.

Default methods

A `default` method supplies an inheritable instance implementation in an interface. Static and private interface methods can also have bodies, but serve different roles.

Suppose many classes already implemented the `DrawingTool` interface. For example `pencil`, `pen`, `brush`, `fuzzybrush` etc. They all draw in different ways when `draw()` is called.

```
interface DrawingTool {
    String draw(Curve curve);
    Color getColor();
    void setColor(Color color);
}
```

Then you decide that all tools should be always able to draw with a default color. That means:

- you need to add the `drawWithDefaultColor` method to the `DrawingTool` interface **and modify all implementing classes**
- **if implementing classes are a lot this could become very time-consuming!**.

The key motivation behind default methods in interfaces is supporting backward compatibility. `default` methods provide a quicker solution in that you don't have to implement them in all classes.

```
interface DrawingTool {
    static final Color DEFAULT_COLOR = Color.BLACK;

    String draw(Curve curve);
    Color getColor();
    void setColor(Color color);

    default String drawWithDefaultColor(Curve curve) {
        Color original = getColor();
        setColor(DEFAULT_COLOR);
        try {
            return draw(curve);
        } finally {
            setColor(original);
        }
    }
}
```

Although default methods are implemented, you cannot invoke them directly from an interface like `DrawingTool.drawWithDefaultColor()`. You still need to have an object of a class that implements the interface:

```
DrawingTool pencil = new Pencil(Color.RED);
pencil.drawWithDefaultColor(new Curve("line"));
```

The `finally` block restores color even if drawing fails. This temporarily mutates the tool and is **not a thread-safety guarantee**. A future API might accept color as an argument instead. Adding default methods can also create conflicts between interfaces: they are not an unconditional compatibility guarantee.

Review: an assistant omits `return` and restores the color only after a normal draw. Identify both the compilation problem and the failure-path bug. A `finally` block restores the color even if drawing throws an exception; restoration only after the call would be skipped on failure.

Private methods

Sometimes default methods are huge. To make it possible to decompose such methods, **Java allows declaring private methods inside an interface**:

```java
interface Feature {
    default void action() {
        String answer = subAction();
        IO.println(answer);
    }

    private String subAction() {
        return "Default action";
    }
}
```

Resources

E3.5 — Abstract Classes and Composition

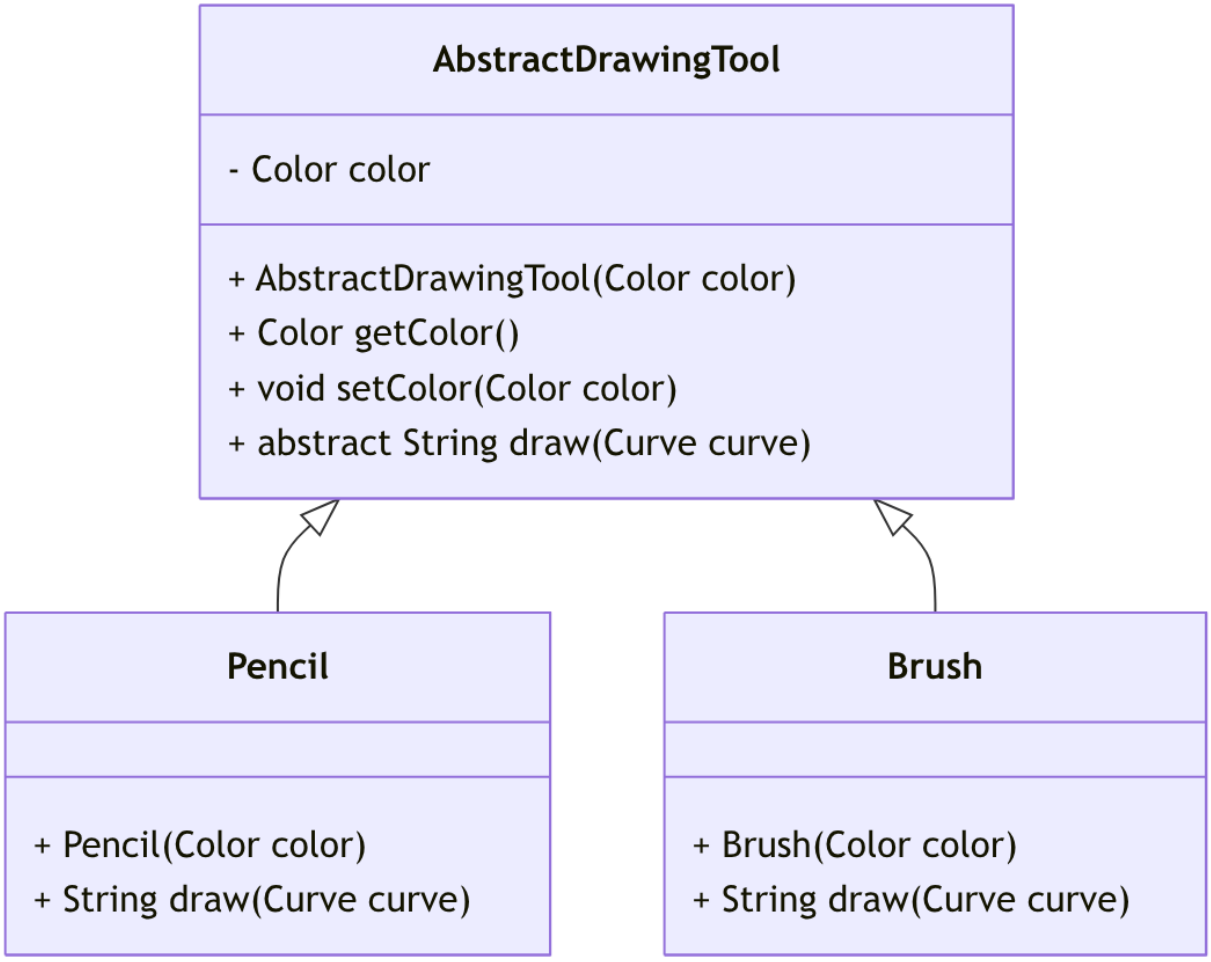
The drawing examples use `import java.awt.Color;` and the following supporting type. Compile alternative drawing-tool definitions separately.

```
record Curve(String description) {
    public String draw() {
        return description;
    }
}
```

Abstract classes

An **abstract class**, declared using the `abstract` keyword, can share state and implementation across related classes while leaving some operations to subclasses. An interface does not inherently require duplicated code: default methods or a collaborator can also provide reuse. Choose a superclass only when the inheritance relationship and its contract are appropriate, not merely because two methods look similar.

- It can define **both abstract methods** (without implementation) and **concrete methods** (with implementation).
- It can include **constructors**, which are typically used by its subclasses.
- You **cannot create instances** of an abstract class directly — it may contain methods that are not fully implemented.
- An abstract class can **extend another class**, including another abstract class.




```java
public abstract class AbstractDrawingTool {

    protected Color color;

    protected AbstractDrawingTool(Color color) {
        this.color = color;
    }

    public Color getColor() {
        return color;
    }

    public void setColor(Color color) {
        this.color = color;
    }

    public abstract String draw(Curve curve); // an abstract method
}
```

Since `AbstractDrawingTool` is an abstract class we cannot create instances of this class:

```
AbstractDrawingTool pencil = new AbstractDrawingTool(Color.RED); // this throws a
↪  compile time error
```

The method `draw()` is declared abstract because, at this level of abstraction, its implementation is unknown. Concrete subclasses of the class `AbstractDrawingTool` should have an implementation of this method.

```
class Pencil extends AbstractDrawingTool {

    // It can have additional fields as well

    public Pencil(Color color) {
        super(color);
    }

    @Override
    public String draw(Curve curve) {
        return "Pencil drawing a " + curve.draw();
    }
}
```

```
class Brush extends AbstractDrawingTool {

    // It can have additional fields as well

    public Brush(Color color) {
        super(color);
    }

    @Override
    public String draw(Curve curve) {
        return "Brush drawing a " + curve.draw();
    }
}
```

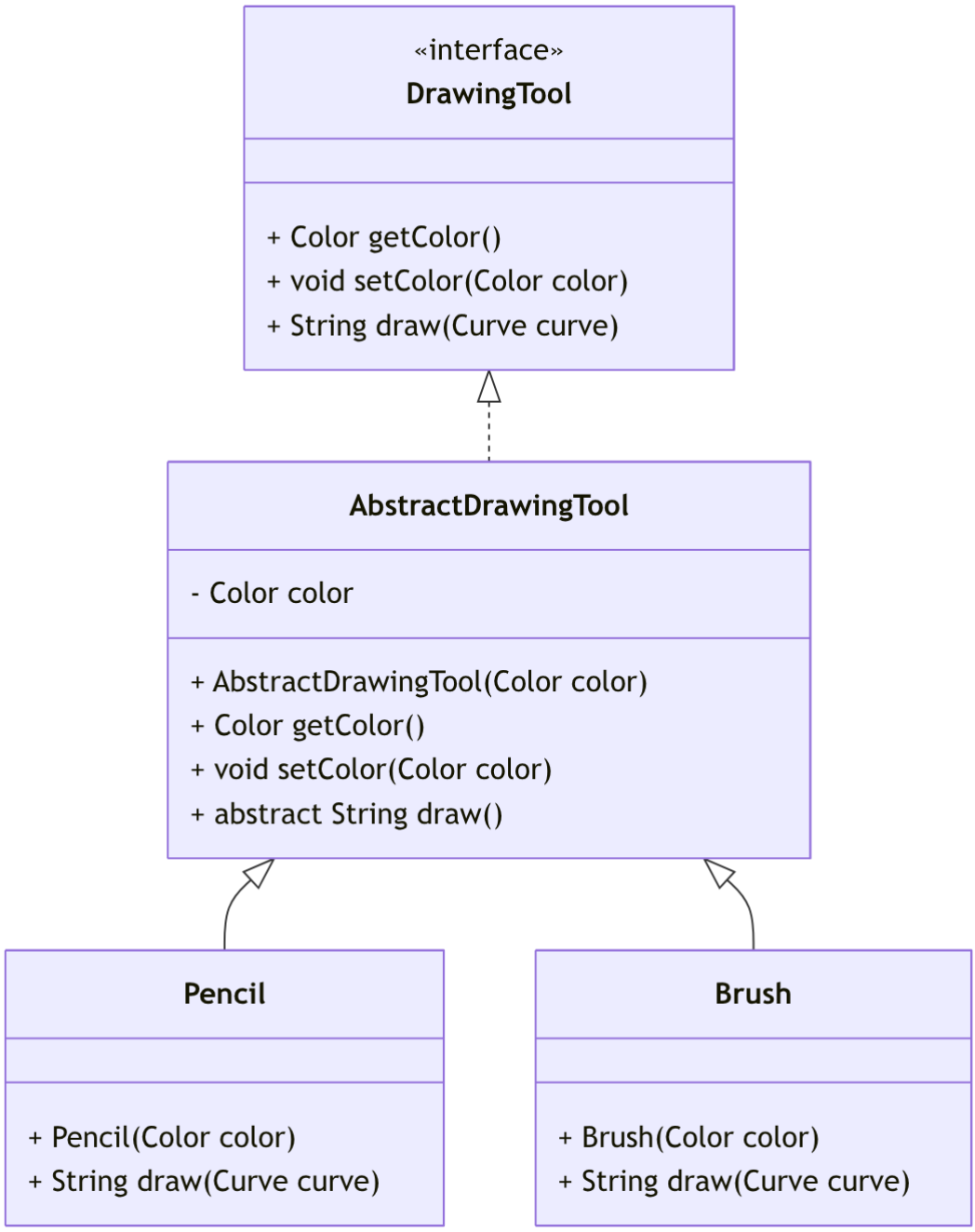
In practice

Example 1

Interfaces and abstract classes can be combined to create a flexible and reusable class hierarchy.

- **Interface**: Defines the core behavior that all drawing tools must provide (e.g., a `draw` method).
- **Abstract Class**: Implements common state and behavior (like managing the tool's color) and provides a foundation for concrete drawing tools.

- **Concrete Classes**: Specialize behavior for each type of tool (e.g., `Pencil` or `Brush`) and provide specific implementations of the `draw` method.




```java
interface DrawingTool {
    String draw(Curve curve);
    Color getColor();
    void setColor(Color color);
}
```

```java
abstract class AbstractDrawingTool implements DrawingTool {
    protected Color color;

    protected AbstractDrawingTool(Color color) {
        this.color = color;
    }

    public Color getColor() {
        return color;
    }

    public void setColor(Color color) {
        this.color = color;
    }

    public abstract String draw(Curve curve); // an abstract method
}
```

```java
class Pencil extends AbstractDrawingTool {

    // It can have additional fields as well

    public Pencil(Color color) {
        super(color);
    }

    @Override
    public String draw(Curve curve) {
        return "Pencil drawing a " + curve.draw();
    }
}
```

```java
class Brush extends AbstractDrawingTool {

    // It can have additional fields as well

    public Brush(Color color) {
        super(color);
    }

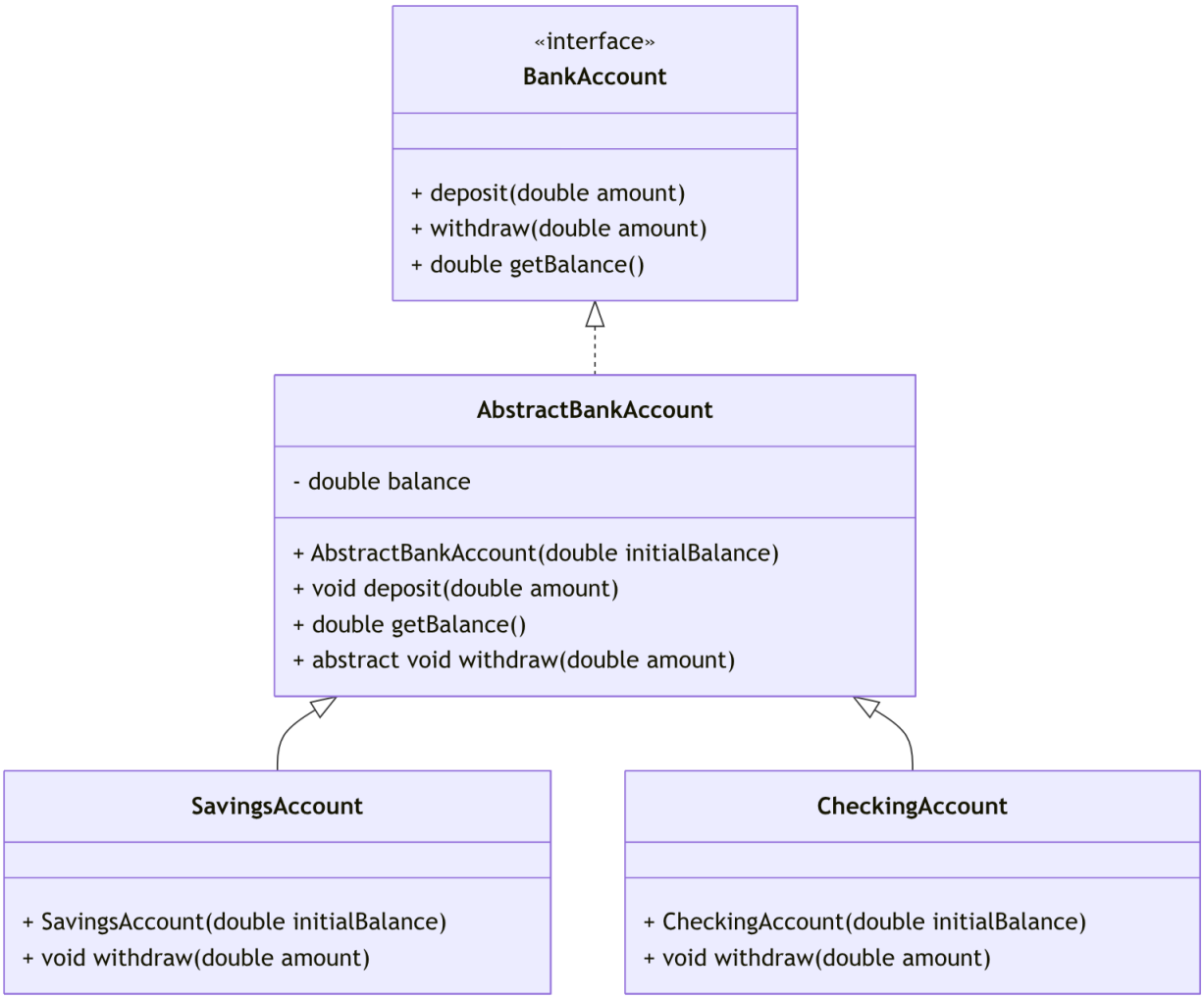
    @Override
    public String draw(Curve curve) {
        return "Brush drawing a " + curve.draw();
    }
}
```

Example 2

Note — transfer the same pattern to another domain. The account example uses double only to focus on abstraction, not as a production money model. Assume a documented

simplified currency unit; finite-value and amount validation follow below.

Here is another example of **mixing abstract classes and interfaces**, this time to model **two types of bank accounts**: a **SavingsAccount** and a **CheckingAccount**. The interface defines general bank operations, and the abstract class handles shared logic (e.g., balance management).




```java
interface BankAccount {
    void deposit(double amount);
    void withdraw(double amount);
    double getBalance();
}
```

```java
abstract class AbstractBankAccount implements BankAccount {
    protected double balance;

    public AbstractBankAccount(double initialBalance) {
        if (!Double.isFinite(initialBalance) || initialBalance < 0) {
            throw new IllegalArgumentException("Invalid initial balance");
```

```
        }
        this.balance = initialBalance;
    }

    @Override
    public void deposit(double amount) {
        if (!Double.isFinite(amount) || amount <= 0 || !Double.isFinite(balance +
        ↪  amount)) {
            throw new IllegalArgumentException("Invalid deposit");
        }
        balance += amount;
    }

    @Override
    public double getBalance() {
        return balance;
    }

    @Override
    public abstract void withdraw(double amount);
}
```

```
class SavingsAccount extends AbstractBankAccount {

    public SavingsAccount(double initialBalance) {
        super(initialBalance);
    }

    @Override
    public void withdraw(double amount) {
        if (Double.isFinite(amount) && amount > 0 && balance >= amount) {
            balance -= amount;
        } else {
            throw new IllegalArgumentException("Insufficient funds in SavingsAccount");
        }
    }
}
```

```
class CheckingAccount extends AbstractBankAccount {

    private static final double OVERDRAFT_LIMIT = 100.0;

    public CheckingAccount(double initialBalance) {
        super(initialBalance);
    }

    @Override
    public void withdraw(double amount) {
        if (Double.isFinite(amount) && amount > 0 && balance - amount >=
        ↪  -OVERDRAFT_LIMIT) {
            balance -= amount;
        } else {
```

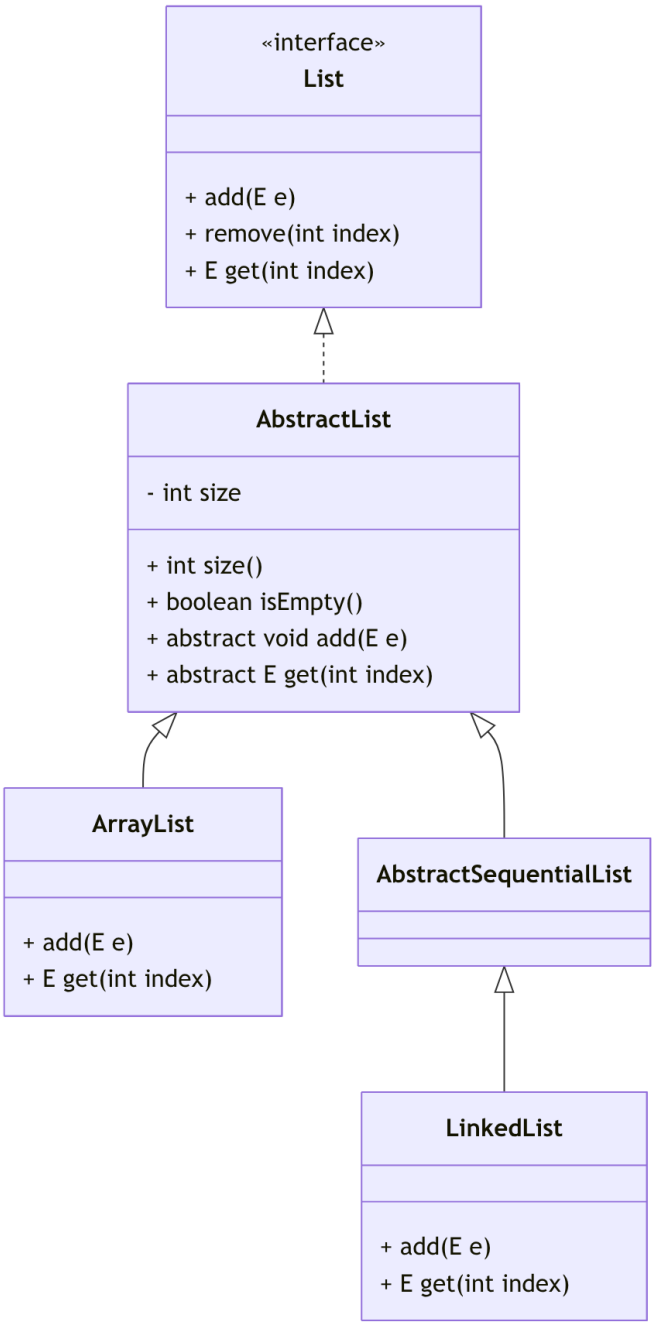
```
            throw new IllegalArgumentException("Overdraft limit exceeded in
             ↪  CheckingAccount");
        }
    }
}
```

Example 3

Reference — simplified collection hierarchy. `LinkedList` extends `AbstractSequentialList`, which itself extends `AbstractList`. The diagram shows that intermediate class explicitly; method lists are illustrative excerpts, not literal JDK declarations.

The **Java Collections Framework** is a classic example of how **interfaces and abstract classes are combined** to create a **flexible and extensible hierarchy**.

- **Interfaces** define *what* a collection can do (e.g., add, `remove`, `iterate`).
- **Abstract classes** provide *reusable logic* and *partial implementations* to reduce code duplication.
- **Concrete classes** specialize behavior for different use cases (e.g., lists vs. sets vs. queues).



Resources

- https://dev.java/learn/interfaces/
- https://www.baeldung.com/java-interface-vs-abstract-class
- https://www.baeldung.com/java-interface-default-method-vs-abstract-class
- https://www.baeldung.com/java-abstract-class

E3.6 — Object Equality and Hashing

The Object class

The Java Standard Library provides a class named `Object`, which is the root of the Java class hierarchy.

Every class that does not explicitly extend another class implicitly extends `Object`.

For example:

```
class A { }
```

is effectively a subclass of `Object`.

We can also explicitly write:

```
class A extends Object { }
```

but this is unnecessary.

The `Object` class can refer to an instance of any class because every class ultimately extends `Object`.

```
Object anObject = new Object();
```

For example:

```
Long number = 1_000_000L;
Object obj1 = number;

String str = "str";
Object obj2 = str;
```

These assignments are examples of **upcasting**.

The `Object` class provides methods that are inherited by all classes, including:

- **object identity and equality**: `equals()`, `hashCode()`;
- **string representation**: `toString()`;
- **runtime type information**: `getClass()`;
- **thread coordination**: `wait()`, `notify()`, `notifyAll()`;
- **object copying**: `clone()` is a protected method of `Object` and has special rules.

> **Note:** `finalize()` is still present in Java 25 but is deprecated for removal. Java 18 deprecated finalization for removal; it did not remove the method. Use explicit ownership and try-with-resources for resources, not finalization or garbage-collector timing.

equals()

`equals()` is a method defined by `Object` and is used to determine whether two objects are considered equal.

The default implementation inherited from `Object` compares object identity. Therefore, two different objects are not equal unless a subclass overrides `equals()` with a different definition.

```
// Example: equals() not overridden

class Person {
  String name;

  Person(String name) {
    this.name = name;
  }
}

public class EqualsFailExample {
  public static void main(String[] args) {
    Person p1 = new Person("Alice");
    Person p2 = new Person("Alice");

    // Object.equals() uses object identity
    IO.println(p1.equals(p2)); // false
  }
}
```

Explanation: p1 and p2 have the same name, but they are two different `Person` objects. Since `Person` does not override `equals()`, the implementation inherited from `Object` considers them unequal.

To define logical equality, we can override `equals()`:

```
// Example 2: Using equals() with custom objects

class Person {
  String name;

  Person(String name) {
    this.name = name;
  }

  // Equality policy for this example: exact runtime class and equal names.
  @Override
  public boolean equals(Object obj) {
    if (this == obj) return true;
```

```
    if (obj == null || getClass() != obj.getClass()) return false;

    Person other = (Person) obj;
    return java.util.Objects.equals(name, other.name);
  }

  @Override
  public int hashCode() {
    return java.util.Objects.hash(name);
  }
}

public class EqualsExample {
  public static void main(String[] args) {
    Person p1 = new Person("Alice");
    Person p2 = new Person("Alice");
    Person p3 = new Person("Bob");

    IO.println(p1.equals(p2)); // true
    IO.println(p1.equals(p3)); // false
  }
}
```

The general contract of `equals()` is:

1. **Reflexive**: for any non-null reference `x`, `x.equals(x)` must return `true`.
2. **Symmetric**: for any non-null references `x` and `y`, `x.equals(y)` must return `true` if and only if `y.equals(x)` returns `true`.
3. **Transitive**: for any non-null references `x`, `y`, and `z`, if `x.equals(y)` is `true` and `y.equals(z)` is `true`, then `x.equals(z)` must also be `true`.
4. **Consistent**: repeated calls to `x.equals(y)` should consistently return the same result as long as no information used by the comparison changes.
5. **Non-nullity**: for any non-null reference `x`, `x.equals(null)` must return `false`.

> **NOTE:** Whenever `equals()` is overridden, `hashCode()` should also be overridden so that equal objects have equal hash codes.

hashCode()

`hashCode()` is a method defined by `Object` that returns an `int`.

It is used primarily by hash-based collections such as `HashMap` and `HashSet`.

The most important rule of the `hashCode()` contract is:

> If two objects are equal according to `equals()`, they **must** have the same hash code.

The reverse is not required:

> Two objects with the same hash code do **not** have to be equal.

Different objects can therefore have the same hash code. This is called a **hash collision**.

```
// Example: hashCode() not overridden

class Person {
  String name;

  Person(String name) {
    this.name = name;
  }
}

public class HashCodeFailExample {
  public static void main(String[] args) {
    Person p1 = new Person("Alice");
    Person p2 = new Person("Alice");

    IO.println(p1.hashCode());
    IO.println(p2.hashCode());
  }
}
```

The two values may be different because `Person` does not override `hashCode()`.

> **NOTE:** The default `Object.hashCode()` is not specified as a memory address. Its implementation is JVM-dependent. Therefore, it is better not to describe the default hash code as a memory address.

If `equals()` is overridden, `hashCode()` should be overridden consistently:

```
import java.util.Objects;

// Example 2: Overriding hashCode() for custom objects

class Person {
  String name;

  Person(String name) {
    this.name = name;
  }

  @Override
  public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;

    Person other = (Person) obj;
    return Objects.equals(name, other.name);
  }
```

```
  @Override
  public int hashCode() {
    return Objects.hash(name);
  }
}

public class HashCodeExample {
  public static void main(String[] args) {
    Person p1 = new Person("Alice");
    Person p2 = new Person("Alice");
    Person p3 = new Person("Bob");

    IO.println(p1.hashCode() == p2.hashCode()); // true
    IO.println(p1.hashCode() == p3.hashCode()); // false for these names and this
↪   implementation
    // Different objects are allowed to have the same hash code.
  }
}
```

For this implementation and these inputs, the last comparison is false. The general contract does **not** require different hashes for unequal objects: collisions are allowed.

equals() and hashCode() consistency problem

Consider the following incorrect implementation:

```
import java.util.Objects;
import java.util.HashSet;

class Person {

  private int id;
  private String name;

  public Person(int id, String name) {
    this.id = id;
    this.name = name;
  }

  // Two persons are equal if they have the same id
  @Override
  public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Person)) return false;

    Person p = (Person) o;
    return id == p.id;
  }

  // WRONG: hashCode uses an extra field (name)
  @Override
```

```
  public int hashCode() {
    return Objects.hash(id, name);
  }
}
```

```
public class Test {

  public static void main(String[] args) {

    Person p1 = new Person(1, "Alice");
    Person p2 = new Person(1, "Bob");

    IO.println(p1.equals(p2)); // true

    HashSet<Person> set = new HashSet<>();
    set.add(p1);
    set.add(p2);

    IO.println(set.size()); // 2 (unexpected)
  }
}
```

Explanation

- `equals()` says that **p1 and p2 are equal** because they have the same `id`.
- `hashCode()` uses both `id` and `name`, so the two objects can produce different hash codes.
- A `HashSet` uses the hash code to determine where an object should be stored.
- If equal objects have different hash codes, they may be placed in different hash buckets, so the set can fail to recognize them as equal.

The correct implementation must use the **same equality-significant state** in both methods:

```
@Override
public int hashCode() {
  return Integer.hashCode(id);
}
```

This repair ensures equal IDs produce equal hashes. Hash functions may legally use only a subset of equality-significant fields, but must never distinguish equal objects. Do not mutate equality-significant state while an object is a hash-map key or hash-set element.

Review: a generated `equals` compares only `id`, while its generated `hashCode` includes `name`. Test two instances with the same ID and different names. Checking only `equals` misses the collection-level failure.

Testing equality as a contract

In the E2.8 practice project, save the **name-based `Person` implementation with both `equals` and `hashCode`** from “Overriding hashCode() for custom objects” as `src/main/java/com/example/Pe

Add `package com.example;` before its import. Do not copy the accompanying `HashCodeExample` class or the later ID-based example: these are alternative examples, not parts of the same source file.

Save this as `src/test/java/com/example/PersonTest.java`:

```java
package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class PersonTest {
    @Test
    void equalNamesDefineEqualityAcrossDifferentInstances() {
        Person first = new Person("Alice");
        Person second = new Person("Alice");
        Person third = new Person("Alice");

        assertNotSame(first, second);
        assertEquals(first, first);   // reflexivity
        assertEquals(first, second);
        assertEquals(second, first); // symmetry
        assertEquals(second, third);
        assertEquals(first, third);  // transitivity for these instances
        assertEquals(first.hashCode(), second.hashCode());
    }

    @Test
    void differentNamesNullAndUnrelatedTypesAreNotEqual() {
        Person person = new Person("Alice");

        assertNotEquals(person, new Person("Bob"));
        assertFalse(person.equals(null));
        assertFalse(person.equals("Alice"));
    }
}
```

`assertEquals` checks logical equality, while `assertNotSame` distinguishes it from identity. Do **not** assert that unequal persons have different hash codes: that is not part of the contract. These examples exercise selected cases, not all possible objects; add further cases for the equality policy your application actually defines.

The collection-level tests in E4.2 check that this equality policy also works when instances are stored and looked up in a set.

Connection to collections

The equality and hashing contract determines element uniqueness in hash-based sets and key lookup in hash-based maps. Equal objects must have equal hash codes; unequal objects may

still share a hash code. Define equality before choosing a hash-based container; do not rely on object identity accidentally.

Resources

E3.7 — Immutability and Records

Compile alternative record and class definitions separately. Fragments belong inside the indicated class or `main`; import the standard types used by each example.

Immutability

An object is **immutable** when its observable state cannot change after construction.

- `final` fields help prevent reassignment but do not freeze referenced arrays, lists, or other mutable objects.
- A changed value is represented by another object rather than by mutating the original.
- `String` is a familiar immutable type. Its operations return a result without modifying the original; they do not necessarily allocate a new object every time.

The class `Patient` is not immutable because it is possible to change any field of an object.

```java
public class Patient {
    // field to store the patient's name
    public String name;

    // other stuff

}
```

```
Patient patient = new Patient();

patient.name = "Mary";
patient.name = "Alice";
```

Immutable Classes

Passing immutable data reduces accidental changes through shared references. **It does not establish validity by itself**: construction must enforce the domain rules.

- Classes to simply hold data, such as database results, or information from a service.

For a simple immutable value class:

- prevent subclasses from adding incompatible mutable behavior, for example using a final class;

- **private, final field** for each piece of data
- defensively copy mutable inputs and outputs where needed;
- **getter** for each field (**setters** are useless in this case)
- **public constructor** with a corresponding argument for each field
- **equals method** that returns true for objects of the same class when all fields match
- **hashCode method** that returns the same value when all fields match
- **toString method** that includes the name of the class and the name of each field and its corresponding value

Equality, hashing, accessors, and string representation make a value class convenient; they are not themselves requirements for immutability. Avoid printing sensitive state in generated `toString` methods.

For example, we can create an immutable *Person* data class with a name and address:

```
import java.util.Objects;

public final class Person {
    private final String name;
    private final String address;

    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, address);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        } else if (!(obj instanceof Person)) {
            return false;
        } else {
            Person other = (Person) obj;
            return Objects.equals(name, other.name)
              && Objects.equals(address, other.address);
        }
    }

    @Override
```

```
    public String toString() {
        return "Person [name=" + name + ", address=" + address + "]";
    }
}
```

Records

Records are concise value-oriented data carriers with **final component fields**, not a guarantee of deep immutability. The compiler supplies a canonical constructor, component accessors, and implementations of `equals`, `hashCode`, and `toString` unless explicitly provided.

```
public record Person (String name, String address) {}
```

The accessors are `name()` and `address()`, not `getName()` and `getAddress()`. The canonical constructor has the record's accessibility; it is public for this public record. Records are final and can implement interfaces, but cannot extend an application superclass.

Defensive copies and compact constructors

Predict: if a record stores a caller's mutable list directly, can that caller change the record's observable contents?

```
import java.util.List;
import java.util.Objects;

public record Team(String name, List<String> members) {
    public Team {
        Objects.requireNonNull(name);
        members = List.copyOf(members);
    }
}
```

The compact constructor validates and replaces the incoming list with an unmodifiable snapshot before fields are assigned. Here the elements are immutable strings, so this is sufficient for immutable list contents. If elements were mutable, a shallow snapshot would not freeze them.

Review: an assistant proposes `Collections.unmodifiableList(members)` instead. Test a later change through the caller's original list. A read-only view and an independent snapshot are different contracts.

> **Note:** array record components still use array identity in generated equality, not automatic element-wise comparison. Choose representations and custom methods deliberately when value equality must include array contents.

Record patterns

Note — reading modern Java 25 code: record patterns deconstruct a record into its components. This feature has been final since Java 21; it does not require preview flags in Java 25.

```
record Coordinates(int x, int y) {}

class DescribeCoordinates {
    static String describe(Object value) {
        return switch (value) {
            case Coordinates(int x, int y) -> "(" + x + ", " + y + ")";
            case null -> "No coordinates";
            default -> "Another type";
        };
    }
}
```

Here `int x` matches an `int` record component. This is not the preview feature that generalizes primitive patterns to other type-testing contexts. A record pattern reads components; it does not make their objects immutable.

Resources

- https://www.baeldung.com/java-record-keyword

E3.8 — Enums and State Modelling

Enums

An **enum** declares a finite set of named instances. It improves type checking and makes a closed vocabulary explicit. It does not automatically validate transitions between states, and enum references can still be null.

```
public class Pizza {
    public enum PizzaStatus {
        NOT_ASSIGNED,
        ORDERED,
        READY,
        DELIVERED
    }
    private PizzaStatus status = PizzaStatus.NOT_ASSIGNED;

    public boolean isDeliverable() {
        return status == PizzaStatus.READY;
    }
}
```

Values and transitions are different contracts

`PizzaStatus` restricts the possible values, but it does not encode which change is allowed next. For this example, suppose the required workflow is:

```
NOT_ASSIGNED -> ORDERED -> READY -> DELIVERED
```

Add these operations **inside the preceding `Pizza` class**. Rejecting a transition before assignment leaves the old state intact. The enum remains a vocabulary; the object owns the transition rules.

```
public void placeOrder() {
    if (status != PizzaStatus.NOT_ASSIGNED) {
        throw new IllegalStateException("Order already placed");
    }
    status = PizzaStatus.ORDERED;
}

public void markReady() {
```

```
    if (status != PizzaStatus.ORDERED) {
        throw new IllegalStateException("Pizza has not been ordered");
    }
    status = PizzaStatus.READY;
}

public void deliver() {
    if (status != PizzaStatus.READY) {
        throw new IllegalStateException("Pizza is not ready");
    }
    status = PizzaStatus.DELIVERED;
}
```

Predict: after `placeOrder()` and `markReady()`, what does `isDeliverable()` return? What if `deliver()` is called on a new pizza, or twice on the same pizza? Throwing `IllegalStateException` rejects an invalid transition before changing the state, preserving the state invariant.

Resources

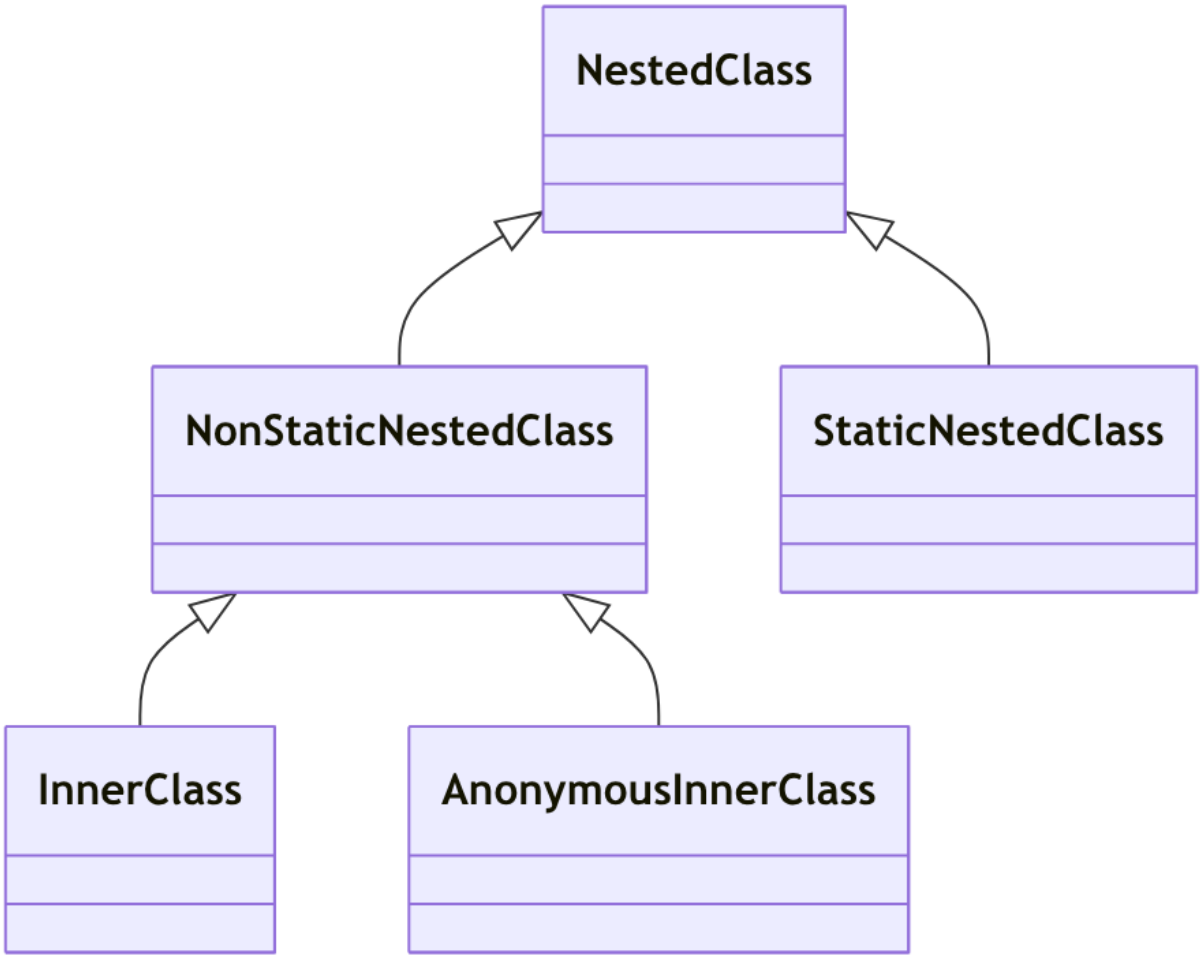
- https://www.baeldung.com/a-guide-to-java-enums

E3.9 — Nested Classes

Nested classes

You can call a class **nested** when it is declared inside another class.

Reference: static nested classes do not require an enclosing instance; non-static member inner classes do. Local and anonymous classes provide additional forms. The following diagram groups concepts; it is not an inheritance hierarchy of Java runtime types.



Anonymous Inner Classes

Let's assume we have the following interface with two methods:

```
interface SpeakingEntity {

    void sayHello();

    void sayBye();
}
```

Here, two anonymous classes are defined to represent an English-speaking person and a cat, respectively.

The expression creates an instance of an **anonymous implementing class**, not an instance of the interface itself:

```
SpeakingEntity englishSpeakingPerson = new SpeakingEntity() {

    @Override
    public void sayHello() {
        IO.println("Hello!");
    }

    @Override
    public void sayBye() {
        IO.println("Bye!");
    }
};
```

```
SpeakingEntity cat = new SpeakingEntity() {

    @Override
    public void sayHello() {
        IO.println("Meow!");
    }

    @Override
    public void sayBye() {
        IO.println("Meow!");
    }
};
```

Now, we can invoke overridden methods:

```
englishSpeakingPerson.sayHello();
cat.sayBye();
```

Inner Classes

Note — use an inner class when its association with an enclosing instance is meaningful. Otherwise a static nested class may avoid an unnecessary enclosing reference.

In this example, the Bow class will be a non-static inner class of the Cat class. The Bow class will have access to the instance variables of the Cat class, such as the `color` and `size` of the bow.

```
class Cat {
    private String name = "Whiskers";  // Instance variable of the Cat class
    private String color = "Red";      // Instance variable for Bow color

    // Non-static inner class Bow
    class Bow {
        void tieBow() {
            // Accessing outer class's instance fields
            IO.println(name + " wears a " + color + " bow.");
        }
    }

    void createBow() {
        // Creating and using the Bow instance
        Bow bow = new Bow();
        bow.tieBow();  // Calling the method in the Bow class
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an instance of the Cat class
        Cat cat = new Cat();
        cat.createBow();  // Using the inner Bow class
    }
}
```

- The Bow class is a non-static inner class of `Cat`. It can access the `name` and `color` instance variables of `Cat`.
- The `createBow()` method in the `Cat` class creates an instance of `Bow` and calls the `tieBow()` method to display a message.
- The `main()` method creates an instance of `Cat` and invokes `createBow()` to use the inner Bow class.

Static Nested Classes

Now make Bow a **static nested class**. It has no implicit `Cat` instance, so it needs an explicit `Cat` reference to access instance members. It can directly access the enclosing class's static members.

```java
class Cat {
    private static String type = "Persian";  // Static field in Cat class

    // Static nested class Bow
    static class Bow {
        void tieBow() {
            // Accessing outer class's static field
            IO.println("The cat is a " + type + " and wears a bow.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an instance of the static Bow class without an instance of Cat
        Cat.Bow bow = new Cat.Bow();
        bow.tieBow();  // Calling the method in the static Bow class
    }
}
```

- Bow is a static nested class of `Cat`. It directly accesses `type` without an enclosing instance.
- The `main()` method in `Main` creates an instance of the Bow class directly using `Cat.Bow`, without needing an instance of `Cat`.
- The `tieBow()` method of Bow is called to display a message about the cat type and the bow.

Resources

E3.10 — SOLID Principles

SOLID groups five design principles intended to make software easier to understand and change. Robert C. Martin helped formulate and popularize these principles; the underlying ideas have multiple origins.

> SOLID is a set of design heuristics, not a requirement to create an interface for every class or to maximize the number of types.

The bank-account, bird, and report examples below retain the original progression. Compile each section's types separately; some sections introduce alternative designs. Method fragments belong inside their indicated class.

Single responsibility principle (SRP)

A module should have **one coherent reason to change**: changes driven by different concerns or stakeholders should not be unnecessarily coupled.

A class can have several related methods and still have one responsibility.

Consider a bank account that manages its balance, writes SQL, and sends SMS messages:

```
BankAccount
 ├─ deposit / withdraw / calculateInterest
 ├─ saveBankAccountDetails
 └─ sendSmsNotification
```

An interest rule, a database schema, and a messaging provider can change independently. Keeping all three in one class creates unrelated edit paths.

Separate state rules from infrastructure

This balance example uses **integer cents in one assumed currency**, with no interest, fees, or concurrency. It is not a full banking model.

```java
public final class BankAccount {
    private long balance;

    public BankAccount(long initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Negative initial balance");
```

```
        }
        balance = initialBalance;
    }

    public long getBalance() {
        return balance;
    }

    public void deposit(long amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance = Math.addExact(balance, amount);
    }

    public void withdraw(long amount) {
        if (amount <= 0 || amount > balance) {
            throw new IllegalArgumentException("Invalid withdrawal");
        }
        balance -= amount;
    }
}
```

Persistence and notification can be separate collaborators:

```
class SQLBankAccountRepository {
    public void save(BankAccount account) {
        // Infrastructure boundary: database implementation omitted.
    }
}

class NotificationService {
    public void sendSmsNotification(BankAccount account, String message) {
        // Infrastructure boundary: SMS implementation omitted.
    }
}

class BankAccountService {
    private final SQLBankAccountRepository repository;
    private final NotificationService notifications;

    public BankAccountService(SQLBankAccountRepository repository,
                              NotificationService notifications) {
        this.repository = java.util.Objects.requireNonNull(repository);
        this.notifications = java.util.Objects.requireNonNull(notifications);
    }

    public void deposit(BankAccount account, long amount) {
        account.deposit(amount);
        repository.save(account);
        notifications.sendSmsNotification(account, "Deposit: " + amount);
    }
}
```

The empty infrastructure bodies mark omitted integrations, not working database or SMS code. The service coordinates the workflow while the account owns balance rules.

Review: if saving succeeds but notification fails, should the deposit be repeated? Separating classes does not by itself provide transactions, idempotency, or a failure policy.

Open closed principle (OCP)

Software can be **open to a selected kind of extension without requiring changes throughout existing client code**.

This does not mean “never modify existing code.” Choose extension points for changes that are plausible, not every imaginable future feature.

The original account example calculates interest by checking account-type strings. A new interest rule would require another branch. One alternative is to make that rule a collaborator:

```
interface InterestPolicy {
    double calculate(double balance);
}

final class SavingsInterest implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.03;
    }
}

final class CheckingInterest implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.01;
    }
}

final class FixedDepositInterest implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.05;
    }
}

final class InterestService {
    public double calculate(double balance, InterestPolicy policy) {
        if (!Double.isFinite(balance) || balance < 0) {
            throw new IllegalArgumentException("Invalid balance");
        }
        return policy.calculate(balance);
    }
}
```

This independent example uses `double` to show varying formulas, not production monetary arithmetic. Policy implementations assume non-negative finite balances; clients supply a non-null policy. A real implementation needs units, periods, and rounding rules.

Adding a policy need not change `InterestService`. Configuration or construction code still chooses the policy.

> **Note:** a short switch over a stable, closed set may be clearer than several strategy classes. OCP is not a ban on switches or a mandate to replace simple constants with hierarchies.

Liskov Substitution Principle (LSP)

A subtype should be usable wherever its supertype is expected **without breaking the client's documented expectations**.

Matching signatures is necessary, but not sufficient. Preconditions, postconditions, and invariants matter too.

Suppose `Bird.fly()` promises that a valid call performs flight. A `Penguin` override that always throws breaks that promise even if the code compiles.

Instead, model the capability only where it applies:

```
class Bird {
    public void eat() {
        IO.println("Eating");
    }
}

interface Flying {
    void fly();
}

final class Parrot extends Bird implements Flying {
    @Override
    public void fly() {
        IO.println("Flying");
    }
}

final class Penguin extends Bird {
}
```

Predict: which objects can be supplied to a method taking `Bird`? Which can be supplied to one taking `Flying`?

Check your prediction

Both objects satisfy `Bird`; only `Parrot` satisfies `Flying`. We corrected the advertised capability rather than adding a runtime “unsupported” branch to every caller.

Review: a subclass accepts fewer valid input values than the superclass contract. Does its `@Override` annotation establish substitutability? Design a contract test that runs against both implementations.

Interface segregation principle (ISP)

Clients should not be forced to depend on operations they do not need. Organize contracts around coherent client roles.

If a loan account cannot offer ordinary withdrawal, forcing it to implement a broad account interface may produce a dummy method, a silent no-op, or an unexpected exception.

An alternative is to expose separate capabilities:

```
interface BalanceView {
    long getBalance();
}

interface Depositable {
    void deposit(long amount);
}

interface Withdrawable {
    void withdraw(long amount);
}

final class CashMachine {
    public void withdraw(Withdrawable account, long amount) {
        account.withdraw(amount);
    }
}
```

An ordinary account can implement all three; another product can implement only its relevant capabilities. The cash machine depends only on withdrawal.

Activity: adapt the earlier `BankAccount` to implement these interfaces without changing its balance rules. Then identify whether a particular client actually benefits from the split.

> **Note:** splitting every method into its own interface can create needless indirection. Segregate by client needs, not by an arbitrary maximum method count.

Dependency inversion principle (DIP)

High-level policy should not be tightly coupled to low-level implementation details. Both can depend on an abstraction shaped by the policy's needs.

In the report example, a service that constructs both PDF and HTML generators must know each implementation and its setup.

Introduce a common contract and supply the implementation from outside:

```
interface ReportGenerator {
    String generateReport();
}

final class PDFReportGenerator implements ReportGenerator {
    @Override
    public String generateReport() {
        return "PDF report description";
    }
}

final class HTMLReportGenerator implements ReportGenerator {
    @Override
    public String generateReport() {
        return "HTML report description";
    }
}

final class ReportService {
    private final ReportGenerator generator;

    public ReportService(ReportGenerator generator) {
        this.generator = java.util.Objects.requireNonNull(generator);
    }

    public String generateReport() {
        return generator.generateReport();
    }
}
```

These generators return descriptions, not actual PDF or HTML documents. The example isolates the dependency boundary.

Client construction can be explicit:

```
ReportService service = new ReportService(new HTMLReportGenerator());
IO.println(service.generateReport());
```

Constructor injection does not require a framework. A test can supply `() -> "test report"` and verify delegation without a document renderer.

Dependency injection is a technique; dependency inversion is a design principle. Injecting a concrete, infrastructure-specific dependency does not automatically invert the architectural dependency.

Resources

- https://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod
- https://refactoring.guru/

E4.1 — Collections Lists and Iterators

Examples use `java.util` types; add the required imports. Put statements inside `main` and method fragments inside a class. Simplified interface excerpts show selected signatures, not replacement library types. Alternative definitions of `Person` are compiled separately.

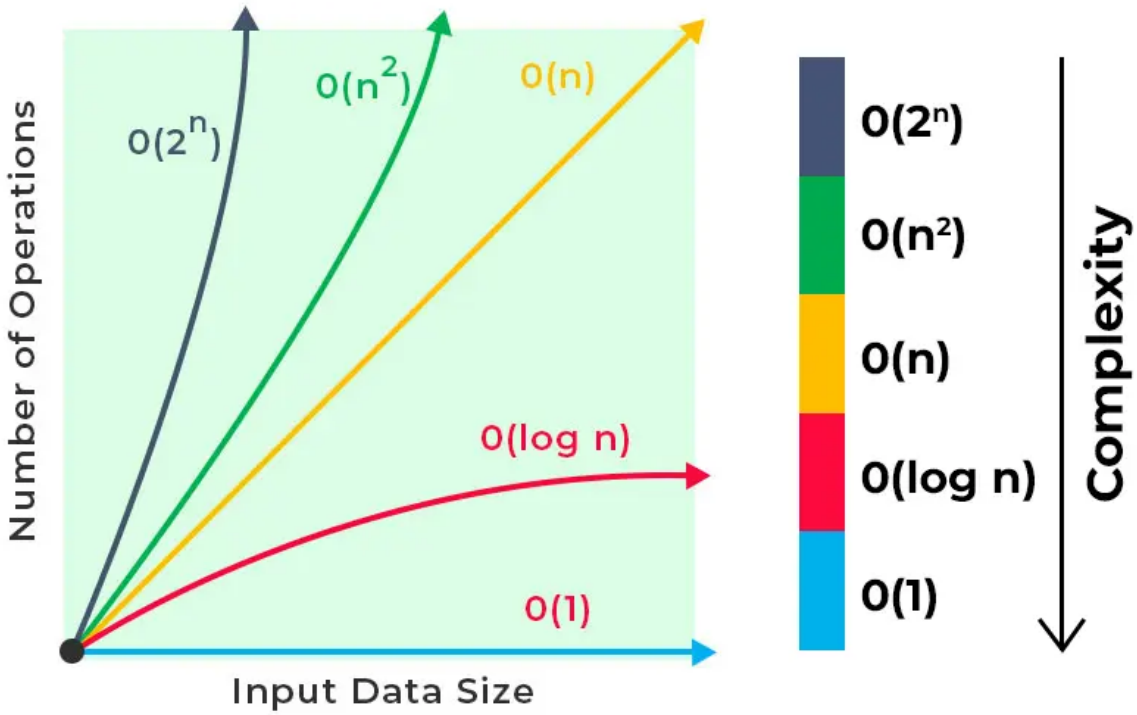
The **Java Collections Framework (JCF)** is a unified architecture for representing and manipulating collections of objects.

It provides:

- **interfaces** defining common collection abstractions and operations
- **abstract classes** providing partial implementations and reusable code
- **concrete classes** implementing the collection interfaces
- **algorithms** for manipulating collections, mainly provided by `java.util.Collections`

Complexity

Reference — compare relevant operations, not just collection names.



When choosing a data structure, it is important to consider the complexity of the operations that will be performed most frequently.

Typical complexities include:

- **O(1)** — constant time
- **O(log n)** — logarithmic time
- **O(n)** — linear time
- **O(n log n)** — typically associated with efficient comparison-based sorting algorithms

The actual performance of a collection also depends on factors such as memory layout, cache locality, resizing, hashing, and the particular implementation.

Key technologies

Dynamic array

The main limitation of a regular array is that its size is fixed at the time of allocation.

In other words, if an array is allocated with size n, it cannot accommodate the (n+1)-th element without creating a new array.

One possible workaround is to allocate a large array in advance, but this can lead to **significant memory waste**.

This limitation is addressed by the concept of a dynamic array.

A dynamic array maintains:

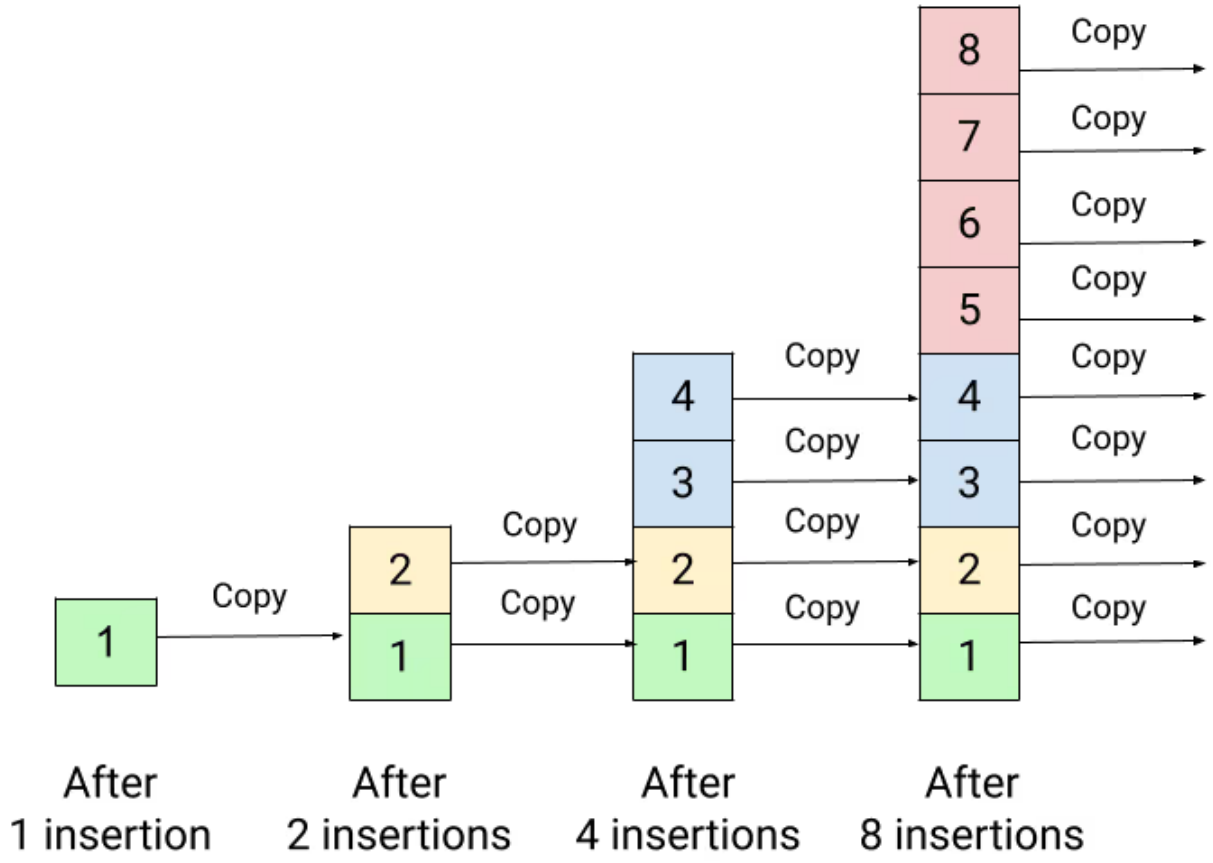
- a backing array
- a current **size**, representing the number of stored elements
- a **capacity**, representing the number of elements that can currently fit in the backing array

When the capacity is exceeded, a new, larger array is allocated and the existing elements are copied over.

Resizing itself takes **O(n)** time, but with a suitable growth strategy, appending elements is **amortized O(1)**.

Dynamic arrays provide:

- efficient random access: **O(1)**
- efficient iteration
- amortized **O(1)** append operations
- occasional **O(n)** resizing operations



Linked list

A linked list is a linear collection of elements whose order is determined by **links** between nodes rather than by contiguous memory locations.

A linked list consists of a sequence of **nodes**, where each node typically contains:

- the stored element
- a reference to the next node
- and, in a doubly linked list, a reference to the previous node

A linked list does not provide efficient random access. To access the element at position `i`, the list generally has to traverse the preceding nodes.

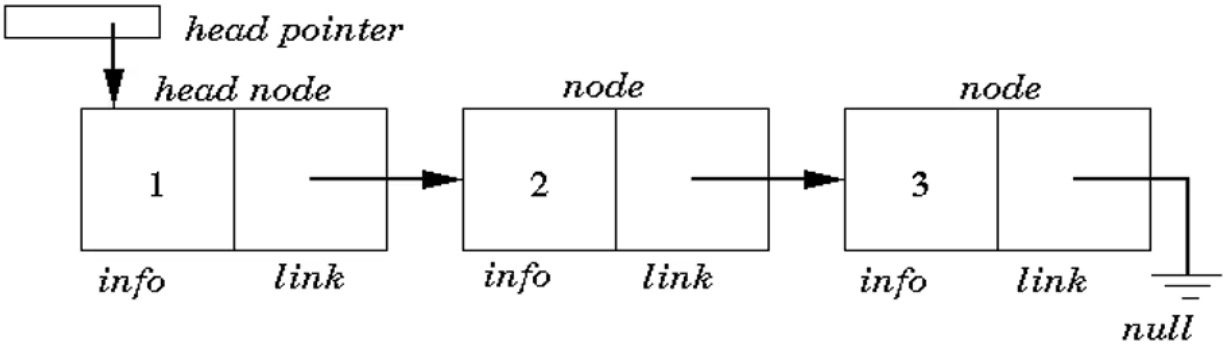
Therefore:

- access by index is **O(n)**
- searching is generally **O(n)**
- insertion/removal can be **O(1)** when the relevant node/position is already known
- finding an arbitrary position by index is **O(n)**

Linked lists can also have **poorer cache locality** than arrays because their nodes are separate objects that may be located in different memory areas.

More advanced variants include:

- **doubly linked lists**
- **circular linked lists**



Binary tree

A tree is a hierarchical data structure composed of **nodes** connected by **edges**.

A tree has a **root node** at the top. Nodes can have **child nodes**, forming branches, while nodes without children are called **leaves**.

A **binary tree** is a tree in which each node has at most two children.

A **binary search tree (BST)** maintains an ordering relationship between the elements. In a balanced BST, the height remains approximately logarithmic in the number of elements.

Self-balancing binary search trees, such as:

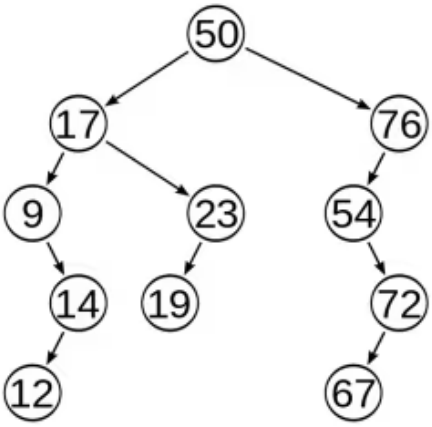
- **AVL trees**
- **Red-Black trees**

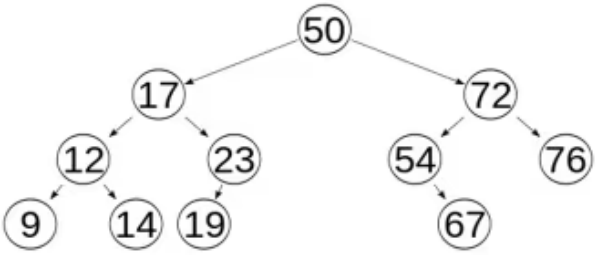
maintain a bounded tree height and provide **O(log n)** worst-case search, insertion, and deletion.

Trees are widely used in applications such as:

- file systems
- databases
- search algorithms
- indexing structures

- hierarchical data representation





Hash table

A hash table is a data structure that implements an **associative array**, mapping **keys** to **values**.

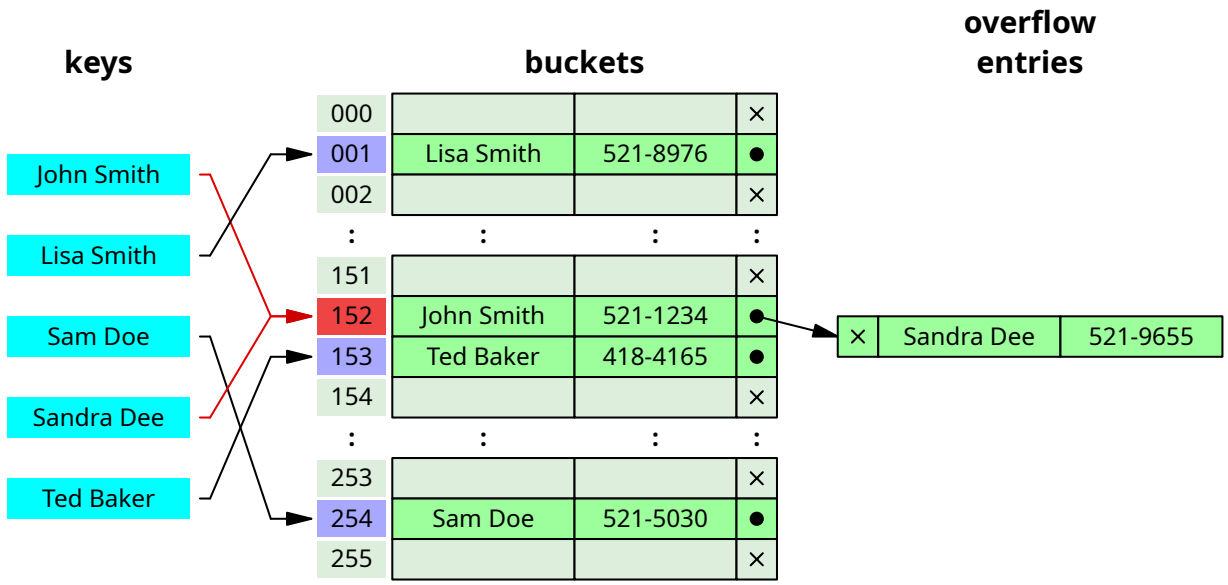
A hash table uses a **hash function** to transform a key into a hash value. The hash value is then used to determine an appropriate location, or **bucket**, in an internal table.

During a lookup operation:

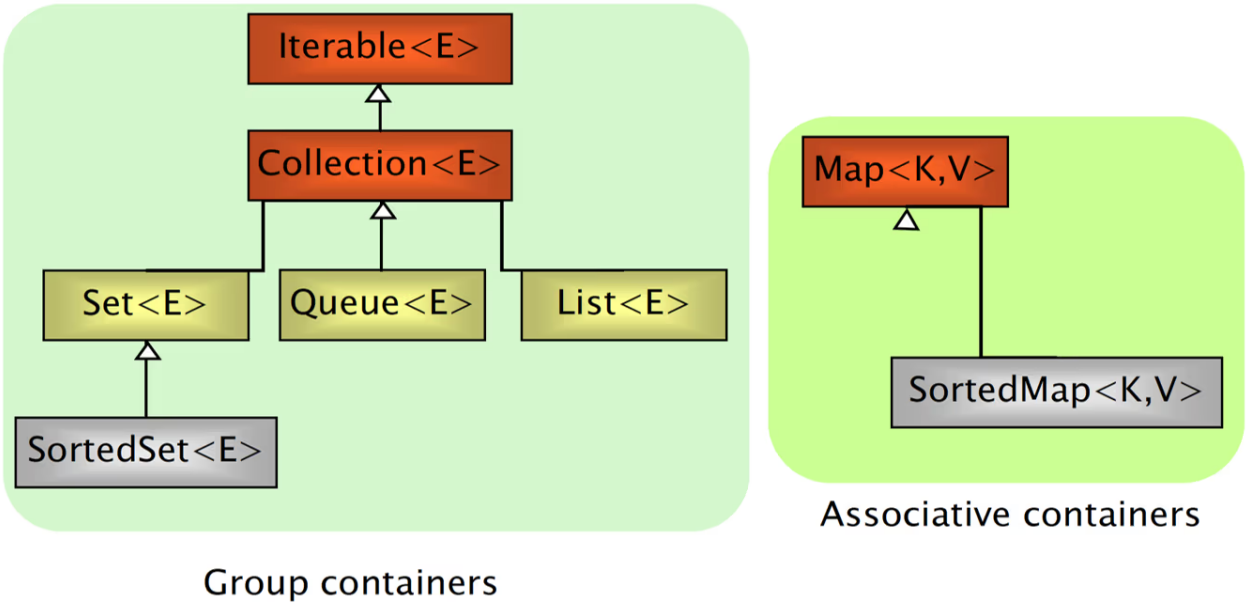
1. the key is hashed
2. the hash value is used to identify candidate bucket(s)
3. the stored key is compared with the requested key
4. the associated value is returned if the key is found

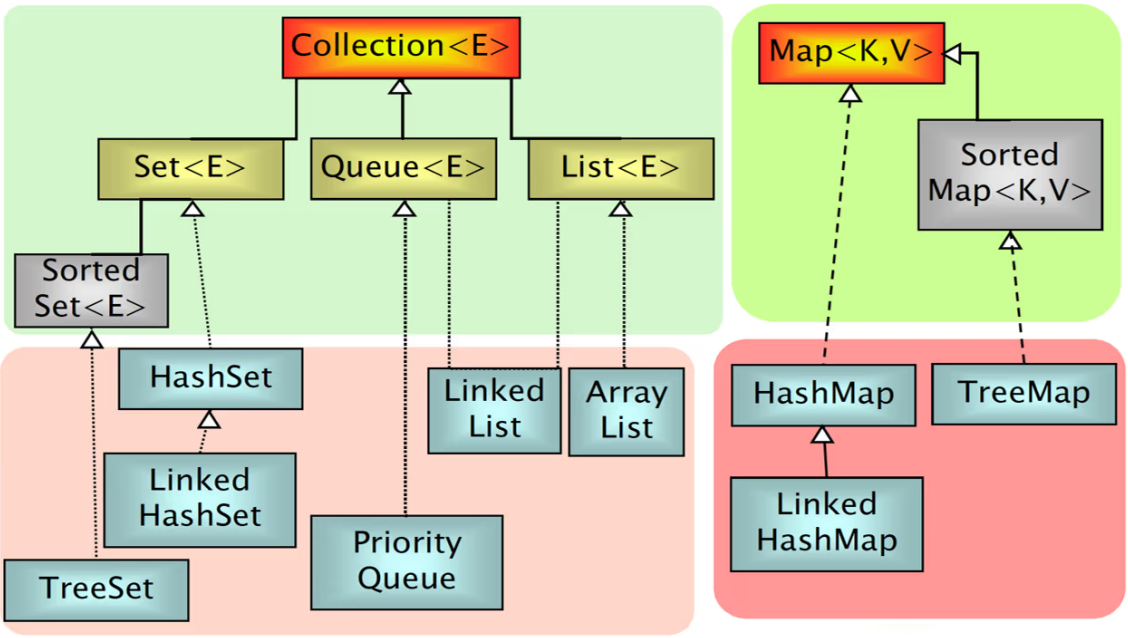
Hash tables typically provide **O(1) average-case** insertion, deletion, and lookup, although the exact complexity depends on the implementation and the distribution of hash values.

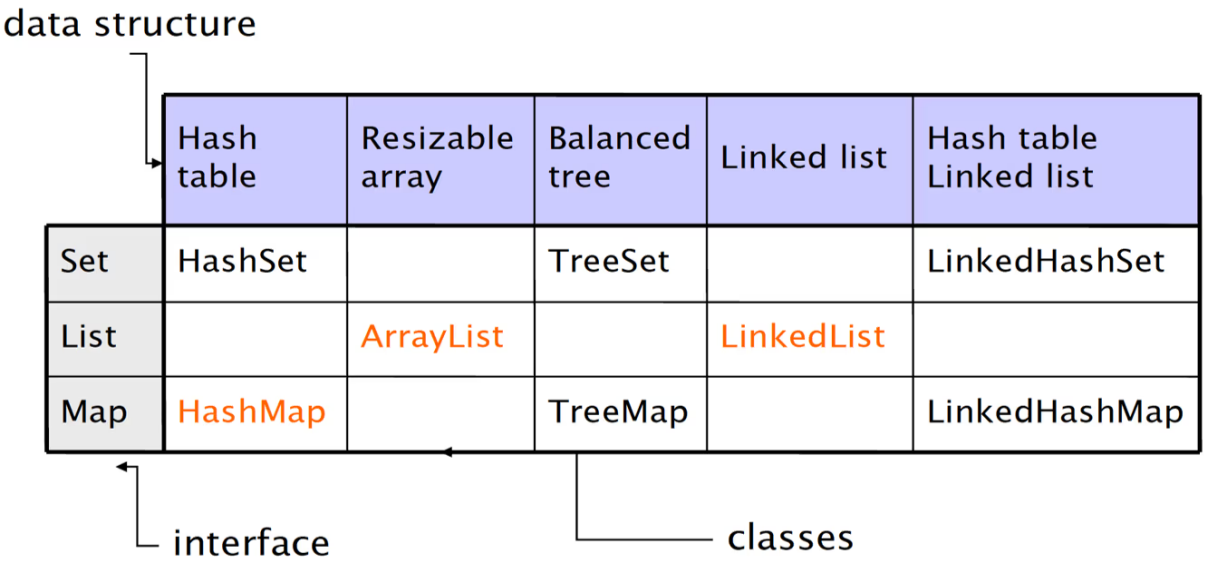
Poor hashing or excessive collisions can degrade performance.



Interfaces and implementations







Iterable and Iterator Interfaces

The **`Iterable<T>`** interface (`java.lang.Iterable`) represents an object whose elements can be traversed by obtaining an `Iterator<T>`.

Objects implementing `Iterable` can be used in **enhanced `for` loops**:

```java
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

For example:

```java
List<Object> l = new ArrayList<>();

for (Object o : l) {
    // do something
}
```

For an `Iterable`, enhanced `for` uses iterator-based traversal. Arrays also support enhanced `for`, but are not `Iterable` and use a different translation.

The **`Iterator`** interface encapsulates the traversal of a collection in a separate object.

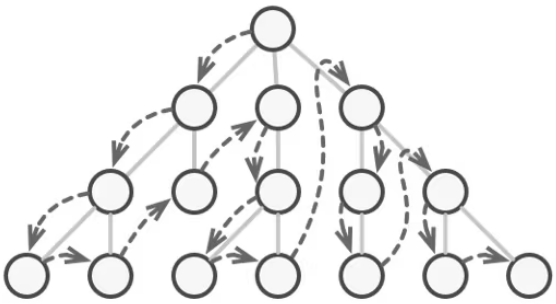
```java
public interface Iterator<T> {
    boolean hasNext();
    T next();
    void remove();
}
```

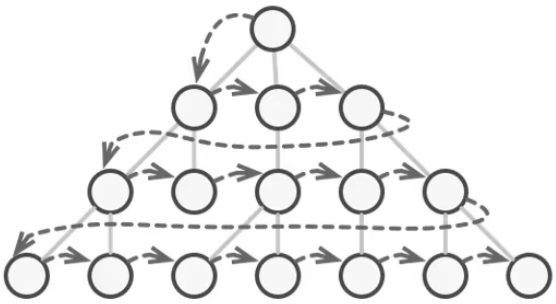
For example:

```java
ArrayList<Object> l = new ArrayList<>();

for (Iterator<Object> i = l.iterator(); i.hasNext();) {
    Object o = i.next();
    // do something
}
```

The `remove()` method allows an iterator to remove the last element returned by `next()`, when the underlying collection supports this operation.





See more: Design Patterns: Iterator - Refactoring.guru

> **Note:** `Iterable` is not the root interface of the Java Collections Framework. `Collection` is the main root interface for collections, while `Iterable` is a more general interface for objects that can be iterated over.

Collection Interface

The **`Collection`** interface represents a group of objects, known as its elements.

Some collections:

- allow duplicate elements
- do not allow duplicates
- preserve a defined order
- do not guarantee a particular iteration order

The `Collection` interface is useful when code should work with a collection without depending on a specific implementation.

Collection main methods

Reference — the generic forms of the main methods are:

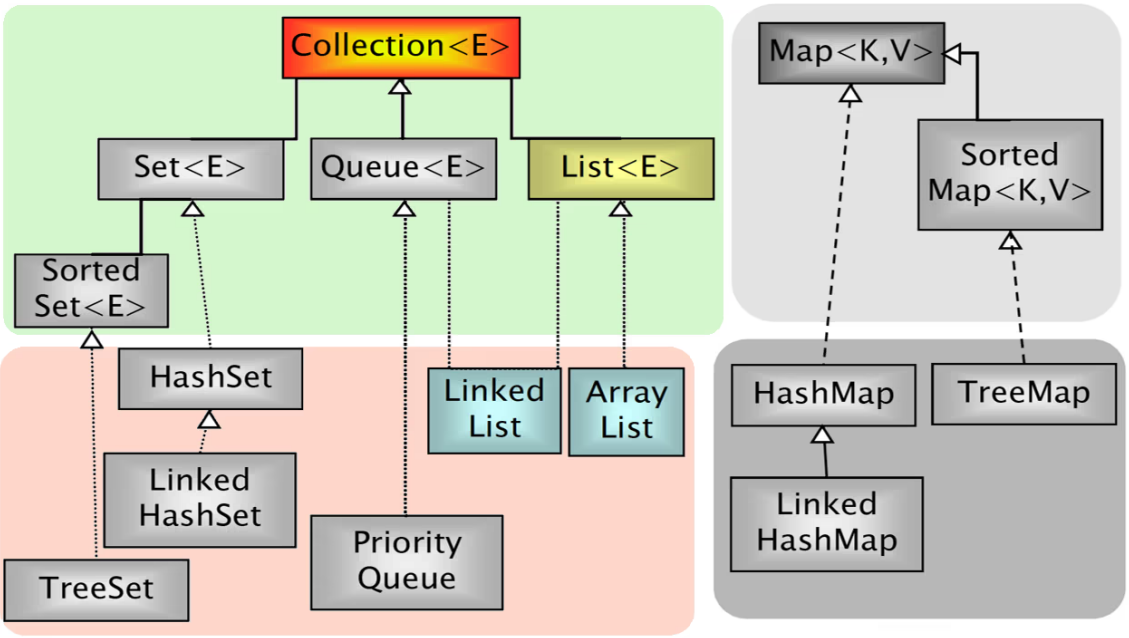
- `int **size**()`
- `boolean **isEmpty**()`
- `boolean **contains**(Object element)`
- `boolean **containsAll**(Collection<?> c)`
- `boolean **add**(E element)`
- `boolean **addAll**(Collection<? extends E> c)`
- `boolean **remove**(Object element)`
- `boolean **removeAll**(Collection<?> c)`
- `void **clear**()`
- `Object[] **toArray**()`
- `<T> T[] **toArray**(T[] a)`
- `Iterator<E> **iterator**()`

List Interface

A **`List`** is an ordered collection in which:

- duplicate elements are allowed
- **insertion order is preserved**
- elements can be accessed **by position**

- elements can be inserted or removed at arbitrary positions



List main methods

- `E **get**(int index)`
- `E **set**(int index, E element)`
- `E **remove**(int index)`
- `void **add**(int index, E element)`
- `boolean **addAll**(int index, Collection<? extends E> c)`
- `int **indexOf**(Object o)`
- `int **lastIndexOf**(Object o)`
- `List<E> **subList**(int fromIndex, int toIndex)`

List implementations

Two general-purpose `List` implementations are **`ArrayList`** and **`LinkedList`**.

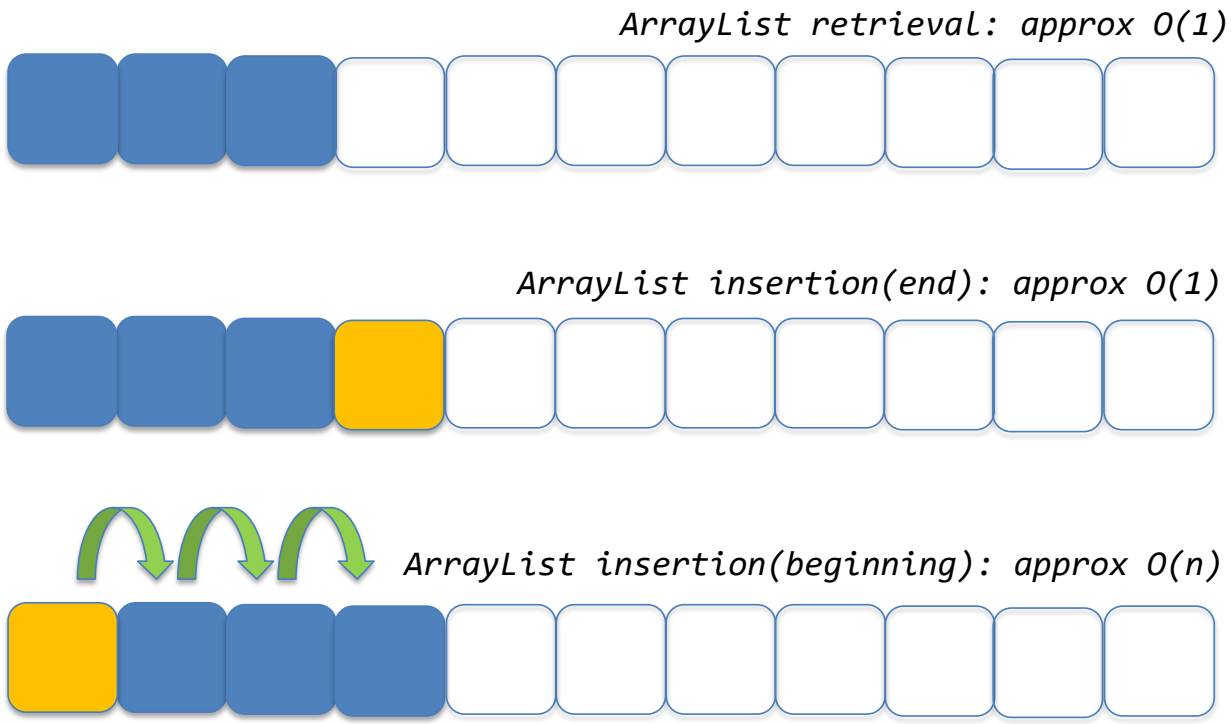
In most situations, `ArrayList` is the preferred general-purpose implementation because it provides:

- constant-time positional access
- efficient iteration
- good cache locality
- efficient bulk element movement when elements have to be shifted

`LinkedList` can be useful when elements are frequently inserted or removed through an iterator or when its Deque operations are required. However, indexed access is linear, and its node-based representation generally has worse memory locality and higher per-element overhead than `ArrayList`.

ArrayList implements **List**

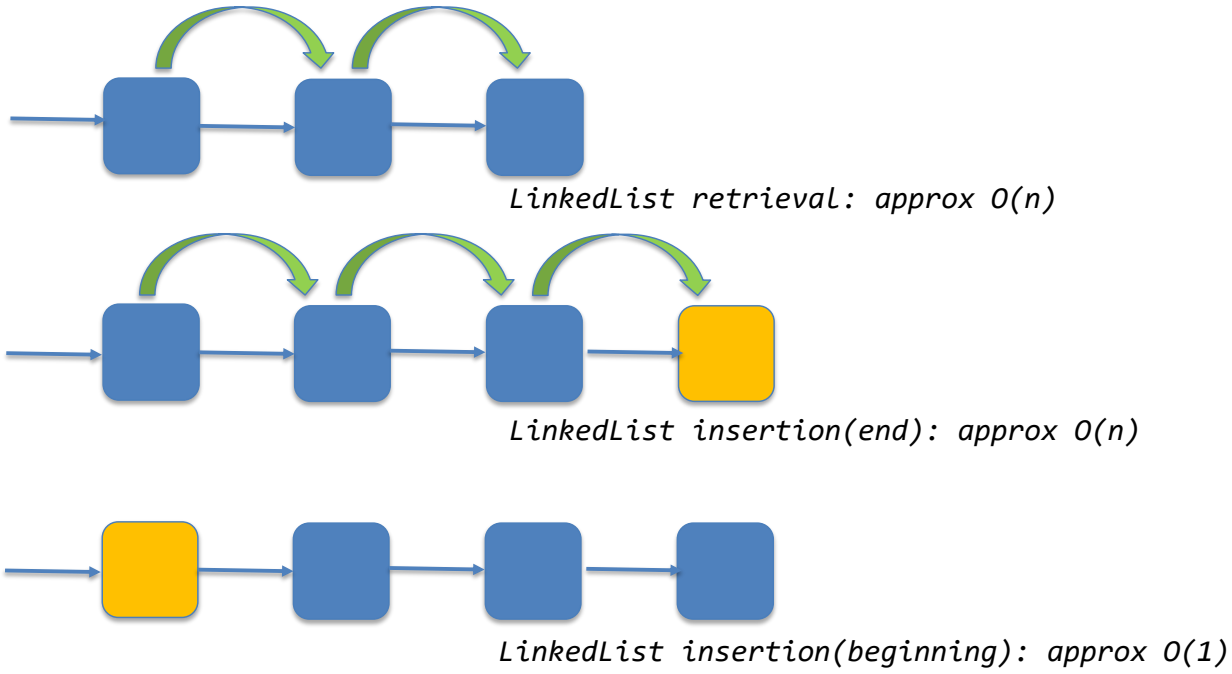
- `get(index)` → **O(1)**
- `add(obj)` at the end → **amortized O(1)**
- `add(index, obj)` → **O(n)** in the general case



LinkedList implements **List, Deque**

- `get(index)` → **O(n)**
- `add(obj)` at the end → **O(1)**
- `add(index, obj)` → **O(n)** because locating the position is linear

Once the relevant node is already known, insertion/removal can be performed in **O(1)**.



List initialization

Declaring an interface type keeps clients independent of one implementation. A replacement may require only one construction change **if** its ordering, null, mutation, and performance behavior meets the client's requirements:

```
/* plain, simple, long */
List<Integer> l = new ArrayList<>();
l.add(14);
l.add(73);
l.add(18);

/* more compact version (mutable) */
List<Integer> fromArray = new ArrayList<>(Arrays.asList(14, 73, 18));
List<Integer> fromValues = new ArrayList<>(List.of(14, 73, 18));

/* more compact version (unmodifiable) */
List<Integer> readOnly = List.of(14, 73, 18);
```

`List.of(...)` returns an **unmodifiable** list. In particular, operations such as add, remove, and set are not supported.

Mutability and ownership

Unmodifiable collections can still contain mutable objects. An unmodifiable **view** can also reflect changes made through its backing collection.

```
List<String> source = new ArrayList<>(List.of("Ada"));
List<String> view = Collections.unmodifiableList(source);
List<String> snapshot = List.copyOf(source);
source.add("Grace");
IO.println(view);
IO.println(snapshot);
```

Predict both outputs before running.

Check your prediction

The view prints `[Ada, Grace]`; the snapshot prints `[Ada]`. Neither permits mutation through its own API, but only the snapshot is independent of subsequent source-list structure changes. Neither deep-copies elements.

> **Note:** `List.of`, `Set.of`, and `Map.of` reject nulls. Their `copyOf` counterparts also reject null entries/elements. `Arrays.asList` is a fixed-size array-backed view, not an immutable snapshot.

Sequenced collections in Java 25

Note — modern API vocabulary: `SequencedCollection`, `SequencedSet`, and `SequencedMap`, introduced in Java 21 and available in Java 25, describe collections with a defined encounter order. Operations include access at both ends and `reversed()` views. `List` extends `SequencedCollection`; `LinkedHashSet` and `LinkedHashMap` support the corresponding sequenced interfaces.

```
List<String> names = new ArrayList<>(List.of("Ada", "Grace"));
IO.println(names.getFirst()); // Ada
IO.println(names.reversed()); // [Grace, Ada]
names.reversed().set(0, "Barbara");
IO.println(names); // [Ada, Barbara]: reversed is a view
```

Do not assume every implementation supports every optional mutation. A general `HashSet` or `HashMap` does not promise encounter order.

Manipulating Collections

Using Iterators

A collection should generally **not be structurally modified directly while it is being traversed by an iterator or enhanced `for` loop**.

For many standard collections, modifying the collection directly during iteration causes a `ConcurrentModificationException` on a subsequent iterator operation.

For example:

```
public static void main(String[] args) {

    List<Double> l =
        new LinkedList<>(List.of(10.8, 11.1, 13.2, 30.2));

    int count = 0;

    for (double i : l) {
        if (count == 1) {
            l.remove(count);
        }

        if (count == 2) {
            l.add(22.3);
        }

        count++;
    }

    // Typically results in ConcurrentModificationException
}
```

The important distinction is that **the iterator itself should be used when the operation is supported by the iterator**.

The `Iterator` interface provides forward traversal and supports removal of the last element returned by `next()`:

- `boolean **hasNext**()`
- `E **next**()`
- `void **remove**()`

```
public static void main(String[] args) {

    // With Iterator, remove all elements with a value of 11.1
    List<Double> l =
        new LinkedList<>(List.of(10.8, 11.1, 13.2, 30.2));

    for (Iterator<Double> i = l.iterator(); i.hasNext();) {
        double value = i.next();

        if (value == 11.1) {
            i.remove();
        }
    }

    IO.println(l);
    // [10.8, 13.2, 30.2]
}
```

> `ConcurrentModificationException` is part of the **fail-fast behavior** of many collection iterators. It should not be treated as a synchronization mechanism or as a guaranteed result for every collection implementation.

For straightforward conditional removal, `l.removeIf(value -> value == 11.1)` expresses the same exact-value predicate on the mutable list. The exact floating-point literal comparison here is a teaching choice, not a general tolerance policy.

The `ListIterator` interface extends `Iterator` and provides traversal in **both directions**, as well as modification operations:

- `boolean **hasNext**()`
- `boolean **hasPrevious**()`
- `E **next**()`
- `E **previous**()`
- `void **add**(E e)`
- `void **set**(E e)`
- `void **remove**()`
- `int **nextIndex**()`
- `int **previousIndex**()`

```java
public static void main(String[] args) {

    // With ListIterator, replace all elements
    // with value 11.1 with their double
    List<Double> l =
        new LinkedList<>(List.of(10.8, 11.1, 13.2, 30.2));

    for (ListIterator<Double> i = l.listIterator(); i.hasNext();) {
        double value = i.next();

        if (value == 11.1) {
            i.set(value * 2);
        }
    }

    IO.println(l);
    // [10.8, 22.2, 13.2, 30.2]
}
```

Using `set()` is the simplest way to replace the last element returned by `next()`.

Resources

- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/SequencedCollection.html
- https://www.baeldung.com/java-collections

- https://www.baeldung.com/java-hashmap
- https://dev.java/learn/api/collections-framework/arraylist-vs-linkedlist/

E4.2 — Sets

Set Interface

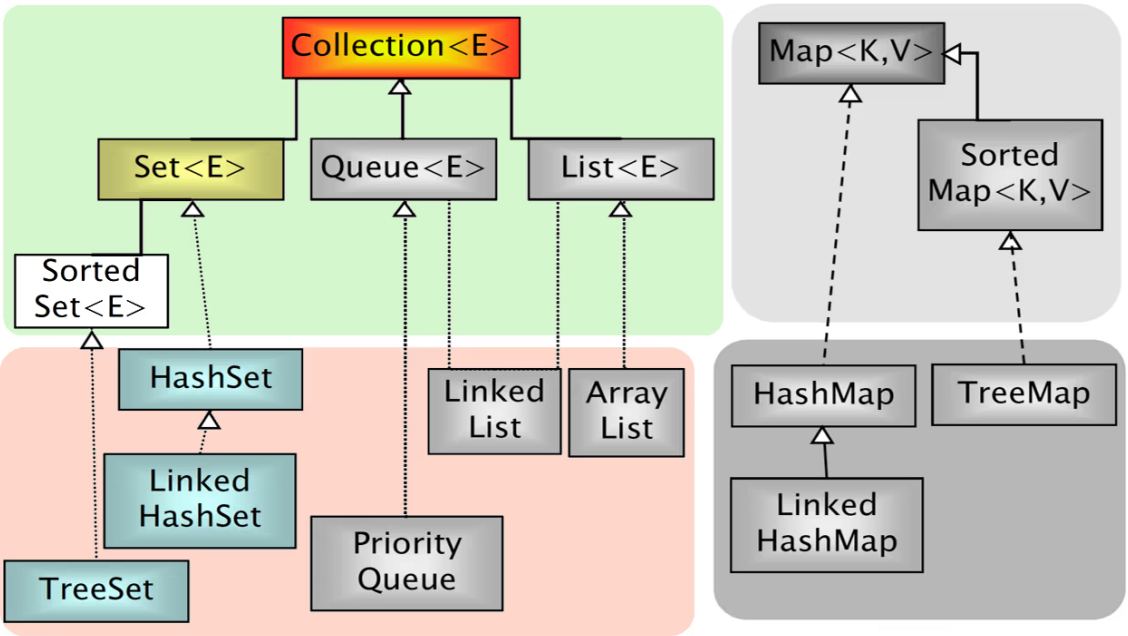
A **Set** is a collection that contains **no duplicate elements**.

More formally, a set cannot contain two elements e1 and e2 such that:

```
e1.equals(e2)
```

The Set interface models the mathematical set abstraction.

The Set interface itself does not guarantee a particular iteration order.



Set main methods

For ordinary set operations, use the inherited `Collection` methods; `Set` refines their contracts around uniqueness. It also provides static factories such as `of` and `copyOf`.

Its main characteristic is therefore the **uniqueness constraint**.

Set implementations

- **HashSet** implements **Set**
  - hash-table-based implementation
  - no guaranteed iteration order
  - typically provides O(1) average-time basic operations
- **LinkedHashSet** extends **HashSet**
  - maintains a linked structure between entries
  - preserves insertion order during iteration
- **TreeSet** implements **NavigableSet**, which extends **SortedSet** and Set
  - based on a balanced search tree
  - maintains elements according to a defined ordering
  - typically provides O(log n) basic operations
  - ordering can be defined by natural ordering or a `Comparator`

Set initialization

```java
public static void main(String[] args) {
    List<String> l = List.of(
        "Nicola", "Denise", "Agata", "Marzia", "Agata"
    );

    Set<String> hs = new HashSet<>(l);
    IO.println(hs);
    // Order is unspecified

    Set<String> lhs = new LinkedHashSet<>(l);
    IO.println(lhs);
    // [Nicola, Denise, Agata, Marzia]

    Set<String> ts = new TreeSet<>(l);
    IO.println(ts);
    // [Agata, Denise, Marzia, Nicola]
}
```

HashSet does not preserve insertion order, so its printed order must **not** be relied upon.

LinkedHashSet preserves insertion order.

TreeSet maintains its elements according to its ordering.

```java
/* more compact version (unmodifiable) */
Set<String> set = Set.of("Nicola", "Denise", "Agata", "Marzia");
```

> **Important:** `Set.of(...)` does not allow duplicate elements. Passing duplicate elements causes an `IllegalArgumentException`.

Removing duplicate elements from a List

```
public static void main(String[] args) {
    List<String> l = List.of(
        "Nicola", "Denise", "Agata", "Marzia", "Agata"
    );

    List<String> noDuplicatesUnordered =
        new ArrayList<>(new HashSet<>(l));

    IO.println(noDuplicatesUnordered);
    // Order is unspecified

    List<String> noDuplicatesOrdered =
        new ArrayList<>(new LinkedHashSet<>(l));

    IO.println(noDuplicatesOrdered);
    // [Nicola, Denise, Agata, Marzia]
}
```

Union and intersection between Sets

```
public static void main(String[] args) {
    Set<Integer> s1 = new HashSet<>(List.of(1, 2, 3, 4, 5));
    Set<Integer> s2 = new HashSet<>(List.of(4, 5, 6, 7, 8));

    Set<Integer> union = new HashSet<>(s1);
    Set<Integer> intersection = new HashSet<>(s1);

    union.addAll(s2);
    intersection.retainAll(s2);

    IO.println(union);
    // Contains: 1, 2, 3, 4, 5, 6, 7, 8

    IO.println(intersection);
    // Contains: 4, 5
}
```

HashSet vs TreeSet

- HashSet does not guarantee any particular iteration order, whereas TreeSet maintains its elements according to an ordering.

- `HashSet` generally provides **O(1) average-time** `add`, `remove`, and `contains`.
- `TreeSet` generally provides **O(log n)** `add`, `remove`, and `contains`.
- `TreeSet` provides additional navigational operations such as:
  - `first()`
  - `last()`
  - `ceiling()`
  - `floor()`
  - `higher()`
  - `lower()`

`HashSet` permits a single `null` element. A `TreeSet` using natural ordering generally cannot store `null`, because `null` cannot be compared with ordinary elements. A custom comparator may define a different policy.

Review: a generated deduplication replaces `LinkedHashSet` with `HashSet` because it is "faster." Is input encounter order part of the requirement? Tests should assert the promised order, not the accidental order of one hash-table run.

Testing uniqueness and promised order

Continue the E2.8 practice project with the **name-based `Person` class tested in E3.6**. Save this as `src/test/java/com/example/SetTest.java`:

```
package com.example;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class SetTest {
    @Test
    void hashSetRecognizesEqualObjectsAsOneElement() {
        Set<Person> people = new HashSet<>();

        assertTrue(people.add(new Person("Alice")));
        assertFalse(people.add(new Person("Alice")));

        assertEquals(1, people.size());
        assertTrue(people.contains(new Person("Alice")));
        assertFalse(people.contains(new Person("Bob")));
    }

    @Test
    void linkedHashSetKeepsFirstEncounterOrderWhenRemovingDuplicates() {
```

```
        List<String> input = List.of("Nicola", "Denise", "Agata", "Denise");

        List<String> actual = new ArrayList<>(new LinkedHashSet<>(input));

        assertEquals(List.of("Nicola", "Denise", "Agata"), actual);
        assertEquals(List.of("Nicola", "Denise", "Agata", "Denise"), input);
    }
}
```

The first test checks uniqueness and lookup using **different but equal instances**, not repeated use of one reference. Together with the hash-code assertion in E3.6, it helps detect an inconsistent `equals`/`hashCode` implementation. Do not mutate the name of a `Person` while it is in this set.

The second test checks an ordering requirement, so it deliberately compares **lists**: set equality ignores iteration order. For a `HashSet`, assert membership or set equality instead of assuming a printed order. Neither test inspects internal buckets or capacity.

Practice: add a test that removes an element through a different but equal `Person` instance, then verifies that lookup fails and the set is empty.

Resources

E4.3 — Queues and Deques

Queue/Deque Interface

Queue

A **`Queue`** is a collection designed to hold elements prior to processing.

Queues typically follow a **FIFO (first-in, first-out)** policy, although the interface itself does not require FIFO behavior. For example, `PriorityQueue` orders elements according to priority.

Queue operations are available in two variants:

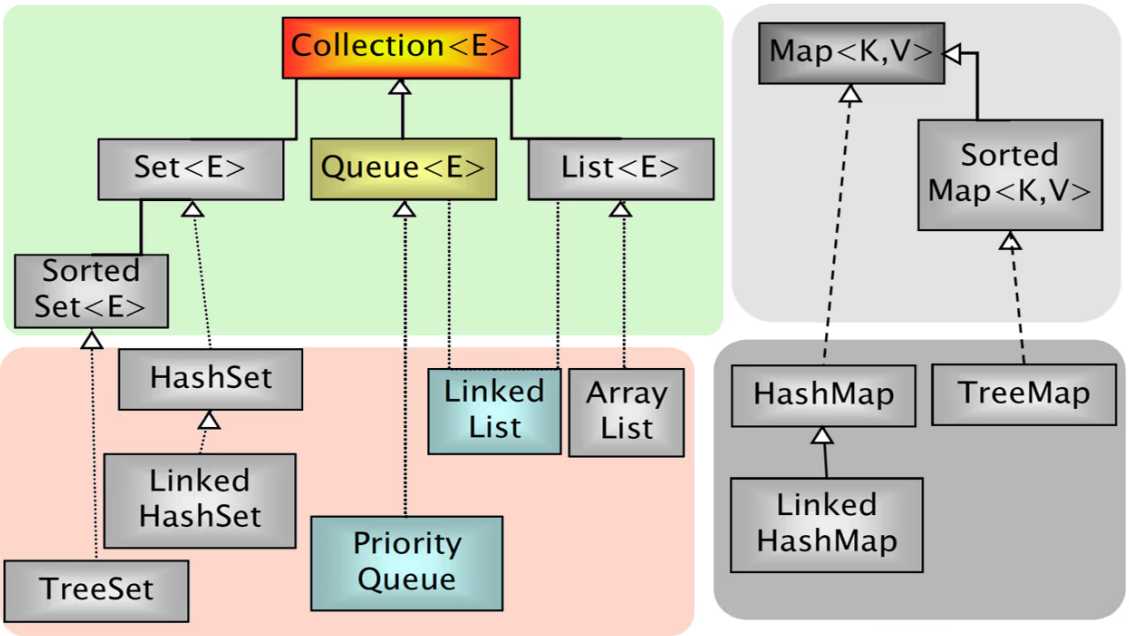
- one variant throws an exception when the operation fails
- the other returns a special value such as `null` or `false`

Deque

A **`Deque`** ("double-ended queue") supports insertion and removal of elements at **both ends**.

It can therefore be used as:

- a queue
- a stack



Queue main methods

	Throws exception	Special value
Insert	`add(e)`	`offer(e)`
Remove	`remove()`	`poll()`
Examine	`element()`	`peek()`

Deque main methods

	Exception (H)	Special value (H)	Exception (T)	Special value (T)
Insert	`addFirst(e)`	`offerFirst(e)`	`addLast(e)`	`offerLast(e)`
Remove	`removeFirst()`	`pollFirst()`	`removeLast()`	`pollLast()`
Examine	`getFirst()`	`peekFirst()`	`getLast()`	`peekLast()`

Here **H** means *head* and **T** means *tail*.

Queue/Deque Implementations

Use `ArrayDeque` as a usual starting point for a single-threaded FIFO queue or stack. It is not thread-safe. Concurrent queues support access by multiple threads; blocking queues additionally provide operations that wait for an element or available capacity. Choose those variants only when their coordination contract is required.

- **`LinkedList`** implements **List, Deque**
  - preserves insertion order as a `List`
  - supports queue and deque operations
  - no fixed capacity restriction
- **`ArrayDeque`** implements **Deque**
  - array-based deque
  - efficient insertion/removal at both ends
  - no fixed capacity restriction
  - does not permit `null` elements
- **`PriorityQueue`** implements **Queue**
  - elements are ordered according to their natural ordering or a supplied `Comparator`
  - the head is the least element according to that ordering
  - iteration is **not guaranteed to produce sorted order**
  - no fixed capacity restriction

- **`LinkedBlockingDeque`** implements **`BlockingDeque`**
  - supports blocking operations
  - optionally bounded by a capacity
- **`ArrayBlockingQueue`** implements **`BlockingQueue`**
  - array-based
  - bounded by a fixed capacity
- **`ConcurrentLinkedDeque`** implements **`Deque`**
  - thread-safe, non-blocking deque
  - designed for concurrent access

Queue/Deque Example

```
public class QueueExample {

    public static final List<Integer> VALUES =
        List.of(5, 1, 2, 4, 3);

    public static void fillQueue(Queue<Integer> queue) {
        for (Integer integer : VALUES) {
            queue.add(integer);
        }
    }

    public static void emptyQueue(Queue<Integer> queue) {
        IO.println(queue.getClass().getName());

        while (!queue.isEmpty()) {
            IO.print(queue.remove() + ", ");
        }

        IO.println();
    }

    public static void main(String[] args) {
        Queue<Integer> fifo = new LinkedList<>();
        Queue<Integer> fifoArray = new ArrayDeque<>();
        Queue<Integer> pqueue = new PriorityQueue<>();

        fillQueue(fifo);
        fillQueue(fifoArray);
        fillQueue(pqueue);

        emptyQueue(fifo);      // 5, 1, 2, 4, 3
        emptyQueue(fifoArray); // 5, 1, 2, 4, 3
        emptyQueue(pqueue);    // 1, 2, 3, 4, 5
    }
}
```

The `PriorityQueue` example produces sorted values when repeatedly removing the head. This does **not** mean that iterating directly over a `PriorityQueue` produces sorted order.

Resources

E4.4 — Maps

Map Interface

A **`Map`** stores associations between **keys and values**.

A map:

- cannot contain duplicate keys
- associates each key with at most one value
- can contain duplicate values
- provides views of its keys, values, and key-value mappings

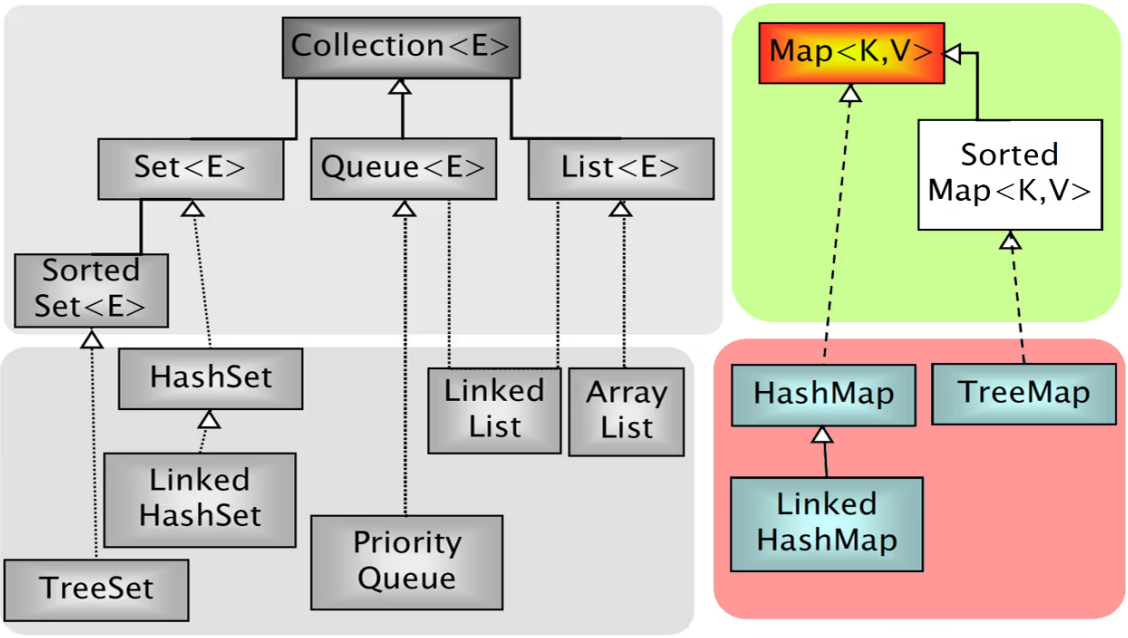
The main views are:

- `keySet()` → `Set<K>`
- `values()` → `Collection<V>`
- `entrySet()` → `Set<Map.Entry<K, V>>`

Unlike `Collection`, `Map` does not represent a collection of individual elements and therefore does not extend `Collection`.

The views are backed by the map: supported removals through them remove mappings. A `get` result of null can mean an absent key **or** a key mapped to null in implementations such as `HashMap`; use `containsKey` when that distinction matters.

Core: keys must remain stable with respect to equality and hashing while stored in a hash map. Mutating an equality-significant field can make a stored mapping unreachable through an ordinary lookup.



Map main methods

For `Map<K, V>`:

- `V **put**(K key, V value)`
- `V **get**(Object key)`
- `V **remove**(Object key)`
- `boolean **containsKey**(Object key)`
- `boolean **containsValue**(Object value)`
- `Set<K> **keySet**()`
- `Collection<V> **values**()`
- `Set<Map.Entry<K, V>> **entrySet**()`
- `int **size**()`
- `boolean **isEmpty**()`
- `void **clear**()`

Map Implementations

- **`HashMap`** implements **Map**
  - hash-table-based
  - no guaranteed iteration order
  - typically provides O(1) average-time basic operations
- **`LinkedHashMap`** extends **HashMap**
  - maintains a linked ordering between entries
  - normally preserves insertion order during iteration
  - can also be configured for access order

- **TreeMap** implements **NavigableMap**, which extends `SortedMap` and `Map`
  - tree-based
  - maintains keys according to an ordering
  - typically provides O(log n) basic operations

```
public static void main(String[] args) {
    Map<Integer, String> src;

    src = new HashMap<>();
    src.put(77, "Nicola");
    src.put(17, "Marzia");
    src.put(22, "Agata");

    IO.println(src);
    // Order is unspecified

    src = new LinkedHashMap<>();
    src.put(77, "Nicola");
    src.put(17, "Marzia");
    src.put(22, "Agata");

    IO.println(src);
    // {77=Nicola, 17=Marzia, 22=Agata}

    src = new TreeMap<>();
    src.put(77, "Nicola");
    src.put(17, "Marzia");
    src.put(22, "Agata");

    IO.println(src);
    // {17=Marzia, 22=Agata, 77=Nicola}
}
```

Map initialization

```
public static void main(String[] args) {

    /* more compact version (mutable) */
    Map<String, Integer> m =
        new HashMap<>(
            Map.of(
                "Agata", 2,
                "Marzia", 3,
                "Denise", 4,
                "Nicola", 1
            )
        );

    /* note well:
```

```
       Map.of(...) requires unique keys.

       The following code throws an IllegalArgumentException
       because "Agata" appears twice.
    */

    // Run separately to observe the failure:
    // Map.of("Agata", 2, "Agata", 3);

    /* more compact version (unmodifiable) */
    Map<String, Integer> immutable =
        Map.of(
            "Agata", 2,
            "Marzia", 3,
            "Denise", 4,
            "Nicola", 1
        );
}
```

`Map.of(...)` returns an **unmodifiable map**.

For an existing mutable map, put with an equal key replaces its value instead of creating a second mapping. Factory rejection of duplicate input and ordinary map update are different operations.

Map Example

```
public static void main(String[] args) {

    Map<String, Integer> m =
        new HashMap<>(
            Map.of(
                "Agata", 2,
                "Marzia", 3,
                "Denise", 4,
                "Nicola", 1
            )
        );

    // contains key
    if (m.containsKey("Denise")) {
        IO.println(m.get("Denise"));
    }

    // looping keys and accessing values
    Set<String> keys = m.keySet();

    for (String key : keys) {
```

```
        IO.println(key + " -> " + m.get(key));
    }

    // looping values
    Collection<Integer> values = m.values();

    for (int value : values) {
        IO.println(value);
    }

    // looping entries
    for (Map.Entry<String, Integer> entry : m.entrySet()) {
        IO.println(entry.getKey());
        IO.println(entry.getValue());
    }
}
```

When both the key and the value are needed, iterating over `entrySet()` is generally preferable to iterating over `keySet()` and performing an additional `get()`.

Resources

E4.5 — Collection Ordering and Search

Using `java.util.Collections`

`java.util.Collections` is a utility class containing static methods for operating on collections.

It can be thought of as a counterpart to `java.util.Arrays` for collections.

Common operations include:

- `sort()`
- `binarySearch()`
- `shuffle()`
- `reverse()`
- `rotate()`
- `min()`
- `max()`

```java
public static void main(String[] args) {

    ArrayList<String> l =
        new ArrayList<>(
            List.of("Nicola", "Agata", "Marzia", "Agata")
        );

    Collections.sort(l);
    IO.println(l);
    // [Agata, Agata, Marzia, Nicola]

    Collections.reverse(l);
    IO.println(l);
    // [Nicola, Marzia, Agata, Agata]

    Collections.shuffle(l);
    IO.println(l);
    // Order is random

    Collections.rotate(l, 1);
    IO.println(l);
    // Depends on the order before rotation

    Collections.sort(l);
    IO.println(l);
    // [Agata, Agata, Marzia, Nicola]
```

```
    // The list must be sorted according to the same ordering
    // used by binarySearch().
    IO.println(
        Collections.binarySearch(l, "Nicola")
    );
    // 3

    IO.println(
        Collections.binarySearch(l, "Zane")
    );
    // -5
}
```

For `binarySearch()`:

- the list must be sorted according to the same ordering
- if the element is found, its index is returned
- if it is not found, the result is `-(insertion point) - 1`

For example, if `"Zane"` would be inserted at index 4:

```
-(4) - 1 = -5
```

Sorting Collections

Objects can define a **natural ordering** by implementing the `Comparable<T>` interface.

```
public interface Comparable<T> {
    int compareTo(T obj);
}
```

The `compareTo()` method returns:

- `< 0` if `this` precedes `obj`
- `== 0` if `this` and `obj` are considered equivalent for ordering
- `> 0` if `this` follows `obj`

A collection of T can be sorted according to its natural ordering when T implements `Comparable<? super T>`.

Many standard Java types, such as `String` and the numeric wrapper classes, implement `Comparable`.

Sorting Example

The example below defines the natural ordering for the Person class as the alphabetical order of surnames.

```
public class Person implements Comparable<Person> {

    protected String name;
    protected String lastname;
    protected int age;

    public Person(String name, String lastname, int age) {
        this.name = Objects.requireNonNull(name);
        this.lastname = Objects.requireNonNull(lastname);
        this.age = age;
    }

    @Override
    public int compareTo(Person p) {
        // order by surname
        return lastname.compareTo(p.lastname);
    }
}
```

Eventually, people with the same surname can be sorted by name as well.

```
public class Person implements Comparable<Person> {

    protected String name;
    protected String lastname;
    protected int age;

    public Person(String name, String lastname, int age) {
        this.name = Objects.requireNonNull(name);
        this.lastname = Objects.requireNonNull(lastname);
        this.age = age;
    }

    @Override
    public int compareTo(Person p) {

        // order by surname
        int cmp = lastname.compareTo(p.lastname);

        if (cmp == 0) {
            // if surnames are equal, order by name
            cmp = name.compareTo(p.name);
        }

        return cmp;
    }
}
```

> The comparison must be consistent with the intended ordering. If `compareTo()` returns `0`, sorted collections such as `TreeSet` and `TreeMap` may treat the objects as equivalent for ordering purposes.

These examples inherit identity equality but order by names: they deliberately demonstrate that ordering equivalence is not automatically object equality. For a sorted set implementing the usual `Set` equality contract, choose an ordering consistent with `equals`.

Comparator

Sometimes a class has a natural ordering, but an application needs to sort its objects in a different way.

For example, `Person` could have a natural ordering by surname, while a particular operation might require sorting by age.

The **`Comparator<T>`** interface allows an ordering to be defined independently of the class being compared.

```
public interface Comparator<T> {
    int compare(T obj1, T obj2);
}
```

A `Comparator` can be implemented by a separate class:

```
public class SortByAge implements Comparator<Person> {

    @Override
    public int compare(Person o1, Person o2) {
        return Integer.compare(o1.age, o2.age);
    }
}
```

Using `Integer.compare(...)` is preferable to subtracting the two ages directly because subtraction can overflow for arbitrary integer values.

A `Comparator` can also be provided anonymously where it is needed:

```
public static void main(String[] args) {

    ArrayList<Person> l = new ArrayList<>();

    l.add(new Person("Mario", "Rossi", 68));
    l.add(new Person("Luca", "Bianchi", 28));
    l.add(new Person("Carlo", "Antoni", 34));

    // natural ordering (Comparable)
```

```
    Collections.sort(l);

    // Comparator with named class
    Collections.sort(l, new SortByAge());

    // Comparator with anonymous class
    Collections.sort(l, new Comparator<Person>() {

        @Override
        public int compare(Person o1, Person o2) {
            return Integer.compare(o1.age, o2.age);
        }
    });

    // Comparator with lambda
    Collections.sort(l, (o1, o2) ->
        Integer.compare(o1.age, o2.age)
    );
}
```

A lambda expression is often the most concise way to provide a comparator when the comparison logic is simple.

For example:

```
l.sort(Comparator.comparingInt(person -> person.age));
```

Resources

E5.1 — Exceptions and Propagation

An exception is an object used to report an exceptional outcome and transfer control to a matching handler. It is thrown during execution; a compilation error is a different kind of failure.

Put statement fragments inside `main` and method declarations inside a class. File examples need the relevant `java.io`, `java.nio.file`, `java.nio.charset`, or `java.util` imports. Blocks labeled as pseudocode explain control flow, not compilable Java.

Why exceptions?

Imagine we need to write a method for loading a file in memory:

First approach (ignore error checking)

- Short
- Readable
- Failure behavior is unspecified
- **Not dependable**

```
void loadFile()  {
  open file;
  determine file size;
  allocate memory;
  read file into memory;
  close file;
}
```

Second approach (error checking with magic numbers)

- **Long**
- **Obscure**
- Can make failures explicit, if callers check the result
- Cleanup still needs a policy

```
int loadFile()  {
  open file;
  if (operationFailed)
    return -1;

  determine file size;
  if (operationFailed)
    return -2;
```

```
  allocate memory;
  if (operationFailed)
    return -3;

  read file into memory;
  if (operationFailed)
    return -4;

  close file;
  if (operationFailed)
    return -5;
  return 0;
}
```

Third approach (error checking with exceptions)

- Reusable
- Dependable
- Readable

```
void loadFile() {
  try {
      open file;
      determine file size;
      allocate memory;
      read file into memory;
      close file;
  } catch (fileOpenFailed) {
      doSomething;
  } catch(determineSizeFailed) {
    doSomething;
  } catch (memoryAllocationFailed) {
      doSomething;
  } catch (readFailed) {
      doSomething;
  } catch (fileCloseFailed) {
      doSomething;
  }
}
```

These sketches illustrate reporting styles, not complete resource-safe implementations. Exceptions alone do not guarantee cleanup: owned resources must be closed on both success and failure, typically with try-with-resources. A named result type can also be appropriate for expected outcomes; error codes are not intrinsically wrong.

Exceptions examples

ArithmeticException

Suppose you are writing a program that reads two integers from the standard input and then outputs the result of the integer division of the first number by the second one. Look at the code below.

```
import java.util.Scanner;

public class ArithmeticExceptionDemo {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int a = scanner.nextInt();
        int b = scanner.nextInt();

        // ArithmeticException here if b == 0
        IO.println(a / b);
        IO.println("finished");
    }
}
```

NumberFormatException

Another situation to consider is when you are trying to convert a string into an integer number. If the string has an unsuitable format, the code will throw an exception.

The following program reads a line from the standard input and then outputs the number that follows it.

```
import java.util.Scanner;

public class NumberFormatExceptionDemo {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();

        // NumberFormatException here if user presses 121a
        int number = Integer.parseInt(input);
        IO.println(number + 1);
    }
}
```

ArrayIndexOutOfBoundsException

This is a fairly common exception that occurs while working with arrays. It is caused by attempting to access a non-existent element of the array.

```
int[] array = { 1, 2, 3 }; // an array of ints

int n1 = array[2];  // n1 is 3
int n2 = array[3];  // ArrayIndexOutOfBoundsException
int n3 = array[-1]; // ArrayIndexOutOfBoundsException
```

NullPointerException (NPE)

`null` is a reference value identifying no object, not an uninitialized local variable. Dereferencing it or implicitly unboxing it can cause `NullPointerException`. Choose an explicit absence policy; checking null is not always preferable to rejecting an invalid argument at a boundary.

```
String someString = null; // a reference type can be null

int size = someString.length(); // NullPointerException (NPE)
```

Hierarchy of exceptions

Exceptions are considered objects of special classes organized into a class hierarchy. The following picture illustrates the simplified hierarchy of exceptions:

```
Throwable
├─ Error (unchecked)
│    ├─ OutOfMemoryError
│    ├─ StackOverflowError
│    └─ ... (many more)
└─ Exception
     ├─ RuntimeException (unchecked)
     │    ├─ NullPointerException
     │    ├─ ArrayIndexOutOfBoundsException
     │    ├─ ArithmeticException
     │    └─ ... (many more)
     ├─ IOException (checked)
     │    └─ FileNotFoundException
     ├─ SQLException (checked)
     └─ ... (other checked exception classes)
```

The `Throwable` class has several important subclasses:

- **`Error`** – represents serious problems in the JVM, e.g., `OutOfMemoryError`, `StackOverflowError`.

- **`Exception`** – represents application-level events that can be handled, e.g., `SQLException`, `IOException`.
- **`RuntimeException`** – a subclass of `Exception` representing **unchecked exceptions**, e.g., `ArithmeticException`, `NumberFormatException`, `NullPointerException`.

Checked exceptions

Checked exceptions are exception types other than `RuntimeException` and its subclasses, and other than the `Error` branch. The compiler requires eligible checked failures to be caught or declared. Whether a failure is recoverable depends on the caller and context, not just on this classification.

With checked exceptions, the compiler forces the programmer to make a choice:

Handle the exception using a `try-catch` block:

```
public static void displayFile() {
    try {
        String text = java.nio.file.Files.readString(java.nio.file.Path.of("file.txt"));
        IO.println(text);
    } catch (java.io.IOException e) {
        System.err.println("Cannot display file.txt: " + e.getMessage());
    }
}
```

Propagate it by declaring `throws` in the calling method.

```
public static String readTextFromFile() throws java.io.IOException {
    return java.nio.file.Files.readString(java.nio.file.Path.of("file.txt"));
}
```

The first method is a console boundary that reports failure; the second leaves the decision to its caller. Returning an empty string after an I/O failure would make “empty file” indistinguishable from “could not read.” `Files.readString` owns and closes the underlying file operation.

Unchecked Exceptions

Java introduced **unchecked exceptions** to address situations where **forcing the programmer to declare or handle every possible error would be cumbersome or unnecessary**.

```
int[] array = { 1, 2, 3 }; // an array of ints

int n2 = array[3];  // ArrayIndexOutOfBoundsException is unchecked
```

`RuntimeException` and its subclasses are unchecked exceptions; `Error` and its subclasses are also unchecked throwable types. The compiler does not require catching or declaring them. That does not make them safe to ignore or imply that catching every `Throwable` is sensible.

```java
public static Long convertStringToLong(String str) {
    return Long.parseLong(str); // may throw NumberFormatException
}
```

This code **compiles successfully** without using the `throws` keyword because `NumberFormatException` is **unchecked**.

Throwing exceptions

The throw keyword

Any object of the `Throwable` class and all its subclasses can be thrown using the **throw** statement.

```java
public class Main {
    public static void main(String[] args) {
        throw new RuntimeException("Something's bad.");
    }
}
```

With no handler in this example, the main thread terminates and the default uncaught-exception handling prints a stack trace:

```
Exception in thread "main" java.lang.RuntimeException: Something's bad.
    at Main.main(Main.java:3)
```

Throwing checked exceptions

```java
public static String readTextFromFile(String path) throws java.io.IOException {
    if (path.isBlank()) {
        throw new IllegalArgumentException("Blank path");
    }
    return java.nio.file.Files.readString(java.nio.file.Path.of(path));
}
```

The `throws` keyword following the method parameters is required since an `IOException` is a checked exception. Otherwise, the code won't compile.

If a method throws two or more checked exceptions, they must be separated by a comma in the declaration:

```
public static void method() throws ExceptionType1, ExceptionType2, ExceptionType3 {
  // ...
}
```

If a method is declared as throwing an exception, it can also throw any subclass of the specified exception. In the example below, `FileNotFoundException` does not require a specific annotation because it is a subclass of `IOException`:

```
public static String readLine(String filename) throws IOException {
  try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
    return reader.readLine(); // null at end-of-file; IOException on read failure
  }
}
```

Throwing unchecked exceptions

The `Account` class contains the method called `deposit`, that adds the specified amount to the current balance. If the amount is not positive or exceeds the limit, the method throws an `IllegalArgumentException`.

```
class Account {

    private long balance = 0;

    public void deposit(long amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Incorrect sum " + amount);
        }

        if (amount >= 100_000_000L) {
            throw new IllegalArgumentException("Too large amount");
        }

        balance = Math.addExact(balance, amount);
    }

    // ...
}
```

If a method throws an unchecked exception such as `IllegalArgumentException`, the keyword `throws` is not required in the method declaration.

```
public class AccountUtils {

  /**
   * Tries to deposit a given amount into the account.
   * Prints a message whether the deposit succeeded or failed.
   */
  public static void safeDeposit(Account account, long amount) {
    try {
      account.deposit(amount);
      IO.println("Deposit successful: " + amount);
    } catch (IllegalArgumentException e) {
      IO.println("Deposit failed: " + e.getMessage());
```

```
    }
  }

  public static void main(String[] args) {
    Account myAccount = new Account();

    // Test deposits
    safeDeposit(myAccount, 1000);          // valid
    safeDeposit(myAccount, -50);           // invalid: negative
    safeDeposit(myAccount, 200_000_000L);  // invalid: too large
  }
}
```

When to throw an exception?

Technically, throwing an exception is a rather straightforward task. But the question is, when do you need to do this? The answer is not always obvious.

Throw when the method cannot fulfill its contract and its API represents that failure with an exception. Broken preconditions are one case; external I/O failures, timeouts, and unavailable services can also occur with valid arguments.

- A date parser can throw `DateTimeParseException` for input that does not match its supported date format.
- A method reading a non-existing file throws a `FileNotFoundException`.

Choose a relevant standard type, such as `IllegalArgumentException`, rather than a generic `Exception` when possible. There is no standard `java.lang.InvalidArgumentException`. Expected absence may instead use a documented empty result or `Optional`; do not use exceptions as routine loop control.

Testing expected exceptions

In E2.8 — Unit Testing with JUnit, tests checked returned values. An exception can also be the expected outcome of a method's contract.

Save the following as `src/test/java/com/example/ParsingTest.java` in the practice project from E2.8:

```
package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ParsingTest {
    @Test
    void parseIntRejectsNonNumericText() {
        assertThrows(NumberFormatException.class,
```

```
                () -> Integer.parseInt("12x"));
    }
}
```

- `NumberFormatException.class` identifies the expected exception type.
- `() -> Integer.parseInt("12x")` supplies an action for JUnit to execute. This is a lambda expression; its general form is covered in E7. For now, read it as “run this call when JUnit asks.”
- The assertion passes if that action throws the expected type or a subtype; it fails if nothing is thrown or a different type is thrown. Use `assertThrowsExactly` when the contract requires the exact type.

Keep the action narrow so an unrelated failing operation cannot satisfy the assertion. Do not catch and ignore the exception inside the test: the test must fail if the expected rejection does not occur. Test valid input with ordinary value assertions too.

Resources

E5.2 — Error Handling and Contracts

Handling exceptions

The try-catch statement

```
try {
    // code that may throw an exception
} catch (Exception e) {
    // code for handling the exception
}
```

The `try` block is used to wrap the code that may throw an exception.

The `catch` block is a handler for the specified type of exception and all of its subclasses. This block is executed when an exception of the corresponding type occurs in the `try` block.

```
public static void main(String[] args) {
  IO.println("before the try-catch block"); // it will be printed

  try {
    IO.println("inside the try block before an exception"); // it will be printed
    IO.println(2 / 0); // it throws ArithmeticException
    IO.println("inside the try block after the exception"); // it won't be printed
  } catch (ArithmeticException e) {
    IO.println("Division by zero!"); // it will be printed
  }

  IO.println("after the try-catch block"); // it will be printed

}
```

The output:

```
before the try-catch block
inside the try block before an exception
Division by zero!
after the try-catch block
```

The program does not print "`inside the try block after the exception`" since the `ArithmeticException` aborted the normal flow of the execution. Instead, it executes the print statement in the `catch` block. After completion of the `catch` block, the program executes the next statement (printing "`after the try-catch block`") without returning to the `try` block again.

Catching multiple exceptions

Java supports the use of several handlers inside the same `try` block.

```
try {
    // code that throws exceptions
} catch (IOException e) {
    // handling the IOException and its subclasses
} catch (Exception e) {
    // handling the Exception and its subclasses
}
```

The more specialized handlers (like `IOException`) must be written before the more general ones (like `Exception`).

Catch only failures that this layer can meaningfully handle, translate, or report. A broad catch that substitutes a plausible default may hide a bug.

Read a stack trace and preserve the cause

Start with the exception type and message, locate the relevant application frame, and follow any `Caused by` chain. Source line numbers depend on the actual file.

```java
public static int parseCount(String input) {
    try {
        return Integer.parseInt(input);
    } catch (NumberFormatException cause) {
        throw new IllegalArgumentException("Invalid count: " + input, cause);
    }
}
```

This example adds domain context while preserving the original cause. In production, avoid putting secrets or sensitive payloads in exception messages. Logging and rethrowing at every layer can duplicate the same failure many times.

Custom exceptions

To create a custom exception you need to extend the `Exception` (checked) or `RuntimeException` (unchecked) classes.

```java
public class MalformedRequestException extends Exception {
  public MalformedRequestException() {
  }

  public MalformedRequestException(String message) {
    super(message);
```

```
  }

  public MalformedRequestException(String message, Throwable cause) {
    super(message, cause);
  }

  public MalformedRequestException(Throwable cause) {
    super(cause);
  }

  public MalformedRequestException(String message, Throwable cause, boolean
   ↪  enableSuppression, boolean writableStackTrace) {
    super(message, cause, enableSuppression, writableStackTrace);
  }
}
```

In the example above, a new class of exceptions is declared. It is a checked exception because it extends the `Exception` class. The declared class has five constructors for creating instances, and they call the corresponding constructor of the base class. They can be all auto-generated in IntelliJ.

Now, we can throw an instance of the class:

```
public static void someMethod() throws MalformedRequestException {
    throw new MalformedRequestException("Something bad");
}
```

Best practices for custom exceptions

- In general, **it is advisable to use standard exceptions whenever possible**:
  - Standard exceptions are widely known by other programmers. One can understand the type of problem just by looking at the name of the exception.
  - By opting for standard exceptions, you follow the reusability principle. It makes your code clearer and more professional.

Exceptions and loops

For errors affecting a single iteration (or items of a collection!), the try-catch blocks is nested in the loop. In case of exception the execution goes to the catch block and then proceed with the next iteration.

```
while (something) {
  try {
    // potential exceptions
  } catch (Exception e) {
    //  discard iteration (continues the loop)
```

```
  }
}
```

For errors compromising the whole loop, the loop is nested within the try block. In case of exception the execution goes to the catch block, thus exiting the loop.

```
try {
  while (something) {
    //  potential exceptions
  }
} catch (Exception e) {
  // discard whole loop
}
```

Testing rejection without corrupting state

An error contract describes both **how failure is reported** and **what state remains afterward**. Reuse the validated `Car` class from E3.2 and the `assertThrows` syntax introduced in E5.1.

Save this as `src/test/java/com/example/CarContractTest.java` in the practice project:

```java
package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CarContractTest {
    @Test
    void constructorRejectsNegativePrice() {
        assertThrows(IllegalArgumentException.class,
                () -> new Car("AB123CD", -1, 90.0));
    }

    @Test
    void constructorRejectsNullPlate() {
        assertThrows(IllegalArgumentException.class,
                () -> new Car(null, 12_000, 90.0));
    }

    @Test
    void rejectedPriceUpdatePreservesPreviousState() {
        Car car = new Car("AB123CD", 12_000, 90.0);

        assertThrows(IllegalArgumentException.class,
                () -> car.changePrice(-1));

        assertEquals(12_000, car.getPrice());
        assertEquals("AB123CD", car.getLicensePlate());
```

```
    }

    @Test
    void rejectedPlateUpdatePreservesPreviousState() {
        Car car = new Car("AB123CD", 12_000, 90.0);

        assertThrows(IllegalArgumentException.class,
                () -> car.setLicensePlate("invalid"));

        assertEquals("AB123CD", car.getLicensePlate());
        assertEquals(12_000, car.getPrice());
    }
}
```

Checking only the exception would miss an implementation that assigns the invalid value and then throws. The assertions after `assertThrows` make the state-preservation requirement observable. The valid boundary `0` is already covered by `CarTest`: test both sides of the rule, not only rejection.

Do not require an exact exception message unless its wording is part of the API contract. For wrappers such as `parseCount` above, also check the preserved cause when error translation is part of the requirement.

Practice: add constructor tests for negative horsepower, `Double.NaN`, and positive infinity. Then move price validation after assignment in `changePrice` and verify that the rejection test detects the corrupted state.

Resources

- https://dev.java/learn/exceptions/
- https://www.baeldung.com/java-exceptions
- https://www.baeldung.com/java-checked-unchecked-exceptions

E5.3 — Resource Ownership

Closing resources

When a file is opened, the JVM requests a **file descriptor** from the OS, which is a limited resource. If an exception occurs before `close()` is called, the file descriptor may not be released, potentially causing **resource leaks**. Properly closing streams ensures that system resources are freed for reuse.

```
Reader reader = null;

try {
    // code which may throw an exception
    reader = new FileReader("file.txt");

    reader.close();
} catch (IOException e) {
    // do something
}
```

The older **try-finally** pattern places cleanup on both normal and exceptional control-flow paths. `finally` is not literally guaranteed under every circumstance, such as abrupt JVM termination. Prefer try-with-resources for owned `AutoCloseable` resources.

```
Reader reader = null;

try {
    // code which may throw an exception
    reader = new FileReader("file.txt");
} catch (IOException e) {
    // do something
} finally {
    if (reader != null) {
      reader.close(); // Still needs an IOException policy; can mask an earlier failure.
    }
}
```

try-with-resources

try-with-resources has two parts:

- **Round brackets** – declare and create one or more resources (e.g., input streams).
- **Curly brackets** – contain the code that uses these resources.

The `close()` method is automatically invoked for successfully initialized non-null resources, in reverse declaration order. A resource must implement `AutoCloseable`; closure itself can fail.

```
try (Reader reader = new FileReader("file.txt")) {
    // some code
}

try (Reader reader1 = new FileReader("file1.txt");
     Reader reader2 = new FileReader("file2.txt")) {
    // some code
}
```

Core: use try-with-resources for resources this scope owns. Do not close a borrowed stream merely because a helper can wrap it; for example, closing a scanner over `System.in` closes that shared input source.

```
try (Scanner scanner = new Scanner(new File("file.txt"))) {
    int first = scanner.nextInt();
    int second = scanner.nextInt();
    IO.println("arguments: " + first + " " + second);
}
```

> **Note:** if the body and cleanup both throw, the body failure remains primary and cleanup failures are suppressed exceptions, available through `getSuppressed()`. This preserves information more reliably than a naive finally block.

Resources

E6.1 — Generic Classes and Methods

Examples use `java.util` types and `java.awt.Point` where a concrete unrelated type is useful; add the corresponding imports. Compile alternative class versions separately. Signature excerpts and deliberately rejected calls are labeled; they are not complete programs.

Reuse of code and type-safety

Integer-based resizable array

Let's start from an enhanced array for integers.

These three teaching containers expose allocated length, not a list's logical size. Setting an index beyond the current length grows storage, leaving default-valued gaps. They assume practical allocation sizes; a production implementation also needs overflow and maximum-capacity handling. Prefer standard collections for application work.

```
public class EnhancedResizableArray {
    public static final int DEFAULT_CAPACITY = 4;
    private int[] v;

    public EnhancedResizableArray() {
        this.v = new int[DEFAULT_CAPACITY];
    }

    public int get(int index) {
        return v[index];
    }

    public void set(int index, int value) {
        if (index >= v.length) {
            int[] tmp = new int[Math.max(index + 1, v.length * 2)];
            System.arraycopy(v, 0, tmp, 0, v.length);
            v = tmp;
        }
        v[index] = value;
    }

    public boolean contains(int value) {
        for (int i : v) {
            if (i == value) {
                return true;
            }
        }
```

```
        return false;
    }

    public void fill(int value) {
        Arrays.fill(v, value);
    }

    public int[] toArray() {
        return Arrays.copyOf(v, v.length);
    }

    public int length() {
        return v.length;
    }
}
```

Writing a slightly different version of the EnhancedResizableArray class for each possible type it could manage would represent a massive violation of the **DRY (Don't Repeat Yourself)** principle.

Object-based resizable array

To mitigate this issue we could use *Object[]* instead of *Integer[]*!

```
public class EnhancedResizableArrayObject {
    public static final int DEFAULT_CAPACITY = 4;
    private Object[] v;

    public EnhancedResizableArrayObject() {
        this.v = new Object[DEFAULT_CAPACITY];
    }

    public Object get(int index) {
        return v[index];
    }

    public void set(int index, Object value) {
        if (index >= v.length) {
            Object[] tmp = new Object[Math.max(index + 1, v.length * 2)];
            System.arraycopy(v, 0, tmp, 0, v.length);
            v = tmp;
        }
        v[index] = value;
    }

    public boolean contains(Object value) {
        for (Object i : v) {
            if (Objects.equals(i, value)) {
                return true;
            }
        }
```

```
        return false;
    }

    public void fill(Object value) {
        Arrays.fill(v, value);
    }

    public Object[] toArray() {
        return Arrays.copyOf(v, v.length);
    }

    public int length() {
        return v.length;
    }
}
```

Now we are talking! We can store every kind of Object within the same container!

However, when we use it, the compiler complains.

```
public static void main(String[] args) {
    EnhancedResizableArrayObject resizableArrayObject = new
 ↪  EnhancedResizableArrayObject();
    resizableArrayObject.set(0, "Hello");
    resizableArrayObject.set(1, "World");
    // The compiler complains here!
    // String s = resizableArrayObject.get(0); // Deliberately rejected: Object is not
     ↪  String
}
```

The declared return type is `Object`. Assigning it to a `String` requires a compatible checked cast:

```
String s = (String) resizableArrayObject.get(0);
```

There is no contract that could guarantee that the return type of the list is a *String*. The array could hold any object. It can only guarantee that it is an *Object* and therefore requires an explicit downcast to be manipulated as a string.

The cast can fail if the actual object is not a string. Adding type checks everywhere treats symptoms; a generic container can enforce the intended element type at compile time.

```
public static void main(String[] args) {
    EnhancedResizableArrayObject resizableArrayObject = new
 ↪  EnhancedResizableArrayObject();
    resizableArrayObject.set(0, "Hello");
    resizableArrayObject.set(1, new Point(2,3));

    // Run-time error! java.lang.ClassCastException
    String s = (String)resizableArrayObject.get(1);
}
```

Generic resizable array

It would be much easier if programmers could express their intention to use specific types and the compiler ensured the correctness of such types. This is the core idea behind generics.

```
List<Integer> list = new ArrayList<>();
```

`List<Integer>` declares a parameterized type. The diamond `<>` on `new ArrayList<>()` asks the compiler to infer constructor type arguments from context.

```
public class EnhancedResizableArrayGeneric<T> {
    public static final int DEFAULT_CAPACITY = 4;
    private Object[] v;

    public EnhancedResizableArrayGeneric() {
        this.v = new Object[DEFAULT_CAPACITY];
    }

    @SuppressWarnings("unchecked")
    public T get(int index) {
        return (T) v[index];
    }

    public void set(int index, T value) {
        if (index >= v.length) {
            Object[] tmp = new Object[Math.max(index + 1, v.length * 2)];
            System.arraycopy(v, 0, tmp, 0, v.length);
            v = tmp;
        }
        v[index] = value;
    }

    public boolean contains(T value) {
        for (Object i : v) {
            if (Objects.equals(i, value)) {
                return true;
            }
        }
        return false;
    }

    public void fill(T value) {
        Arrays.fill(v, value);
    }

    public Object[] toArray() {
        return Arrays.copyOf(v, v.length);
    }

    public int length() {
        return v.length;
    }
}
```

```
public static void main(String[] args) {
    EnhancedResizableArrayGeneric<String> resizableArrayGeneric = new
↪   EnhancedResizableArrayGeneric<>();
    resizableArrayGeneric.set(0, "Hello");
    // Compile-time error!
    // resizableArrayGeneric.set(1, new Point(2,3)); // Deliberately rejected

    String s = resizableArrayGeneric.get(0);
}
```

We have replaced a potential run-time error with a much safer and easier-to-handle compile-time error — a clear improvement!

Keep unchecked operations local

The generic version uses private `Object[]` storage. Its cast is justified by an invariant: only values accepted as T enter through `set` and `fill`; raw-type misuse is excluded. `get` can still return null from an unfilled slot.

The `toArray` method honestly returns `Object[]`. Casting an `Object[]` to `T[]` would not create a real runtime `String[]`, and could fail at a caller expecting one. Standard collection APIs accept a destination array or array generator when a typed result is needed.

Review: an assistant fixes an unchecked warning by adding `@SuppressWarnings` to the entire class. Ask which invariant makes each cast safe. A warning suppression is not evidence of correctness.

Type parameters

A *generic type* is a class (or interface) that is parameterized over types. To declare a generic class, we need to declare a class with the type parameter section delimited by angle brackets < > following the class name.

Remember that **only reference types can be used for generics**. This means that instead of primitive types, we use wrapper classes such as `Integer`, `Double`, `Boolean`, and so on.

This rule is unchanged in Java 25. Preview features involving primitive patterns do not introduce primitive generic type arguments or make `List<int>` legal.

```
// Deliberately rejected:
// List<int> invalid = new ArrayList<>();

// OK!
List<Integer> list = new ArrayList<>();
```

Single type parameter

In the following example, Box has a single type parameter named T. This is a teaching container, not `java.util.Optional`.

```
public class Box<T> {
    private final T value;

    /**
     * Takes a value of "some type" and assigns it to the field
     */
    public Box(T t) {
        this.value = t;
    }

    /**
     * Returns a value of "some type"
     */
    public T get() {
        return value;
    }

    /**
     * Returns true is value is not null
     */
    public boolean isPresent() {
        return value != null;
    }
}
```

After being declared, a type parameter can be used inside the class body as an ordinary type. For instance, the above example uses the type parameter T as:

- a type for a *field*
- a *constructor* argument type
- an *instance method* return type

We can now use Box with different types:

```
Box<Integer> o1 = new Box<>(5);
Box<String> o2 = new Box<>("Hello!");

Integer i = o1.get();  // i = 5
String s = o2.get();   // s = "Hello!"
```

> **Note:** `java.util.Optional` has a stronger absence contract and factory methods rather than a public constructor. Its empty `get()` throws; this box's `get()` simply returns its stored value, possibly null.

Multiple type parameters

A class can have any number of type parameters. For example, the following class has two. Most generic classes have just one or two type parameters.

```
public class Pair<K, V> {
    K key;
    V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public void setKey(K key) {
        this.key = key;
    }

    public V getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }
}
```

- T -- Type
- S, U, V etc. -- 2nd, 3rd, 4th types
- E -- Element (used extensively in collections)
- K -- Key
- V -- Value
- N -- Number

Reference — selected Java API signatures, not complete declarations:

```
interface List<E> {
    boolean add(E e);
    E remove(int index);
    // ...
}

interface Map<K,V> {
    Set<K> keySet();
    Collection<V> values();
    // ...
}
```

```
class Collections {
    static <T extends Comparable<? super T>> void sort(List<T> list);
    // ...
}
```

Type Inference

Type inference is a Java compiler's ability to look at each method invocation and corresponding declaration to determine the type argument (or arguments) that make the invocation applicable.

The inference algorithm determines the types of the arguments and, if available, the type that the result is being assigned, or returned.

Finally, the inference algorithm tries to find the **most specific** type that works with all the arguments.

```
public class TypeInferenceWithGenerics {

    @SafeVarargs
    public static <T> List<T> toList(T... elements) {
        return Arrays.asList(elements);
    }

    public static void main(String[] args) {
        // the most specific type is String
        List<String> fruits = toList("apple", "banana", "cherry");
        // the most specific type is Integer
        List<Integer> numbers = toList(1, 2, 3, 4, 5);

        IO.println(fruits);   // [apple, banana, cherry]
        IO.println(numbers);  // [1, 2, 3, 4, 5]
    }
}
```

```
import java.util.*;

class Animal {}
class Dog extends Animal {}
class Cat extends Animal {}

public class TypeInferenceExample {

    // Generic method that takes variable arguments and returns a list
    @SafeVarargs
    public static <T> List<T> toList(T... elements) {
        return Arrays.asList(elements);
    }

    public static void main(String[] args) {
        Dog dog = new Dog();
```

```
        Cat cat = new Cat();

        // Animal is the most specific type
        List<Animal> pets = toList(dog, cat);

        IO.println(pets);  // [Dog@..., Cat@...]
    }
}
```

Generic Methods

Generic static methods

Static methods cannot use type parameters of their class! If you want to use type parameters in a static method, declare this method's own type parameters.

```
public static <T> T doSomething(T t) {
    return t;
}
```

```
public static <T, U> void method(T t, U u) {
    // do something
}
```

We frequently use generic static methods for different operations with arrays and collections: sorting an array, searching for a value in a collection, reversing an array, and so on.

```
// Selected Collections signatures, shown for reference:
public static <T> void fill(List<? super T> list, T obj);
public static <T> boolean replaceAll(List<T> list, T oldVal, T newVal);
public static void shuffle(List<?> list);
public static <T extends Comparable<? super T>> void sort(List<T> list);
```

Generic instance methods

A generic method declares its own type parameters, independently of whether its enclosing class is generic. A method that only uses its class's T is not itself a generic method.

```
// A simple generic class with type parameter T
public class SimpleClass<T> {

    // Method using the class-level type parameter T
    public T doStuff(T t) {
        return t;
    }
```

```
    // Method introducing its own type parameter U
    // Can use both the class-level T and method-level U
    public <U> T getParameterizedObject(T t, U u) {
        return t;
    }
}
```

As another example, consider the method *toArray()* defined within the java.util.List interface.

```
interface List<E> {
    boolean add(E e);
    E remove(int index);
    <T> T[] toArray(T[] a);
    // ...
}
```

```
public static void main(String[] args) {
    List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);

    Integer[] a1 = list.toArray(new Integer[]{});
    Number[] a2 = list.toArray(new Number[]{});
    Integer[] a3 = list.toArray(Integer[]::new); // Typed array generator
}
```

The destination component type still matters: asking this integer list for `String[]` compiles through the method's independent type parameter but fails with `ArrayStoreException`. Generics do not eliminate every runtime check.

Resources

E6.2 — Bounds and Wildcards

Type bounds

Constraining Type Parameters (class definition)

Imagine that we have a generic Stack<E> class that can contain objects of any class.

```
public class Stack<E> {
    private List<E> items = new ArrayList<>();

    public void push(E e) {
        items.addLast(e);
    }

    public E pop() {
        return items.removeLast();
    }

    public boolean isEmpty() {
        return items.isEmpty();
    }
}
```

We can put any type of object inside Stack<E>. However, there are some situations in which we want to restrict these objects. We can say, for example, that the stack has to be able to contain only CharSequence objects (i.e., a common ancestor of both String and StringBuilder).

```
public class Stack<E extends CharSequence> {
    // ...
}
```

Now creating three Stack objects will lead to different results:

```
public static void main(String[] args) {
    Stack<String> strings = new Stack<>(); // ok
    Stack<StringBuilder> builders = new Stack<>(); // ok
    // Stack<Point> points = new Stack<>(); // Deliberately rejected by the bound
}
```

It is worth noting that, a type variable may have a single type bound:

```
<T extends A>
```

or multiple type bounds:

```
<T extends A & B & C & ...>
```

The first type bound (”A”) can be a class or an interface. The rest of the type bounds (”B” onwards) must be interfaces.

Constraining Type Parameters (method definition)

Consider a generic method that finds the average of the values in a List. How can you compute averages when you know nothing about the element type? You need to have a mechanism for measuring the elements.

```
public interface Measurable {
    double getMeasure();
}
```

We can constrain the type of the elements, requiring that the type implement the Measurable type. In Java, this is achieved by adding the clause extends Measurable after the type parameter. We can express this concept with two different syntaxes:

```
// Signature sketch; the complete implementation follows.
public static <T extends Measurable> double average(List<T> objects);
```

We are saying that T is a subtype of `Measurable`. Here is the complete method, assuming non-null elements and finite measurements. Returning zero for an empty list is an explicit teaching policy, not a mathematical average of an empty set:

```
public static <T extends Measurable> double average(List<T> objects) {
    if (objects.size() == 0) {
        return 0.0;
    }
    double sum = 0;
    for (T obj : objects) {
        sum += obj.getMeasure();
    }
    return sum / objects.size();
}
```

Which syntax?

Reference — equivalent API choices, not overloads to declare together. The wildcard and type-parameter versions below have the same erasure. Compiling both versions in one class would cause a name clash. The empty bodies illustrate signatures, not working algorithms.

- **Use <?> (wildcards) when you don't need to relate the types of multiple parameters or return values.**
- **Use <T> (type parameters) when there are constraints or relationships between types**, such as:
  - The type of one parameter depends on the type of another.
  - The return type depends on the parameter type.

```
// ------------------------------------------------
// Using wildcards: we just need a list; we don't
// care about element type relationships
// ------------------------------------------------
public static void reverse(List<?> list) {}
public static void shuffle(List<?> list, Random rnd) {}
public static void swap(List<?> list, int i, int j) {}

// ------------------------------------------------
// Using type parameters: works, but unnecessary here
// since no relationships between parameters exist
// ------------------------------------------------
public static <T> void reverse(List<T> list) {}
public static <T> void shuffle(List<T> list, Random rnd) {}
public static <T> void swap(List<T> list, int i, int j) {}
```

- `List<?>` means a list of an unknown element type. The compiler permits adding only `null` through that reference; the implementation may still reject nulls or all additions.
- Works fine when **the method doesn't need to know the type or relate types**.

```
// Type T links parameters and/or return values
public static <T> void sort(List<T> list, Comparator<? super T> c) {}
public static <T> void fill(List<? super T> list, T obj) {}
public static <T> void copy(List<? super T> dest, List<? extends T> src) {}
```

- **<T>** declares a type parameter for the method.
- **`sort`**: The list and comparator must be compatible—`Comparator<? super T>` can compare T or its supertypes.
- **`fill`**: You can fill a `List` of T or any of its supertypes with an object of type T.
- **`copy`**: Ensures type safety: the source contains elements of T or a subtype, and the destination can accept T or a supertype.

Examples with subclasses

Core: generic types are invariant. Although `Integer` extends `Number`, `List<Integer>` is not a subtype of `List<Number>`. Otherwise code could insert a `Double` through the number-list reference into an integer-only list.

Suppose we want to add a method `pushAll()` that takes a sequence of elements and pushes them all onto the stack. Here's a first attempt:

```
public class Stack<E> {
    // ...
    public void push(E e) {
        items.addLast(e);
    }

    public void pushAll(Iterable<E> src) {
        for (E e : src) {
            push(e);
        }
    }
    // ...
}
```

This method compiles cleanly, but it isn't entirely satisfactory. It does not work with subtypes!

```
public static void main(String[] args) {
    Stack<Number> numberStack = new Stack<>();

    // Ok
    numberStack.pushAll(new ArrayList<Number>(List.of(1,2,3)));

    // Deliberately rejected: ArrayList<Integer> is NOT a subtype of ArrayList<Number>.
    // numberStack.pushAll(new ArrayList<Integer>(List.of(1,2,3)));
}
```

The type of the input parameter to pushAll should not be *Iterable of E* but *Iterable of some subtype of E* and there is a wildcard type that means precisely that: *Iterable<? extends E>*.

```
public class Stack<E> {
    // ...
    public void pushAll(Iterable<? extends E> src) {
        for (E e : src) {
            push(e);
        }
    }

    // ...
}
```

Now suppose you want to write a *popAll()* method to go with *pushAll()*. The popAll method pops each element off the stack and adds the elements to the given collection:

```java
public class Stack<E> {
    // ...
    public E pop() {
        return items.removeLast();
    }

    public void popAll(Collection<E> dst) {
        while (!isEmpty()) {
            dst.add(pop());
        }
    }
    // ...
}
```

Again, this compiles cleanly and works fine if the element type of the destination collection exactly matches that of the stack. But again, it does not seem entirely satisfactory when using subtypes.

```java
public static void main(String[] args) {
    Stack<Integer> integerStack = new Stack<>();
    integerStack.pushAll(new ArrayList<>(List.of(1,2,3)));

    // Ok
    integerStack.popAll(new ArrayList<Integer>());

    // Deliberately rejected: ArrayList<Number> is NOT a supertype of
    ↪  ArrayList<Integer>.
    // integerStack.popAll(new ArrayList<Number>());
}
```

The type of the input parameter to *popAll()* should not be *Collection of E* but *Collection of some supertype of E* (where supertype is defined such that E is a supertype of itself). Again, there is a wildcard type that means precisely that: *Collection<? super E>*.

```java
public class Stack<E> {
    // ...
    public void popAll(Collection<? super E> dst) {
        while (!isEmpty()) {
            dst.add(pop());
        }
    }
    // ...
}
```

Producer extends, consumer super (PECS)

- Read values from a producer: `? extends E`.
- Supply values to a consumer: `? super E`.
- Use an exact type when the operation needs both directions with that same type.

`extends` does not make the collection immutable; it restricts what can be written through that reference. `super` permits writing compatible values, but reading only guarantees `Object`.

Predict: can a `Stack<Number>` receive an `Iterable<Integer>` through the revised `pushAll`? Can a `Stack<Integer>` empty into a `Collection<Number>` through the revised `popAll`? Explain both using data flow rather than memorizing punctuation.

Resources

E6.3 — Type Erasure

Type Erasure

Reference — runtime model and migration compatibility. Ordinary API use does not require reproducing the compiler transformation. It is still legal to use collection types and other generic types without supplying type parameters, but **you should not do it**.

If you use raw types, you lose all the safety and expressiveness benefits of generics.

```
public static void main(String[] args) {
    List l = new ArrayList();
    l.add("Hello");
    l.add("World");
    l.add(2.3);

    for (Object o : l) {
        String s = (String) o;
        // runtime error when 2.3 is processed!
        s.length();
    }
}
```

Given that you shouldn't use raw types, why did the language designers allow them? **To provide compatibility**.

The Java platform was about to enter its second decade when generics were introduced, and there was an enormous amount of Java code in existence that did not use generics. **It had to be legal to pass instances of parameterized types (new types) to methods that were designed for use with old types, and vice versa**. This requirement, known as *migration compatibility*, drove the decision to support raw types.

Java implements generics primarily through **erasure**: a type variable erases to its leftmost bound, or `Object` when unbounded. The compiler inserts casts and may generate bridge methods. Generic signatures can remain as metadata for reflection; erasure does not mean all generic information vanishes from class files.

You cannot use `new T[]`, `T.class`, or `instanceof List<String>` as though the element parameter were a reified runtime class. `instanceof List<?>` is permitted.

```
public class Pair<K, V> {
    K key;
    V value;

    public Pair(K key, V value) {
```

```
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public void setKey(K key) {
        this.key = key;
    }

    public V getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }
}
```

Conceptual source-level sketch after erasure (not the literal emitted bytecode):

```
public class Pair {
    Object key;
    Object value;

    public Pair(Object key, Object value) {
        this.key = key;
        this.value = value;
    }

    public Object getKey() {
        return key;
    }

    public void setKey(Object key) {
        this.key = key;
    }

    public Object getValue() {
        return value;
    }

    public void setValue(Object value) {
        this.value = value;
    }
}
```

In the example below, `Measurable` is the upper bound of the generic type E. It supplies the operation used to compare items:

```
public interface Measurable {
    double getMeasure();
}
```

Assume non-null items with finite measures. The method returns an item with the smallest measure:

```
public static <E extends Measurable> E min(List<E> list) {
    if (list.isEmpty()) {
        throw new IllegalArgumentException();
    }

    E smallest = list.get(0);
    for (E item : list) {
        if (item.getMeasure() < smallest.getMeasure()) {
            smallest = item;
        }
    }
    return smallest;
}
```

Conceptual source-level sketch with explicit casts:

```
public static Measurable min(List list) {
    if (list.isEmpty()) {
        throw new IllegalArgumentException();
    }

    Measurable smallest = (Measurable) list.get(0);
    for (Object value : list) {
        Measurable item = (Measurable) value;
        if (item.getMeasure() < smallest.getMeasure()) {
            smallest = item;
        }
    }
    return smallest;
}
```

Resources

- https://dev.java/learn/generics/
- https://www.baeldung.com/java-generics
- https://www.baeldung.com/java-generics-vs-extends-object
- https://www.baeldung.com/java-type-erasure

E7.1 — Lambdas and Functional Interfaces

Use imports from `java.util`, `java.util.function`, `java.util.random`, and `java.util.logging` as needed. Method fragments belong inside a class; alternative type definitions and lambdas illustrate separate examples. Interface excerpts show selected API methods, not replacement standard-library types.

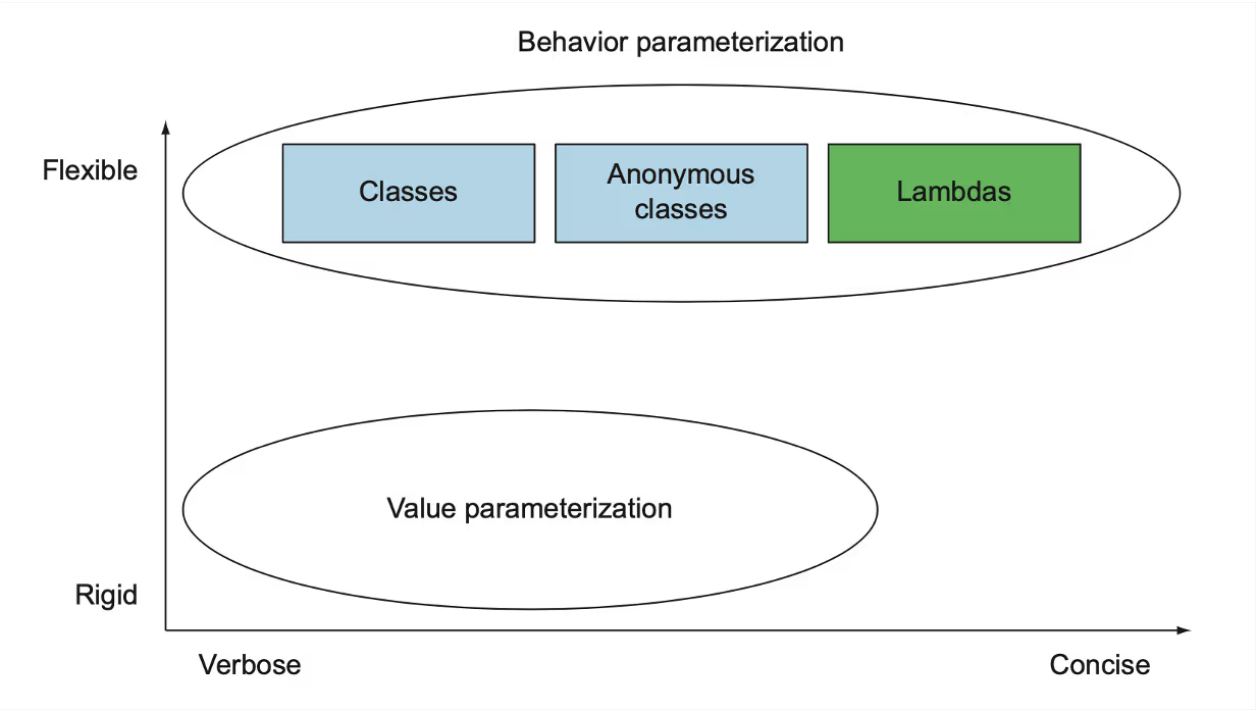
Functional vs imperative programming

Imperative code expresses steps and state changes explicitly. Our first filter fixes both the traversal and the selection rule in its body; values vary between calls, but the rule does not.

```
// imperative style
public static List<Integer> filterEven(List<Integer> numbers) {
    List<Integer> result = new ArrayList<>();
    for (Integer n : numbers) {
        if (n % 2 == 0) {   // behaviour is hardcoded
            result.add(n);
        }
    }
    return result;
}
```

Functional programming emphasizes applying and composing functions. Java supports a mixed style: **behavior parameterization** lets us supply the rule independently of the traversal.

```
// functional style
public static List<Integer> filter(List<Integer> numbers, Predicate<Integer> predicate)
 ↪  {
    List<Integer> result = new ArrayList<>();
    for (Integer n : numbers) {
        if (predicate.test(n)) {  // behaviour is a parameter
            result.add(n);
        }
    }
    return result;
}
```



The second method still contains an imperative loop. Passing behavior is useful without making the whole program purely functional. These introductory filters assume non-null lists, predicates, and integer elements.

Lambda expressions

A lambda expression supplies an implementation of a **target functional interface**. In Java it is not a standalone function with an independent function type.

The most general form of a lambda expression looks like this:

```
(parameters) -> expression // single-line expression
```

```
(parameters) -> { statements; } // block body, with return when required
```

The part before `->` is the list of parameters (like in methods), and the part after that is the expression that can return a value.

Examples

A boolean expression: `(List<String> list) -> list.isEmpty()`

Creating objects: `() -> new Apple(45)`

Consuming from an object: `(Apple a) -> IO.println(a.getWeight())`

Select/extract a field from an object: `(String s) -> s.length()`

Multiply two integers: `(int a, int b) -> a * b`

Compare two objects: `(Apple a1, Apple a2) -> a1.getWeight().compareTo(a2.getWeight())`

These `Apple` shapes are illustrative and assume a suitable class with an `Integer getWeight()` method. Prefer `Integer.compare` for primitive weights. A lambda needs a target type: `Predicate<String> empty = s -> s.isEmpty();` is valid, while `var empty = s -> s.isEmpty();` has no target functional interface and is rejected.

Capturing variables

Local variables captured by a lambda must be final or **effectively final**:

```
int threshold = 10;
java.util.function.Predicate<Integer> above = value -> value > threshold;
// threshold++; // Rejected: would make the captured local non-effectively-final
IO.println(above.test(12));
```

This restricts reassignment of the local, not mutation of objects it references. A lambda that mutates a captured list still has a side effect and is not automatically thread-safe.

What is a Functional Interface?

For ordinary use, a *functional interface* has one abstract operation, possibly inherited. Default and static methods do not add abstract operations; declarations matching public `Object` methods do not count toward this test. `@FunctionalInterface` asks the compiler to check the actual rules.

To explicitly declare an interface as functional, Java provides the `@FunctionalInterface` annotation. Although optional, using this annotation allows the compiler to check that the interface does not accidentally define more than one abstract method.

```
@FunctionalInterface
public interface MyAction {
    void execute();
}
```

This interface can be used with a lambda expression like this:

```
public static void main(String[] args) {
    MyAction action = () -> IO.println("Action executed!");
    // prints Action executed!
    action.execute();
}
```

Functional interfaces are the cornerstone of Java's functional features and are widely used in the standard library — such as `Runnable`, `Callable`, `Comparator`.

```java
@FunctionalInterface
public interface Runnable {
    void run();
}
```

```java
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

```java
@FunctionalInterface
public interface Comparator<T> {
    int compare(T o1, T o2);
}
```

`java.util.function` package

The standard interfaces describe common input/output shapes. Reuse a suitable shape instead of inventing a new interface for every lambda. A domain-specific name can still help when it expresses a meaningful contract.

These predefined interfaces, found in the `java.util.function` package, include:

- `Function<T, R>` – takes an input of type T and returns a result of type R
- `Predicate<T>` – takes an input of type T and returns a boolean (useful for conditions and filters)
- `Consumer<T>` – takes an input of type T and returns nothing (useful for performing actions)
- `Supplier<T>` – takes no input and returns a result of type T

Function

Core: understand `Function`, `Predicate`, `Consumer`, and `Supplier`. The Bi-variants and specialized forms below are reference material once those four are familiar.

The most general case of a lambda is a functional interface with a method that receives one value and returns another. This function is represented by the *Function* interface, which is parameterized by the types of its argument and a return value:

```java
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

```java
public static void main(String[] args) {
    // if it has only one argument "()" are optional
    Function<Integer, Integer> adder1 = x -> x + 1;

    // with multiple statements
    Function<Integer, Integer> adder5 = x -> {
        x += 2;
        x += 3;
        return x;
    };

    // with two different types
    Function<String, Integer> length = s -> s.length();

    // use
    IO.println(adder1.apply(5));         // 6
    IO.println(length.apply("nicola"));  // 6
}
```

```java
public static void main(String[] args) {
    // without lambdas

    // single operation
    Function<Integer, Integer> adder1 = new Function<Integer, Integer>() {
        @Override
        public Integer apply(Integer x) {
            return x + 1;
        }
    };

    // multiple statements
    Function<Integer, Integer> adder5 = new Function<Integer, Integer>() {
        @Override
        public Integer apply(Integer x) {
            x += 2;
            x += 3;
            return x;
        }
    };

    // two different types
    Function<String, Integer> length = new Function<String, Integer>() {
        @Override
        public Integer apply(String s) {
            return s.length();
        }
    };

    // use
    IO.println(adder1.apply(5));         // 6
    IO.println(length.apply("nicola"));  // 6
}
```

Predicate

The `Predicate` functional interface represents a function that takes a single argument and returns a `boolean` result.

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

And here are some examples of how to use it:

```
public static void main(String[] args) {
    // basic usage: checks if a number is positive
    Predicate<Integer> isPositive = x -> x > 0;

    // testing with strings
    Predicate<String> isEmpty = s -> s.isEmpty();

    // use the predicate
    IO.println(isPositive.test(5));         // true
    IO.println(isEmpty.test(""));           // true
}
```

A typical use case of the `Predicate` interface is to filter a collection of values:

```
public static void main(String[] args) {
    List<String> names = new ArrayList<>(List.of("Angela", "Aaron", "Bob", "Claire",
    ↪  "David"));
    names.removeIf(s -> s.startsWith("B"));  // "Angela", "Aaron", "Claire", "David"
}
```

```
interface List<E> {
    // ...
    boolean removeIf(Predicate<? super E> filter);
    // ...
}
```

Consumer

The `Consumer` functional interface represents an operation that takes a single input and returns no result. It's typically used to perform side effects — like printing values, updating external state, or writing to a file.

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```
public static void main(String[] args) {
    // basic usage: print the input value
    Consumer<String> printer = s -> IO.println(s);

    // side-effect: adding values to a list
    List<String> names = new ArrayList<>();
    Consumer<String> addToList = name -> names.add(name);

    // use the consumers
    printer.accept("Hello, world!");
    addToList.accept("Alice");
    addToList.accept("Bob");
    IO.println(names); // [Alice, Bob]
}
```

Consumer is used by stream terminal operations and collection APIs. `Iterable.forEach`, used below, accepts one without creating a stream:

```
public static void main(String[] args) {
    List<String> names = new ArrayList<>(List.of("Angela", "Aaron", "Bob", "Claire",
    ↪  "David"));
    names.forEach(name -> IO.println("Hello, " + name));  // Hello Angela...
}
```

```
interface Iterable<T> {
    // ...
    void forEach(Consumer<? super T> action);
    // ...
}
```

Supplier

The Supplier accepts nothing and returns a generified result. There is no requirement that a new or distinct result be returned each time the supplier is invoked.

```
@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

For instance, we can consume items coming from different suppliers.

```
public static void main(String[] args) {
    Supplier<String> supplierBasic = () -> "Hello World!";

    Supplier<String> supplierRandom = () -> {
        RandomGenerator rnd = RandomGenerator.getDefault();
        return "Hello World!-" + Integer.valueOf(rnd.nextInt(100));
```

```
    };

    // use the supplier
    IO.println(supplierBasic.get()); // "Hello World!"
    IO.println(supplierRandom.get()); // "Hello World!-72"
}
```

Some logging methods support **lazy evaluation** by accepting a `Supplier<String>`. The `Supplier` is called **only if the log level requires it**, so the message is computed lazily.

```
public static void testLog() {
    Logger logger = Logger.getLogger("MyLogger");
    logger.info("This message is computed immediately");
    logger.info(() -> "This message is computed lazily");
}
```

BiFunction

Reference — two inputs rather than one.

BiFunction has both arguments and a return type generified, while ToDoubleBiFunction and others allow us to return a primitive value.

```
@FunctionalInterface
public interface BiFunction<T,U,R> {
    R apply(T t, U u);
}
```

```
public static void main(String[] args) {
    // add two integers
    BiFunction<Integer, Integer, Integer> adder = (a, b) -> a + b;

    // concatenate two strings
    BiFunction<String, String, String> joiner = (s1, s2) -> s1 + s2;

    // sum the length of two strings
    BiFunction<String, String, Integer> sumLength = (s1, s2) -> s1.length() +
↪   s2.length();

    // use the bifunctions
    IO.println(adder.apply(3, 4));           // 7
    IO.println(joiner.apply("Hello, ", "World!")); // Hello, World!
    IO.println(sumLength.apply("Hello", "World")); // 10
}
```

One of the typical examples of using this interface is in the *Map.replaceAll()* method, which allows replacing all values in a `Map<K,V>` with a BiFunction:

```
interface Map<K,V> {
    // ...
    default void replaceAll(BiFunction<? super K,? super V,? extends V> function);
    // ...
}
```

```
public static void main(String[] args) {
    Map<String, Integer> salaries = new HashMap<>(Map.of(
            "John", 40000,
            "Freddy", 30000,
            "Samuel", 50000
    ));

    salaries.replaceAll((name, salary) ->
            name.startsWith("F") ? salary * 2 : salary + 2);

    // Contains Samuel=50002, John=40002, Freddy=60000; HashMap order is unspecified.
    IO.println(salaries);
}
```

BiConsumer

BiConsumer has both arguments generified and returns void. One of its use cases is iterating through the entries of a map.

```
@FunctionalInterface
public interface BiConsumer<T,U> {
    void accept(T t, U u);
}
```

One of the typical examples of using this interface in the standard API is in the *Map.forEach()* method, which performs the given action for each entry.

```
interface Map<K,V> {
    // ...
    default void forEach(BiConsumer<? super K,? super V> action);
    // ...
}
```

```
public static void main(String[] args) {
    Map<String, Integer> salaries = new HashMap<>(Map.of(
            "John", 40000,
            "Freddy", 30000,
            "Samuel", 50000
    ));

    salaries.forEach((name, salary) -> IO.println(name + " earns " + salary + "
↪   dollars/year!"));
```

```
    // prints:
    // The order of these HashMap entries is unspecified.
    // Samuel earns 50000 dollars/year!
    // John earns 40000 dollars/year!
    // Freddy earns 30000 dollars/year!
}
```

Operators

Operator interfaces are special cases of a function that receive and return the same value type. The UnaryOperator interface receives a single argument.

```
@FunctionalInterface
public interface UnaryOperator<T> {
    T apply(T t);
}
```

This shows the functional shape. In the actual API, `UnaryOperator<T>` extends `Function<T, T>` and inherits `apply`.

One of its use cases in the Collections API is the *List.replaceAll()* method replacing all items in a list with some computed values of the same type:

```
interface List<E> {
    // ...
    default void replaceAll(UnaryOperator<E> operator);
    // ...
}
```

```
public static void main(String[] args) {
    List<String> names = new ArrayList<>(List.of("Angela", "Aaron", "Bob", "Claire",
    ↪  "David"));
    names.replaceAll(name -> name.toUpperCase(Locale.ROOT));
}
```

Method references

A method reference adapts an existing method or constructor to a compatible functional-interface target. It does not invoke that method immediately.

```
// lambda expression
BiFunction<Integer, Integer, Integer> max = (x, y) -> Integer.max(x, y);
```

```
// method reference
BiFunction<Integer, Integer, Integer> max = Integer::max;
```

This code works because the definition of the method `int max(int a, int b)` fits the type `BiFunction<Integer, Integer, Integer>`: they both mean taking two integer arguments and returning an integer value.

Now we have the `max` object that can be used as a function by invoking the `apply` method. Let's invoke it!

```
IO.println(max.apply(50, 70)); // 70
```

Use method references when they clarify the operation. A shorter expression is not automatically easier to understand or test; test the behavior whichever syntax supplies it.

Kinds of method references

It's possible to write method references to both static and instance (non-static) methods.

In general, there are four kinds of method references:

- reference to a static method;
- reference to an instance method of an existing object;
- reference to an instance method of an object of a particular type;
- reference to a constructor.

Reference to a static method

```
ClassName :: staticMethodName
```

```
Function<Double, Double> sqrt = (x) -> Math.sqrt(x);
```

```
Function<Double, Double> sqrt = Math::sqrt;
```

Reference to an instance method of an object

```
objectName :: instanceMethodName
```

```
String whatsGoingOnText = "What's going on here?";

Function<String, Integer> indexWithinWhatsGoingOnText = (text) ->
↪  whatsGoingOnText.indexOf(text);
```

```
String whatsGoingOnText = "What's going on here?";

Function<String, Integer> indexWithinWhatsGoingOnText = whatsGoingOnText::indexOf;
```

Reference to an instance method of an object of a particular class

```
ClassName :: instanceMethodName
```

```
Function<Long, Double> converter = (l) -> l.doubleValue();
```

```
Function<Long, Double> converter = Long::doubleValue;
```

Reference to a constructor

```
ClassName :: new
```

For example, let's consider our custom class `Person` with a single field `name`.

```
class Person {
    String name;

    public Person(String name) {
        this.name = name;
    }
}
```

Here is a reference to the constructor of this class:

```
Function<String, Person> personGenerator = Person::new;
```

```
Person johnFoster = personGenerator.apply("John Foster"); // we have a John Foster
 ↪  object
```

A case study: filtering students

We need to select all students with a given average, and also print a string representation of the selected students.

Use this supporting type for the grade-filtering examples:

```
record Student(String name, double average) {
    public double getAverage() {
        return average;
    }
}
```

Assume finite averages in the supported grading range and non-null inputs. Exact equality below is an explicit selection rule; an application using computed averages may instead need a tolerance or threshold policy.

Imperative approach

```
public static List<Student> filterStudentsByGrade(List<Student> students, double
↪  average) {
    List<Student> result = new ArrayList<>();
    for (Student s : students) {
        if (s.getAverage() == average) {
            result.add(s);
        }
    }
    return result;
}
```

After a while, requirements change, and you need to select all the students with an average comprised within a given range. You can add an alternative method.

```
public static List<Student> filterStudentsByGradeRange(List<Student> students, double
↪  low, double high) {
    List<Student> result = new ArrayList<>();
    for (Student s : students) {
        if (s.getAverage() >= low && s.getAverage() <= high) {
            result.add(s);
        }
    }
    return result;
}
```

However, this approach breaks the DRY (Don't Repeat Yourself) principle. The two methods vary only in one line!

Functional approach

Parameterize only what varies here: the selection rule. This generic method filters without transforming or printing; a caller can choose those separate actions.

```
public static <T> List<T> filter(List<T> list, Predicate<? super T> predicate) {
    List<T> result = new ArrayList<>();
    for (T entity : list) {
        if (predicate.test(entity)) {
            result.add(entity);
        }
    }
    return result;
}
```

```
public static void main(String[] args) {
    List<Student> students = List.of(new Student("Ada", 28), new Student("Lin", 24));
    List<Student> filteredStudents = filter(students,
```

```
            s -> s.getAverage() >= 26 && s.getAverage() <= 30);
    filteredStudents.forEach(IO::println);
}
```

Resources

E7.2 — Function Composition and Optional

Composing behavior

```
Predicate<Integer> nonNegative = value -> value >= 0;
Predicate<Integer> below100 = value -> value < 100;
Predicate<Integer> valid = nonNegative.and(below100);
Function<String, String> clean = String::strip;
Function<String, Integer> lengthAfterCleaning = clean.andThen(String::length);
IO.println(valid.test(50)); // true
IO.println(lengthAfterCleaning.apply(" Ada ")); // 3
```

Note: `IntPredicate`, `IntUnaryOperator`, and `ToIntFunction<T>` can avoid boxing in appropriate APIs. Standard `Function` and `Predicate` methods do not declare checked exceptions; a lambda cannot transparently throw an `IOException` through those contracts. Handle it meaningfully or choose an API designed to propagate it.

Composing behavior

Optional values

Like many programming languages, Java uses `null` to represent the absence of a value. Frequently, this approach leads to **Null Pointer Exceptions**. The British computer scientist Tony Hoare, the inventor of the `null` concept, even describes introducing `null` as a **"billion-dollar mistake"** since it has led to innumerable errors, vulnerabilities, and system crashes.

```
public class App {
    record Student(String name, String lastName, String email) {}

    public static Student searchStudent(List<Student> students, String name) {
        for (Student student : students) {
            if (name.equals(student.name())) {
                return student;
            }
        }
        return null;
    }
```

```
    public static void main(String[] args) {
        List<Student> students = List.of(
                new Student("John", "Doe", "3314567543"),
                new Student("Liz", "White", "3319876543"),
                new Student("Sarah", "Green", "3317649432")
        );
        IO.println(searchStudent(students, "John").email());
        IO.println(searchStudent(students, "Josh").email()); // NPE!
    }
}
```

`Optional<T>` represents either a present non-null value or absence. An empty optional is an object, not a null reference. Never return null from a method whose contract is to return an `Optional`.

```
public class App {
    record Student(String name, String lastName, String email) {}

    public static Optional<Student> searchStudent(List<Student> students, String name) {
        for (Student student : students) {
            if (name.equals(student.name())) {
                return Optional.of(student);
            }
        }
        return Optional.empty();
    }

    public static void main(String[] args) {
        List<Student> students = List.of(
                new Student("John", "Doe", "3314567543"),
                new Student("Liz", "White", "3319876543"),
                new Student("Sarah", "Green", "3317649432")
        );

        Optional<Student> optionalStudent = searchStudent(students, "Josh");
        if (optionalStudent.isPresent()) {
            Student student = optionalStudent.get();
            IO.println(student.email());
        }
    }
}
```

Creating an Optional

In the following code, we create two Optional objects. The first object represents an empty value (such as `null`), and the second one keeps a string value. The `isPresent` method checks whether an object is empty or not.

```
public static void main(String[] args) {
    Optional<String> absent = Optional.empty();
    Optional<String> present = Optional.of("Hello");

    IO.println(absent.isPresent()); // false
    IO.println(present.isPresent()); // true
}
```

In a situation when you don't know whether a variable is `null` or not, you should pass it to the `ofNullable` method instead of the `of` method. It creates an empty Optional if the passed value is `null`.

```
public static void main(String[] args) {
    Optional<String> optMessage1 = Optional.ofNullable(null);
    Optional<String> optMessage2 = Optional.ofNullable("Hello");

    IO.println(optMessage1.isPresent()); // false
    IO.println(optMessage2.isPresent()); // true

}
```

Getting the value from an Optional

The most obvious thing to do with an Optional is to get its value.

- `get` returns the value if it's present, otherwise throws an exception;
- `orElseThrow` behaves as `get`, but the name describes it better;
- `orElse(T other)` returns the value if it's present, otherwise returns `other`;
- `orElseGet(Supplier<? extends T> other)` returns the value if it's present, otherwise invokes `other` and returns its result.

```
public static void main(String[] args) {
    Optional<String> optName = Optional.ofNullable("John");
    Optional<String> optNull = Optional.ofNullable(null);

    IO.println(optName.get());           // "John"
    IO.println(optName.orElseThrow());   // "John"
    IO.println(optName.orElse("Default name")); // "John"

    // Run separately to observe failure:
    // IO.println(optNull.get());         // throws NoSuchElementException
    // IO.println(optNull.orElseThrow()); // throws NoSuchElementException
    IO.println(optNull.orElse("Default name"));   // "Default name"
}
```

Eager and lazy defaults

Predict: which fallback is evaluated when a value is already present?

```
Optional<String> name = Optional.of("Ada");
String eager = name.orElse(fallback());
String lazy = name.orElseGet(() -> fallback());
```

Use this helper in the same class:

```
static String fallback() {
    IO.println("Computing fallback");
    return "Unknown";
}
```

Check your prediction

The message is printed once: ordinary argument evaluation calls `fallback()` before `orElse`. `orElseGet` invokes its supplier only when absent. Both results are Ada here.

Transform an optional result

```
Optional<String> name = Optional.of(" Ada ");
String display = name.map(String::strip)
        .filter(value -> !value.isEmpty())
        .orElse("Unknown");
IO.println(display); // Ada
```

Prefer composing an optional result when that expresses the requirement better than `isPresent` followed by `get`. Optional is primarily useful for return values representing possible absence, not an automatic replacement for every nullable field or parameter.

Conditional actions

There are also convenient methods that take functions as arguments and perform some actions on values wrapped inside Optional:

- `ifPresent(Consumer<? super T> action)` performs the given action with the value, otherwise does nothing;
- `ifPresentOrElse(Consumer<? super T> action, Runnable emptyAction)` performs the given action with the value, otherwise performs another action.

```java
public static void main(String[] args) {
    Optional<String> optName = Optional.ofNullable("John");
    Optional<String> optNull = Optional.ofNullable(null);

    optName.ifPresent((name) -> IO.println(name.length())); // 4
    optNull.ifPresent((name) -> IO.println(name.length())); // does nothing
}
```

The ”classic” equivalent of these two code snippets looks like the following. This code is more error-prone because it is possible to forget to perform the `null` check explicitly and then get the **NPE**.

```java
public static void printName(String name) {
    if (name != null) {
        IO.println(name.length());
    }
}
```

`ifPresentOrElse` expresses a two-branch action on an optional value. It does not make arbitrary code inside either branch safe or guarantee that the optional reference itself is non-null.

```java
public static void main(String[] args) {
    Optional<String> optName = Optional.ofNullable("John");
    Optional<String> optNull = Optional.ofNullable(null);

    // prints 4
    optName.ifPresentOrElse(
            (name) -> IO.println(name.length()),
            () -> IO.println(0)
    );

    // prints 0
    optNull.ifPresentOrElse(
            (name) -> IO.println(name.length()),
            () -> IO.println(0)
    );
}
```

Resources

- https://dev.java/learn/lambdas/
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Optional.html
- https://www.baeldung.com/java-functional-programming
- https://www.baeldung.com/java-8-functional-interfaces

E7.3 — Stream Pipelines

Import types from `java.util` and `java.util.stream` as needed. Statement fragments belong inside `main`; helper methods belong inside a class. Later `Account` definitions extend the earlier example: compile those alternative versions separately. Assume non-null source elements unless stated otherwise.

The basic concept of streams

The **Stream API** supports declarative pipelines for processing values. It lets us express transformations without spelling out each traversal step. Parallel execution is available, but correctness and useful speedup require additional reasoning; neither follows automatically from using streams.

A **stream** conveys elements from a **source** (a collection, a generator function, a file, an I/O channel, another stream) and then processes the elements by using a sequence of predefined operations combined into a pipeline.

There are three stages of working with a stream:

1. Obtaining the stream from a source.
2. Performing intermediate operations with the stream to process data.
3. Performing a terminal operation to produce a result.



Loops vs streams

All classes associated with streams are located in the `java.util.stream` package. There are several common stream classes: `Stream<T>`, `IntStream`, `LongStream` and `DoubleStream`. While the generic stream works with reference types, others work with the corresponding primitive types.

Let's suppose we have a list of numbers, and we'd like to count the numbers that are greater than 5. A ”traditional” way to do it is to write a loop like the following:

```
import java.util.ArrayList;

public static void main(String[] args) {
    List<Integer> numbers = new ArrayList<>(List.of(1, 4, 7, 6, 2, 9, 7, 8));

    long count = 0;
    for (int number : numbers) {
        if (number > 5) {
            count++;
        }
    }
    IO.println(count); // 5
}
```

Using the Stream API, it becomes:

```
long count = numbers.stream()
        .filter(number -> number > 5)
        .count(); // 5
```

Here we get a stream from the `numbers` list, then filter its elements by using a predicate lambda expression, and then count the numbers that satisfy the condition.

This pipeline separates the selection rule from traversal. Whether it is easier to read depends on the task; a loop remains appropriate when it expresses the algorithm more clearly. For example, we can skip the first four input values:

```
long count = numbers.stream()
        .skip(4)  // skip 1, 4, 7, 6
        .filter(number -> number > 5)
        .count();  // 3
```

Creating streams

The most common way to create a stream is to take it from a collection. Any collection has the `stream()` method for this purpose.

```
List<Integer> famousNumbers = List.of(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55);
Stream<Integer> numbersStream = famousNumbers.stream();

Set<String> usefulConcepts = Set.of("functions", "lazy", "immutability");
Stream<String> conceptsStream = usefulConcepts.stream();
```

It is also possible to obtain a stream from an array:

```
Stream<Double> doubleStream = Arrays.stream(new Double[]{ 1.01, 1d, 0.99, 1.02, 1d,
↪  0.99 });
```

or directly from some values:

```
Stream<String> persons = Stream.of("John", "Demetra", "Cleopatra");
```

or concatenate other streams together:

```
Stream<String> stream1 = Stream.of(/* some values */);
Stream<String> stream2 = Stream.of(/* some values */);
Stream<String> resultStream = Stream.concat(stream1, stream2);
```

Stream operations

All stream operations are divided into two groups:

- **Intermediate operations** are not evaluated immediately when invoking. They simply return new streams to call the next operations on them (**lazy**).
- **Terminal operations** initiate the traversal needed to produce a result or perform an action. A pipeline is consumed at most once; short-circuiting and optimizations can avoid processing some elements or stages.

Once a terminal operation has been evaluated, it is **not permitted** to reuse the stream again.

More precisely, stream reuse is prohibited and **may** be detected with `IllegalStateException`; do not rely on detection in every situation. Create a new stream from the source for another traversal. A stream is a computation, not a reusable container.

Read a lazy pipeline

Predict: when is the filter below evaluated?

```
Stream<Integer> selected = Stream.of(1, 2, 3)
        .filter(value -> {
            IO.println("Checking " + value);
            return value > 1;
        });
IO.println("Pipeline created");
IO.println(selected.findFirst().orElse(-1));
```

Check your prediction

The output is `Pipeline created`, then `Checking 1`, `Checking 2`, and `2`. This sequential short-circuiting traversal stops when it finds a match; not every source element needs to be examined.

Intermediate operations

- `filter` returns a new stream that includes the elements that match a **predicate**;
- `map` returns a new stream that consists of the elements that were obtained by applying a **function** (i.e. transforming each element).
- peek returns the same stream of elements but allows observing the current elements with a **consumer**;
- `sorted` returns a new stream that includes elements sorted according to the natural order or a given **comparator**;
- `limit` returns a new stream that consists of the first `n` elements of this stream;
- `skip` returns a new stream without the first `n` elements of this stream;
- `distinct` returns a new stream consisting of only unique elements according to the results of `equals`;

Terminal operations

- `count` returns the number of elements in the stream as a `long` value;
- `max` / `min` returns an `Optional` maximum/minimum element of the stream according to the given comparator;
- `reduce` combines values from the stream into a single value (an aggregate value);
- `findFirst` / `findAny` returns the first / any element of the stream as an `Optional`;
- `anyMatch` returns `true` if at least one element matches a predicate (see also: `allMatch`, `noneMatch`);
- `forEach` takes a **consumer** and applies it to each element of the stream (for example, printing it);
- `collect` performs a mutable reduction; the result may be a collection, map, string, or another aggregate;
- `toList` returns an unmodifiable list in encounter order, when one exists;
- `toArray` returns an array of the values in the stream.

> **NOTE**: Some terminal operations (findAny, findFirst, max, min, etc.) return an `Optional` because the stream can be empty.

Intermediate operations, in detail

The filter method

The `filter` method returns a new stream consisting only of those elements that match the given predicate.

We'd like to create a new list consisting of prime numbers that belong to the range from 11 to 23 (inclusively).

```
List<Integer> primeNumbers = List.of(2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31);

List<Integer> filteredPrimeNumbers = primeNumbers.stream()
        .filter(n -> n >= 11 && n <= 23)
        .toList();
```

Sometimes, two or more filters are used together. We'd like to count how many programming languages start with an upper letter ignoring all the empty strings.

```
List<String> programmingLanguages = Arrays.asList("Java", "", "scala", "Kotlin", "",
↪  "clojure");

long count = programmingLanguages.stream()
        .filter(lang -> lang.length() > 0)
        .filter(lang -> Character.isUpperCase(lang.charAt(0)))
        .count();
```

These two filter operations can be replaced with a single operation that takes a complex predicate (less readable):

```
List<String> programmingLanguages = Arrays.asList("Java", "", "scala", "Kotlin", "",
↪  "clojure");

long count = programmingLanguages.stream()
        .filter(lang -> lang.length() > 0 && Character.isUpperCase(lang.charAt(0)))
        .count();
```

The map method

`map` is a method of the `Stream` interface that takes a function as a parameter.

Let's divide each number by 2. For that, we can *map* each element of the stream to the element divided by 2 and *collect* it in the new list:

```
List<Double> numbers = List.of(6.28, 5.42, 84.0, 26.0);

List<Double> famousNumbers = numbers.stream()
        .map(number -> number / 2)
        .collect(Collectors.toList());
```

Using map for extracting properties

`map` is often used to get a stream of properties from a stream of objects. For example, given a class `Job`, we can get the list of job titles from a given list of jobs by using the `map` method:

```
public class Job {
    private String title;
    private String description;
    private double salary;

    public Job(String title, String description, double salary) {
        this.title = title;
        this.description = description;
        this.salary = salary;
    }

    public String getTitle() {
        return title;
    }
}
```

```
List<String> titles = jobs.stream()
        // synthetic form of .map(job -> job.getTitle())
        .map(Job::getTitle)
        .collect(Collectors.toList());
```

The flatMap method

The method `stream()` of a `List<List<String>>` returns a `Stream<List<String>>`. In that case, we often need to *flatten* a stream of collections to a stream of elements from these collections.

Flattening refers to merging elements of a list of lists into a single list. For example, if we flatten a list of lists `[["a", "b"], ["c"], ["d", "e"]]`, we will get the list `["a", "b", "c", "d", "e"]`.

The `flatMap` method applies a function to transform each stream element into a new stream and concatenates these streams together.

Let's consider an example with Java books. Each book has a title, a publishing year, and a list of authors:

```
record Book(String title, int year, List<String> authors) {
    public Book {
        authors = List.copyOf(authors);
    }

    public List<String> getAuthors() {
        return authors;
    }
}
```

```
List<Book> javaBooks = List.of(
        new Book("Java EE 7 Essentials", 2013, List.of("Arun Gupta")),
        new Book("Algorithms", 2011, List.of("Robert Sedgewick", "Kevin Wayne")),
```

```
        new Book("Clean code", 2014, List.of("Robert Martin"))
);
```

Now we can obtain a list of all authors from the list of Java books by using `flatMap` method:

```
List<String> authors = javaBooks.stream()
        .flatMap(book -> book.getAuthors().stream())
        .collect(Collectors.toList());
```

The resulting list is:

```
["Arun Gupta", "Robert Sedgewick", "Kevin Wayne", "Robert Martin"]
```

The peek method

Note — diagnostics, not required business actions. peek observes elements as they are consumed. A terminal operation may short-circuit, and an implementation may elide traversal when its result is already known. Therefore, a peek action is not guaranteed to run for every source element.

```
List<Integer> primeNumbers = List.of(2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31);

List<Integer> primeNumbersCopy = primeNumbers.stream()
        // synthetic form of .peek(number -> IO.println(number))
        .peek(IO::println)
        .collect(Collectors.toList());
```

Gatherers available in Java 25

Note — an optional extension after map and flatMap. `Stream.gather` supports intermediate transformations such as grouping consecutive elements into windows. Gatherers became final in Java 24 and are stable in Java 25.

```
List<List<Integer>> batches = Stream.of(1, 2, 3, 4, 5)
        .gather(java.util.stream.Gatherers.windowFixed(2))
        .toList();
IO.println(batches); // [[1, 2], [3, 4], [5]]
```

The final incomplete batch is retained. A gatherer is an **intermediate** operation, unlike a collector used by terminal `collect`. Custom gatherer implementations are reference material, not required for the introductory pipeline exercises.

Predict: what changes if `limit(2)` appears after `gather` instead of before it? Are you limiting input elements or output batches?

Filtering students, the easy way!

We need to select a group of students with a specific features, accumulate them into a List, and also print a string representation of those selected.

Use this teaching representation with finite grade averages:

```
record Student(String name, String lastname, double average) {
    public String getName() { return name; }
    public String getLastname() { return lastname; }
    public double getAverage() { return average; }
}
```

```
public static void main(String[] args) {
    List<Student> students = new ArrayList<>(List.of(
            new Student("John", "Doe", 24),
            new Student("Dimebag", "Darrell", 25),
            new Student("Eric", "Baret", 28),
            new Student("Tyler", "Durden", 20),
            new Student("Jack", "Napier", 22),
            new Student("Diana", "Krall", 27),
            new Student("Rosa", "Luxemburg", 30)));

    List<Student> selectedStudents = students.stream()
            .filter(s -> s.getAverage() >= 26 && s.getAverage() <= 30)
            .toList();

    selectedStudents.forEach(s -> System.out.printf(java.util.Locale.ROOT,
            "%s_%s_%f%n", s.getLastname(), s.getName(), s.getAverage()));
}
```

Printing is a required action here, so it happens explicitly after selection rather than being hidden inside peek. The result list is unmodifiable; choose `toCollection(ArrayList::new)` if callers need to modify it.

Resources

E7.4 — Stream Collection and Reduction

Terminal operations, in detail

The collect method

`collect` is a terminal operation that can accept an object of the Collector type. The `Collectors` class contains static methods that return the Collector and implement functionality for accumulating stream elements.

Choosing a list result

Core: select the mutability contract deliberately:

```
List<Integer> readOnly = Stream.of(1, 2, 3).toList();
List<Integer> mutable = Stream.of(1, 2, 3)
        .collect(Collectors.toCollection(ArrayList::new));
mutable.add(4);
```

`Stream.toList()` is unmodifiable, not deeply immutable. `Collectors.toList()` does **not** promise a particular list type or mutability; use `toCollection` when a mutable `ArrayList` is required.

Accumulating into collections

Let's consider the `Account` class below:

```
public class Account {
    private long balance;
    private String number;

    public Account(long balance, String number) {
        this.balance = balance;
        this.number = number;
    }

    public long getBalance() {
        return balance;
    }

    public String getNumber() {
```

```
        return number;
    }

    @Override
    public String toString() {
        return "Account{balance=" + balance + ", number='" + number + "'}";
    }
}
```

We can accumulate stream elements:

```
public static void main(String[] args) {

    List<Account> accounts = new ArrayList<>(List.of(
            new Account(1000, "IT001"),
            new Account(2500, "IT002"),
            new Account(500, "IT003"),
            new Account(800, "IT001")));

    // toList
    List<String> numbersList = accounts.stream()
            .map(Account::getNumber)
            .collect(Collectors.toList());

    IO.println("List: " + numbersList);

    // toSet
    Set<String> numbersSet = accounts.stream()
            .map(Account::getNumber)
            .collect(Collectors.toSet());

    IO.println("Set: " + numbersSet);
}
```

Accumulating single values

The `collect()` method is able to accumulate stream elements into a single value. Here you can see some Collectors methods that produce a single value:

- summingInt, summingLong, summingDouble;
- averagingInt, averagingLong, averagingDouble;
- maxBy, minBy;
- counting.

```
long summary = accounts.stream()
// synthetic form of .summingLong(account -> account.getBalance())
.collect(Collectors.summingLong(Account::getBalance));
```

```
double average = accounts.stream()
.collect(Collectors.averagingLong(Account::getBalance));
```

Partitioning

Imagine that we want to divide a collection of accounts into two groups: accounts whose balance is greater than or equal to 10000, and accounts with a balance lower than 10000.

The partitioning operation is presented by the `Collectors.partitioningBy` method that accepts a predicate. It splits input elements into a `Map` of two lists: one list contains elements for which the predicate is true, and the other contains elements for which it is false. The keys of the `Map` has the `Boolean` type.

To illustrate the idea, let's create the following list of accounts:

```
List<Account> accounts = List.of(
        new Account(3333L, "530012"),
        new Account(15000L, "771843"),
        new Account(0L, "681891")
);
```

And partition them into two lists by a `balance >= 10000` predicate:

```
Map<Boolean, List<Account>> accountsByBalance = accounts.stream()
        .collect(Collectors.partitioningBy(account -> account.getBalance() >= 10000));
```

The `accountsByBalance` map contains the following entries:

```
{
    false=[Account{balance=3333, number='530012'}, Account{balance=0, number='681891'}],
     true=[Account{balance=15000, number='771843'}]
}
```

Grouping

Instead of splitting data into two groups based on a predicate, the grouping operation can produce any number of groups based on a *classification function* that maps elements to some key.

The grouping operation is presented by the `Collectors.groupingBy` method that accepts a classification function. The collector `groupingBy` also produces a `Map`. The keys of the `Map` are values produced by applying the classification function to the input elements. The corresponding values of the `Map` are lists containing elements mapped by the classification function.

Let's create the `Status` enum and add field `status` to the `Account` class:

```
enum Status {
    ACTIVE,
    BLOCKED,
    REMOVED
}

public class Account {
    private long balance;
    private String number;
    private Status status;

    public Account(long balance, String number, Status status) {
        this.balance = balance;
        this.number = number;
        this.status = java.util.Objects.requireNonNull(status);
    }

    public long getBalance() {
        return balance;
    }

    public String getNumber() {
        return number;
    }

    public Status getStatus() {
        return status;
    }

    @Override
    public String toString() {
        return "Account{balance=" + balance + ", number='" + number + "'}";
    }
}
```

Also, let's update the list of accounts:

```
List<Account> accounts = List.of(
        new Account(3333L, "530012", Status.REMOVED),
        new Account(15000L, "771843", Status.ACTIVE),
        new Account(0L, "681891", Status.BLOCKED)
);
```

Now, we can divide all account into groups by its `status`:

```
Map<Status, List<Account>> accountsByStatus = accounts.stream()
        .collect(Collectors.groupingBy(Account::getStatus));
```

The `accountsByStatus` map contains the following entries:

```
{
    BLOCKED=[Account{balance=0, number='681891'}],
    REMOVED=[Account{balance=3333, number='530012'}],
    ACTIVE=[Account{balance=15000, number='771843'}]
}
```

Downstream collectors

In addition to a predicate or a classification function, `partitioningBy` and `groupingBy` collectors can accept a **downstream** collector. Such a collector is applied to the results of another collector.

Let's create the following list of accounts:

```
List<Account> accounts = List.of(
        new Account(3333L, "530012", Status.ACTIVE),
        new Account(15000L, "771843", Status.BLOCKED),
        new Account(15000L, "234465", Status.ACTIVE),
        new Account(8800L, "110011", Status.ACTIVE),
        new Account(45000L, "462181", Status.BLOCKED),
        new Account(0L, "681891", Status.REMOVED)
);
```

And calculate the total balances of `blocked`, `active`, and `removed` accounts using a downstream collector:

```
Map<Status, Long> sumByStatuses = accounts.stream()
        .collect(Collectors.groupingBy(
                Account::getStatus,
                Collectors.summingLong(Account::getBalance))
         );
```

The code above groups accounts by the `status` field and applies a downstream `summingLong` collector to the `List` values created by the `groupingBy` operator. The resulting map contains the following entries:

```
{ REMOVED=0, ACTIVE=27133, BLOCKED=60000 }
```

The downstream collector accumulates per group; it does not require building an intermediate list for each key. Printed map and set order is not guaranteed by these default collectors. The displayed entries describe contents, not a required iteration order.

> **Note:** `Collectors.toMap` without a merge function rejects duplicate keys. Decide whether duplicates are invalid, should be combined, or should replace a value before choosing a merge function. Do not silence the failure by arbitrarily keeping one result.

Numeric streams and reduction

```
long total = java.util.stream.LongStream.of(10, 20, 30).sum();
int sum = Stream.of(1, 2, 3).reduce(0, Integer::sum);
IO.println(total); // 60
IO.println(sum);   // 6
```

For reduction, choose an identity and an associative operation compatible with it. Subtraction is not associative, and floating-point addition is not exactly associative. Parallel grouping can change floating-point rounding results. Primitive streams also avoid boxing for their numeric operations.

Source interference and parallelism

Core: lambdas should be non-interfering with the stream source and usually stateless. Do not modify the source list from inside its pipeline or append parallel results into an ordinary shared `ArrayList`.

Review: an assistant replaces `stream()` with `parallelStream()` and collects results using `forEach(result::add)`. Identify the shared mutable state and use a suitable collection operation instead. `forEach` does not preserve encounter order in parallel execution; `forEachOrdered` addresses order, not arbitrary shared-state safety.

> **Note:** parallel streams are not a default speed optimization. Consider input size, splitting cost, blocking operations, ordering, the shared execution pool, and measured performance. A sequential pipeline is often the clearest starting point.

Streams backed by resources

Note — close owned resource-backed streams with try-with-resources:

```
public static long countNonBlankLines(java.nio.file.Path path) throws
 ↪  java.io.IOException {
    try (Stream<String> lines = java.nio.file.Files.lines(path)) {
        return lines.filter(line -> !line.isBlank()).count();
    }
}
```

Close an owned file-backed stream even when short-circuiting. Streams from ordinary in-memory collections do not own an external resource. Deferred reading can also fail during traversal, for example with `UncheckedIOException` from `Files.lines`.

Resources

- https://dev.java/learn/api/streams/
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Stream.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Gatherers.html
- https://www.baeldung.com/java-streams

E8.1 — Threads and Execution

Examples use `java.util`, `java.util.concurrent`, `java.util.concurrent.atomic`, and `java.util.random` types; add the imports required by each block. Fragments belong in a class or method as indicated. Deliberately unsafe examples demonstrate a bug, not recommended application patterns. Console ordering is unspecified unless an explicit coordination rule establishes it.

OS perspective

Reference — background for the Java execution model.

Process

An OS process is an executing program with its own address space and associated resources. Processes normally use IPC to communicate, including pipes, sockets, and explicitly shared memory; separate address spaces do not mean shared memory is impossible.

`ProcessBuilder` launches an external program and returns a `Process` handle. The Java API does not promise a particular fork/exec implementation strategy.

```
// Linux-specific command; surrounding method declares IOException and InterruptedException.
Process process = new ProcessBuilder("/bin/ls", "-al", "/")
        .redirectErrorStream(true).start();
try (BufferedReader in = new BufferedReader(new InputStreamReader(process.getInputStream()))
    String line;
    while ((line = in.readLine()) != null) {
        IO.println(line);
    }
}
IO.println(process.waitFor()); // Wait for exit before retrieving its status.
```

Thread

Threads are frequently called lightweight processes. Like processes, **each thread has its own stack, program counter, and local variables**. However, **threads within the same process share the same address space and, consequently, can share variables and objects**.

Sharing variables is a simple and fast way threads use for communicating but **frequently causes bugs unseen in single-thread programs**

JVM and OS

The OS scheduler shares CPU execution among runnable platform threads. **Concurrency** means work can overlap in time; **parallelism** means work actually executes simultaneously on multiple processing units. No fixed scheduling time slice should be assumed by the application.

Platform threads are typically backed by OS threads. Java 25 also supports **virtual threads**, a stable feature since Java 21, scheduled by the JVM onto carrier platform threads. Both use the Thread API; the same shared-state correctness rules apply.

Why Threads?

There are many reasons to use threads in your Java programs. If you use Android, Swing, JavaFX, Servlets, RMI you may already be using threads without realizing it. The main reasons for using threads:

- **keep UIs responsive while performing background operations**
- **take advantage of multiprocessor systems**

Case study

Imagine a stockbroker application with three key functionalities:

- Download stock prices from a remote REST service and store them in a local database
- Use the data contained in the database to make short-term price predictions
- Provide a JavaFX GUI for the user

In a single-threaded process actions execute one after another on a single thread. An action happens only when the previous one is completed.

- If the download takes 10 minutes, should the user enjoy an unresponsive UI for 10 minutes?
- If price analysis takes additional 10 minutes the result may come too late. Prices could already have changed significantly!

In a multithreaded process actions execute on different threads:

- the download can execute in background (i.e. in another thread)
- the analysis, too, can execute in background (i.e. possibly in multiple threads)
- the user can enjoy a responsive UI while waiting for notifications (buy/sell signals)

Summarizing

The good of multi-threading

- Enable parallelism
- Lighter than processes for both
  - Creation(i.e., fork())
  - Communication (i.e., r/w pipes ...)

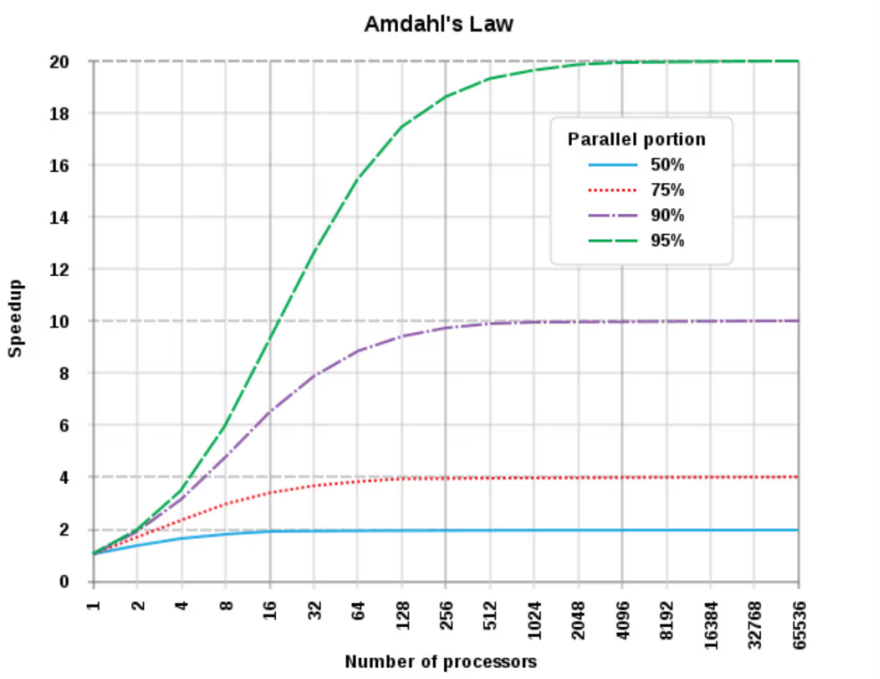
The bad of multi-threading

- Hard for most programmers
- Even for experts, development is often painful
- Shared mutable state couples components unless concurrency responsibilities are encapsulated deliberately.

The Amdahl's law

Reference — a bound on speedup, not a promise of performance.

In computer architecture, Amdahl's law is a formula which gives the theoretical speedup of the execution of a task that can be expected from a system whose resources are improved. It states that *the overall performance improvement gained by optimizing a single part of a system is limited by the fraction of time that the improved part is actually used*.



Threads in Java

Every Java program has at least one thread, which is called the *main thread*, created automatically by the JVM process to execute statements inside the *main method*. All Java programs have

some other default threads as well (for example, a separate thread for the garbage collector).

Throughout the stages of development of the Java language, the approach to multithreading has changed towards the use of high-level abstractions (i.e., Executors, Tasks, etc...). However, understanding the fundamentals remains very important for a good developer.

Any thread has a name, an identifier, a priority, and some other characteristics that can be obtained through its methods. The example below demonstrates how to obtain the characteristics of the *main thread*:

```
public class MainThreadDemo {
    public static void main(String[] args) {
        Thread t = Thread.currentThread(); // main thread

        IO.println("Name: " + t.getName());
        IO.println("ID: " + t.threadId());
        IO.println("Alive: " + t.isAlive());
        IO.println("Priority: " + t.getPriority());
        IO.println("Daemon: " + t.isDaemon());

        t.setName("my-thread");
        IO.println("New name: " + t.getName());
    }
}
```

```
Name: main
ID: 1
Alive: true
Priority: 5
Daemon: false
New name: my-thread
```

This is illustrative output; do not assert a fixed thread ID. `threadId()` is the modern API; `getId()` has been deprecated since Java 19.

Creating threads

Java has two primary ways to create a new thread:

- by extending the *Thread* class and overriding its *run* method;
- by implementing the *Runnable* interface and passing the implementation to the constructor of the *Thread* class.

```
class HelloThread extends Thread {

    @Override
    public void run() {
        String helloMsg = String.format("Hello, i'm %s", getName());
        IO.println(helloMsg);
    }
}
```

```
class HelloRunnable implements Runnable {

    @Override
    public void run() {
        String helloMsg = String.format("Hello, i'm %s",
        ↪  Thread.currentThread().getName());
        IO.println(helloMsg);
    }
}
```

```
Thread t1 = new HelloThread(); // a subclass of Thread

Thread t2 = new Thread(new HelloRunnable()); // passing runnable
```

Both versions supply a `run` implementation. Prefer expressing the **task** as a `Runnable` and choosing its execution mechanism separately, rather than subclassing `Thread` just to attach work. The task form fits lambdas:

```
Thread t3 = new Thread(() -> {
    IO.println(String.format("Hello, i'm %s", Thread.currentThread().getName()));
});
```

Starting threads

`start()` schedules a new thread to execute `run()`. It does not wait for completion; the new thread may execute before or after `start()` returns. Calling `run()` directly is only an ordinary call on the current thread.

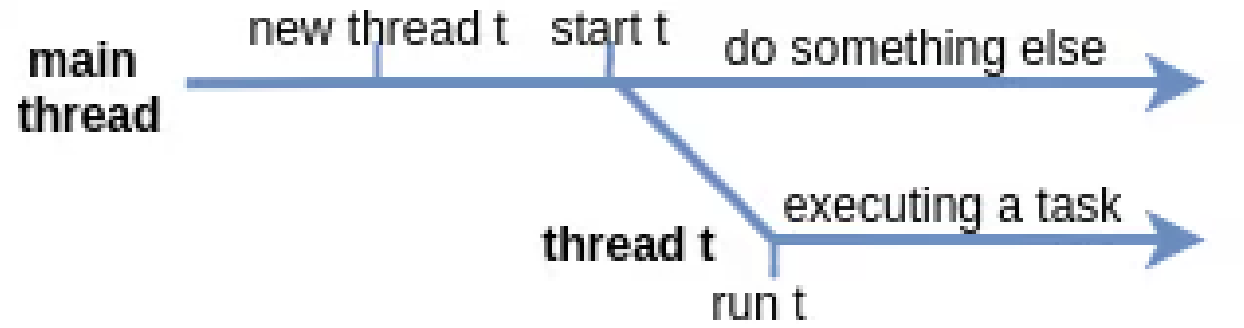
Let's suppose that inside the *main method* you create a *HelloThread* object named *t* and start it.

```
Thread t = new HelloThread(); // an object representing a thread
t.start();
```

Eventually, it prints something like:

```
Hello, i'm Thread-0
```

The picture illustrates one possible schedule. It does not establish a guaranteed delay or ordering between the caller and the new thread.



If you try to start a thread more than once, the *start* method throws *IllegalThreadStateException*.

Execution order

Although each thread follows its own program order, **the relative order of actions in different threads is not guaranteed without coordination**, such as waiting for termination with `join()` or using synchronization.

Consider the following code:

```
public class StartingMultipleThreads {

    public static void main(String[] args) {
        Thread t1 = new HelloThread();
        Thread t2 = new HelloThread();

        t1.start();
        t2.start();

        IO.println("Finished");
    }
}
```

The order of displaying messages may be different. Here is one of them:

```
Hello, i'm Thread-1
Finished
Hello, i'm Thread-0
```

It is even possible that all threads print their text after the *main thread* prints *"Finished"*:

```
Finished
Hello, i'm Thread-0
Hello, i'm Thread-1
```

Summarizing:

- It is not guaranteed that threads will start running in the same order in which their start() methods have been called.
- It is not guaranteed that a thread keeps executing until it's done (it is not guaranteed that its loop completes before another thread begins)
- Completion and fairness are not generally guaranteed: tasks can loop forever, deadlock, block indefinitely, or fail.

Predict: if `t1.start()` is replaced with `t1.run()`, which thread executes that task? Add `Thread.currentThread().getName()` to verify the distinction.

Terminating threads

Threads do not form an automatic lifetime tree. A thread can continue after the thread that created it has finished.

- The JVM ordinarily remains alive while started **non-daemon** threads remain alive.
- Once no non-daemon threads remain, shutdown can begin; daemon threads do not keep the JVM alive.
- Use `join` or task-result APIs to wait for required work, not assumptions about a “parent” thread.
- Interruption is a **cooperative request**, not forced termination.

```
public class StartStopThread extends Thread {
    public StartStopThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        IO.println(getName() + " started");
        while (!isInterrupted()) {
            IO.println(getName());
        }
        IO.println(getName() + " terminated");
    }

    public static void main(String[] args) throws InterruptedException {
        StartStopThread a = new StartStopThread("Homer");
        StartStopThread b = new StartStopThread("Marge");

        /* start children threads */
        a.start();
        b.start();

        /* wait 10 millis */
        Thread.sleep(10L);

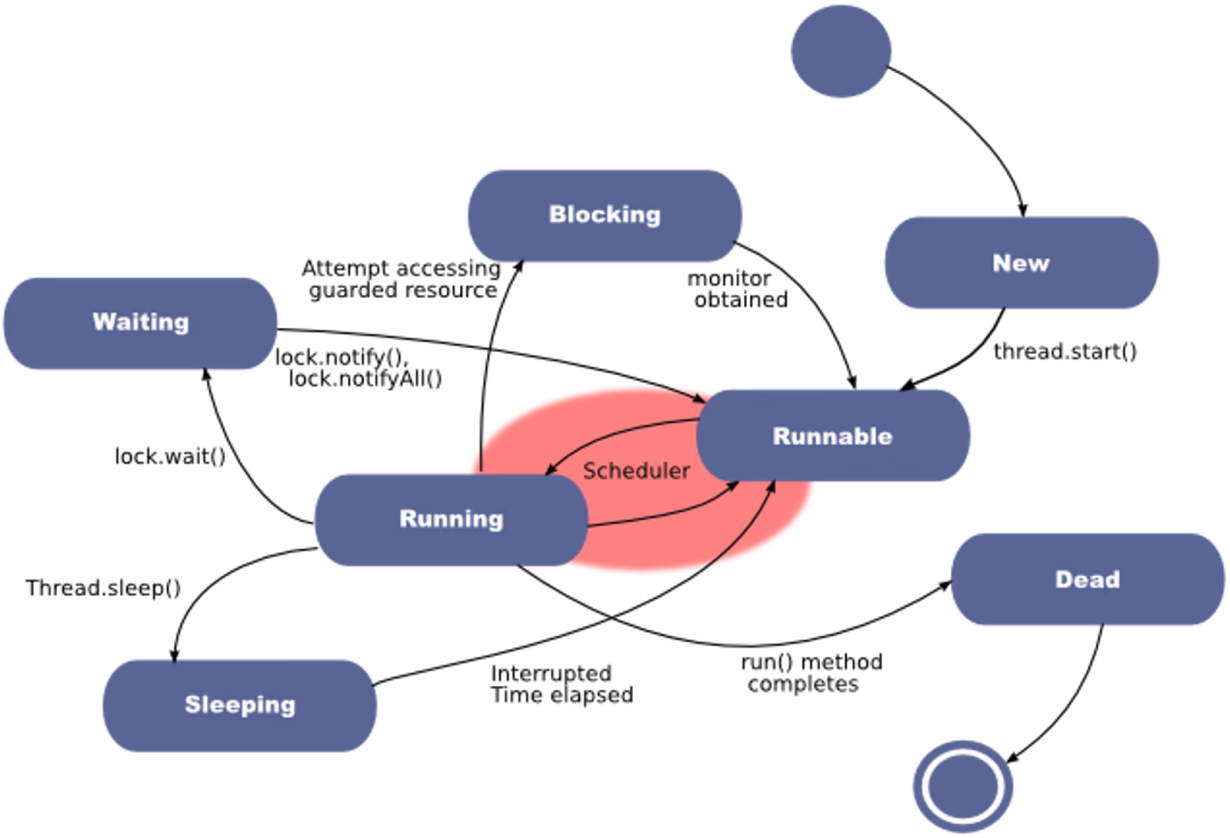
        /* gracefully terminate children threads */
        a.interrupt();
        b.interrupt();
```

```
        /* wait for children before exit */
        a.join();
        b.join();
    }
}
```

The sleep here only lets the demo run briefly; it does not synchronize shared state. Printing in a tight loop is deliberately simple, not a workload or performance model.

`isInterrupted()` observes a thread's flag without clearing it. Static `Thread.interrupted()` checks and clears the current thread's flag. Interruptible methods such as `sleep`, `wait`, and blocking-queue operations can throw `InterruptedException` and clear the flag; propagate it, terminate, or restore the flag when handling it at a task boundary.

Thread states



Reference: the image is a conceptual scheduling diagram. Java's `Thread.State` exposes these six states instead:

State	Meaning
`NEW`	Not yet started
`RUNNABLE`	Executing in the JVM or eligible to execute; can include some OS waits
`BLOCKED`	Waiting to acquire an intrinsic monitor
`WAITING`	Indefinite wait, e.g. untimed `wait` or `join`
`TIMED_WAITING`	Timed wait, including `sleep`
`TERMINATED`	Execution completed

Java has no separate `RUNNING` or `SLEEPING` enum state. A state snapshot is diagnostic information, not a synchronization protocol.

Leaving the running state

Common operations that affect progress include:

- **sleep()**: pauses subject to interruption and timer/scheduler accuracy; does not release monitors.
- **yield()**: offers a hint that the scheduler may ignore; no correctness guarantee.
- **join()**: waits for the target thread to terminate, or until an optional timeout/interruption.
- **wait()**: releases the specified monitor while waiting, then reacquires it before returning.

There are additional ways for leaving the running state which are not under the control of the programmer:

- End of `run()` produces `TERMINATED`.
- OS descheduling does not create a separate Java `Thread.State` value.
- I/O blocking is not the same as Java's monitor-specific `BLOCKED` state.

Thread.sleep()

The static method *Thread.sleep()* causes the currently executing thread to suspend execution for the specified number of milliseconds. This is an efficient means of making processor time available for the other threads or processes running on the same node.

```
IO.println("Started");

Thread.sleep(2000L); // suspend current thread for 2000 millis

IO.println("Finished");
```

Thread.yield()

Reference: `yield()` is only a scheduler hint. It is not a replacement for a blocking queue, a lock, or waiting on a condition. Do not use it to “fix” a race or implement an idle consumer.

Thread.join()

The *Thread.join()* method forces the current thread to wait for the completion of the thread for which the method *join* was called. In the following example, the string **”Do something else”** will not be printed until the thread terminates.

```
Thread thread = ...
thread.start(); // start thread

IO.println("Do something useful");

thread.join();  // waiting for thread to die

IO.println("Do something else");
```

There is an overloaded version of this method which takes a waiting time in milliseconds and is used to avoid waiting for too long.

```
// waits at most two seconds
thread.join(2000L);
```

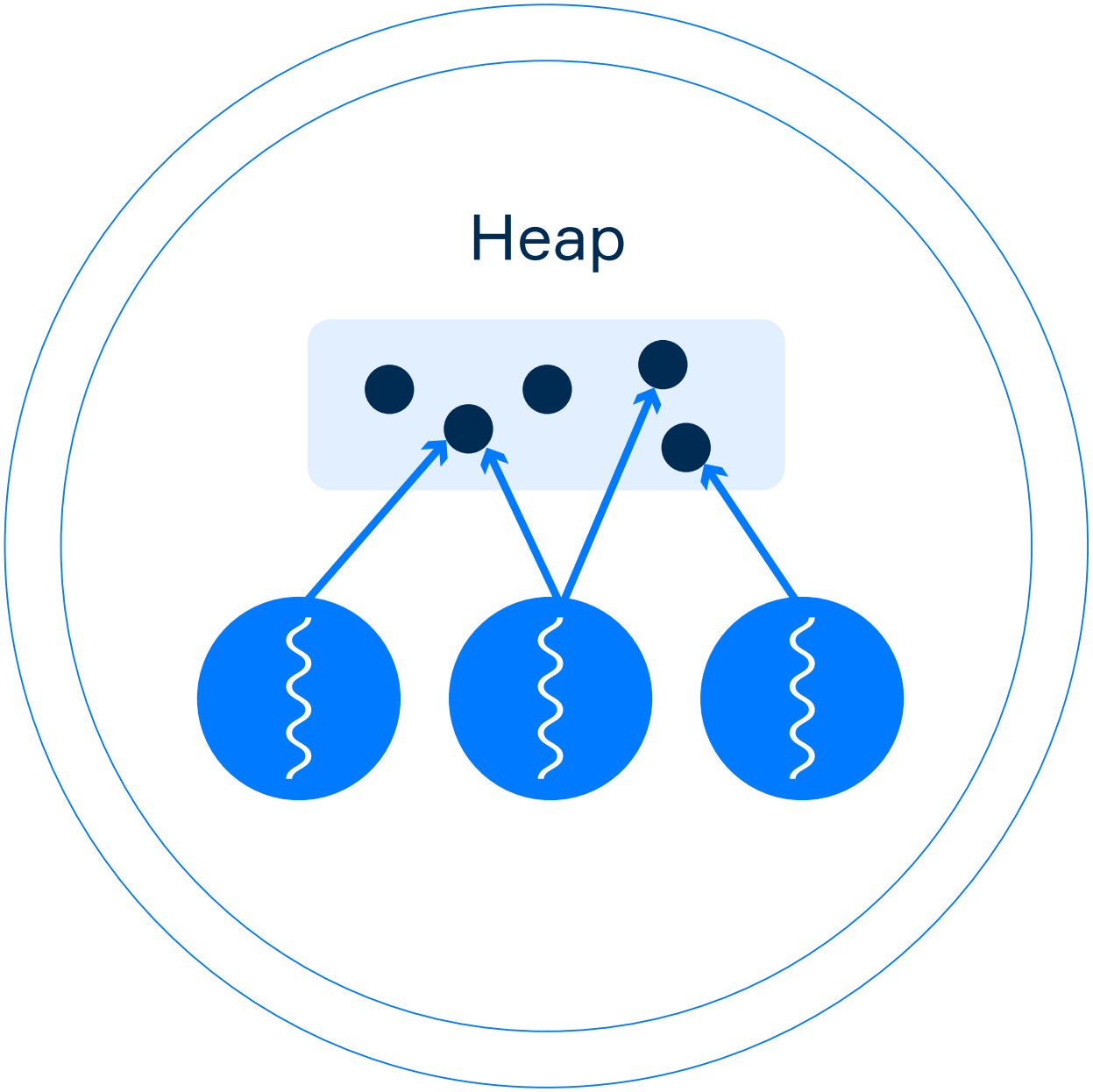
A timed join can return while the target is still alive. Decide how to handle that outcome rather than assuming completion.

Resources

E8.2 — Shared State and Synchronization

Thread Interference

Threads can share reachable objects. They can also communicate through message-passing abstractions such as blocking queues; shared address space does not force every client to coordinate low-level shared variables itself.



When you write your code in different threads that work with the same data concurrently, many things might go wrong. Imagine two people (represented by two threads) each one having an ATM card linked to the same account:

```
class Account {
    private int balance;

    public Account(int balance) {
        this.balance = balance;
    }
```

```
    public int getBalance() {
        return balance;
    }

    public void withdraw(int amount) {
        balance -= amount;
    }
}
```

Each person (i.e., thread) does these steps:

1. Check if the balance is greater than 0
2. Choose an amount to withdraw comprised between 0 and balance
3. Withdraw the chosen amount

```
public void run() {
    RandomGenerator rnd = RandomGenerator.getDefault();
    while (!interrupted()) {
        if (account.getBalance() <= 0) {
            break;
        }

        int amount = rnd.nextInt(account.getBalance() + 1);

        account.withdraw(amount);

        Thread.yield();
    }
}
```

What happens if the scheduler suspends one thread between step 2 and step 3 and the other one gets executed?

1. **Homer enters the status RUNNING**
2. Homer checks that the account is not empty and contains 125$!
3. Homer chooses to withdraw 100$
4. **Homer leaves the status RUNNING**
5. **Marge enters the status RUNNING**
6. Marge checks that the account is not empty and contains 125$!
7. Marge chooses to withdraw 120$
8. Marge withdraws 120$
9. **Marge leaves the status RUNNING**
10. **Homer enters the status RUNNING**
11. Homer subtracts 100 from 5: this implementation can produce a negative balance. The code does not enforce a partial-withdrawal policy.



The intentionally unsafe loop can also fail while choosing a random amount if a racing balance read becomes negative. Repeated balance reads are not a consistent snapshot, and a run with no visible failure does not establish safety.

Critical sections

A critical section accesses shared state that must be protected from conflicting concurrent operations. Mutual exclusion prevents cooperating threads from entering the pro-

tected region simultaneously. It does not stop OS preemption while a thread holds the lock.

The balance check and withdrawal must form one coordinated operation. Synchronizing each getter and setter separately would not make the whole check-then-act sequence atomic.

Atomicity, visibility, and ordering

- **Atomicity:** an operation is indivisible relative to relevant concurrent operations.
- **Visibility:** a thread observes writes made by another thread under the memory model's guarantees.
- **Ordering:** a happens-before relationship establishes constraints on how writes can be observed.

Examples of happens-before edges include actions before `start()` to the started thread, a thread's actions to a successful `join`, and monitor unlock to a subsequent lock of that same monitor. Publishing an object reference without an appropriate protocol is not automatically safe.

Natively Atomic Operations in Java

An individually atomic access does **not** by itself provide visibility for other data or make a compound algorithm race-free.

1. **Reads and writes of primitive types**

- Reads/writes of `byte`, `short`, `int`, `char`, `float`, `boolean`, and references are atomic.
- The specification permits non-atomic reads/writes of non-volatile `long` and `double`; volatile reads/writes of those types are atomic.
- Example:

```
int count = 10; // atomic write
int c = count;  // atomic read
```

Not Natively Atomic

These operations are **not atomic** on their own:

- Compound operations: x++, x = x + 1, x += 5
- Any operation involving multiple steps

```
public class RaceConditionWrong {
  static int times = 10000;
  static int count = 0; // shared variable

  public static void main(String[] args) throws InterruptedException {
    Thread t1 = new Thread(() -> {
      for (int i = 0; i < times; i++) {
```

```
        count++;
      }
    });

    Thread t2 = new Thread(() -> {
      for (int i = 0; i < times; i++) {
        count++;
      }
    });

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    IO.println("Final count: " + count); // Expected: 20000, Actual: often less
  }
}
```

For compound atomic operations, use classes in `java.util.concurrent.atomic`, such as `AtomicInteger`.

`volatile` can support a visibility protocol, such as a stop flag, but **does not make `count++` atomic**. Use an atomic increment or a common lock. Atomic variables also do not automatically coordinate invariants spanning several variables.

```
public class RaceConditionAtomic {
    static int times = 10000;
    static AtomicInteger atomicInteger = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < times; i++) {
                atomicInteger.incrementAndGet();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < times; i++) {
                atomicInteger.incrementAndGet();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        IO.println("Final count: " + atomicInteger.get()); // Always prints 20000
    }
}
```

The *synchronized* keyword

Developers cannot rely on an uninterrupted time slice. Correctness comes from a coordination protocol, not scheduling luck or `yield()`.

`synchronized` acquires an object's intrinsic monitor for a method or a block. Only one thread holds that monitor at a time, and unlocking/locking it provides visibility. All accesses relevant to the shared invariant must participate in the same protocol; an object is not globally protected just because one method is synchronized.

More specifically, every object in Java has ONE built-in lock. **Entering a synchronized non-static method means getting the lock of the object**. If one thread gets the lock, all other threads have to wait to enter ALL the synchronized methods until the lock is released (the first thread exits the synchronized method).

The monitor is reentrant: the owning thread can acquire it again. A static synchronized method locks the declaring `Class` object, not an arbitrary instance.

```
public synchronized void doStuff() {
    IO.println("synchronized");
}

/* is equivalent to… */

public void doStuff() {
    synchronized(this) {
        IO.println("synchronized");
    }
}
```

Locking granularity

First, repair the earlier balance operation by keeping the invariant inside the shared object:

```
public final class SafeAccount {
    private long balance;

    public SafeAccount(long initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Negative balance");
        }
        balance = initialBalance;
    }

    public synchronized boolean tryWithdraw(long amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
        if (amount > balance) {
            return false;
        }
```

```
        balance -= amount;
        return true;
    }

    public synchronized long getBalance() {
        return balance;
    }
}
```

The check and update share one lock, and all balance access follows that protocol. For a starting balance of 125, concurrent requests for 100 and 120 cannot both succeed. Which one succeeds is not predetermined. This example uses integer units and does not model currency conversion or an actual ATM transaction.

Which operations share the lock?

Each call to `A()` sleeps while holding the instance monitor, preventing another thread from entering `A()` or `B()` on that same instance until the monitor is released. **All synchronized instance methods of that object share its monitor.** `C()` does not acquire it.

```
class Actor extends Thread {
    Runnable runnable;

    public Actor(Runnable runnable) {
        super();
        this.runnable = runnable;
    }

    @Override
    public void run() {
        while (!isInterrupted()) {
            runnable.run();
        }
    }
}
```

```
class SharedResource {
    public synchronized void A() {
        IO.println(Thread.currentThread().getName() + " A()");
        try {
            Thread.sleep(100L);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public synchronized void B() {
        IO.println(Thread.currentThread().getName() + " B()");
    }
```

```
    public void C() {
        IO.println(Thread.currentThread().getName() + " C()");
    }
}
```

```
public class LockingGranularity {
    public static void main(String[] args) throws InterruptedException {
        SharedResource resource = new SharedResource();
        Actor a = new Actor(resource::A);
        Actor b = new Actor(resource::B);
        Actor c = new Actor(resource::C);

        a.start();
        b.start();
        c.start();

        Thread.sleep(10);

        a.interrupt();
        b.interrupt();
        c.interrupt();

        a.join();
        b.join();
        c.join();
    }
}
```

Whenever an object lock has been acquired by one thread, other threads can still access the class's non-synchronized methods. Methods that don't access critical data don’t need to be synchronized.

Details:

- Threads going to sleep don't release locks!
- A thread can acquire more than one lock. For example, a thread can enter a synchronized method, then immediately invoke a synchronized method on another object (deadlock prone!)

Resources

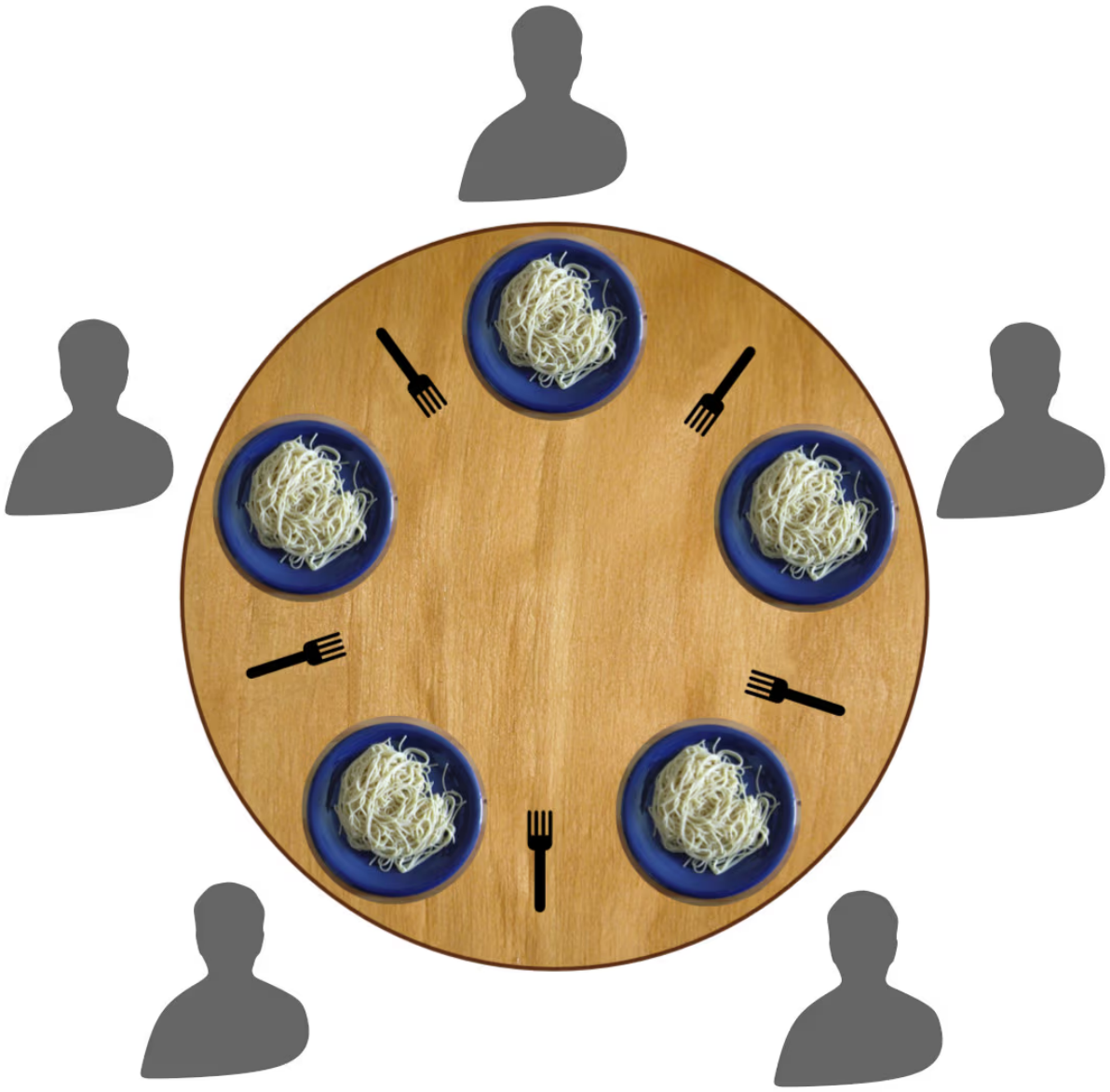
E8.3 — Thread Coordination

The dining philosophers

Reference — deadlock and progress. A consistent global lock-acquisition order is one way to avoid circular waiting. Thread safety is not only about correct final values: tasks must also be able to make progress.

In computer science, the dining philosophers problem is an example problem often used in concurrent algorithm design to illustrate synchronization issues and techniques for resolving them.

It was originally formulated in 1965 by Edsger Dijkstra as a student exam exercise, presented in terms of computers competing for access to tape drive peripherals. Soon after, Tony Hoare gave the problem its present form.



Synchronization using thread-safe objects

Two coordination styles are illustrated below. The polling implementations explain locking but are **not recommended producer/consumer designs**: spinning, exceptions for empty queues, and unbounded growth waste resources.

1. Use the *synchronized* modifier within the *run()* method of each thread to lock the shared object.

```
static class ProducerSafe extends Thread {
    final Deque<Integer> integerDeque;

    public ProducerSafe(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

    @Override
    public void run() {
        int i = 0;
        while (!isInterrupted()) {
            synchronized (integerDeque) {
                integerDeque.addFirst(i++);
            }
        }
    }
}
```

```
static class ConsumerSafe extends Thread {
    final Deque<Integer> integerDeque;

    public ConsumerSafe(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

    @Override
    public void run() {
        while (!isInterrupted()) {
            try {
                synchronized (integerDeque) {
                    integerDeque.removeLast();
                }
            } catch (NoSuchElementException e) {
                Thread.yield();
            }
        }
    }
}
```

2. Encapsulate synchronization in the shared abstraction, or use a suitable concurrent collection.

A thread-safe class preserves its documented contract under concurrent use. Its implementation need not use intrinsic locks: concurrent collections can use other coordination mechanisms. Individual thread-safe operations do not automatically make a multi-operation client sequence atomic.

- Interface List: Vector
- Interface Queue: LinkedBlockingQueue, ArrayBlockingQueue, ConcurrentLinkedQueue
- Interface Map: ConcurrentMap

```java
// Safe only when supplied a suitable thread-safe deque; unsafe with LinkedList.
static class ProducerWithoutExternalLock extends Thread {
    final Deque<Integer> integerDeque;

    public ProducerWithoutExternalLock(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

    @Override
    public void run() {
        int i = 0;
        while (!isInterrupted()) {
            integerDeque.addFirst(i++);
        }
    }
}
```

```java
static class ConsumerWithoutExternalLock extends Thread {
    final Deque<Integer> integerDeque;

    public ConsumerWithoutExternalLock(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

    @Override
    public void run() {
        while (!isInterrupted()) {
            try {
                integerDeque.removeLast();
            } catch (NoSuchElementException e) {
                Thread.yield();
            }
        }
    }
}
```

Synchronization using Object methods

Reference — understand existing low-level code. Prefer blocking collections for routine producer/consumer coordination.

Threads might be able to acquire exclusive access a shared resource but still be unable to progress. For example, a producer with a full queue, or a consumer with an empty queue.

To avoid waste of computational resources we can use the *wait()* and *notify()* methods (which are inherited from Object).

`wait()` requires ownership of that same object's monitor, whether acquired through a block or a synchronized method. It releases that monitor, waits, and reacquires it before returning. It does not release unrelated locks the thread holds.

`notify` and `notifyAll` also require ownership of that monitor. Notification does not transfer the lock immediately or promise that a condition is now true. Always recheck the condition in a **while loop**: other threads can change it, and spurious wakeups are permitted.

```
static class ProducerSafeWaitNotify extends Thread {
    final Deque<Integer> integerDeque;
    final int maxDequeSize = 10;

    public ProducerSafeWaitNotify(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

    @Override
    public void run() {
        int i = 0;
        while (!isInterrupted()) {
            synchronized (integerDeque) {
                try {
                    while (integerDeque.size() >= maxDequeSize) {
                        integerDeque.wait();
                    }
                    integerDeque.addFirst(i++);
                    integerDeque.notifyAll();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }
}
```

```
static class ConsumerSafeWaitNotify extends Thread {
    final Deque<Integer> integerDeque;

    public ConsumerSafeWaitNotify(Deque<Integer> integerDeque) {
        super();
        this.integerDeque = integerDeque;
    }

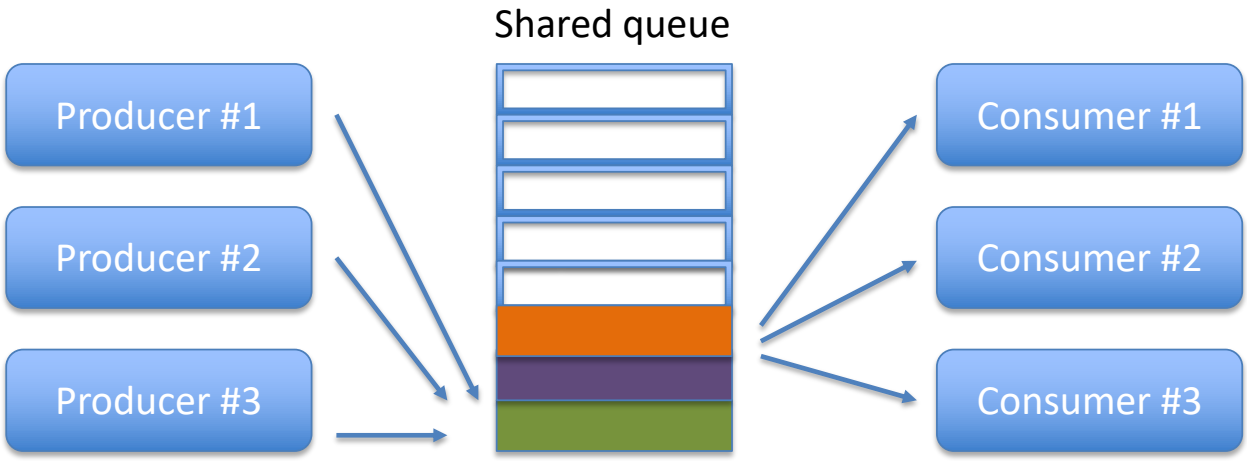
    @Override
    public void run() {
        while (!isInterrupted()) {
            synchronized (integerDeque) {
                try {
                    while (integerDeque.isEmpty()) {
                        integerDeque.wait();
                    }
                    IO.println(integerDeque.removeLast());
                    integerDeque.notifyAll();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
```

```
                    return;
                }
            }
        }
    }
}
```

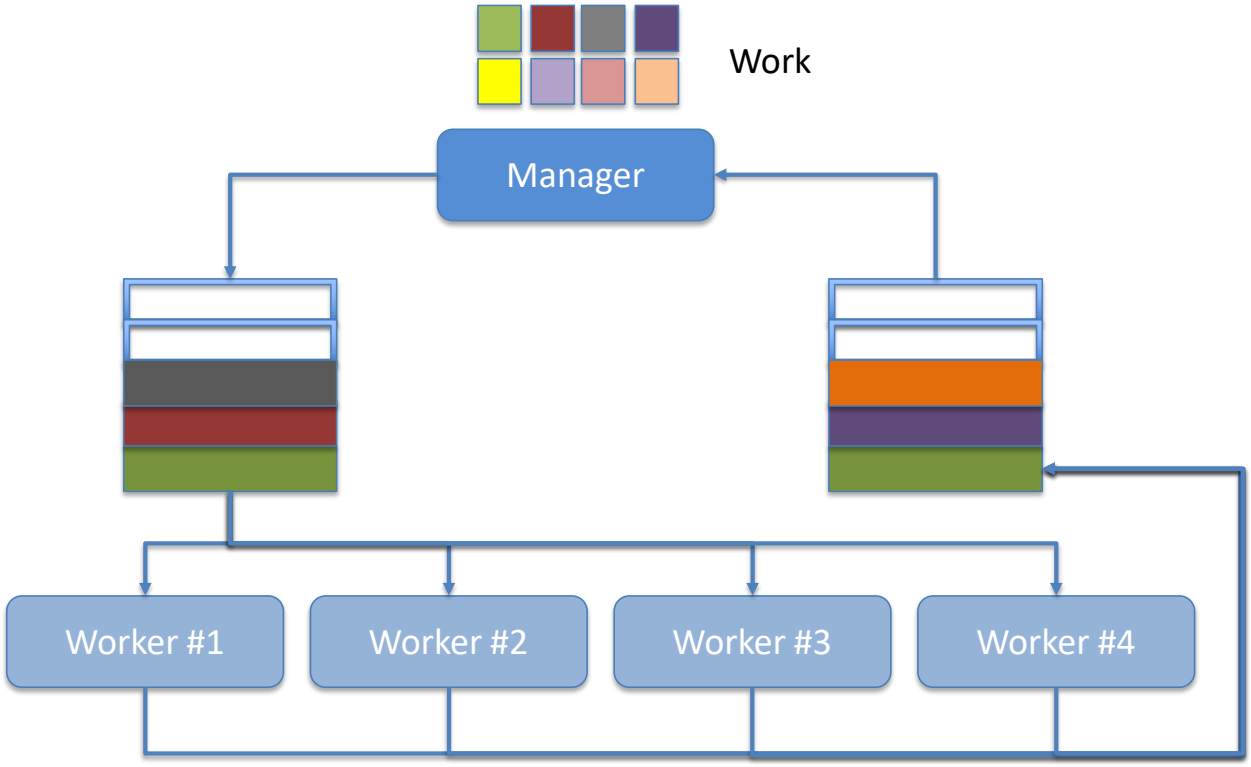
Multi-thread design patterns

Two useful patterns are producer/consumer and manager/workers. They do not exhaust concurrent-program design, but give concrete ways to separate work and coordination:

The producer/consumer pattern, where the producer thread pushes elements into a shared object and the consumer thread fetches (consumes) them. See this producer/consumer example for further details.



The manager/workers pattern, where a manager decomposes a complex task into subtasks, and assigns them to worker threads. See this manager/workers example for further details.



Bounded producer/consumer coordination

Use blocking operations instead of polling an empty queue:

```
import java.util.List;
import java.util.ArrayList;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.Executors;

public class BlockingQueueDemo {
    public static void main(String[] args) throws Exception {
        var queue = new ArrayBlockingQueue<Integer>(2);
        try (var executor = Executors.newFixedThreadPool(2)) {
            var producer = executor.submit(() -> {
                for (int value = 1; value <= 3; value++) {
                    queue.put(value);
                }
                return null;
            });
            var consumer = executor.submit(() -> {
                List<Integer> received = new ArrayList<>();
                for (int i = 0; i < 3; i++) {
                    received.add(queue.take());
                }
                return received;
            });
            try {
```

```
                producer.get();
                IO.println(consumer.get()); // [1, 2, 3]
            } finally {
                producer.cancel(true);
                consumer.cancel(true);
            }
        }
    }
}
```

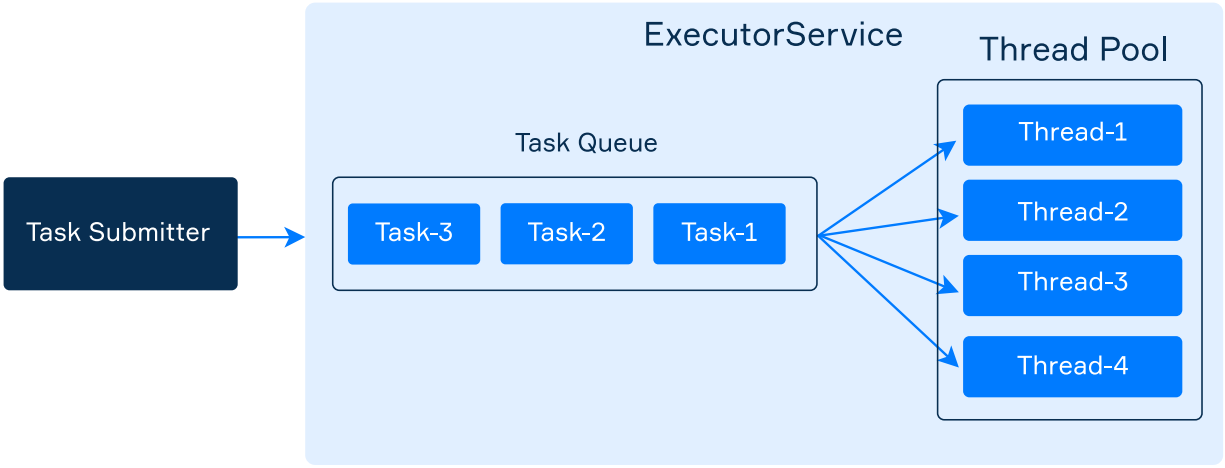
The capacity bounds the buffer, and `put`/`take` wait interruptibly when needed. Both tasks agree on exactly three messages; general pipelines need an explicit completion and failure protocol. Two workers are necessary here: a single worker could block producing before the consumer ever starts. The finally block requests cancellation if the main thread fails while waiting.

Resources

E8.4 — Executors Tasks and Futures

ExecutorService

`ExecutorService` separates **tasks** from execution and lifecycle management. Many implementations use a worker pool and task queue; not every executor has a reusable pool, as the virtual-thread-per-task executor illustrates later.



The executor manages execution mechanics, but the application still owns its lifetime, observes task failures, and controls demand and shared resources.

Creating executors

All types of executors are located in the *java.util.concurrent* package. This package also contains a convenient utility class *Executors* for creating different types of *ExecutorService*.

Reference — choose according to workload and resource limits:

An executor with a single thread - The simplest executor has only a single thread in the pool. It may be enough for async execution of rarely submitted and small tasks.

```
ExecutorService executor = Executors.newSingleThreadExecutor();
```

An executor with a fixed pool - It can execute multiple tasks concurrently and may speed up suitable workloads. A worker that terminates due to failure is replaced if needed to execute subsequent tasks. Choose the pool size based on available CPU capacity, blocking behavior, and external resource limits; measure under a representative workload.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
```

An executor with a growing pool - There is also an executor that automatically increases the number of threads as it needed and reuse previously constructed threads.

```
ExecutorService executor = Executors.newCachedThreadPool();
```

This can suit some short-lived asynchronous workloads, but thread creation can grow excessively under load. Choose a pool and admission policy from the workload and resource constraints rather than treating either factory as a universal default.

`newFixedThreadPool` limits worker threads but uses an **unbounded task queue**. `newCachedThreadPool` can create very many platform threads. Neither is automatic overload protection. Production systems may need bounded queues, rejection policies, timeouts, and limits on external resources.

An executor that schedules a task - If you need to perform the same task periodically or only once after the given delay, use the following executor:

```
ScheduledExecutorService executor = Executors.newSingleThreadScheduledExecutor();
```

The method *scheduleAtFixedRate* submits a periodic *Runnable* task that becomes enabled first after the given *initDelay*, and subsequently with the given *period*.

Scheduled time is not a guarantee of exact execution time. If a periodic invocation throws an uncaught exception, subsequent executions are suppressed; retain and observe the returned future when that failure matters.

```
ScheduledExecutorService executor = Executors.newSingleThreadScheduledExecutor();
executor.scheduleAtFixedRate(() ->
      IO.println(LocalTime.now() + ": Hello!"), 1000, 1000, TimeUnit.MILLISECONDS);
```

This kind of executor also has a method named *schedule* that starts a task only once after the given delay and another method *scheduleWithFixedDelay* that starts the task with a fixed wait after the previous one is completed.

Submitting tasks

An executor has the *submit* method that accepts a *Runnable* task to be executed. Since *Runnable* is a functional interface, it is possible to use a lambda expression as a task.

As an example, here we submit a task that prints *"Hello!"* to the standard output.

```
executor.submit(() -> IO.println("Hello!"));
```

`submit` returns a future without waiting for completion. Depending on the executor, work is queued or assigned to a new task thread. Rejected submissions can fail immediately. Exceptions from a submitted task are captured in its future; ignoring that future can hide a failure.

Stopping executors

Pool workers can remain alive waiting for work after a task finishes. Non-daemon platform workers can keep the JVM alive, so close executors you own. Virtual threads are daemon threads and have different JVM-lifetime behavior, but required work must still be awaited.

There are two methods for stopping executors:

1. `shutdown()` initiates orderly shutdown: it rejects new submissions and lets previously submitted tasks complete. It **does not wait**. Use `awaitTermination` to wait with a limit and handle timeout or interruption.

```
public static void main(String[] args) throws InterruptedException {
    ExecutorService executor = Executors.newFixedThreadPool(4);

    // submitting tasks

    executor.shutdown();

    boolean terminated = executor.awaitTermination(60, TimeUnit.MILLISECONDS);

    if (terminated) {
        IO.println("The executor was successfully stopped");
    } else {
        IO.println("Timeout elapsed before termination");
    }
}
```

2. `shutdownNow()` attempts to interrupt active tasks, stops processing waiting tasks, and returns tasks that had not started. It does **not** guarantee that running work stops or that the executor has terminated.

Scope-based executor ownership in Java 25

```
import java.util.concurrent.Executors;

public class ScopedExecutor {
    public static void main(String[] args) throws Exception {
        try (var executor = Executors.newFixedThreadPool(2)) {
            var result = executor.submit(() -> 21 * 2);
            IO.println(result.get()); // 42; task failures are observed here
        }
    }
}
```

`ExecutorService` has been `AutoCloseable` since Java 19. In Java 25, `close()` initiates orderly shutdown and waits for termination; it is **not a bounded-time shutdown policy**. A task that never finishes can prevent scope exit. At an application boundary, replace the broad demonstration `throws Exception` with an explicit failure policy.

An example: names of threads and tasks

In the following example, we create one executor with a pool consisting of four threads. We submit ten tasks to it and then analyze the results. Each task prints the name of the actual thread executing it, as well as the name of the task.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorDemo {
    private final static int POOL_SIZE = 4;
    private final static int NUMBER_OF_TASKS = 10;

    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(POOL_SIZE);

        for (int i = 0; i < NUMBER_OF_TASKS; i++) {
            int taskNumber = i;
            executor.submit(() -> {
                String taskName = "task-" + taskNumber;
                String threadName = Thread.currentThread().getName();
                System.out.printf("%s executes %s\n", threadName, taskName);
            });
        }

        executor.shutdown();
    }
}
```

If you launch this program many times, you will get a different output. Below is one of the possible outputs:

```
pool-1-thread-1 executes task-0
pool-1-thread-2 executes task-1
pool-1-thread-4 executes task-3
pool-1-thread-3 executes task-2
pool-1-thread-3 executes task-7
pool-1-thread-3 executes task-8
pool-1-thread-3 executes task-9
pool-1-thread-1 executes task-6
pool-1-thread-4 executes task-5
pool-1-thread-2 executes task-4
```

This run illustrates worker reuse. Which worker handles each task, and whether every worker performs visible work, are not contractual guarantees.

For CPU-bound work, available processors can be a starting point for measurement, not a universal sizing formula. I/O waits, resource limits, container configuration, and competing workloads also matter.

```
int poolSize = Runtime.getRuntime().availableProcessors();
ExecutorService executor = Executors.newFixedThreadPool(poolSize);
```

Callable and Future

Sometimes you need not only to execute a task in an executor but also to return a result of this task to the calling code. Java supports another class of tasks named *Callable* that returns the result.

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Callable is a generic interface where the type parameter *V* determines the type of the result. Since it is a **functional interface**, we can use it together with lambda expressions. Here is a *Callable* that emulates a long-running task and returns a number that was ”calculated”.

```
Callable<Integer> generator = () -> {
    TimeUnit.SECONDS.sleep(5);
    return 700000;
};
```

Submitting a Callable and obtaining a Future

When we submit a *Callable* to an executor service, it cannot return a result directly since the *submit* method does not wait until the task completes. Instead, an executor returns a special object called *Future* that wraps the actual result that may not even exist yet. This object represents the result of an asynchronous computation (task).

```
ExecutorService executor = Executors.newSingleThreadExecutor();

Future<Integer> future = executor.submit(() -> {
    TimeUnit.SECONDS.sleep(5);
    return 700000;
});
```

`isDone()` reports that the future reached a terminal state, including successful completion, failure, or cancellation. It does not mean that a successful result is available. Immediately after submission either true or false is possible.

```
IO.println(future.isDone()); // most likely it is false
```

Getting the actual result of a task

`get()` waits for completion and either returns the result or reports failure. `ExecutionException` wraps a task failure, `InterruptedException` reports interrupted waiting, and `CancellationException` reports cancellation. A successful `get` also provides the task-to-caller memory visibility guarantee.

```
int result = future.get();
```

To prevent this, waiting too much time, there is also an overloaded version of *get* with a waiting timeout. In this case, the calling thread waits for 10 seconds most for the computation to complete. If the timeout ends, the method throws a *TimeoutException*.

```
int result = future.get(10, TimeUnit.SECONDS); // it blocks the current thread
```

Cancelling a task

The *Future* class provides an instance method named *cancel* that attempts to cancel the execution of a task. An attempt will fail if:

- the task has already completed
- the task has already been canceled
- the task could not be canceled for some other reason

If cancellation succeeds before execution, the task should not run. If execution has already begun, cancelling the future does not undo side effects or prove that task code has stopped.

The boolean controls whether an attempt to interrupt a running task is permitted. `cancel(false)` can mark a running future cancelled without interrupting its task; it is not a reliable “cancel only if queued” test.

```
future1.cancel(true);  // try to cancel even if the task is executing now
future2.cancel(false); // do not interrupt a task that is already executing
```

With `cancel(true)`, termination remains cooperative and can be delayed by non-interruptible operations or ignored interrupts. No general forced-stop guarantee is provided.

If someone invokes *future.get()* at a successfully canceled task, the method throws an unchecked *CancellationException*. If you do not want to deal with it, you may check whether a task was canceled by invoking *isCancelled()*.

State checks can race with completion or cancellation. Code that requires the result should handle the actual outcome from `get`, not assume a preceding check makes failure impossible. A timeout from timed `get` does not automatically cancel the task.

Virtual threads in Java 25

Note — modern execution choice, same correctness rules:

```
import java.util.concurrent.Executors;

public class VirtualThreadDemo {
    public static void main(String[] args) throws Exception {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            var result = executor.submit(() -> {
                Thread.sleep(10); // Stand-in for blocking work, not synchronization
                return "virtual=" + Thread.currentThread().isVirtual();
            });
            IO.println(result.get()); // virtual=true
        }
    }
}
```

Virtual threads can make large numbers of blocking tasks practical. They do not make CPU computation intrinsically faster, remove races, or eliminate limits on database connections and remote services. Do not pool virtual threads just to reuse them; limit scarce resources explicitly.

> **Note:** JDK 24 removed monitor-related virtual-thread pinning (JEP 491). In Java 25, blocking in a `synchronized` method or block is no longer by itself a reason to replace the monitor with `ReentrantLock`. Contention still serializes access, and native/foreign calls or class initialization can still involve pinning. Choose lock APIs for required semantics, not outdated blanket advice. The old `jdk.tracePinnedThreads` property no longer has an effect; use current JFR diagnostics when investigating pinning.

Scoped values in Java 25

Note — bounded context propagation, not shared mutable state. `ScopedValue` is a stable Java 25 API for making a context binding available during a dynamically bounded call. It is useful for some read-only request-context uses of `ThreadLocal`, not a replacement for ordinary method parameters everywhere.

```
public class RequestContextDemo {
    private static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();

    private static void handle() {
        IO.println(REQUEST_ID.get());
    }

    public static void main(String[] args) {
        ScopedValue.where(REQUEST_ID, "request-42").run(RequestContextDemo::handle);
        IO.println(REQUEST_ID.isBound()); // false: binding ended with run
    }
}
```

The output is `request-42`, then `false`. The binding has a bounded lifetime, but a mutable object stored in it would still need its own concurrency policy. Binding does not automatically propagate into arbitrary threads or tasks submitted to an executor; this example deliberately stays on one thread.

Reference — stable versus preview: virtual threads and scoped values are stable in Java 25. `StructuredTaskScope` belongs to the fifth preview of structured concurrency (JEP 505), not the stable executor API. The try-with-resources executor examples above do not use structured concurrency. Preview APIs require an explicit opt-in and version-specific code; none of this lesson's examples require `--enable-preview`.

Resources

- https://dev.java/learn/new-features/virtual-threads/
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html
- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html
- https://www.baeldung.com/java-completablefuture